BUILDING AN INTERACTIVE ENGINEERING PORTFOLIO

A Textbook and Guided Build of the Website, from the Theory of the Frontend to the Last Line of Code

Gopolang Makgothi

Edition 0.5, living document

Revised as the website grows

Contents

Introduction . . . 14

How to Use This Book . . . 16

Book Map . . . 18

Part I. Foundations: The Theory of the Frontend . . . 19

Chapter 1. How the Web Works . . . 20

1.1 Meet the Client and the Server . . . 20

1.2 Addresses: URLs and DNS . . . 20

1.3 The Conversation: HTTP . . . 21

1.4 What the Server Sends . . . 23

1.5 Static Sites and Applications . . . 23

1.6 What Travels on the Wire . . . 24

1.7 The Cost of a Request . . . 25

1.8 Caching . . . 26

1.9 Hosting a Static Site . . . 27

Chapter 2. The Browser as a Machine . . . 29

2.1 From Bytes to Pixels: The Critical Rendering Path . . . 29

2.2 JavaScript and the Parser . . . 31

2.3 The Main Thread and the Frame Budget . . . 32

2.4 The Graphics Processor . . . 33

2.5 Layout Thrashing . . . 33

2.6 Measure, Don't Guess . . . 34

Chapter 3. Three Languages, Three Jobs . . . 37

3.1 Structure: HTML . . . 37

3.2 Presentation: CSS . . . 38

3.3 Behaviour: JavaScript . . . 40

3.4 The Document Tree Connects Them . . . 41

3.5 Progressive Enhancement . . . 42

Chapter 4. Space and Size 45
4.1 What Is a Pixel, Really? 45
4.2 The Canvas Problem 46
4.3 The Box Model 47
4.4 Flow, Flex and Grid 48
4.5 Units 49
4.6 Responsive Design 51

Chapter 5. Events, State and the Event Loop 53
5.1 JavaScript Does One Thing at a Time 53
5.2 The Stack, the Queues and the Loop 53
5.3 Events and Handlers 55
5.4 State 57
5.5 Rendering as a Function of State 57

Chapter 6. Time, Motion and Animation 61
6.1 The Animation Loop 61
6.2 Delta Time: Motion That Does Not Depend on Your Computer 62
6.3 Interpolation and Easing 63
6.4 Smoothing Towards a Target 64
6.5 Timelines and Sequences 66
6.6 CSS Animation 66
6.7 Motion with Manners 67

Chapter 7. Graphics on the Web 69
7.1 Raster and Vector 69
7.2 The Canvas 2D Model 70
7.3 SVG in Brief 71
7.4 Three Dimensions: The Idea 72
7.5 Meshes, Materials and Light 75
7.6 Keeping a Scene Fast 76

Chapter 8. Quality: Access, Speed, Safety and Tools 79
8.1 Accessibility 79

8.2 Performance . . . 82
8.3 Security . . . 83
8.4 Tools . . . 84

Part II. The Tech Stack . . . 86

Chapter 9. HTML: The Skeleton . . . 87
Words You Will Meet . . . 88
9.1 The Skeleton of Every Page . . . 88
9.2 How the Parser Thinks . . . 90
9.3 Text and Structure . . . 91
9.4 Landmarks and Sectioning . . . 93
9.5 Links, Images and Media . . . 94
9.6 Tables . . . 96
9.7 Forms . . . 97
9.8 The Head: Metadata That Matters . . . 99
9.9 Accessibility: When Native Elements Are Not Enough . . . 100
9.10 Hooks for CSS and Script . . . 101
9.11 Validation and Debugging . . . 101
Worked Example: A Small Page, Built Properly . . . 102
Worked Example: A Specification Table and a Form . . . 102
Practice Problems A: The Skeleton and the Parser . . . 103
Practice Problems B: Content, Links, Images, Tables and Forms . . . 104
Practice Problems C: The Head, Accessibility and Hooks . . . 104
Mini Project: A Documentation Page for a Robot . . . 105

Chapter 10. CSS: The Look . . . 107
Words You Will Meet . . . 108
10.1 How the Browser Reads Your Rules . . . 109
10.2 Writing Selectors Well . . . 109
10.3 The Box, Margins and Display . . . 110
10.4 Layout: Flow, Flex and Grid . . . 111
10.5 Position and Stacking . . . 113

10.6 Colour, Type and Space 114
10.7 Pseudo-elements and Pseudo-classes 116
10.8 Transforms, Transitions and Animations 117
10.9 Responsive Design in Practice 118
10.10 Organising a Stylesheet 119
Worked Example: A Responsive Card Row 120
Worked Example: A Panel From the Website, Explained 120
Practice Problems A: Selectors, the Cascade and the Box 121
Practice Problems B: Layout, Position and Stacking 121
Practice Problems C: Colour, Type, Motion and Organisation 122
Mini Project: Style the Robot Page 122

Chapter 11. JavaScript: The Behaviour 124
Words You Will Meet 125
11.1 Running JavaScript 125
11.2 Values and Types 126
11.3 Variables and Scope 127
11.4 Operators and Expressions 128
11.5 Control Flow 129
11.6 Functions 130
11.7 Arrays 132
11.8 Objects, Map and Set 134
11.9 Strings and Numbers in Practice 135
11.10 Classes and Prototypes 136
11.11 Errors and Debugging 138
11.12 The DOM in Practice 139
11.13 Asynchronous Code: Promises and async/await 140
11.14 Timers, Animation Frames and Rate Limiting 142
11.15 Modules 143
11.16 Working with Data 143
11.17 Writing Code That Lasts 144
Practice Problems A: Values, Variables and Flow 145

Practice Problems B: Functions, Arrays and Objects . . . 146
Practice Problems C: Classes, Errors, the DOM and Async . . . 146
Practice Problems D: Modules, Data and Tests . . . 147
Mini Projects . . . 147

Chapter 12. React: Interfaces from Components . . . 150
Words You Will Meet . . . 151
12.1 What React Is, and Is Not . . . 151
12.2 Components and JSX . . . 152
12.3 Props Down, Events Up . . . 153
12.4 State . . . 154
12.5 Rendering Lists and Keys . . . 155
12.6 Effects: Talking to the Outside World . . . 155
12.7 Thinking in React: What Is State? . . . 156
12.8 Forms and Controlled Inputs . . . 157
12.9 Refs: Things Outside React . . . 157
12.10 The Render Cycle . . . 158
12.11 Getting React Into a Page . . . 159
Worked Example: A Stopwatch With a Clean Effect . . . 159
Practice Problems A: Components, Props and State . . . 160
Practice Problems B: Lists, Effects, Forms and Refs . . . 160
Mini Project: A Part Picker . . . 161

Chapter 13. Canvas and SVG: Drawing in the Browser . . . 163
Words You Will Meet . . . 163
13.1 The Drawing Recipe . . . 164
13.2 Paths: Lines, Arcs and Curves . . . 164
13.3 Mapping Data to Pixels, and Ticks . . . 166
13.4 Transforms . . . 167
13.5 Clipping, Gradients, Text and Images . . . 167
13.6 Offscreen Canvases and Layers . . . 168
13.7 Animation on a Canvas . . . 169

13.8 Interaction: Hit Testing, Hover and Drag . . . 169
13.9 SVG: The Practical Bits . . . 170
13.10 Choosing, and Mixing . . . 172
Worked Example: A Bar Chart on a Canvas . . . 172
Worked Example: A Block Diagram in SVG . . . 173
Practice Problems A: Canvas . . . 173
Practice Problems B: SVG and Choosing . . . 174
Mini Project: A Live Oscilloscope . . . 174

Chapter 14. GSAP: Animation Without the Headaches . . . 176
Words You Will Meet . . . 177
14.1 Loading It . . . 177
14.2 Tweens . . . 178
14.3 Easing . . . 178
14.4 Staggers . . . 179
14.5 Timelines . . . 179
14.6 ScrollTrigger . . . 181
14.7 Reduced Motion, Responsiveness and Cleaning Up . . . 182
14.8 When GSAP Earns Its Place . . . 183
Practice Problems A: Tweens and Timelines . . . 183
Practice Problems B: Scroll, Manners and Judgement . . . 183
Mini Project: A Scroll Story, Twice . . . 184

Chapter 15. three.js: A 3D World in Your Page . . . 186
Words You Will Meet . . . 187
15.1 The Smallest Scene . . . 188
15.2 The Renderer: Size, Pixel Ratio, Colour and Tone . . . 189
15.3 Cameras . . . 189
15.4 Geometry . . . 191
15.5 Materials . . . 192
15.6 Lights, Shadows and the Environment . . . 193
15.7 The Scene Tree . . . 194

15.8 Animation . . . 195
15.9 Picking: What Did the Visitor Click? . . . 195
15.10 Textures and Models . . . 196
15.11 Keeping It Fast . . . 196
15.12 Bundling a three.js Page . . . 197
Worked Example: A Needle Gauge in 3D . . . 197
Practice Problems A: Scene, Camera and Tree . . . 197
Practice Problems B: Materials, Lights, Picking and Speed . . . 198
Mini Project: An Instrument You Can Take Apart . . . 198

Chapter 16. Build Tools and Git: The Workshop . . . 201
Words You Will Meet . . . 202
16.1 Node.js and npm . . . 202
16.2 Version Numbers . . . 203
16.3 The Bundler . . . 204
16.4 Git: A Time Machine for Your Project . . . 205
16.5 Branches and Merging . . . 206
16.6 Undoing Things . . . 207
16.7 Remotes and GitHub . . . 208
16.8 Publishing a Static Site . . . 208
16.9 The Editor, the Formatter and the Linter . . . 209
16.10 When the Build Breaks . . . 209
Worked Example: From Nothing to a Bundled Page . . . 209
Practice Problems A: Packages and Builds . . . 210
Practice Problems B: Git . . . 210
Mini Project: Set Up the Robot Page for Real . . . 211

Chapter 17. C++: From Your First Program to Your Own Robot . . . 213
Words You Will Meet . . . 214
17.1 Your First Program . . . 214
17.2 Numbers, Types and Arithmetic . . . 216
17.3 Reading Input . . . 218

Practice Problems A: First Steps . . . 219
17.4 Making Decisions . . . 219
17.5 Loops . . . 220
17.6 Functions . . . 222
Practice Problems B: Decisions, Loops, Functions . . . 223
17.7 Arrays and Vectors . . . 223
17.8 Strings and the Command Line . . . 225
17.9 Structs and Classes . . . 226
Practice Problems C: Collections, Text and Classes . . . 227
17.10 Choosing Between States: enum and switch . . . 228
17.11 References, Pointers and Cleaning Up After Yourself . . . 229
17.12 Lambdas and the Standard Algorithms . . . 231
17.13 Reading and Writing Files . . . 232
17.14 Splitting a Program Across Files . . . 233
Practice Problems D: States, Memory, Lambdas, Files . . . 234
17.15 Numerical Simulation: How Wrong Can a Step Be? . . . 235
17.16 A PID Controller in Code . . . 236
17.17 Testing and Debugging . . . 238
17.18 Templates and Operators: Writing Once, Using Many Times . . . 240
Practice Problems E and F: Numerics, PID, Tests, Templates . . . 241
17.19 Five Mini Projects . . . 241
What Comes Next . . . 246

Part III. Six Guided Projects . . . 247

Chapter 18. Project 1: A Landing Page (HTML and CSS) . . . 248
18.1 Step 1: The Skeleton and the Content . . . 248
18.2 Step 2: Reset and Variables . . . 249
18.3 Step 3: Layout . . . 249
18.4 Step 4: Hover, Focus and Phone . . . 250

Chapter 19. Project 2: A Filterable Gallery (JavaScript and State) . . . 251
19.1 Step 1: The Data and the State . . . 251

19.2 Step 2: The Page . . . 251
19.3 Step 3: One Function That Draws Everything . . . 252
19.4 Step 4: One Listener on the Parent . . . 252

Chapter 20. Project 3: An Animated Scroll Story (CSS, IntersectionObserver, GSAP) . . . 253
20.1 Step 1: The Hidden State in CSS . . . 253
20.2 Step 2: Noticing When a Section Appears . . . 254
20.3 Step 3: The Same Page with GSAP . . . 254

Chapter 21. Project 4: An Interactive Physics Chart (Canvas 2D) . . . 256
21.1 Step 1: Check the Numbers by Hand First . . . 256
21.2 Step 2: The Page . . . 256
21.3 Step 3: Computing the Curve . . . 256
21.4 Step 4: Drawing It . . . 258

Chapter 22. Project 5: A 3D Model You Can Take Apart (three.js) . . . 260
22.1 Step 1: Plan the Data First . . . 260
22.2 Step 2: The Scene . . . 261
22.3 Step 3: Explode and Resize . . . 261
22.4 Step 4: Picking and the Panel . . . 262
22.5 Step 5: Bundle and Run . . . 262

Chapter 23. Project 6: A Cart That Brakes Before the Wall (C++) . . . 263
23.1 Step 1: Check the Numbers by Hand First . . . 263
23.2 Step 2: The Constants and the Cart . . . 264
23.3 Step 3: The Loop and the State Machine . . . 265
23.4 Step 4: Plot It . . . 265
23.5 Step 5: The Soft Stop (Pause) . . . 266

Part IV. Advanced Topics . . . 267

Chapter 24. Advanced CSS . . . 268
24.1 Cascade Layers . . . 268
24.2 Container Queries . . . 268
24.3 Logical Properties, :has() and Custom Property Tricks . . . 269

Chapter 25. Advanced JavaScript 270
25.1 Modules, Bundling and Dynamic Import 270
25.2 Closures, `this` and Pitfalls 270
25.3 Web Workers 270
Chapter 26. Advanced React 272
26.1 Effects, Cleanup and Refs 272
26.2 Avoiding Wasted Renders 272
26.3 Custom Hooks and Context 273
Chapter 27. Advanced Canvas 274
27.1 Layers and Offscreen Canvases 274
27.2 Only Draw What the Visitor Can See 274
27.3 Avoiding Blurry Lines 275
Chapter 28. Advanced GSAP 276
28.1 Timelines as State Machines 276
28.2 Scrubbed Scroll Animations 276
28.3 Cleaning Up 276
Chapter 29. Advanced three.js 278
29.1 Physically Based Materials and Studios 278
29.2 Post-processing: Ambient Occlusion and Tone Mapping 278
29.3 Instancing and Merging 278
29.4 Disposal and Warm-up 279
Chapter 30. Advanced Accessibility and Performance 280
30.1 Auditing Seriously 280
30.2 Performance Budgets 280
30.3 Measure on Real, Modest Hardware 280
Chapter 31. Advanced C++ 281
31.1 When a Time Step Is Too Big 281
31.2 A Twin to Prove It Right 282
31.3 Habits That Keep C++ Honest 283

31.4 Where This Goes Next 283

Part V. The Guided Build of the Website 284

Chapter 32. The Shape of the Whole Site 285

32.1 What Is in the Folder 285

32.2 What Each Page Is Built With 286

32.3 The Shared Design Language 287

Chapter 33. The HTML of a Page, Line by Line 288

33.1 The Head 288

33.2 The Style Block 289

33.3 The Body: Navigation and Hero 290

33.4 The Main Content 291

Chapter 34. The CSS, Rule by Rule 292

34.1 The Equation Box 292

34.2 The Instrument Panel 293

34.3 The Slider 294

34.4 Responsive Behaviour 295

Chapter 35. The JavaScript: the Spiral Climb, Function by Function 296

35.1 Making the Canvas Sharp 296

35.2 The Physics: Where the Numbers Come From 297

35.3 The Loop That Draws It 298

Chapter 36. The Stability Model, Function by Function 300

36.1 Building the Equations 300

36.2 The Characteristic Polynomial 301

36.3 The Routh Array 302

Chapter 37. The Panels: How the Page Fits Together 303

37.1 One Shared State and a List of Subscribers 303

37.2 Drawing Only When Visible 304

Chapter 38. The Patrick Page: One Model Drives Everything 305

38.1 The Motor . . . 306
38.2 The Wheel Grip, and the Rubber Bands . . . 307
38.3 The Arm: Two Circles Meeting . . . 308
38.4 Posing the Robot . . . 309
38.5 Looking Around and Picking . . . 310

Chapter 39. Build and Publish . . . 312
39.1 The Two Builds . . . 312
39.2 Publishing . . . 313

Chapter 40. React and GSAP in This Site . . . 314
40.1 The React File . . . 314
40.2 What the Portfolio Does Instead: Scroll Scrubbing by Hand . . . 314
40.3 Where GSAP Would Fit . . . 315

Chapter 41. The Claude Skills Behind the Site . . . 317
41.1 Anatomy: Isometric Figures . . . 317
41.2 Teardown: Interactive 3D Machines . . . 318

Chapter 42. Building Patrick From Scratch: The Robot's Program in Nine Stages . . . 321
42.1 Stage 1: One Motor . . . 322
42.2 Stage 2: A Wheel on the Ground . . . 324
42.3 Stage 3: Two Wheels, and Where the Robot Is . . . 326
42.4 Stage 4: The Robot Learns to See . . . 327
42.5 Stage 5: Approach, Brake, and Stop Where the Sensor Says . . . 329
42.6 Stage 6: The Arm and the Claw . . . 330
42.7 Stage 7: One Ingredient, Pantry to Table . . . 331
42.8 Stage 8: Five Ingredients . . . 333
42.9 Stage 9: The Soft Stop, and Proving It Right . . . 334
Try It Yourself . . . 336

Answer Key . . . 337

Introduction

Welcome. If you picked up this book because you want to know how a website like this one gets built, you are in the right place. If you picked it up because you are a little afraid of the whole subject, you are in an even better place, because this book was written for you.

The website in question is an engineering portfolio. In its pages you can fly a spiral climb, watch a phugoid oscillation draw itself across a graph, and pull a small robot named Patrick to pieces to see what each part does. It looks like the work of a big team with a big budget. It is not. It is a handful of plain files, a browser, a text editor, and a pile of ideas that are far simpler than they first look. This book hands you those ideas, one at a time, and then shows you exactly how they were used.

Why this book explains everything twice

Most coding books tell you what to type. This one tells you **why**. Every time you meet a function, a setting, or a decision, the book stops and answers three questions: why is it done this way, why does that matter, and what goes wrong if you do it some other way? You will see the answer in a shaded box with a heading that says *Why Do It This Way?*

That sounds like extra reading, and it is. It is also the difference between a person who can copy a recipe and a person who can cook. If you only know *what* to type, the first time something breaks you are stuck. If you know *why* it was typed, you can usually see the problem before you finish reading the error message.

How the book is built

The book has five parts, and they build on each other.

Part I is theory. It explains how the web works and how a browser thinks, in plain English, before you write any serious code. Every chapter ends with questions to check what you have taken in.

Part II goes through the tools one at a time, and each is a complete course: HTML, CSS, JavaScript, React, canvas and SVG drawing, GSAP, three.js, the build tools with Git, and C++, from a first program to a controller for a robot of your own. Each tool gets a list of words you will meet, the theory of how it works, diagrams, worked examples, practice problems with full answers, and mini projects that have nothing to do with the website. A skill you can only use on one project is not really a skill. Every program whose output is printed in these chapters was run to produce it; the files are in the `Guide/code` folder.

Part III is six guided projects, from a first page up to a 3D model you can take apart and a C++ cart that brakes before a wall. Each one uses what you learned in Part II, and each ends with a short quiz.

Part IV covers the advanced material for each tool, including stable simulation and twin testing in C++.

Part V is a guided tour of the real website, one section, function and decision at a time. It includes a guided project that builds the robot's whole C++ program from an empty file in nine runnable stages, and covers the Claude skills that helped build it.

A living book

This is Edition 0.5. It will change. Whenever the website grows or a better way of explaining something turns up, the book grows with it. The Book Map after this introduction shows what the book contains today.

A promise about honesty

You will notice that this book sometimes says "the site does not do this". For example, the website does not use GSAP, and the React file in the project is not yet connected to any page. The book says so plainly, because a guide that pretends a project does things it does not do is worse than no guide at all. Where a tool is not used, Part V tells you where it would fit and why.

How to Use This Book

The little boxes

You will see five kinds of shaded boxes. Learn to love them.

> **TIP**
>
> A Tip is a shortcut or a habit that saves you time or trouble. You can skip a Tip and the book will still make sense, but you will probably wish you had read it.

> **WARNING**
>
> A Warning marks something that will bite you. If a Warning is there, someone (usually the author) has already been bitten.

> **REMEMBER**
>
> A Remember box repeats an idea that the rest of the chapter leans on. If you forget it, the next page will feel harder than it should.

> **TECHNICAL STUFF: Technical Stuff**
>
> Technical Stuff is for the curious. It explains what is going on underneath. You can safely skip it on a first reading and come back later.

> **WHY DO IT THIS WAY? The Why Box**
>
> **What it is.** The thing being explained: a function, a setting, a line of code.
>
> **Why it is done this way.** The reason this approach was chosen over the alternatives.
>
> **Why it matters.** What this buys you: speed, safety, clarity, or something else.
>
> **What happens if you do not.** The failure you would see. This is often the most useful line of the four.

Two more boxes frame each chapter. **In This Chapter** lists what you will be able to do by the end, and **What You Learned** is the chapter in ten lines, for revision. The figures are numbered within each chapter and drawn from the same numbers as the text, so a curve in a figure is the curve the formula gives.

Questions and answers

Part I ends every chapter with **Check Your Knowledge** (short questions) and **Review Problems** (longer ones, often with a calculation). Part II chapters have several sets of **Practice Problems** and end with **Mini Projects**, and Part III ends each project with a **Check Yourself** quiz. All the answers are at the back of the book, worked line by line. Please try each question before you peek. Your brain only remembers what it has had to dig for.

Typography

The text is in Times New Roman at 11 points, with 1.5 line spacing, justified. Chapter and section headings are 13 points and subsection headings are 12. Code and tables are set tighter than the running text, because a code listing at 1.5 spacing is harder to read, not easier.

What you need

A modern browser, a text editor (Visual Studio Code is a good one), and a willingness to try things. You do not need any programming experience. You will meet Node.js and npm in Part II, when you first need them, and not before.

How to read it

New to all this? Start at Chapter 1 and do the problems. Already building websites? Skim Part I, pick the Part II chapters you are weakest in, and jump to the projects. Only want to know how this particular site works? Part V stands on its own, and points back to earlier chapters for any idea it needs.

Book Map

The table lists every part of the book and where it stands today.

Part	What is in it	State
I. Foundations	The theory of the frontend: the web, the browser, the three languages, space and size, events, animation, graphics, and quality. Eight chapters with objectives, diagrams, worked examples, questions and summaries.	Written, second edition
II. The Tech Stack	Nine full courses: HTML, CSS, JavaScript (17 sections, four runnable mini projects), React (every example rendered for real), canvas and SVG, GSAP (timelines checked by GSAP), three.js (numbers checked by three.js), build tools with Git (real builds and a real history), and C++ (18 sections, five mini projects). Each with words, theory, diagrams, worked examples, problem sets with line answers and mini projects.	Written, second edition
III. Six Guided Projects	A landing page, a responsive gallery, an animated story, an interactive physics chart, a 3D model you can take apart, and a C++ cart that brakes before a wall. Each ends with a quiz.	Written, first edition
IV. Advanced Topics	Advanced CSS, JavaScript, React, canvas, GSAP, three.js, accessibility with performance, and C++ (stable simulation, twin testing).	Written, first edition
V. The Guided Build	The real website: its shape, the HTML, CSS and JavaScript of the flight dynamics page, the stability model, the Patrick robot page, the robot's C++ program built from scratch in nine stages, the build pipeline, React and GSAP (honestly), and the Claude skills.	Written, first edition

"First edition" means the chapter is complete enough to learn from, and will be deepened as the site grows. Parts I and II are now at their second edition. Part IV is the next to be deepened, and Part V will grow as more of the site's pages are read line by line.

Part I. Foundations: The Theory of the Frontend

Eight chapters on the ideas that everything else rests on

Chapter 1. How the Web Works

Here is a secret that most beginners are never told: a website is not a thing. It is a conversation. One computer asks, another computer answers, and the page you see is just what happens when the answer arrives. Once you picture that conversation, a great deal of web development stops being mysterious. This chapter introduces the two speakers, the language they use, and what each exchange costs.

In This Chapter

- Tell a client from a server, and explain why one machine can be both.
- Take a URL apart into its six pieces, and say which piece never leaves the browser.
- Read an HTTP request and response, including the status code and the `Content-Type` header.
- Explain what a packet, a connection and an encrypted connection are, and why each costs a round trip.
- Estimate how long a file takes to arrive from its size, the bandwidth and the round-trip time.
- Say what caching is, what goes wrong with it, and how a changed file name fixes that.

1.1 Meet the Client and the Server

The two speakers have names. The **client** is whoever asks. In this book the client is always a web browser on a phone or a laptop. The **server** is whoever answers: a computer, usually far away, that keeps files and hands them to anyone who asks correctly.

Notice that these are *jobs*, not kinds of machine. Your laptop can run a browser (a client) and a tiny test server at the same moment, and that is exactly what happens when you try out the pages of this website on your own computer.

> **REMEMBER**
> The server does not draw the page. It only hands over text and data. Your browser reads that text and does all the drawing. That is why a page looks the same in Johannesburg and in Oslo, and it is why the work the browser has to do *after* the files arrive is so important. Chapter 2 is all about that work.

1.2 Addresses: URLs and DNS

Every file on the web has an address, called a **URL** (uniform resource locator). A URL looks like one long word, but it has parts, and you should be able to name them.

```
https://example.com:443/pages/flight-dynamics.html?view=modes#routh
\___/   \_________/ \_/ \_____________________________/ \________/ \____/
scheme     host    port           path                   query   fragment
```

The **scheme** says which language to speak (`https` means "the web language, with encryption"). The **host** is the name of the server. The **port** picks one of the many services a server might run, and `443` is

the default for `https`, so you almost never see it. The **path** names the file. The **query** passes extra settings. The **fragment** names a spot inside the page, and here is a detail that trips people up: the fragment is *never sent to the server*. The browser keeps it and uses it to scroll.

A host like `example.com` is a name for humans. Computers use numbers called **IP addresses**, and a kind of phone book called the **Domain Name System (DNS)** converts one into the other. Before your browser can say anything to `example.com`, it has to look the name up, and that takes a moment. Browsers remember the answer for a while, which is why a second visit to a site is quicker than the first.

1.3 The Conversation: HTTP

Once the browser knows where the server is, it sends an **HTTP request**. A request is plain text with a simple shape: the **method** and the path, followed by **headers** that describe the client.

```
GET /flight-dynamics.html HTTP/1.1
Host: example.com
Accept: text/html
Accept-Encoding: gzip, br
```

`GET` means "please send me this". The server answers with an **HTTP response**: a **status code**, some headers, and then the **body**, which is the file itself.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 28457
Cache-Control: max-age=600

<!DOCTYPE html> ...
```

The status code is a quick summary of how things went. Codes in the 200s mean success. The 300s mean "it moved". The 400s mean *you* made a mistake (the famous `404 Not Found` means you asked for a path that does not exist). The 500s mean the *server* made a mistake.

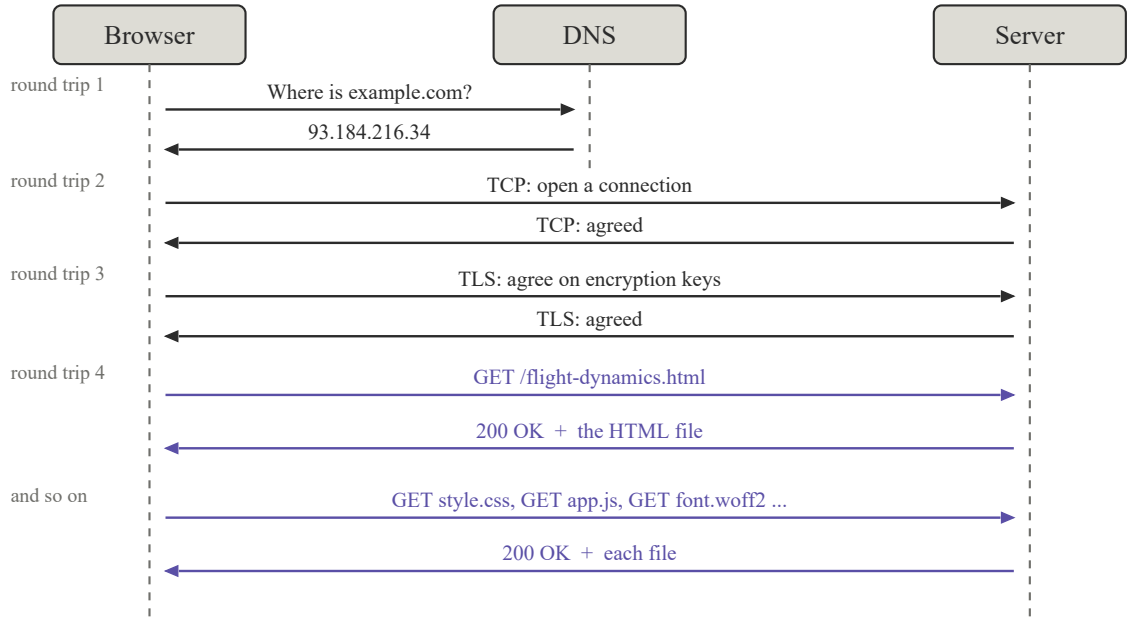


Figure 1.1. The whole conversation for one page: a name lookup, a connection, an encryption handshake, and only then the request for the page, followed by a request for every file the page mentions. Each pair of arrows is one round trip.

Code	Name	What it means
200	OK	Here is what you asked for.
301 / 302	Moved	Ask at this other address instead (permanently / for now).
304	Not Modified	Your cached copy is still good; use it.
400	Bad Request	The request itself was malformed.
403	Forbidden	You are not allowed to see this.
404	Not Found	No such path. The most common error on the web, and usually a typo in a link.
500	Internal Server Error	The server's program crashed.
503	Service Unavailable	The server is overloaded or down for maintenance.

WHY DO IT THIS WAY? Sending a Content-Type header

What it is. A header on the response that says what kind of thing the body is, for example `text/html` or `text/css` or `application/javascript`.

Why it is done this way. A file is just a pile of bytes. The server has no other way to tell the browser whether those bytes are a page, a picture, or a script.

Why it matters. The browser decides what to *do* with the bytes based on this header. HTML gets parsed into a page. CSS gets applied. A script gets run.

> **What happens if you do not.** The wrong header gives baffling failures. A stylesheet sent as plain text is ignored, and a script sent as plain text may refuse to run. When a page looks completely unstyled for no reason you can see, check the Content-Type before you check anything else.

One page, many requests

A page is never a single request. When your browser reads the first HTML file, it spots references to other files (stylesheets, scripts, images, fonts) and asks for each one. Those may mention still more files. The flight dynamics page of this website starts with one HTML file and then asks for a font stylesheet, the KaTeX stylesheet, two KaTeX scripts, and its own script. You can watch it happen in the **Network** tab of your browser's developer tools, which lists every request with its status, size and time. Open it now on any page. It is oddly satisfying.

1.4 What the Server Sends

The files a browser receives come in a few kinds. This book is mostly about the first three.

Kind	Typical file	What the browser does with it
HTML	`page.html`	Builds the structure and content of the page.
CSS	`style.css`	Works out how everything looks and where it goes.
JavaScript	`app.js`	Runs it, so the page can react and change.
Images	`.png`, `.jpg`, `.svg`	Decodes them and draws them.
Fonts	`.woff2`	Loads them so text can use the chosen typeface.
Data	`.json`	Hands it to a script.

HTML, CSS and JavaScript are not three versions of one thing. They are three languages with three different jobs, and Chapter 3 is about keeping those jobs apart.

1.5 Static Sites and Applications

A **static site** is one whose files are ready-made and identical for every visitor. The server finds the right file and sends it, and that is all it does. A **dynamic site** builds a different page for different visitors, often by running a program and asking a database for information every time. In between are **single-page applications**, which load one HTML file and a big script, and then build everything in the browser.

This website is static, and that was a deliberate choice.

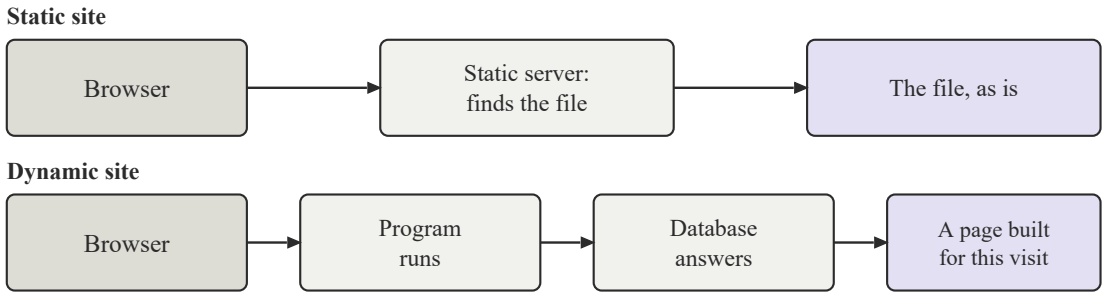


Figure 1.2. A static server has one job: find the file and send it. A dynamic server runs a program and often asks a database before it can answer, so every visit costs work on the server.

WHY DO IT THIS WAY? Making the site static

What it is. Every page is a finished file sitting on a server. There is no program running on the server and no database.

Why it is done this way. A portfolio shows the same thing to everyone. There is nothing to calculate per visitor, so there is nothing to put on a server.

Why it matters. A static site has nothing to hack, because there is no program to attack and no database to steal. It costs almost nothing to host, because any service that can serve files will do. It is fast, because the server's whole job is to hand over a file.

What happens if you do not. A dynamic site must be maintained, patched, backed up and paid for, and every one of those is a place for something to go wrong. For a portfolio that is a lot of trouble for no benefit. The pages are still interactive, by the way. The interactivity lives in JavaScript that runs in the visitor's browser.

1.6 What Travels on the Wire

It helps to know what is really happening underneath, because the costs in the next section come straight from it. Three layers are stacked under every request.

Packets. Nothing travels in one piece. A file is cut into **packets** of about 1 500 bytes, each stamped with the address it is going to and the address it came from, and each finds its own way across the network. They may arrive out of order, and some may not arrive at all.

TCP. The **Transmission Control Protocol** turns that unreliable stream of packets into a reliable conversation. It numbers the packets, puts them back in order, asks again for any that went missing, and slows down when the network is congested. Before any of that can start, the two computers exchange a small "hello, hello back, thanks" called the **handshake**, which costs one round trip.

TLS. The **Transport Layer Security** protocol is what the *s* in `https` stands for. The browser and the server agree on secret keys so that nobody in between can read or alter what passes. Agreeing takes another round trip (older versions took two). Everything after that, including the URL path and the headers, is encrypted; only the host name is visible to the network.

Only when all three layers are ready does the first `GET` go out. That is why the worked example below counts several round trips before a single byte of the file arrives, and why a page that needs files from five different servers pays this set-up five times.

> **TECHNICAL STUFF: Technical Stuff**
> Newer versions of HTTP change the arithmetic a little. **HTTP/2** sends many requests over one connection at the same time, so the "six at a time" limit of HTTP/1.1 goes away and small files stop queuing. **HTTP/3** replaces TCP with a protocol called QUIC that folds the connection and encryption handshakes together, saving a round trip. The ideas in this chapter still hold: set-up costs round trips, transfer costs bandwidth, and the fastest request is the one you never make.

1.7 The Cost of a Request

Every page load pays a toll, and once you understand the toll you can predict whether a page will feel quick. Two things matter. **Latency** is how long a message takes to go to the server and back (the round-trip time, or RTT). **Bandwidth** is how many bits per second the connection can carry. A tiny file is all latency, because the request spends nearly all its time travelling. A huge file is mostly bandwidth, because most of the time goes on the download.

There is also set-up. Before the first byte of a file can even be requested over a secure connection, your browser has to look up the name, open a connection, and agree on encryption keys. That costs several round trips before anything useful happens.

Worked Example: Time to load one script

The bundle that runs the robot page of this website is about 686 kilobytes. A visitor has 10 megabits per second of bandwidth and an 80 millisecond round-trip time. How long does the file take on a first visit?

Step 1. Convert to bits. 686 kilobytes is 686 000 bytes. Each byte is 8 bits, so the file is 686 000 × 8 = 5 488 000 bits, or 5.488 megabits.

Step 2. Transfer time. At 10 megabits per second, 5.488 ÷ 10 = 0.549 seconds.

Step 3. Set-up. Assume three round trips before the request can be sent (name lookup, connection, encryption) and one more for the request and the first byte back. That is 4 × 0.080 = 0.320 seconds.

Step 4. Total. 0.549 + 0.320 = 0.869 seconds.

What the answer tells you. More than a third of the time is set-up, and no change to the file can remove it. Halve the file and the total only falls to about 0.59 seconds. That is why a page with many small requests feels slower than one with a few bigger ones. It is also why servers compress text files, which can shrink a script like this one to a third of its size.

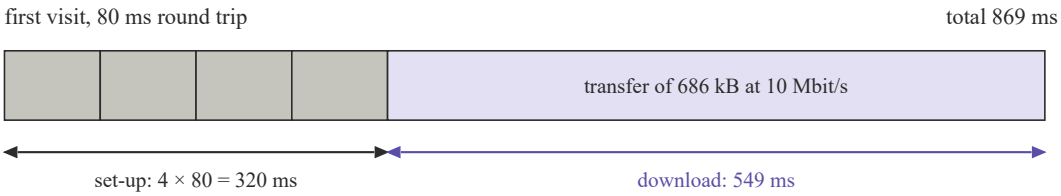


Figure 1.3. The 869 milliseconds of the worked example, drawn to scale. The four grey blocks are round trips that depend only on distance; the violet block is the only part that shrinks when the file does.

Worked Example: A page with a chain of files

A page is made of `index.html` (20 kB), which links to `style.css` (10 kB) and `app.js` (60 kB), and the stylesheet names a font file `font.woff2` (40 kB). The connection is already open and encrypted. The round-trip time is 80 ms and the bandwidth is 10 Mbit/s. How long until the font arrives, and what is the longest chain?

Step 1. Find the chain. The browser cannot ask for a file until it has seen the file that mentions it. So: HTML first; then CSS and JS, which it discovers while reading the HTML; then the font, which it discovers only while reading the CSS. That is three *rounds* of requests, one after another.

Step 2. Round 1, the HTML. One round trip for the request plus the transfer: 20 000 × 8 / 10 000 000 = 0.016 s. Total 0.080 + 0.016 = 0.096 s.

Step 3. Round 2, CSS and JS together. They are requested at the same moment and share the bandwidth. One round trip plus the transfer of 70 kB: 70 000 × 8 / 10 000 000 = 0.056 s. Total 0.080 + 0.056 = 0.136 s.

Step 4. Round 3, the font. It cannot be requested until the CSS has been read. One round trip plus 40 000 × 8 / 10 000 000 = 0.032 s. Total 0.112 s.

Step 5. Add up the chain. 0.096 + 0.136 + 0.112 = 0.344 s before the text can be drawn in the right font.

What the answer tells you. The sizes add up to only 130 kB, which would take 0.104 s to transfer on its own. The chain of discovery costs more than the bytes do. That is why this website's pages use `preconnect` and load fonts early (Part V), and why a stylesheet that imports another stylesheet is a quiet performance bug: it adds a whole round to the chain.

1.8 Caching

The fastest request is the one you never make. A server can send a `Cache-Control` header that says "you may keep this file and reuse it for a while". Next time, the browser uses its own copy.

The catch is staleness. If you change a file and the visitor's browser still holds the old copy, they see the old page. That is where the famous advice "try a hard refresh" comes from.

> **WHY DO IT THIS WAY? Changing a file's name when its content changes**

What it is. Instead of always serving `app.js`, you serve `app.3f9a1c.js`, where the odd bit changes whenever the file's contents change.

Why it is done this way. The browser treats a different name as a different file. It has nothing cached under the new name, so it fetches the new one at once.

Why it matters. It lets you tell browsers "keep this for a year" without any risk, because the name changes when the file does.

What happens if you do not. You have to choose between a short cache (slow pages for everyone) and a long cache (visitors stuck on old versions after you update).

1.9 Hosting a Static Site

Because a static site is only files, publishing it means putting the files where a server can reach them. The source of this website lives in a Git repository, and a repository host can serve a repository's files as a website. Other static hosts exist too. The choice changes the address, the speed and the price. It does not change the pages. The same folder can be opened from your own disk, served from your laptop, or served from a data centre on another continent, and a well-built page behaves the same in all three.

TIP

Whenever a page does something odd, open the developer tools first (press F12) and look at the Console for red errors and the Network tab for failed requests. Nine times out of ten, the answer is sitting there.

What You Learned

- A website is a conversation: the **client** asks with an HTTP request and the **server** answers with a status code, headers and a body. The browser does all the drawing.
- A URL has a scheme, host, port, path, query and fragment. The fragment is never sent.
- The `Content-Type` header tells the browser what the bytes are. The wrong type gives silent failures.
- Underneath, packets travel unreliably, TCP makes them reliable, and TLS makes them private. Each layer costs a round trip before the first request can be sent.
- Time = set-up (round trips × latency) + transfer (bits ÷ bandwidth). Small files are all latency; big files are all bandwidth; chains of discovery multiply the round trips.
- Caching makes the fastest request the one you never send, and a changed file name makes long caching safe.

Check Your Knowledge

1. What is the difference between a client and a server? Can one computer be both?

2. Name the six parts of the URL `https://example.com:8080/a/b.html?x=1#top`, and say which part is never sent to the server.
3. A server sends a JavaScript file with the header `Content-Type: text/plain`. Why might the browser refuse to run it?
4. State the difference between latency and bandwidth, and say which one dominates the time to fetch a very small file.
5. Give two advantages of a static site over a dynamic one, and one thing a static site cannot do by itself.

Review Problems

1. A page loads a 2.4 megabyte image over a 20 megabits per second connection with a 50 millisecond round-trip time. Using three round trips for connection set-up and one for the request, estimate the load time.
2. Compression reduces a 686 kilobyte script to 31% of its size. Recompute the transfer time of the worked example on a 10 megabits per second link, and state the new total including the 0.320 second set-up.
3. A page needs 12 separate small files. Each file takes one round trip (80 milliseconds) to fetch, and the browser can fetch only 6 at a time. How long do the 12 fetches take if they are perfectly parallel within each batch of 6? How long would they take if the files were combined into one file of the same total size, ignoring transfer time?
4. Explain why changing a file's name when its content changes allows a server to set a one-year cache lifetime safely.
5. Open any page of this website in a browser, open the developer tools, and choose the Network tab. Reload the page. Record how many requests it makes, the total size transferred, and the slowest single request. Which file would you shrink first, and why?
6. A visitor in a region with a 300 millisecond round-trip time requests the page from the worked example. Recompute the total time with the same bandwidth. By what factor does it change, and what does this suggest about hosting files closer to the visitor?

Chapter 2. The Browser as a Machine

Think of a browser as a very fast short-order cook. Orders come in (HTML, CSS, scripts), and the cook has one pair of hands, a strict timetable, and a customer who walks out if dinner is late. Understand how the kitchen runs and you will write pages that feel smooth almost by accident. Ignore it, and your pages will stutter, and you will not know why.

In This Chapter

- Name the steps that turn bytes into pixels, and say which of them a given CSS change repeats.
- Draw the DOM tree for a small piece of HTML.
- Explain why stylesheets block the first paint and why scripts block the parser, and what `async` and `defer` change.
- Work out the frame budget for any refresh rate.
- Recognise layout thrashing in code and rewrite it.
- Measure before guessing, using the Performance panel.

2.1 From Bytes to Pixels: The Critical Rendering Path

When the first bytes of an HTML file arrive, the browser does not wait for the rest. It starts cooking immediately, in a fixed sequence of steps called the **critical rendering path**.

Step 1: Parse the HTML into the DOM

The browser reads the HTML text and builds a **tree** of objects from it, called the **Document Object Model**, or **DOM**. Each tag becomes a node, and a tag inside another tag becomes its child.

```
<body>                          body
  <h1>Patrick</h1>                |-- h1 -- "Patrick"
  <p>A <em>robot</em>.</p>        `-- p   -- "A ", em -- "robot", "."
</body>
```

The DOM is the browser's working model of your page, and it is the thing your JavaScript reads and changes. Notice that the browser is forgiving. If you forget an end tag, it quietly repairs the tree and carries on. That is friendly, and it is also why a typo in HTML usually gives you a *weird layout* and not an error message.

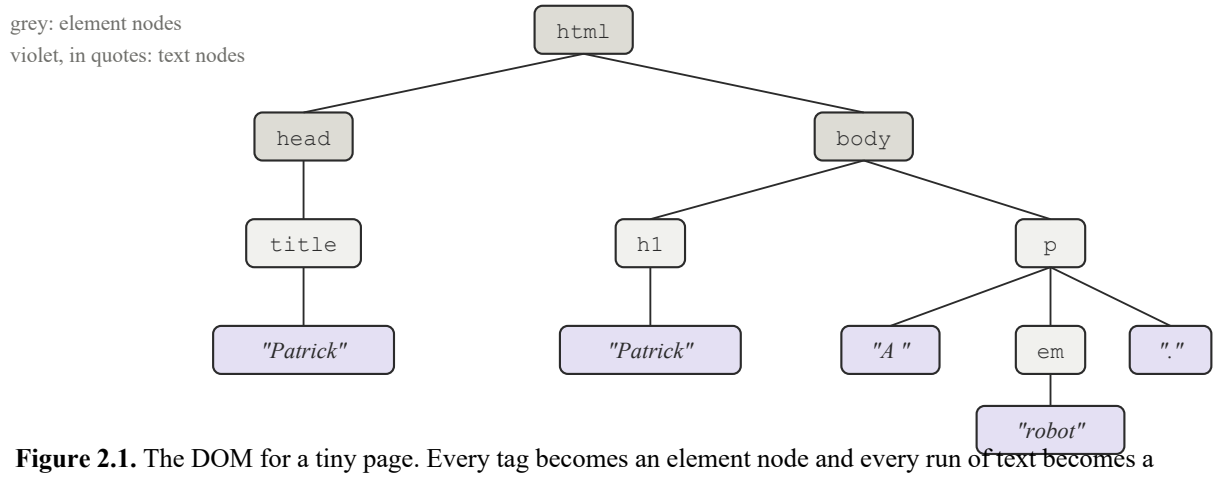


Figure 2.1. The DOM for a tiny page. Every tag becomes an element node and every run of text becomes a text node. The `em` sits *inside* the paragraph's text, between two text nodes, which is exactly how it appears in the source.

Step 2: Parse the CSS into the CSSOM

As stylesheets arrive, the browser builds a second tree called the **CSS Object Model**, which records the rules and their selectors. Here is the surprising part: the browser will not draw anything until its stylesheets have been read. We say stylesheets **block rendering**.

> **WHY DO IT THIS WAY? Stylesheets blocking the first paint**
>
> **What it is.** The browser waits for CSS to load before it paints the first picture of the page.
>
> **Why it is done this way.** The alternative is to paint the page with no styles, then repaint it when the CSS arrives.
>
> **Why it matters.** That alternative produces the "flash of unstyled content": plain text jumps and rearranges as the styles land. It looks broken and it makes the browser do the same work twice.
>
> **What happens if you do not (that is, if you load CSS late).** Visitors see the ugly unstyled version first. That is why the stylesheet link belongs in the `<head>` at the top of the page, so it starts downloading right away.

Step 3: Build the render tree, then layout, paint, composite

The browser combines the DOM and the CSSOM into a **render tree**, which holds only things that will actually appear (an element with `display: none` is in the DOM but not in the render tree), along with their final styles. Then three more steps happen.

1. **Layout** (also called *reflow*) works out the size and position of every box. It is pure geometry: given the width of the window, how wide is each box, where does the text wrap, how tall is each row?
2. **Paint** turns each box into drawing instructions: fill this rectangle, draw this text, draw this border.
3. **Composite** takes the painted pieces, which live on separate **layers**, and stacks them on the screen, usually with the graphics processor (the GPU) doing the work.

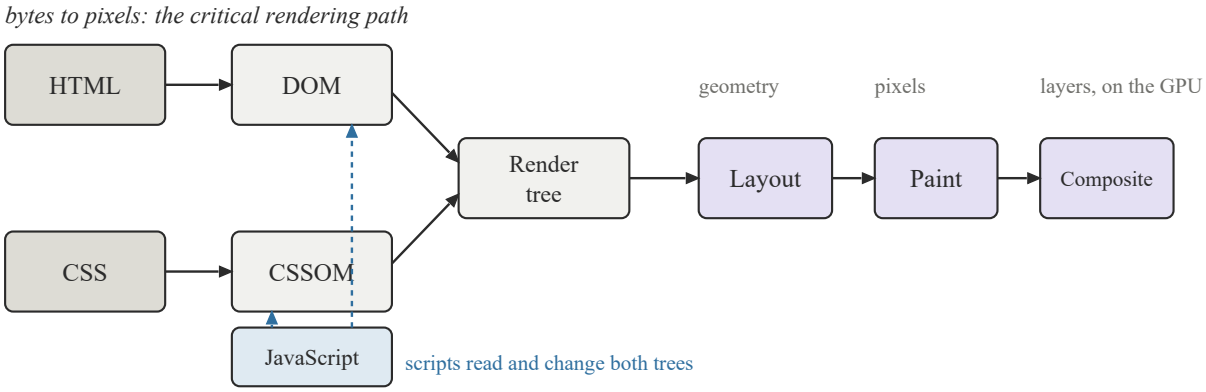


Figure 2.2. The critical rendering path. HTML and CSS are parsed into two trees, combined, laid out, painted and composited. JavaScript can reach into both trees, which is why a script can change anything, and why it can force the later steps to run again.

REMEMBER

This last trio is the most useful idea in the chapter. Changing something that affects **geometry** (like `width`) redoes layout, paint *and* composite. Changing something that only affects **appearance** (like `background-color`) skips layout but still repaints. Changing `transform` or `opacity` on an element with its own layer can be done at the composite step alone, which is by far the cheapest.

WHY DO IT THIS WAY? Animating with transform instead of left

What it is. To slide a box across the screen, you change `transform: translateX(...)` and not `left`.

Why it is done this way. `left` changes geometry, so every frame the browser redoes layout, then paint, then composite. `transform` on a layered element only needs the composite step, and the GPU can do that.

Why it matters. Your frame budget (see below) is tiny. Skipping two of the three steps leaves most of it free for everything else.

What happens if you do not. The animation works on your fast laptop and judders on a phone. It is a classic "works on my machine" bug.

2.2 JavaScript and the Parser

JavaScript complicates things, because a script can change the DOM and the styles while the browser is still building them. When the parser meets a normal `<script>` tag, it has to stop, fetch the script, run it, and only then carry on, because the script might change something that comes later. A slow script near the top of a page therefore holds up everything below it.

Two small attributes fix this. An `async` script downloads while parsing continues and runs the moment it arrives, in no promised order. A `defer` script also downloads while parsing continues, but

it waits and runs only after the whole page has been parsed, and deferred scripts run in the order they appear.

```
<script defer src="katex.min.js"></script>
<script defer src="auto-render.min.js"></script>   <!-- runs after katex.min.js -->
```

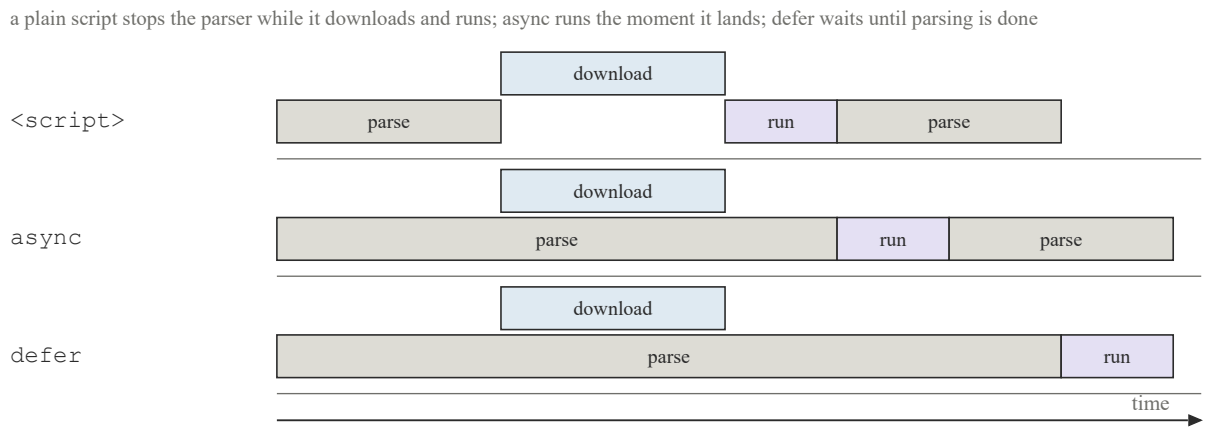


Figure 2.3. What the parser does while a script arrives. A plain script stops parsing until it has downloaded and run. An `async` script downloads alongside parsing and runs the moment it lands. A `defer` script downloads alongside parsing and runs only after the whole document is parsed.

> **WHY DO IT THIS WAY? Marking library scripts defer**
>
> **What it is.** Adding the word `defer` to a script tag.
>
> **Why it is done this way.** The scripts need the page's HTML to exist before they run (they look for elements to fill in), but they do not need to block the page from appearing.
>
> **Why it matters.** The text and layout appear first, and the instruments come to life a moment later. And because deferred scripts run in order, `auto-render` can rely on `katex` having loaded first.
>
> **What happens if you do not.** The visitor stares at a blank page while the scripts download. And a script that runs too early finds no elements to work on, and silently does nothing. The flight dynamics and mechatronics pages of this website load their libraries with `defer`.

2.3 The Main Thread and the Frame Budget

Your page runs on one **main thread**, like our cook's single pair of hands. Parsing HTML, working out styles, layout, painting instructions, and running your JavaScript all take turns on it. While one is busy, the others wait. A script that runs for a long time makes the page unable to respond to clicks, scroll, or animate. Any stretch longer than about 50 milliseconds is called a **long task**, and long tasks are the root of almost every sluggish page.

Smooth motion needs a fresh picture each time the screen refreshes. A 60-hertz screen refreshes 60 times a second, so you get a new frame every 1/60 of a second, which is 16.67 milliseconds. Inside that window the browser has to run your code, work out styles, lay things out, paint and composite. That window is your **frame budget**.

Worked Example: The budget of a frame

A display refreshes at 144 hertz. The robot page of this website redraws a 3D scene and eight charts. What is the budget for one frame, and how much is left if the browser itself needs about 3 milliseconds?

Step 1. Budget. The period is the reciprocal of the frequency: 1000 ms ÷ 144 = 6.94 ms.

Step 2. What is left for you. 6.94 − 3 = 3.94 ms.

Step 3. Compare with 60 hertz. At 60 Hz the budget is 16.67 ms, leaving 13.67 ms for the same code, more than three times as much. A page that is comfortable at 60 Hz can be over budget at 144 Hz.

What happens when you go over? The browser does not squeeze the work in. It misses the frame and shows the old picture again, and the motion appears to stutter. That is why the robot page measures its own frame times and, if they stay too long, lowers its rendering quality a step at a time until it fits.

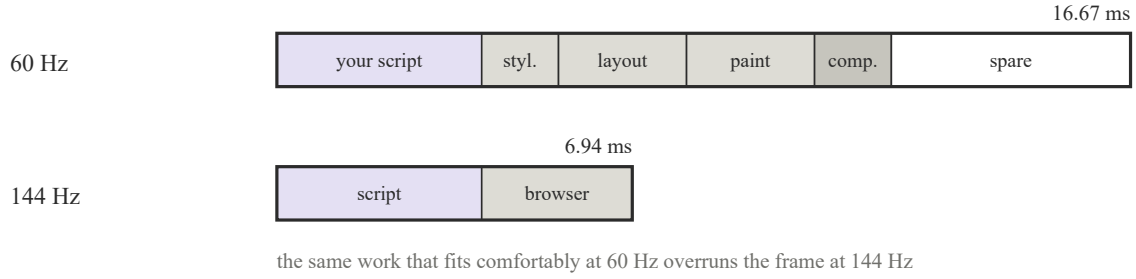


Figure 2.4. One frame at 60 hertz and one at 144 hertz, drawn to the same scale. The browser's own steps (style, layout, paint, composite) come out of the same budget as your script.

2.4 The Graphics Processor

The GPU is a second processor that is brilliant at doing the same simple sum on thousands of pixels at once. Browsers use it to composite layers, and a page can use it directly through **WebGL**, which is how 3D graphics run in a page. The GPU is wonderful at drawing a million triangles and poor at making decisions. So the work is divided: JavaScript on the main thread decides *what* to draw, and the GPU does the drawing.

2.5 Layout Thrashing

There is one sneaky trap worth naming. The browser avoids recalculating layout more often than it has to. If you change several styles in a row, it waits and recalculates once. But if, between your changes, you *read* a value that depends on layout (like `offsetHeight`), the browser has to recalculate immediately to give you a correct answer. Mixing reads and writes forces a recalculation every time round, and the cost piles up.

```
// slow: layout is recalculated on every pass of the loop
```

```
for (const box of boxes) {
  box.style.width = box.parentNode.offsetWidth / 2 + 'px';   // read, then write
}

// fast: read everything first, then write everything
const widths = boxes.map(b => b.parentNode.offsetWidth);
boxes.forEach((b, i) => { b.style.width = widths[i] / 2 + 'px'; });
```

> **WHY DO IT THIS WAY? Reading everything first, then writing**
>
> **What it is.** Doing all your measurements in one batch, then all your changes in another.
>
> **Why it is done this way.** Reading a layout value forces the browser to finish any pending layout work first. Grouping the reads means that happens once, not once per element.
>
> **Why it matters.** On 500 boxes, the slow version recalculates layout 500 times. The fast version does it once.
>
> **What happens if you do not.** Your page does the same amount of "real" work but feels a hundred times slower, and the cause is invisible in the code unless you know the rule.

Worked Example: The cost of a thrashing loop

A page has 500 boxes. Recalculating the layout of the page takes 0.4 ms. How long does each version of the loop above spend in layout, and what does that mean for the frame budget?

Step 1. The slow version. Each pass reads `offsetWidth` after the previous pass wrote a width, so the browser must recalculate layout before it can answer. That is one recalculation per box: 500 × 0.4 = 200 ms.

Step 2. The fast version. All the reads happen first, against one clean layout, so one recalculation serves them all. Then all the writes happen, and the browser lays out once more before it paints: 2 × 0.4 = 0.8 ms.

Step 3. Against the budget. At 60 hertz the budget is 16.7 ms. The slow version misses twelve frames in a row; the visitor sees the page freeze for a fifth of a second. The fast version uses one twentieth of a single frame.

What the answer tells you. The two loops do the same arithmetic and set the same widths. The only difference is the *order* of reads and writes. This is the kind of bug a profile finds and a code review usually misses.

2.6 Measure, Don't Guess

The developer tools have a **Performance** panel. It records a few seconds of your page's life and draws a timeline showing how the main thread spent every millisecond: scripting, rendering, painting. Long tasks are marked in red. The first rule of making a page faster is to measure it. The second is to measure it again after the change, because human intuition about speed is wrong a surprising amount of the time.

> **TIP**
> Before you "optimise" anything, record a profile. A change that feels like it should help often does nothing, and a boring line you never suspected is often the real culprit.

What You Learned

- The browser turns HTML into the **DOM** and CSS into the **CSSOM**, combines them into a render tree, then runs **layout**, **paint** and **composite**.
- Changing geometry repeats all three steps; changing colour repeats paint; changing `transform` or `opacity` on a layer repeats only composite. Animate the cheap ones.
- Stylesheets block the first paint on purpose. Plain scripts block the parser; `defer` lets the page appear first and keeps scripts in order.
- Everything shares one **main thread**. A **long task** (over 50 ms) freezes the page. The **frame budget** is 1000 ms ÷ refresh rate: 16.7 ms at 60 Hz, 6.9 ms at 144 Hz.
- Reading a layout value after writing one forces a recalculation. Batch reads, then writes.
- Measure with the Performance panel. Then measure again.

Check Your Knowledge

1. Name the stages of the critical rendering path in order, from received bytes to pixels on the screen.
2. What is the difference between the DOM and the render tree? Give an example of an element that is in one and not the other.
3. Why do stylesheets block rendering while images usually do not?
4. Explain the difference between `async` and `defer`.
5. Why is animating `transform` cheaper than animating `left`?
6. What is a long task, and what does it do to the page?

Review Problems

1. Compute the frame budget in milliseconds at 30, 60, 90 and 120 hertz.
2. A script takes 22 ms per frame to redraw a chart, and the browser needs 4 ms for its own work. At what refresh rates does the page keep up? What is the highest rate at which it does?
3. A page loads three external scripts without `async` or `defer`. They take 120 ms, 80 ms and 200 ms to fetch and 10 ms, 5 ms and 30 ms to run. If fetches cannot overlap with parsing, how long is parsing blocked in total? If all three are marked `defer` and the fetches run in parallel, how long is parsing blocked by them?
4. The loop in the layout-thrashing example runs over 500 boxes, and each forced layout takes 0.12 ms. Estimate the time lost, and the time if reads and writes are grouped and a single layout is needed taking 0.12 ms.

5. For each of these changes, say whether the browser must repeat layout, paint, or only composite: (a) changing `width`, (b) changing `color`, (c) changing `transform: translateX()` on an element with its own layer, (d) changing `opacity` on the same element.
6. Open the Performance panel on any page of this website, record five seconds while the page animates, and identify the longest task. Report its duration and what the main thread was doing during it.

Chapter 3. Three Languages, Three Jobs

A web page is written in three languages at once. That sounds like a punishment, but it is actually a gift, as long as you keep their jobs apart. **HTML** says what a thing *is*. **CSS** says what it *looks like*. **JavaScript** says what it *does*. Think of a restaurant: the menu says what is on offer, the decor sets the mood, and the waiter actually brings you the food. You would not ask the waiter to repaint the walls.

When each language sticks to its own job, a page is easy to change. You can recolour the whole site without touching its structure, or give a button new behaviour without touching how it looks. When the jobs get mixed, every change is a gamble. This chapter explains each job, and then the thing that connects them: the document tree.

In This Chapter

- State the job of each of the three languages in one sentence each.
- Explain what makes an element *semantic* and why a real `<button>` beats a styled `<div>`.
- Settle a fight between CSS rules using importance, specificity and source order.
- Use inheritance and custom properties to say a thing once.
- Find, change and listen to elements from JavaScript, and explain why scripts should set classes rather than colours.
- Describe progressive enhancement and spot a page that breaks it.

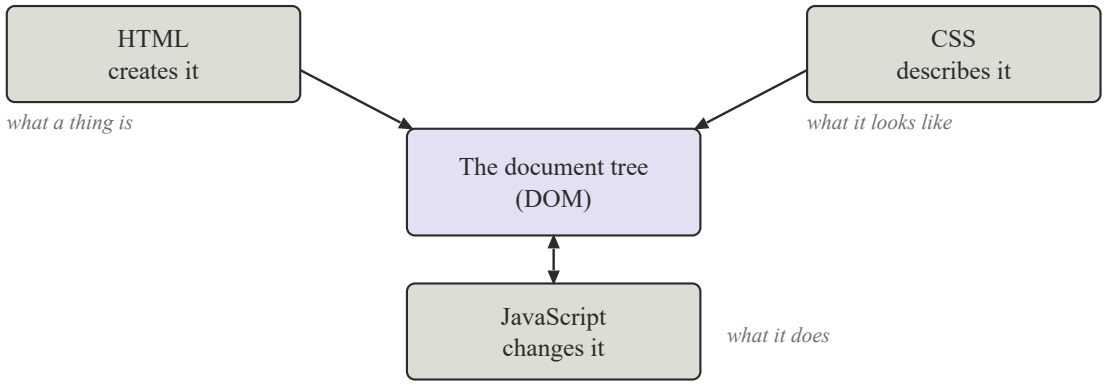


Figure 3.1. The three languages meet in the document tree. HTML builds it, CSS describes how its nodes look, and JavaScript reads and changes it.

3.1 Structure: HTML

HyperText Markup Language describes the *meaning and organisation* of your content. It does not describe looks, even though it is tempting to think a heading is "the big bold text". A heading is a statement: *this text introduces a section*. The browser happens to draw headings big by default, but a screen reader announces them as headings, a search engine uses them to understand your page, and your stylesheet can make them any size you like.

An HTML document is built from **elements**. An element usually has a start tag, some content, and an end tag, and the start tag can carry **attributes**, which are name-and-value pairs that configure it.

```
<a href="flight-dynamics.html" class="chip">Flight dynamics</a>
 \_/ \_____________________________/ \_________/ \_____________/ \__/
 tag        attribute href          attribute      content        end
```

Elements come in families. **Block** elements (`<p>`, `<h1>`, `<div>`) start on a new line and stretch across the available width. **Inline** elements (`<a>`, `<em>`, `<span>`) sit inside a line of text and take only the room they need. **Replaced** elements (`<img>`, `<canvas>`) are boxes whose content comes from somewhere else.

Semantic elements

HTML has elements whose names carry meaning: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<footer>`. A good page uses these instead of a heap of anonymous `<div>` tags.

> **WHY DO IT THIS WAY? Using a button for a button**
>
> **What it is.** When something performs an action, you write `<button>`, not a `<div>` that you have styled to look like one.
>
> **Why it is done this way.** A real button comes with a lot of behaviour that you cannot see: it can be reached with the Tab key, pressed with Space or Enter, and announced by a screen reader as "button".
>
> **Why it matters.** Those free behaviours are exactly what keyboard users and blind users rely on.
>
> **What happens if you do not.** A styled `<div>` looks right and works for mouse users, and is completely dead to everyone else. You would have to rebuild all that behaviour by hand, and most people forget to.

3.2 Presentation: CSS

Cascading Style Sheets describe how things look and where they go. A stylesheet is a list of **rules**. Each rule has a **selector** (which elements it applies to) and a **declaration block** (what to do to them).

```
.panel-head .t {            /* selector: an element of class t inside class panel-he
      ad */
    color: var(--accent);   /* declaration: property, then value */
    letter-spacing: 0.16em;
}
```

The important word is *cascading*. Lots of rules can apply to the same element, and they will not always agree. The cascade is the referee. It settles disagreements in this order: first by **importance** (a declaration marked `!important` beats one that is not), then by **specificity**, then by **order** (a later rule beats an earlier one of equal specificity). Nearly every baffling CSS bug is a case of not knowing which rule the referee picked.

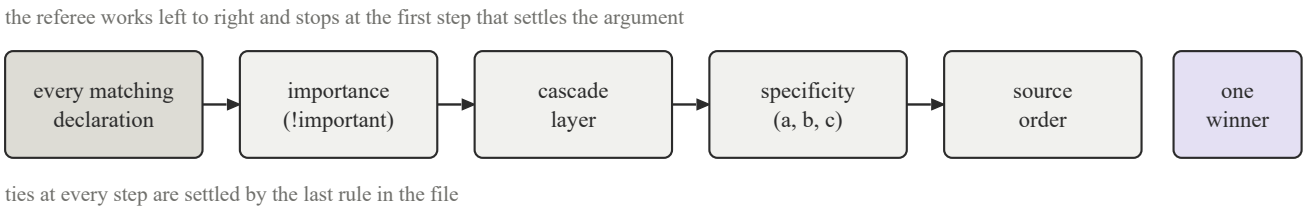


Figure 3.2. The cascade as a sequence of tie-breaks. The referee moves right only when the rules are still tied. Cascade layers, the step between importance and specificity, are covered in Part IV.

Specificity

Specificity is a score for a selector, written as three numbers (a, b, c). Count the **ID selectors** (like `#header`) for a. Count the **class selectors, attribute selectors and pseudo-classes** (like `.panel`, `[type=range]`, `:hover`) for b. Count the **element selectors** (like `p`, `div`) for c. Compare the scores from left to right. One ID beats any number of classes, and one class beats any number of elements.

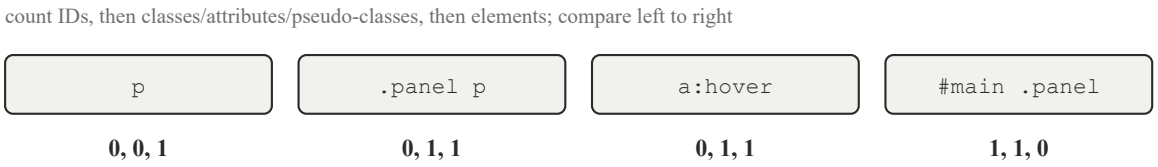


Figure 3.3. Four selectors and their scores. `.panel p` and `a:hover` tie at (0, 1, 1), so between those two the later one in the file wins.

Worked Example: Which colour wins?

An element `<p id="note" class="warn small">` is matched by these four rules. Decide the colour of its text.

```
p              { color: black; }
.warn          { color: orange; }
p.warn.small   { color: red; }
#note          { color: blue; }
```

Step 1. Score each selector. `p` has one element selector: (0, 0, 1). `.warn` has one class: (0, 1, 0). `p.warn.small` has two classes and one element: (0, 2, 1). `#note` has one ID: (1, 0, 0).

Step 2. Compare from the left. The first number decides first. Only `#note` has a non-zero first number, so it outranks the other three whatever they contain.

Step 3. Conclusion. The text is blue. Notice that `p.warn.small` has more selectors than `#note` and still loses, because specificity is compared place by place and not added up.

Moral. Do not fight specificity by adding more selectors or by reaching for `!important`. Keep selectors flat (a single class where possible), and then order alone resolves most conflicts.

> **WARNING**

> Do not fight specificity by piling on more selectors or reaching for `!important`. That is how you end up with a stylesheet nobody dares touch. Keep selectors flat (a single class, if you can) and let source order do the rest.

Inheritance and custom properties

Some properties, like `color` and `font-family`, are **inherited**: leave them unset on an element and it uses its parent's value. Others, like `border` and `margin`, are not. Inheritance is why you can set the typeface once on `body` and the whole page follows. **Custom properties**, also called CSS variables, take the idea further. A name starting with two hyphens holds a value that any rule can read with `var()`.

```
:root { --accent: #a78bfa; --radius: 10px; }
.panel { border-radius: var(--radius); border-color: var(--accent); }
```

> **WHY DO IT THIS WAY? Putting the colours in custom properties**
>
> **What it is.** Defining the palette once at the top (`--accent: #a78bfa`) and using `var(--accent)` everywhere else.
>
> **Why it is done this way.** Each colour then lives in exactly one place.
>
> **Why it matters.** Changing the accent colour of a whole page becomes a one-line edit. And because custom properties follow the cascade, one component can override a value locally and affect only itself.
>
> **What happens if you do not.** The same colour code gets copied into fifty rules. To change it you must find every copy, and you will miss one, so the site ends up with two slightly different purples. Every page of this website defines its palette this way, in one block at the top.

3.3 Behaviour: JavaScript

JavaScript is a general-purpose programming language that browsers can run. On a page, its job is to *respond*: to a click, a key press, data arriving, time passing. It is the only one of the three languages that can make decisions, repeat things and remember things. That power is exactly why you should use it only for behaviour.

> **WHY DO IT THIS WAY? Adding a class instead of setting colours in the script**
>
> **What it is.** When a button becomes active, the script does `button.classList.add('active')` and the stylesheet decides what `.active` looks like. It does *not* do `button.style.color = 'red'`.
>
> **Why it is done this way.** Looks are CSS's job. The script only reports a *fact* ("this button is active").
>
> **Why it matters.** A designer can restyle `.active` without touching any JavaScript, and the browser can optimise a class change better than a pile of inline styles.

> **What happens if you do not.** The appearance ends up scattered between the stylesheet and a dozen scripts, and nobody can tell where a colour comes from.

3.4 The Document Tree Connects Them

The three languages meet in the **document tree**, the DOM from Chapter 2. HTML *creates* the tree. CSS *describes* it, and its selectors are queries against it. JavaScript *changes* it, and so changes the page. The browser gives JavaScript functions for finding nodes and altering them.

```
const title = document.querySelector('h1');       // find the first <h1>
title.textContent = 'Meet Patrick';                // change its text
title.classList.add('highlight');                   // hand the styling to CSS
document.body.append(document.createElement('hr')); // add a new node to the tree
```

> **WHY DO IT THIS WAY? querySelector takes a CSS selector**
>
> **What it is.** `document.querySelector('h1')` finds an element using the very same selector language you use in a stylesheet.
>
> **Why it is done this way.** The designers of the web did not want you to learn two different ways of pointing at elements.
>
> **Why it matters.** Every selector skill you have from CSS works in JavaScript, and the other way round.
>
> **What happens if you do not.** Older ways of finding elements (`getElementById` and friends) each have their own rules, and mixing them is how bugs creep in. If nothing matches, `querySelector` returns `null`, and calling a method on `null` is one of the most common errors you will ever see.

Events

The tree can also *announce* things. When the user clicks, the browser makes an event object and delivers it to the element that was clicked. A script that has asked to be told receives it.

```
const button = document.querySelector('#play');
button.addEventListener('click', () => {
    document.body.classList.toggle('paused');   // behaviour: flip a class
});
```

> **WHY DO IT THIS WAY? addEventListener instead of waiting in a loop**
>
> **What it is.** You hand the browser a function and say "call this when a click happens".
>
> **Why it is done this way.** JavaScript on a page can only do one thing at a time (Chapter 5). If your code stood around checking "has it been clicked yet?", nothing else on the page could run.
>
> **Why it matters.** The browser does the waiting and calls you only when something happens, so the page stays responsive.
>
> **What happens if you do not.** A polling loop freezes the whole page.

Worked Example: Untangling three jobs that got mixed

A beginner's page contains this line. Rewrite it so that each language does its own job, and say what each rewrite buys.

```
<div style="color:red; cursor:pointer" onclick="this.style.color='green'">Start</div
      >
```

Step 1. What is it? It is something you click to start an action. That is a button, so the HTML job is `<button>`. A `<div>` says nothing, and cannot be reached with the keyboard.

Step 2. The looks. The red colour and the pointer cursor are looks, so they move to the stylesheet under a class. The green "done" look is a second state, so it gets a second class.

Step 3. The behaviour. The click should record a fact ("it has started"), not paint a colour. The script toggles the class and the stylesheet decides what the class looks like.

```
<button class="start">Start</button>

.start { color: red; }                /* a real button already shows a pointer and t
      akes focus */
.start.started { color: green; }

document.querySelector('.start').addEventListener('click', e => e.target.classList.a
      dd('started'));
```

Step 4. What it bought. Keyboard users can now press Space to start it. A designer can change "started" from green to a tick mark without touching the script. And the script no longer knows or cares about colours, so it keeps working when the design changes.

3.5 Progressive Enhancement

A well-built page works in layers. Layer one is the HTML alone: content, in the right order, with the right meaning, readable without any styling or scripting. Layer two adds CSS, which makes it a designed page. Layer three adds JavaScript, which makes it an interactive page. This is called **progressive enhancement**.

> **WHY DO IT THIS WAY? Building the page in layers**
>
> **What it is.** Making sure the page still works if a later layer fails.
>
> **Why it is done this way.** Networks drop, scripts have bugs, and old browsers exist.
>
> **Why it matters.** If a script fails to load, the visitor gets the explanation and the equations instead of a blank screen. Many pages on this website follow the pattern in miniature: the explanation is HTML, and the instruments are canvases that a script fills in.
>
> **What happens if you do not.** One failed script takes down the whole page, and the visitor sees nothing at all.

What You Learned

- **HTML** says what a thing is, **CSS** says what it looks like, **JavaScript** says what it does. Keep the jobs apart and every change stays small.
- Semantic elements carry behaviour you cannot see: focus, keyboard handling, announcements. Use them.
- The cascade settles fights by importance, then specificity (a, b, c), then source order. Keep selectors flat and let order do the work.
- Inherited properties flow down the tree; custom properties let you name a value once and use it everywhere.
- Scripts find elements with CSS selectors, change them by adding classes, and respond to events with listeners.
- Build in layers: content first, then style, then behaviour, so a failure in a later layer never hides the earlier ones.

Check Your Knowledge

1. State in one sentence the job of each of HTML, CSS and JavaScript.
2. Why should a clickable control that performs an action be a `<button>` and not a styled `<div>`?
3. In what order does the cascade settle a conflict between rules?
4. Compute the specificity of `nav ul li.active a:hover` and of `#menu .item`. Which is more specific?
5. Give an example of an inherited property and a property that is not inherited.
6. What is progressive enhancement, and what problem does it solve?

Review Problems

1. Rank these selectors from least to most specific: `div p`, `.card`, `#card`, `div.card p.lead`, `*`, `a:hover`.
2. An element is matched by `.a { color: red; }` followed later by `.b { color: green; }`, and the element has both classes. What colour is its text? What changes if the order of the two rules is reversed? What changes if the first rule is `.a.a { color: red; }`?
3. Rewrite the following so that the colours live in custom properties defined once on `:root`, and the two rules reuse them.

```
.title { color: #a78bfa; border-bottom: 2px solid #a78bfa; }
.badge { background: #a78bfa; color: #070a1a; }
```

4. A script contains `element.style.color = isActive ? 'red' : 'grey'` executed on every frame of an animation. Explain what is wrong with this design in terms of the three jobs, and rewrite it so that each language does its own work.
5. For each of these tasks, say which language should do it: (a) making the heading larger on narrow screens; (b) opening a menu when a button is pressed; (c) marking a block of text as a quotation; (d) fading a panel in over 300 milliseconds; (e) calculating a value from two sliders.

6. Write a page of at least five lines of HTML containing a heading, a paragraph and a button. Add a stylesheet that colours the heading using a custom property, and a script that adds the class `done` to the paragraph when the button is clicked. State where each of the three jobs lives.

Chapter 4. Space and Size

Every page lives inside a rectangle that you do not control. It might be a phone held upright, a laptop, or a monitor three times as wide, and your page has to look right on all of them. This chapter is about that problem: what a pixel really is, how boxes get their size and position, which units adapt to the screen and which refuse to, and why a drawing on a canvas needs to be told how dense the screen is.

In This Chapter

- Tell a CSS pixel from a device pixel, and size a canvas so that it is sharp on both.
- Draw the box model and compute a box's size under `content-box` and `border-box`.
- Choose between normal flow, Flexbox and Grid for a given layout, and do the arithmetic of `fr`, `gap` and `flex-grow`.
- Pick a unit (`px`, `rem`, `em`, `%`, `vw`, `fr`) on purpose, and evaluate a `clamp()`.
- Make a page respond to width, pointer type and motion preference.

4.1 What Is a Pixel, Really?

The word *pixel* secretly means two different things, and mixing them up causes most blurry canvases. A **device pixel** is one physical dot on the screen. A **CSS pixel** is a unit of length in your stylesheet. On an old monitor they were the same thing. On a modern phone the dots are far smaller, so a CSS pixel is built from several device pixels, which keeps text a readable physical size. The ratio is the **device pixel ratio** (DPR).

device pixels = CSS pixels × DPR

On a DPR 2 screen, something 300 CSS pixels wide covers 600 device pixels. Text and vector shapes are drawn at the full device resolution automatically, so they stay sharp. A **bitmap**, though, has a fixed number of pixels. If it has fewer pixels than the screen area it must cover, the browser stretches it, and it goes soft.

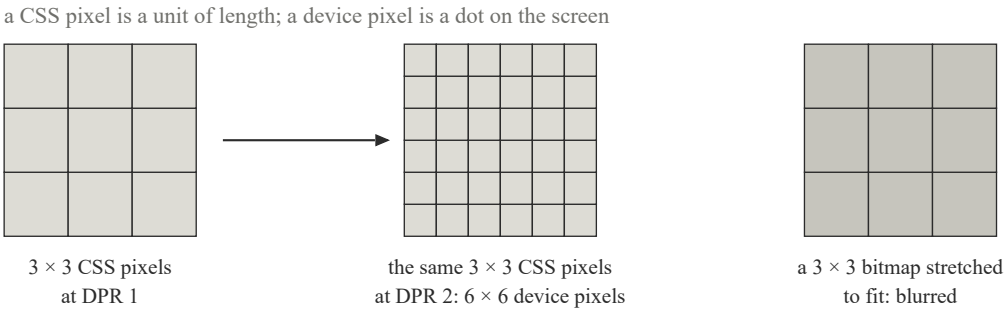


Figure 4.1. The same three-by-three CSS pixels on a DPR 1 screen and a DPR 2 screen. A bitmap drawn with nine pixels has to be stretched over thirty-six, and the stretching is the blur.

4.2 The Canvas Problem

The `<canvas>` element is a bitmap. It has two sizes: the size of its *drawing surface* (the `width` and `height` attributes) and the size it is *shown* at (set by CSS). If they disagree, the browser stretches the surface to fit the box. A canvas shown at 430 by 300 CSS pixels with a 430 by 300 surface, on a DPR 2 screen, gets stretched to double size and looks blurred.

The cure is to give the surface the *device* size, then scale the drawing context so your code can carry on thinking in CSS pixels. Every chart on this website begins with a function that does exactly this.

```
function fit(canvas) {
    const dpr = window.devicePixelRatio || 1;
    const w = canvas.clientWidth, h = canvas.clientHeight;      // size shown, in CS
      S px
    if (canvas.width !== Math.round(w * dpr) || canvas.height !== Math.round(h * dpr
      )) {
        canvas.width  = Math.round(w * dpr);                    // size of the surfa
      ce,
        canvas.height = Math.round(h * dpr);                    // in device px
    }
    const ctx = canvas.getContext('2d');
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);                    // 1 unit drawn = 1 C
      SS px
    ctx.clearRect(0, 0, w, h);
    return { ctx, w, h };
}
```

WHY DO IT THIS WAY? Setting the canvas surface to the device size

What it is. Making `canvas.width` equal to the shown width times `devicePixelRatio`, then calling `setTransform(dpr, ...)` so drawing coordinates stay in CSS pixels.

Why it is done this way. The surface needs enough pixels to cover the screen area one-to-one, and the transform lets you write `lineTo(330, 50)` without ever thinking about DPR.

Why it matters. Lines and text stay razor sharp on every kind of screen, using one piece of code.

What happens if you do not. Charts look fuzzy on phones and modern laptops, and it is hard to see why, because nothing is "wrong" in the code.

WHY DO IT THIS WAY? Only resizing the surface when the size changed

What it is. The `if` that compares the old and new sizes before assigning `canvas.width`.

Why it is done this way. Assigning `canvas.width` always wipes the canvas and reallocates its memory, even if you assign the same number.

Why it matters. This function runs every frame. Reallocating a big bitmap 60 times a second wastes a lot of time and memory.

What happens if you do not. The page runs hot, stutters, and may run out of memory on phones.

Worked Example: Sizing a canvas for a sharp chart

A chart canvas is displayed 430 CSS pixels wide and 300 CSS pixels tall on a laptop whose DPR is 2. What should the `width` and `height` attributes be? If the code draws a line from the point (100, 50) to

(330, 50), where does it appear on the surface, and how thick is a line drawn with `lineWidth = 1`?

Step 1. Surface size. 430 × 2 = 860 and 300 × 2 = 600, so the drawing surface is 860 by 600 device pixels.

Step 2. Transform. The call `setTransform(2, 0, 0, 2, 0, 0)` multiplies every coordinate by 2. The point (100, 50) is therefore drawn at device pixel (200, 100), and (330, 50) at (660, 100).

Step 3. Line thickness. A `lineWidth` of 1 is 1 CSS pixel, which is 2 device pixels, so the line is two device pixels thick and crisp. On a DPR 1 screen the same code gives a one-pixel line with the same appearance, which is the point: one piece of code, sharp everywhere.

Step 4. Cost. The surface now holds 860 × 600 = 516 000 pixels instead of 129 000, four times as many. A page with many large canvases on a high-density phone can run short of memory or time, and this is why the robot page of this website caps the pixel ratio of its three-dimensional canvas (to 1.6 on a capable machine and to 1 on a weak one).

4.3 The Box Model

Every element is a rectangular **box** made of four nested layers. In the middle is the **content**. Around it is **padding** (breathing room that shows the box's own background). Around that is the **border**. Outside the border is the **margin**, which separates the box from its neighbours and shows nothing.

By default, the `width` you set describes the *content only*, so the box you see is wider than you asked for, by all the padding and border. That is rarely what anyone means, which is why almost every modern stylesheet starts with this rule.

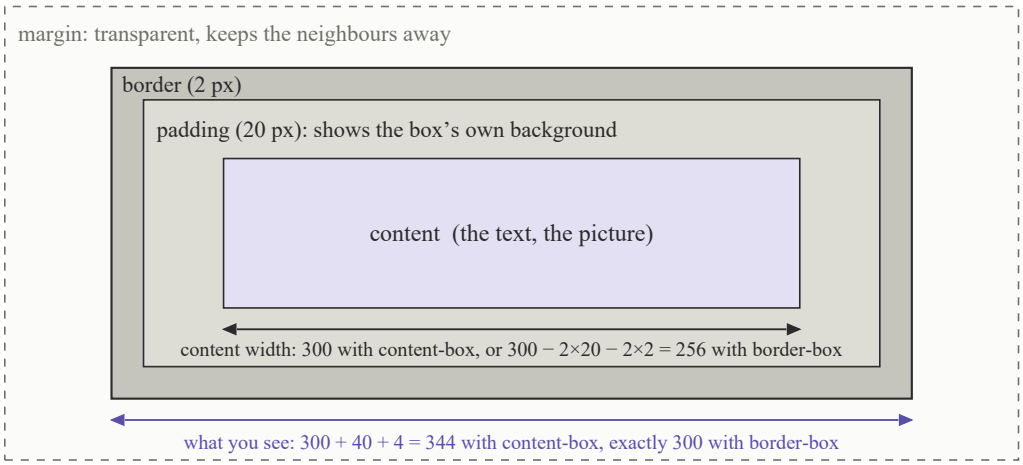


Figure 4.2. The four layers of every box, with the numbers for `width: 300px; padding: 20px; border: 2px solid`. The margin is outside the box and transparent; the padding is inside and shows the background.

```
* { box-sizing: border-box; }    /* width now includes padding and border */
```

WHY DO IT THIS WAY? Starting every stylesheet with border-box

> **What it is.** One rule that makes the declared `width` include padding and border.
>
> **Why it is done this way.** The width you write becomes the width you get. A `width: 300px` box is 300 pixels wide in total, whatever its padding.
>
> **Why it matters.** Layout arithmetic becomes simple. Two boxes of `width: 50%` really do fit side by side.
>
> **What happens if you do not.** Add padding to a box and it quietly gets wider, pushes its neighbour onto the next line, and you spend an evening wondering why 50% plus 50% does not equal 100%. The flight dynamics and mechatronics pages both start with this rule.

4.4 Flow, Flex and Grid

The browser has three main ways of arranging boxes.

Normal flow. Block boxes stack downwards, inline boxes run along a line and wrap. This is what you get if you write no layout CSS at all, and for text it is exactly right.

Flexbox (`display: flex`) arranges children along one direction and shares out the spare room. It is perfect for a row of buttons or a navigation bar. The properties you will reach for are `justify-content`, `align-items`, `gap` and `flex-wrap`.

Grid (`display: grid`) arranges children in rows *and* columns at once. It is the right tool for page layouts and sets of cards.

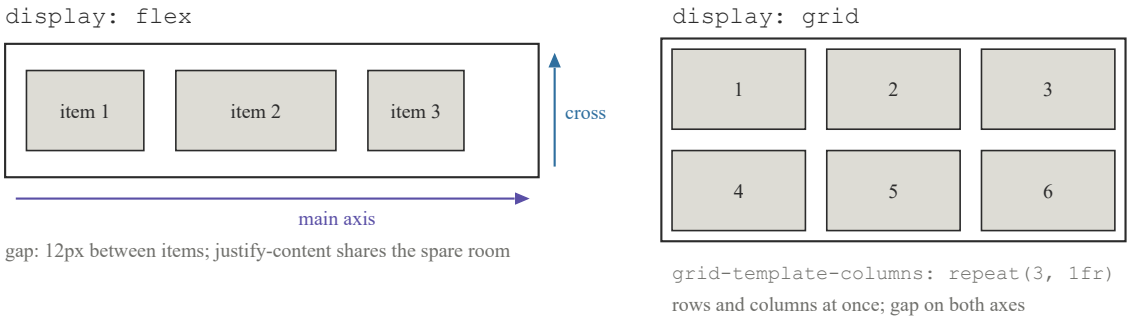


Figure 4.3. Flexbox lays items along one **main axis** and aligns them on the **cross axis**; Grid defines tracks in both directions and places items in the cells.

How Flexbox shares the room

Flexbox has a small piece of arithmetic that is worth knowing by heart, because it explains every "why is this item that wide?" puzzle. Each item starts at its **basis** (its `flex-basis`, or its content width). The container adds up the bases and the gaps; whatever is left over is the **free space**. If the free space is positive, it is handed out in proportion to each item's `flex-grow`. If it is negative, the shortfall is taken away in proportion to `flex-shrink` (weighted by basis). The shorthand `flex: 1` means `flex: 1 1 0`: a basis of zero, grow 1, shrink 1, which makes all the items equal.

Worked Example: Three items, three different grows

A row is 600 px wide with `gap: 20px`. Its three items each have `flex-basis: 100px` and `flex-grow` of 1, 2 and 1. How wide is each?

Step 1. The free space. Bases 3 × 100 = 300, gaps 2 × 20 = 40, so 600 − 300 − 40 = 260 px to share.

Step 2. The shares. The grows add to 4, so one share is 260 / 4 = 65 px. The items get 65, 130 and 65.

Step 3. The widths. 100 + 65 = 165, 100 + 130 = 230, 100 + 65 = 165.

Step 4. Check. 165 + 230 + 165 + 40 = 600. √ The middle item is not twice as wide as the others (230 against 165); only its *share of the spare room* is twice as big. If you want "twice as wide", give all three a basis of zero.

```
.cards {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
    gap: 1rem;
}
```

> **WHY DO IT THIS WAY? auto-fit with minmax**
>
> **What it is.** "Make as many columns as will fit, each at least 190 pixels wide, and share any leftover space equally."
>
> **Why it is done this way.** You describe the *rule* ("never narrower than 190") and let the browser work out how many columns fit.
>
> **Why it matters.** As the screen narrows, columns drop away on their own. No media queries, no special cases. The readout strips and control rows on this website all come from this one line.
>
> **What happens if you do not.** You write a media query for each screen size you can think of, and then a visitor arrives with a screen size you did not think of.

4.5 Units

CSS offers many units, and choosing one is choosing how the element responds to its surroundings.

Unit	Meaning	Use it for
`px`	A CSS pixel, a fixed length	Borders, hairlines, shadows
`rem`	A multiple of the root font size (16 px by default)	Font sizes and spacing
`em`	A multiple of the element's own font size	Spacing that should grow with its text
`%`	A fraction of the parent's size	Widths inside a flexible container
`vw`, `vh`	1% of the viewport width or height	Things sized to the screen
`fr`	A share of leftover space in a grid	Grid columns

> **WHY DO IT THIS WAY? Sizing text in rem, not px**
>
> **What it is.** Writing `font-size: 1.05rem` and not `font-size: 17px`.

Why it is done this way. A visitor can set a larger default text size in their browser. Every `rem` grows with that setting. A `px` size ignores it.

Why it matters. Someone who cannot read small text can make your page readable without any help from you.

What happens if you do not. Your carefully chosen small text stays small for the person who most needs it bigger.

Fluid sizes with clamp

`clamp(min, preferred, max)` gives a size that follows a preferred value but never leaves a safe range. It is how the headings on this website scale smoothly with no media queries.

```
.hero h1 { font-size: clamp(2.4rem, 6vw, 4.2rem); }
```

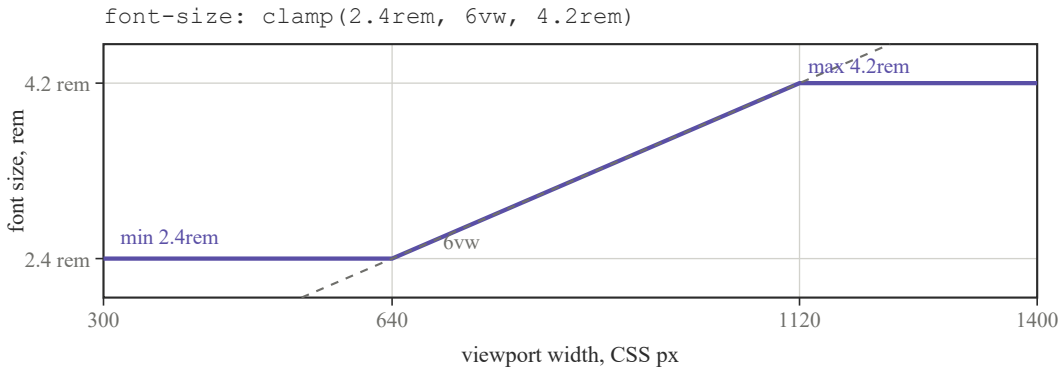


Figure 4.4. The size the heading takes at every viewport width. It follows the sloping `6vw` line between the two limits and is pinned to the limits outside them. With a 16 px root, `6vw` meets 2.4 rem at 640 px and 4.2 rem at 1 120 px.

Worked Example: Evaluating a fluid heading

The heading above is set on a page whose root font size is 16 pixels. Find its size at viewport widths of 320, 700 and 1400 CSS pixels.

Step 1. Convert the limits. The minimum is 2.4 × 16 = 38.4 px, and the maximum is 4.2 × 16 = 67.2 px.

Step 2. Evaluate the preferred value. 6vw is 6% of the width: 19.2 px at 320, 42.0 px at 700, and 84.0 px at 1400.

Step 3. Clamp. At 320 the preferred value 19.2 is below the minimum, so the heading is 38.4 px. At 700 the value 42.0 lies between the limits, so the heading is 42.0 px. At 1400 the value 84.0 exceeds the maximum, so the heading is 67.2 px.

Step 4. Interpret. The heading grows with the screen only between viewport widths of 38.4 ÷ 0.06 = 640 px and 67.2 ÷ 0.06 = 1120 px, and is constant outside that range. Choosing the preferred value therefore chooses *where* the fluid range sits.

4.6 Responsive Design

A **responsive** page adapts to the space it is given. Three tools do almost all the work. First, a viewport declaration in the page head tells phones to use their real width and not pretend to be a wide desktop.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

> **WHY DO IT THIS WAY? The viewport meta tag**
>
> **What it is.** One line in the `<head>`.
>
> **Why it is done this way.** Early phones pretended to be about 980 pixels wide so that old desktop sites would fit, then zoomed out. That pretence is still the default.
>
> **Why it matters.** The line switches the pretence off, so your CSS sees the phone's true width.
>
> **What happens if you do not.** Your carefully responsive page appears as a tiny zoomed-out desktop layout on a phone, and your media queries never fire.

Second, **media queries** apply rules only when a condition holds, most often a width.

```
@media (max-width: 720px) {
    .nav-status { display: none; }      /* on narrow screens, drop a decoration */
    .panel-canvas { height: 420px; }
}
```

Third, **fluid layout** (percentages, `fr`, `auto-fit`, `clamp`) does most of the adapting with no query at all. A good rule: let fluid layout do as much as it can, and add a media query only when the design truly has to change shape.

Pointer and motion preferences

Screens are not the only thing that varies. A phone has no hover, so an interface that shows things only on hover is useless on it. Media queries can ask about this: `(pointer: coarse)` matches touch, and `(prefers-reduced-motion: reduce)` matches someone who has asked their system for less animation. The 3D page on this website checks both.

> **TIP**
>
> Test responsiveness by dragging your browser window narrow, and by using the device toolbar in the developer tools to pretend to be a phone. It takes ten seconds and catches most layout bugs.

What You Learned

- Device pixels = CSS pixels × DPR. A canvas needs a surface of device size and a transform back to CSS units, or it blurs.
- Every element is content, padding, border, margin. `box-sizing: border-box` makes the width you write the width you get.
- Normal flow for text, Flexbox for one direction, Grid for two. Flexbox shares free space by `flex-grow`; Grid shares it by `fr`.

- `rem` respects the visitor's text size; `px` for hairlines; `%` and `fr` for flexible widths; `clamp()` for fluid sizes without queries.
- The viewport meta tag, media queries and fluid layout are the three tools of responsive design. Let fluid layout do as much as it can.
- Screens vary in more than width: ask about pointer type and motion preference too.

Check Your Knowledge

1. What is the relationship between CSS pixels, device pixels and the device pixel ratio?
2. A canvas is shown 600 CSS pixels wide on a DPR 3 screen. How many device pixels wide should its drawing surface be, and what happens if it is left at 600?
3. List the four layers of the box model from the inside outwards.
4. What does `box-sizing: border-box` change?
5. Explain what `repeat(auto-fit, minmax(190px, 1fr))` does as the container narrows.
6. Why is `rem` generally better than `px` for font sizes?

Review Problems

1. A canvas is shown at 512 by 384 CSS pixels. Give the drawing-surface size, and the number of pixels in the surface, at DPR 1, 1.5 and 2.
2. Using `clamp(1rem, 2.5vw + 0.5rem, 2rem)` with a 16 px root, compute the font size at viewport widths of 360, 800 and 1600 pixels. At what width does the size stop growing?
3. A box has `width: 200px; padding: 20px; border: 5px solid; margin: 10px`. Find the total horizontal space it occupies (a) with `content-box` and (b) with `border-box`.
4. A grid uses `repeat(auto-fit, minmax(190px, 1fr))` with a gap of 16 px in a container 900 px wide. How many columns are made, and how wide is each?
5. A page shows a 4 megapixel image (2000 by 2000) in a box that is 400 CSS pixels square on a DPR 2 screen. How many device pixels cover the box, and is the image sharp or soft? What is the smallest square image that would be sharp?
6. Write the CSS for a navigation bar that is a row of links with equal gaps on wide screens and a vertical stack of links on screens narrower than 600 px.

Chapter 5. Events, State and the Event Loop

A page that only shows things is a document. A page that responds to you is a program, and every program needs three things: a way to hear what you do, a place to remember what has happened, and a rule for turning what it remembers into what you see. Those three are **events**, **state** and **rendering**. Underneath all of them is a little piece of machinery called the event loop, which decides the order your code runs in. It is the single most useful thing to understand about JavaScript, and also the least obvious. Take a deep breath. It is simpler than it sounds.

In This Chapter

- Explain what "single-threaded" means and why it is both a gift and a limit.
- Describe the call stack, the task queue, the microtask queue and the loop that ties them together, and predict the order of a mixed script.
- Register handlers, read the event object, and use capture, bubble and delegation on purpose.
- Handle mouse, touch and pen with one set of pointer events.
- Decide what belongs in state, keep each fact once, and draw the screen as a function of it.

5.1 JavaScript Does One Thing at a Time

JavaScript on a page is **single-threaded**. It runs one piece of code at a time, all the way to the end, before it starts another. Picture a cashier with one till and a very polite queue. Nobody is served simultaneously.

That is a gift, because two of your functions can never change the same variable at the same instant, so a whole family of nasty bugs simply cannot happen. It is also a limit, because while one function is running, nothing else on the page can react, which is the long-task problem from Chapter 2.

So how does a one-at-a-time page wait for a click, a timer *and* a network reply? It does not wait. It *schedules*. The browser keeps lists of jobs that are ready, and a loop picks the next one when the current one finishes.

5.2 The Stack, the Queues and the Loop

Three structures work together.

The **call stack** records what is running right now. Calling a function pushes it on; returning pops it off.

The **task queue** holds jobs that are ready but waiting their turn: a click handler, a timer that has gone off, a message from somewhere else.

The **microtask queue** holds smaller follow-up jobs, mainly the next step of a **promise**. It has higher priority than the task queue.

The **event loop** is the rule that ties them together. It repeats forever:

1. Run the oldest task to completion.

2. Run *every* microtask, including any new ones they add, until none are left.
3. If it is time, let the browser redraw the screen.
4. Go back to step 1.

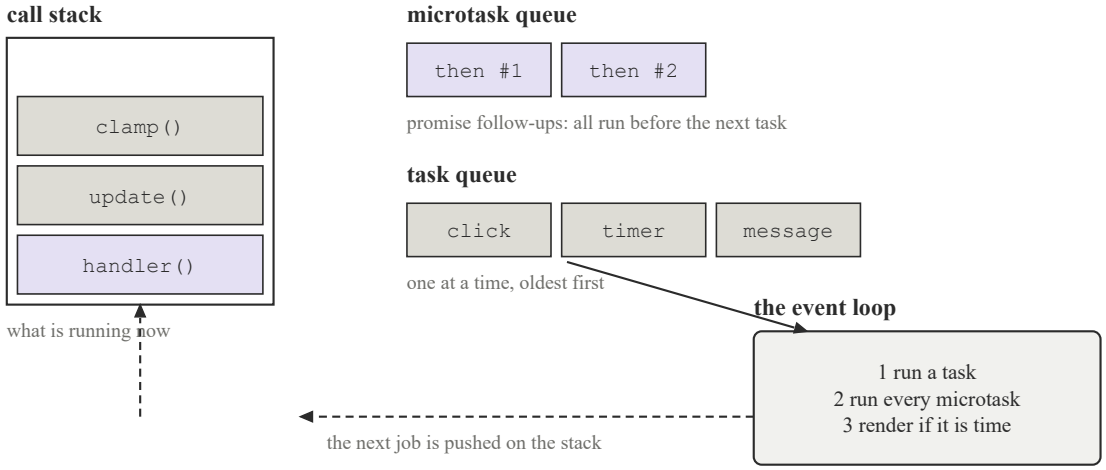


Figure 5.1. The three structures and the loop. Whatever is on the stack runs to the end. Then every microtask runs. Only then does the next task (a click, a timer) get its turn, and only between tasks can the browser redraw.

> **REMEMBER**
>
> The current code always finishes first. Then every waiting microtask runs. Then the next task begins. If you remember only that sentence from this chapter, you can predict almost any ordering puzzle JavaScript throws at you.

Worked Example: Predicting the order of output

What does this script print, and in what order?

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Step 1. Run the script itself. The script is one task. It executes from top to bottom without interruption. `console.log('A')` prints A at once.

Step 2. Schedule the timer. `setTimeout(..., 0)` does not run its callback now. It asks the browser to put the callback in the *task* queue as soon as possible, which is after the current task. So B is queued as a task.

Step 3. Schedule the promise. `Promise.resolve().then(...)` queues the callback as a *microtask*. So C is queued as a microtask.

Step 4. Finish the script. `console.log('D')` prints D. The task is now complete.

Step 5. Drain the microtasks. Before any new task, the loop runs all microtasks, so C prints.

Step 6. Take the next task. Only now does the loop run the timer's task, and B prints.

Answer: A, D, C, B. The timer's delay of zero does not mean "immediately". It means "after the current work and after all microtasks", and the surprise that C precedes B is the whole lesson.

> **WHY DO IT THIS WAY? setTimeout with zero is not "now"**
>
> **What it is.** `setTimeout(fn, 0)` asks for `fn` to run "as soon as possible".
>
> **Why it is done this way.** "As soon as possible" has to mean "after the code that is running now", because JavaScript never interrupts itself.
>
> **Why it matters.** It is a useful way to say "do this after the current work, and after the browser has had a chance to breathe".
>
> **What happens if you do not understand it.** You write code that assumes the timer fires immediately, and get bugs that appear only sometimes.

5.3 Events and Handlers

An **event** is a signal that something happened: a click, a key press, a finger moving, a page finishing loading, a timer expiring. You respond by registering a **handler**, a function the browser will call when it happens.

```
canvas.addEventListener('pointerdown', (event) => {
    console.log('pressed at', event.clientX, event.clientY);
});
```

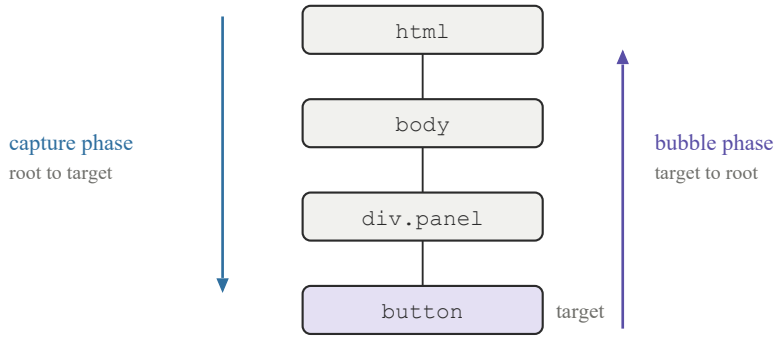
The event object passed to your handler carries the details: where the pointer was, which key, which element was the target.

> **WHY DO IT THIS WAY? Keeping handlers small**
>
> **What it is.** A handler records what happened and does little else. The heavy work happens elsewhere, once per frame.
>
> **Why it is done this way.** Handlers run as tasks on the single main thread.
>
> **Why it matters.** A slow handler blocks everything, including the next click, scrolling and animation.
>
> **What happens if you do not.** The page feels frozen after every click.

Propagation: capture and bubble

An event does not just arrive at the element you clicked. The page is a tree, and the event travels through it in three phases. In **capture**, it travels down from the root to the target. At the **target**, it fires on the element itself. In **bubble**, it climbs back up through every ancestor. A normal handler listens during the bubble phase, so clicking a button inside a panel inside the page triggers the button's handler, then the panel's, then the page's.

This is handy. Instead of attaching a handler to each of a hundred chips in a list, you attach *one* handler to the list and look at `event.target` to see which chip was clicked. That is called **event delegation**.



a normal addEventListener handler listens on the way back up, so one listener on div.panel hears every button inside it

Figure 5.2. A click on the button travels down the tree in the capture phase, fires at the target, and climbs back up in the bubble phase. A normal listener on `div.panel` hears the click on its way back up.

> **WHY DO IT THIS WAY? One handler on the parent instead of many on the children**
>
> **What it is.** `list.addEventListener('click', ...)`, then checking `event.target.closest('li')`.
>
> **Why it is done this way.** The children's events bubble up to the parent anyway.
>
> **Why it matters.** One handler uses less memory than a hundred, and it keeps working for chips you add later. Handlers attached to individual chips would not exist on the new ones.
>
> **What happens if you do not.** You forget to attach a handler to the new items, and some buttons do nothing.

Two methods give you control. `event.stopPropagation()` stops the event climbing further. `event.preventDefault()` cancels the browser's default action (such as following a link).

Pointer events

Mouse, touch and pen used to need three sets of handlers. **Pointer events** (`pointerdown`, `pointermove`, `pointerup`, `pointercancel`) unify them. One handler serves a mouse, a finger and a stylus. A further call, `setPointerCapture`, tells the browser to keep sending the pointer's events to one element even if the pointer wanders outside it.

```
let dragging = false, lastX = 0;
canvas.addEventListener('pointerdown', e => {
    dragging = true; lastX = e.clientX; canvas.setPointerCapture(e.pointerId);
});
canvas.addEventListener('pointermove', e => {
    if (!dragging) return;
    azimuth += (e.clientX - lastX) * 0.01;      // turn the camera by the drag dista
      nce
    lastX = e.clientX;
});
canvas.addEventListener('pointerup', () => { dragging = false; });
```

WHY DO IT THIS WAY? Capturing the pointer when a drag starts

What it is. Calling `canvas.setPointerCapture(e.pointerId)` inside `pointerdown`.

Why it is done this way. Without it, `pointermove` events only arrive while the pointer is over the element.

Why it matters. Drag fast and the cursor races ahead of the element. With capture, the element keeps receiving moves until you let go.

What happens if you do not. Dragging "sticks" as soon as the cursor leaves the element, and orbiting a 3D model feels broken. The viewers on this website use this exact pair of calls.

5.4 State

State is the data that, at any moment, completely describes what the page should show: which tab is selected, how far a slider has moved, whether a panel is open, how many seconds of an animation have passed. It is the page's memory.

The most common cause of bugs in interactive pages is state kept in *two places* that fall out of step. If one variable says the tab is "power" and the screen shows the "sound" tab because a class got left on the wrong element, the page is lying. The cure is to keep each fact **once**, and treat everything on screen as a *consequence* of it, never a second copy.

WHY DO IT THIS WAY? One state object for the whole page

What it is. The robot page keeps its choices (which part is selected, whether the rubber bands are on, how far the robot is taken apart) in a single object. Every control changes a field. Every instrument reads from it.

Why it is done this way. Then there is only one place to look when something is wrong, and only one place to change.

Why it matters. The slider that takes the robot apart and the automatic tour that does the same thing both simply set one number, and everything else follows. Neither needs to know about the other.

What happens if you do not. The slider moves one copy and the tour moves another, they disagree, and the page flickers between two truths.

5.5 Rendering as a Function of State

This brings us to the idea that organises modern interface programming. Think of the screen as a function of the state:

screen = render(state)

Given the same state, you get the same screen. You do not write "when the tab changes, hide this and show that". You write "given the current tab, here is what the page looks like", and run it again whenever the state changes. The first style piles up special cases. The second has none, because the

page simply *is* whatever the state says.

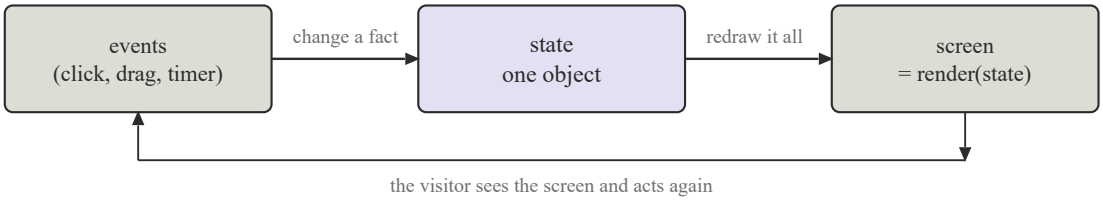


Figure 5.3. The loop every interactive page runs. Events change one fact in the state; the screen is redrawn from the state; the visitor sees it and acts again. Nothing on screen is a second copy of anything.

Worked Example: Deciding what is state

A panel shows one of three charts (speed, power, sound), chosen by three chip buttons, and has a "pause" button that freezes the animation. List the state, and say what is *not* state.

Step 1. List the facts that can change. Which chart is chosen: one of three values. Whether the animation is paused: true or false. How far the animation has run: a time in seconds. That is the whole state: `{ chart: 'speed', paused: false, t: 0 }`.

Step 2. Find the impostors. "Which chip has the highlighted class" is not state. It is a consequence of `chart`. "Whether the pause button says Pause or Resume" is not state; it follows from `paused`. "The pixels of the chart" are not state; they are drawn from `chart` and `t`. Each of these is something `render` works out.

Step 3. Write the render. `render(state)` sets the highlighted class on the chip whose `data-chart` matches `state.chart`, writes the button label from `state.paused`, and draws the chart named by `state.chart` at time `state.t`.

Step 4. Check it. Could the page ever show the power chart with the speed chip highlighted? Only if `render` has a bug, because both come from the same field. Before, with the class toggled by hand in three different click handlers, it was one forgotten line away.

Part II shows how the React library automates this for a whole page. A canvas animation already does it in its purest form: every frame it clears the canvas and redraws everything from the current state, with no memory of the previous frame.

```
function frame() {
    draw(state);                    // the whole picture, from scratch
    requestAnimationFrame(frame);   // and again on the next frame
}
```

> **WHY DO IT THIS WAY? Redrawing everything from scratch each frame**
>
> **What it is.** The canvas is cleared and the whole picture is rebuilt from `state` on every frame.
>
> **Why it is done this way.** It sounds wasteful, but it removes a huge class of bugs: leftover pixels, half-erased shapes, "ghosts" of old frames.

> **Why it matters.** The code is simple and the picture is always correct, because it depends only on the current state.
>
> **What happens if you do not.** You have to track what changed and erase just that, which gets complicated fast.

The next chapter takes up this loop.

What You Learned

- JavaScript runs one thing at a time. The running code always finishes, then every microtask, then the next task, then (if it is time) a redraw.
- `setTimeout(fn, 0)` means "after the current work", never "now". Promise follow-ups run before any timer.
- A handler receives an event object with the details. Keep handlers small; do the heavy work once per frame.
- Events capture down and bubble up. One listener on a parent can serve every child, including children added later.
- Pointer events cover mouse, touch and pen. Capture the pointer when a drag starts.
- State is the set of facts the page needs. Keep each fact once and draw everything from it: screen = render(state).

Check Your Knowledge

1. Why is JavaScript's being single-threaded both an advantage and a limitation?
2. State, in order, the four steps the event loop repeats.
3. What is the difference between the task queue and the microtask queue, and which has priority?
4. Describe the three phases of event propagation, and say in which phase a normal handler runs.
5. What is event delegation, and why does it keep working when elements are added later?
6. What does `setPointerCapture` do, and why is it useful when dragging?
7. Why is it a bug to hold the same fact in two places in a page's state?

Review Problems

1. Predict the output of the following and explain each step.

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3)).then(() => console.log(4));
setTimeout(() => console.log(5), 0);
console.log(6);
```

2. A click handler contains a loop that takes 300 milliseconds. A visitor clicks, and 100 ms later clicks a second button. When does the second button's handler run? Why?
3. A list contains 200 `<li>` elements. Compare the number of handlers needed to react to a click on any of them with and without event delegation. Write the delegating handler.

4. A page has a `<div class="outer"><button class="inner"></button></div>`. Both have click handlers that print their class name. In what order do they print on a click of the button? Add one line to the inner handler that makes the outer handler never print.
5. A page has three tabs and a panel for each. The first version stores the selected tab as a class on the tab button *and* as a class on the panel. Describe a sequence of events that could leave the page in an inconsistent state, and redesign it to hold the selection once.
6. A drag handler registered on `pointermove` at the element (without pointer capture) is used on a slider. The user drags quickly to the right and the cursor leaves the element. What goes wrong, and how is it fixed?

Chapter 6. Time, Motion and Animation

Animation is a magic trick. It is a series of still pictures shown fast enough that your brain sees motion. Whether the trick works depends on two things: *when* each picture appears, and *which in-between values* you choose. This chapter covers both: the loop that drives every animation on the web, the maths of easing and smoothing, and the good manners a page should show to visitors who would rather it sat still.

In This Chapter

- Write an animation loop with `requestAnimationFrame` and explain why a timer is the wrong tool.
- Make motion independent of the frame rate with delta time, and clamp it.
- Interpolate between two values and shape the motion with an easing function.
- Smooth a value towards a moving target in a way that behaves the same at every frame rate.
- Choose between a CSS transition, a CSS animation and a script loop.
- Honour `prefers-reduced-motion`.

6.1 The Animation Loop

An animation needs a fresh picture every time the screen refreshes. The wrong way to ask for one is a timer. `setInterval(draw, 16)` fires about every 16 milliseconds, but its rhythm has nothing to do with the screen's. Frames land just before or just after a refresh, and the motion shivers. The right way is to ask the browser itself, with `requestAnimationFrame`. You hand it a function, and the browser calls it once, just before the next paint, with a timestamp. Your function draws a frame and asks again.

```
let last = null;
function frame(now) {                       // now: milliseconds since the page star
      ted
    const dt = last === null ? 0 : (now - last) / 1000;   // seconds since the last
      frame
    last = now;
    update(dt);                              // advance the state by dt
    draw();                                  // draw it
    requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

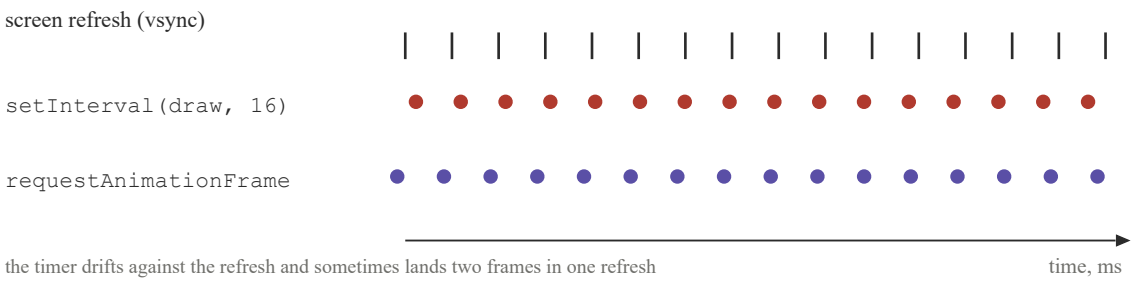


Figure 6.1. A timer set to 16 ms slowly drifts against a 16.67 ms refresh: sometimes two of its frames land inside one refresh and one is thrown away, sometimes a refresh gets none and the old picture repeats. `requestAnimationFrame` is called once per refresh, just before the paint.

> **WHY DO IT THIS WAY? requestAnimationFrame instead of a timer**
>
> **What it is.** The browser's own "call me before the next paint" service.
>
> **Why it is done this way.** Only the browser knows exactly when the next refresh is. A timer is just guessing.
>
> **Why it matters.** Frames arrive at the right moment, so motion is smooth. As a bonus, the browser pauses the calls when the tab is hidden, so a background tab costs you nothing.
>
> **What happens if you do not.** A timer keeps burning battery in hidden tabs, and its animation judders because it drifts out of step with the screen.

6.2 Delta Time: Motion That Does Not Depend on Your Computer

Imagine moving a ball with `x += 5` on every frame. On a 60-hertz screen it travels 300 pixels a second. On a 144-hertz screen the same code runs 144 times a second, so the ball travels 720 pixels a second, more than twice as fast. A game that runs at different speeds on different machines is broken.

The fix is to tie motion to *time*, not frames. Choose a speed in units per second, and move by that speed multiplied by the time the frame took.

$$x \leftarrow x + v \times \Delta t$$

Here v is the speed (say 300 pixels per second) and Δt is the **delta time**: the seconds since the last frame. At 60 hertz Δt is 0.0167, so the ball moves 5 pixels a frame. At 144 hertz Δt is 0.0069, so it moves 2.08 pixels a frame. Either way it covers 300 pixels each second. This is **frame-rate independence**.

One more precaution is needed, because the loop above trusts Δt blindly. A tab hidden for a minute would return with a Δt of sixty seconds and fling everything off the screen. Robust code **clamps** the delta time to a small maximum, as in `dt = Math.min(dt, 0.1)`, and the animation loops of the flight dynamics and mechatronics pages of this website do so.

> **WHY DO IT THIS WAY? Multiplying by delta time, and clamping it**
>
> **What it is.** Using `speed * dt` for movement, and capping `dt` at about 0.1 seconds.

Why it is done this way. Time is the same on every computer. Frames per second is not.

Why it matters. The animation runs at the same speed on a phone, a laptop and a gaming monitor. The clamp stops a long pause from turning into one gigantic leap.

What happens if you do not. The page runs twice as fast on a fast monitor, and objects teleport when you switch back to the tab.

6.3 Interpolation and Easing

An animation between two values must decide what happens in between. **Linear interpolation**, often called *lerp*, blends a start value a and an end value b by a fraction t between 0 and 1:

lerp(a, b, t) = a + (b − a) × t

With t = 0 you get a, with t = 1 you get b, and at t = 0.5 you get the middle. If t grows steadily with time, the motion is steady and **linear**: it starts and stops with a jolt and looks mechanical. Real objects speed up and slow down. We imitate that by passing t through an **easing function**.

The simplest useful one is **smoothstep**:

smoothstep(t) = 3t² − 2t³, for t between 0 and 1.

It is 0 at the start, 1 at the end, and its slope is zero at both ends, so motion eases out of rest and settles back into it. CSS has a more flexible family, the cubic Bézier curves (`cubic-bezier(0.16, 1, 0.3, 1)` is a popular "quick start, long gentle stop"). A good habit: ease things *out* when they arrive (fast, then settling) and *in* when they leave (slow, then fast).

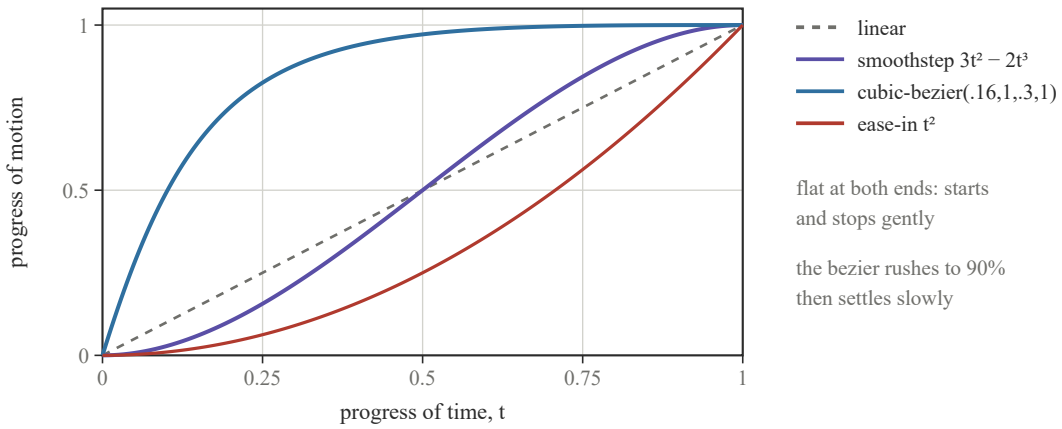


Figure 6.2. Four ways to get from 0 to 1 in the same time. The horizontal axis is how much of the time has passed; the vertical axis is how much of the distance has been covered. The slope at any point is the speed.

TECHNICAL STUFF: Technical Stuff

> A `cubic-bezier(x1, y1, x2, y2)` curve is drawn by a point that moves from (0, 0) to (1, 1) while being pulled towards two control points, (x1, y1) and (x2, y2). The browser treats the curve as a graph of progress against time, so the x values must stay between 0 and 1, but the y values may overshoot: `cubic-bezier(0.34, 1.56, 0.64, 1)` goes past 1 and comes back, which is how a "bouncy" arrival is made. The four CSS keywords are shorthands: `ease` is `cubic-bezier(0.25, 0.1, 0.25, 1)`, `ease-in` is `(0.42, 0, 1, 1)`, `ease-out` is `(0, 0, 0.58, 1)` and `ease-in-out` is `(0.42, 0, 0.58, 1)`.

Worked Example: Smoothstep against linear

An element slides from x = 0 to x = 200 over one second. Compare its position at t = 0.25 and t = 0.5 using linear motion and smoothstep.

Step 1. Linear. Position = 200 t. At t = 0.25 it is 50, at t = 0.5 it is 100.

Step 2. Smoothstep. At t = 0.25: 3(0.0625) − 2(0.015625) = 0.1875 − 0.03125 = 0.15625, so the position is 200 × 0.15625 = 31.25. At t = 0.5: 3(0.25) − 2(0.125) = 0.75 − 0.25 = 0.5, so the position is 100.

Step 3. Compare. At t = 0.25 the eased element has covered only 31.25 of the 200 pixels, against 50 for the linear one: it is starting gently. At the midpoint they agree. By symmetry, the eased element is also behind at the end of its journey in the sense of slowing down: at t = 0.75 it is at 168.75, ahead of the linear 150, and it then settles into 200.

Why it matters. The two motions take the same time, but the eased one avoids the jolt at each end, and the eye reads it as more natural.

> **WHY DO IT THIS WAY? Easing instead of linear motion**
>
> **What it is.** Passing the progress value through smoothstep or a Bézier curve before using it.
>
> **Why it is done this way.** Real objects have mass, so they accelerate and decelerate.
>
> **Why it matters.** The same movement in the same time reads as natural instead of robotic.
>
> **What happens if you do not.** Panels snap into motion and stop dead. Nothing is broken, but it feels cheap, and viewers notice even if they cannot say why.

6.4 Smoothing Towards a Target

Sometimes there is no fixed duration. A camera follows a moving robot. A reading chases a value the user keeps changing. Here the motion is better described as "always move towards the target". The standard trick is **exponential smoothing**: each frame, close a fraction of the remaining gap.

x ← x + (target − x) × k

With k = 0.1, each frame closes a tenth of the gap. It moves fast when far away and slows as it nears the target, and it adapts at once if the target moves.

The trouble is that this is frame-rate dependent: more frames per second means the same k closes the gap faster. To fix it, ask what the rule means in continuous time. Closing the gap at a steady *rate* is the equation dx/dt = s(target − x), and after a time Δt it closes the fraction

$$\mathbf{k = 1 - e^{-s\,\Delta t}}$$

where s is a "snappiness" in units of 1 per second. Compute k from Δt every frame and the motion is identical on every screen.

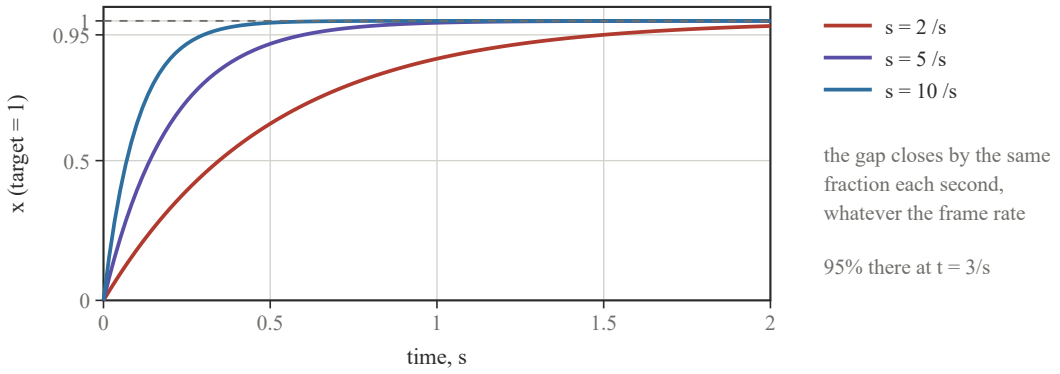


Figure 6.3. Exponential smoothing towards a target of 1 for three values of s. The curve never quite arrives, but it is 95 percent of the way there after 3/s seconds, and 99 percent after 4.6/s.

Worked Example: How long until the camera settles?

The robot page's camera uses s = 5.5 per second. How long does it take to close half the gap? To be within 1 percent of the target? And if a designer asks for it to feel "twice as snappy", what should s become?

Step 1. Half the gap. The remaining gap after time t is e^{-st}. Half remains when $e^{-st} = 0.5$, so t = ln 2 / s = 0.693 / 5.5 = 0.126 s. (This is the smoothing's *half-life*.)

Step 2. Within 1 percent. $e^{-st} = 0.01$ gives t = ln 100 / s = 4.605 / 5.5 = 0.837 s.

Step 3. Twice as snappy. Every time in this motion scales as 1/s, so doubling s to 11 halves every settling time: 0.063 s to half and 0.419 s to 1 percent.

What the answer tells you. With this kind of smoothing you never choose a duration. You choose a *rate*, and all the durations follow. That is why one number, s, is enough to tune the whole feel of the camera.

Worked Example: A frame-rate independent camera

The camera of the robot page eases towards its target with snappiness s = 5.5 per second. Find the fraction of the gap closed per frame at 30, 60 and 144 hertz, and the fraction closed in half a second.

Step 1. Per-frame fraction. k = 1 − exp(−5.5 × Δt), with Δt = 1/30, 1/60 and 1/144.

Step 2. Evaluate. At 30 Hz: 5.5/30 = 0.1833, exp(−0.1833) = 0.8325, so k = 0.1675. At 60 Hz: 5.5/60 = 0.0917, exp(−0.0917) = 0.9124, so k = 0.0876. At 144 Hz: 5.5/144 = 0.0382, exp(−0.0382) = 0.9625, so k = 0.0375.

Step 3. Check that they agree. At 30 Hz, half a second is 15 frames, and the gap remaining is (1 − 0.1675)^15 = 0.8325^15 = 0.0639. At 60 Hz it is 30 frames of 0.9124: 0.9124^30 = 0.0639. At 144 Hz it is 72 frames of 0.9625: also 0.0639. The remaining gap is the same on all three screens: 6.4%.

Step 4. Closed form. The fraction closed in half a second is $1 - e^{-5.5 \times 0.5} = 1 - e^{-2.75} = 1 - 0.0639 = 0.936$, matching the product of per-frame factors, as it must. After half a second the camera is 93.6% of the way there.

> **WHY DO IT THIS WAY? Computing k from delta time**
>
> **What it is.** Using `k = 1 - Math.exp(-s * dt)` instead of a fixed number like `0.1`.
>
> **Why it is done this way.** It is the exact solution of "approach the target at a steady rate", so it does not matter how long each frame is.
>
> **Why it matters.** The robot page's camera glides the same way on a 30 hertz phone and a 144 hertz monitor.
>
> **What happens if you do not.** The camera is sluggish on slow devices and jumpy on fast ones, and you will never see the bug on your own machine.

6.5 Timelines and Sequences

Many animations are scripts: first this, then that two seconds later, then both together. The simplest way to build one is a list of **keyframes**, each saying "at time t, do this", walked through as the clock advances. The automatic tour of the robot page works exactly this way: a list of entries like "at 19 seconds, select the sensor", plus a counter that advances by Δt each frame and fires each entry as its time passes. Because the clock is just a number, you can pause the script, restart it, or jump around in it by changing that number. Libraries such as GSAP, covered in Part II, polish this machinery, but the idea is no bigger than this.

6.6 CSS Animation

Not every animation needs JavaScript. A **transition** animates a property from its old value to its new one whenever it changes.

```
.chip { background: transparent; transition: background 150ms ease, color 150ms ease
      ; }
.chip:hover { background: var(--accent); color: #0b0820; }
```

An **animation** uses **keyframes** to describe a longer or repeating motion.

```
@keyframes blink { 50% { opacity: 0.15; } }
.live-dot { animation: blink 1.2s steps(2) infinite; }
```

> **WHY DO IT THIS WAY? Using CSS for simple state changes**
>
> **What it is.** Letting a stylesheet do hover effects and blinking dots, with no script.
>
> **Why it is done this way.** The browser can run many CSS animations without involving the main thread at all.
>
> **Why it matters.** Your JavaScript stays free for the real work, and the animation stays smooth even when the page is busy.
>
> **What happens if you do not.** A script-driven hover effect stutters whenever the page is doing something heavy. Use JavaScript for motion that depends on data or on a continuous calculation, and CSS for everything else.

6.7 Motion with Manners

Some visitors find motion distracting. For some it is physically unpleasant: big movements and constant background animation can cause dizziness and nausea for people with vestibular disorders. Phones and laptops let a person say "less motion, please", and the browser passes that on to your page as `prefers-reduced-motion: reduce`. A well-mannered page listens.

```
@media (prefers-reduced-motion: reduce) {
    .live-dot { animation: none; }
    html { scroll-behavior: auto; }
}
```

In JavaScript you read it with `window.matchMedia('(prefers-reduced-motion: reduce)').matches`.

> **WHY DO IT THIS WAY? Honouring reduced motion**
>
> **What it is.** Checking the preference and, if it is set, showing a still frame instead of starting animation loops, and skipping the automatic tour.
>
> **Why it is done this way.** The visitor has told you, through their own system, what they need.
>
> **Why it matters.** It can be the difference between a page someone can use and a page that makes them ill. The instruments still work: every control still redraws the picture. Only the *unrequested* motion goes away.
>
> **What happens if you do not.** Some visitors simply cannot use your site, and they will not tell you. They will just leave.

What You Learned

- Ask the browser for frames with `requestAnimationFrame`; it calls you once per refresh and stops when the tab is hidden.
- Move by speed × Δt, never by a fixed amount per frame, and clamp Δt so a long pause cannot become a leap.
- `lerp(a, b, t)` blends two values; an easing function reshapes t so motion starts and stops like a real object.

- To follow a moving target, close a fraction $k = 1 - e^{-s\Delta t}$ of the gap each frame. The rate s sets every settling time.
- Timelines are lists of keyframes walked by a clock. CSS transitions and animations handle simple state changes without script.
- Honour `prefers-reduced-motion`: keep the controls working, drop the unrequested motion.

Check Your Knowledge

1. Why is `requestAnimationFrame` better than `setInterval` for animation?
2. What is delta time, and what problem does it solve? Why should it be clamped?
3. Write the linear interpolation formula and say what t = 0, 0.5 and 1 give.
4. What is special about the slope of smoothstep at the ends of its range?
5. What problem does exponential smoothing solve, and why is `x += (target - x) * 0.1` frame-rate dependent?
6. What does `prefers-reduced-motion` tell a page, and what should the page do about it?

Review Problems

1. An object moves at 240 pixels per second. How far does it move in one frame at 30, 60 and 120 hertz when written with delta time? How far does `x += 4` move it per second at each of those rates?
2. Compute smoothstep at t = 0.1, 0.5 and 0.9, and the position of an element travelling from 0 to 80 at each.
3. A value eases towards its target with k = 0.1 per frame at 60 Hz. What value of snappiness s gives this behaviour in continuous time? What is k per frame at 144 Hz for the same s?
4. A smoothed value starts 100 units from its target with s = 4 per second. How far is it from the target after 0.25 s, 0.5 s and 1 s?
5. A tour script has entries at 0, 7, 12, 16 and 19 seconds. The clock is currently at 14.3 seconds and a frame lasts 0.016 seconds. Which entries have fired, and which is next? How many frames until the next one fires, if the frame length stays constant?
6. Write the JavaScript that reads the reduced-motion preference and, if it is set, draws a single frame and does not start the animation loop. Explain why the instrument still responds to the visitor.

Chapter 7. Graphics on the Web

HTML and CSS are great at documents: text, boxes, pictures. But when a page needs to draw a chart of a phugoid, the path of a spiral climb, or a robot you can take apart, it has to draw pictures *from data*. The browser gives you three ways to do that, and they are different in kind. This chapter describes them, and then explains the maths that turns a 3D model into the flat picture on your screen. That maths is the same whether you write it yourself or let a library do it, so it is worth knowing.

In This Chapter

- Choose between SVG, Canvas 2D and WebGL for a given drawing, and explain retained against immediate mode.
- Use the canvas state machine: paths, the current transform, `save` and `restore`.
- Map data coordinates to pixels with one pair of functions.
- Follow a 3D point from model space to the screen, and compute where it lands with the perspective formula.
- Explain depth precision, Lambert lighting, physically based materials, and why draw calls, not triangles, usually limit speed.

7.1 Raster and Vector

A **raster** image is a grid of coloured pixels. A photograph is a raster. It has a fixed resolution and goes soft when you enlarge it. A **vector** image is a list of *instructions*: draw a circle here, a curve there, fill with this colour. It can be drawn at any size without loss, because the drawing is worked out afresh each time.

Tool	Kind	Model	Best for
SVG	Vector	A tree of shapes in the DOM; each shape is an element	Diagrams, icons, a few hundred shapes you want to style and click
Canvas 2D	Raster, drawn with vector commands	A bitmap you paint on; it keeps no memory of shapes	Charts and animation with thousands of marks redrawn every frame
WebGL	Raster, drawn by the GPU	Triangles and small programs for the graphics processor	3D scenes and huge amounts of geometry

The big difference between SVG and canvas is **retained** versus **immediate** mode. SVG *retains* its shapes: each is an element the browser remembers, so you can style it with CSS, click it, and move it by changing an attribute. The price is that thousands of elements slow the page. A canvas is **immediate**: you issue commands, pixels are painted, and the canvas forgets how they got there. It cannot tell you which shape you clicked, but it can paint a hundred thousand marks every frame.

> **WHY DO IT THIS WAY? Charts on a canvas, diagrams in SVG**

What it is. The charts of this website are drawn on canvases. The control-system block diagram is an SVG.

Why it is done this way. A chart redraws its curves many times per second as sliders move, and a canvas is made for that. The block diagram is a few dozen static shapes that benefit from being real elements, so they can be styled, animated with CSS and read by tools.

Why it matters. Using the right tool means the chart is fast and the diagram is crisp and light.

What happens if you do not. A chart in SVG with thousands of elements crawls. A diagram on a canvas cannot be styled with CSS or scaled without redrawing.

7.2 The Canvas 2D Model

You use a canvas through a **context**, which behaves like a pen with settings. The settings (colour, line width, font, the current transform) are called the *state*, and every drawing command uses whatever the state currently says.

```
const ctx = canvas.getContext('2d');
ctx.strokeStyle = '#a78bfa';        // set state
ctx.lineWidth = 2;
ctx.beginPath();                    // begin a new path
ctx.moveTo(10, 90);                 // lift the pen and put it here
ctx.lineTo(60, 20);                 // draw a segment
ctx.lineTo(110, 70);
ctx.stroke();                       // paint the path with the current state
```

The origin is the *top-left* corner, x increases to the right, and y increases *downward*, the opposite of what you learned in maths. So a chart that plots larger values higher has to flip the axis, and every chart on this website does so with a tiny mapping function.

```
const X = v => left + (v - xmin) / (xmax - xmin) * width;          // data x -> pixe
      l x
const Y = v => top + height - (v - ymin) / (ymax - ymin) * height; // data y -> pixe
      l y (flipped)
```

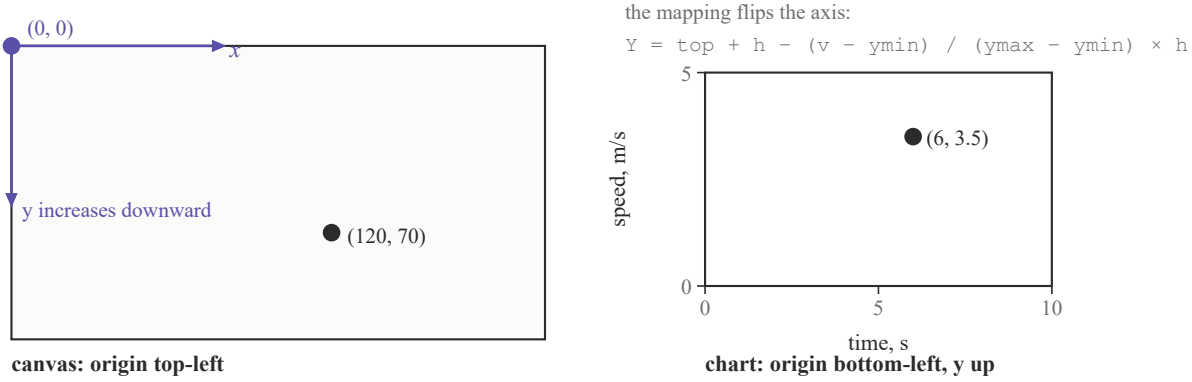


Figure 7.1. The canvas counts from the top-left corner with y pointing down; a chart counts from the bottom-left with y pointing up. The `Y` mapping function is where the flip happens, once, for every mark on the chart.

WHY DO IT THIS WAY? Mapping functions for X and Y

What it is. Two small functions that turn a data value into a pixel position.

Why it is done this way. Your data (speed in metres per second, time in seconds) and the canvas (pixels, y downward) speak different languages. You need one translator, in one place.

Why it matters. Every line, tick and label goes through the same mapping, so they all agree. Changing the axis range changes one line.

What happens if you do not. Pixel arithmetic ends up scattered through the drawing code, and the first time you change an axis range, half the marks stay where they were.

The **transform** is part of the state: `translate`, `rotate` and `scale` change how later coordinates are read. To draw an aircraft symbol turned through an angle, move the origin to the aircraft, rotate the axes, and draw the symbol upright around the origin. Wrap this in `ctx.save()` and `ctx.restore()`, which push and pop the whole state, to undo it afterwards.

```
ctx.save();
ctx.translate(x, y);          // move the origin to the aircraft
ctx.rotate(-pitch);           // turn the axes (negative: y points down on screen)
drawAircraft(ctx);            // drawn upright around the origin
ctx.restore();                // everything back as it was
```

WHY DO IT THIS WAY? save and restore around a transform

What it is. Calling `ctx.save()` before changing the transform and `ctx.restore()` after.

Why it is done this way. The transform is *sticky*. It stays in force until you undo it.

Why it matters. Everything you draw afterwards is unaffected.

What happens if you do not. The axes of your chart end up rotated and shifted because you drew an aircraft symbol earlier, and it is very hard to tell why.

7.3 SVG in Brief

An SVG graphic is XML written right in the page. Its shapes are elements, and its coordinates live in a **viewBox**, a private coordinate system that the browser scales to whatever size the graphic is shown at. That is what makes it resolution-independent.

```
<svg viewBox="0 0 100 60" width="300">
  <rect x="5" y="5" width="40" height="20" rx="3" fill="none" stroke="currentColor"/
      >
  <path d="M45 15 H80 V45" fill="none" stroke="currentColor"/>
  <circle cx="80" cy="45" r="4" fill="#a78bfa"/>
</svg>
```

The control-system block diagram on the flight dynamics page is an SVG animated purely with CSS: the dashed signal lines have their `stroke-dashoffset` changed by a keyframe animation, so the dashes seem to flow along the wires.

WHY DO IT THIS WAY? Animating the diagram with CSS only

What it is. A `@keyframes` rule that shifts the dash pattern, with no JavaScript.

Why it is done this way. It is a decoration that never changes in response to data.

> **Why it matters.** It costs nothing on the main thread and keeps working even if a script fails.
>
> **What happens if you do not.** A script that animates a decoration is one more thing to break and one more thing to slow the page.

7.4 Three Dimensions: The Idea

A 3D scene is a collection of points in space, and the screen is flat. So every 3D program has to answer one question: given a point in space, where on the screen does it appear? The answer is a chain of transformations.

Coordinates and handedness

Space is described by three axes. In three.js, x points right, y points up, and z points *towards you*. That makes the system **right-handed**: curl the fingers of your right hand from x towards y and your thumb points along z. Other tools choose differently, and many CAD programs put z upward. A model from another tool may need a turn to stand upright, and a simulation with its own convention needs a deliberate conversion at the boundary.

> **WHY DO IT THIS WAY? Naming your frames and converting in one place**
>
> **What it is.** On the robot page, the robot is described in its own frame (x forward, y up, z to its right), and the world flips the sign of z so that "left" in the mission is left on screen.
>
> **Why it is done this way.** The physics was written with its own natural axes, and the drawing library has others. Instead of bending either, there is one conversion line.
>
> **Why it matters.** The mission model can say "turn left" and the picture turns left.
>
> **What happens if you do not.** Models come out mirrored and turns come out backwards, and people "fix" it by flipping signs in random places until it looks right, and then it breaks again.

From model to screen

A point passes through a series of **coordinate spaces**, each described by a matrix.

1. **Model space.** The point as the object's maker described it, relative to the object's own origin.
2. **World space.** After the **model matrix** places the object in the scene.
3. **Camera (view) space.** After the **view matrix** re-describes the world as seen from the camera, with the camera at the origin looking down the negative z axis.
4. **Clip space and normalised device coordinates.** After the **projection matrix** squeezes the view into a cube where x, y and z each run from −1 to 1. Anything outside the cube is not visible.
5. **Screen space.** After a last scaling to the pixel size of the canvas.

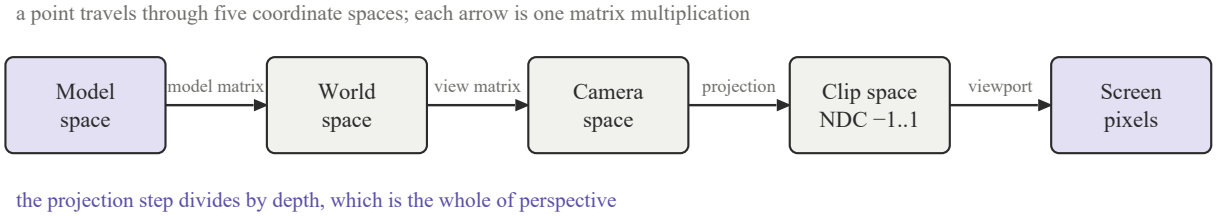


Figure 7.2. The chain of coordinate spaces. Each arrow is a matrix multiplication, and three.js does all of them for you; the perspective division happens in the projection step.

For a **perspective** camera the projection is what makes distant things small. If the vertical field of view is θ and the aspect ratio (width over height) is a, then for a point at camera-space (x, y, z), with z negative in front of the camera:

x_ndc = (f / a) · x / (−z) y_ndc = f · y / (−z) where f = 1 / tan(θ/2)

The division by −z is the whole of perspective: double the distance and the offset from the centre halves. The screen position follows from the canvas width W and height H:

x_px = (x_ndc × 0.5 + 0.5) × W y_px = (−y_ndc × 0.5 + 0.5) × H

The minus in y_px is the flip between the 3D convention (y up) and the canvas convention (y down).

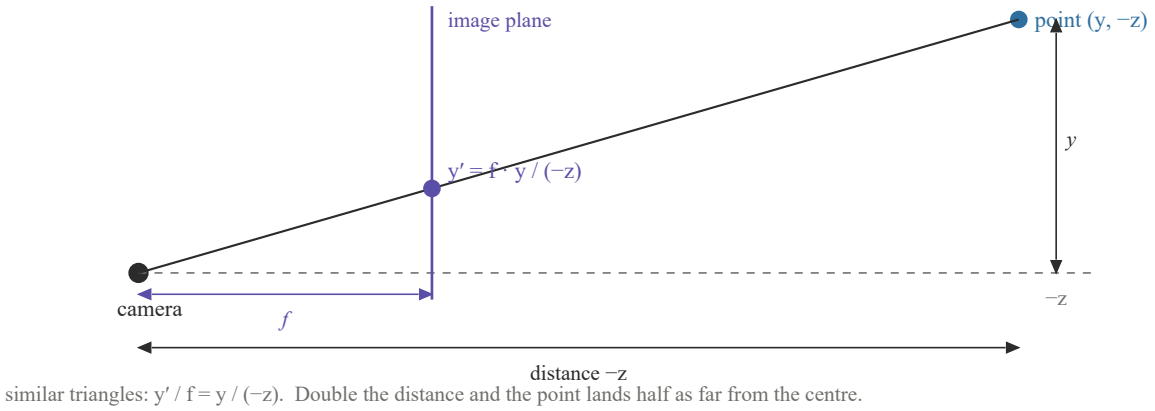


Figure 7.3. Where the perspective formula comes from. The camera sits at the origin and the picture is formed on a plane at distance f. The triangle from the camera to the point and the triangle from the camera to its image are similar, so y′ / f = y / (−z).

Worked Example: Where does a point land on the screen?

A camera has a vertical field of view of 36 degrees, and the canvas is 1000 by 562 pixels. A point lies in front of the camera at camera-space coordinates (0.1, 0.05, −0.5) metres. Find its pixel position.

Step 1. The focal factor. θ/2 = 18°, tan 18° = 0.32492, so f = 1 / 0.32492 = 3.0777.

Step 2. Aspect ratio. a = 1000 / 562 = 1.7794.

Step 3. Normalised coordinates. The distance −z is 0.5. Then x_ndc = (3.0777 / 1.7794) × (0.1 / 0.5) = 1.7296 × 0.2 = 0.3459, and y_ndc = 3.0777 × (0.05 / 0.5) = 0.3078.

Step 4. Pixels. x_px = (0.3459 × 0.5 + 0.5) × 1000 = 0.6730 × 1000 = 673.0. y_px = (−0.3078 × 0.5 + 0.5) × 562 = 0.3461 × 562 = 194.5.

Step 5. Sanity check. The point is to the right of and above the centre of the view, so the pixel should be right of 500 and above (smaller than) 281. It is: (673, 194.5). If the point were twice as far away, at z = −1.0, its normalised offsets would halve and it would land at (586.5, 237.8), nearer the centre.

Depth

The projection also maps depth, so nearer things can hide farther ones. The mapping is **not linear**, so precision is concentrated close to the camera. The depth value runs from −1 at the near plane to +1 at the far plane. With the near plane at 0.05 and the far plane at 9 (the values used by the robot page), it is 0.0056 at 0.1 metres, 0.81 at 0.5 metres, 0.91 at 1 metre and 0.99 at 5 metres. Nine-tenths of the whole range is used up within the first half metre, and the remaining eight and a half metres share the last tenth.

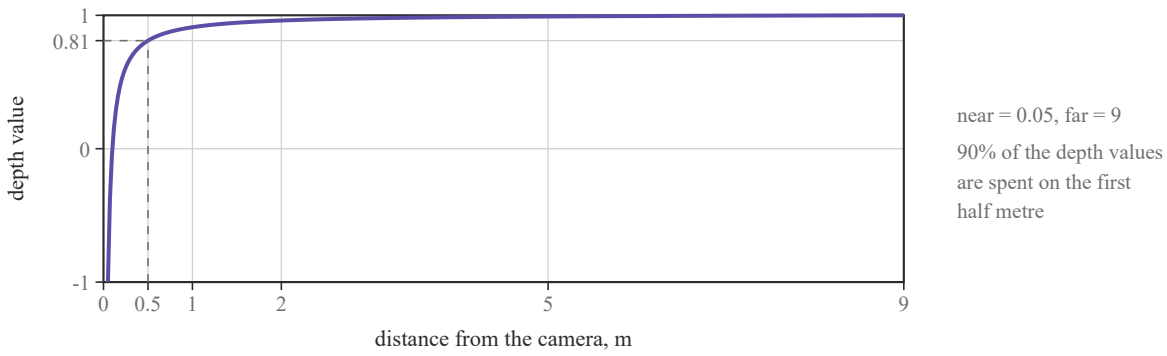


Figure 7.4. Depth value against distance for the robot page's camera. The curve shoots up within the first half metre and is almost flat beyond a metre, which is where two surfaces close together start to receive the same depth value.

> **WHY DO IT THIS WAY? Tight near and far planes**
>
> **What it is.** Setting the camera's `near` and `far` to the smallest and largest distances the scene really needs (0.05 and 9 on the robot page).
>
> **Why it is done this way.** The depth buffer has only so many levels. A tighter range spreads them over the space you actually use.
>
> **Why it matters.** Surfaces that are close together, such as a screw head on a plate, get different depth values and render cleanly.
>
> **What happens if you do not.** With near at 0.001 and far at 10 000, nearby surfaces get the *same* depth value and flicker against each other. This is called z-fighting, and it looks like shimmering static.

7.5 Meshes, Materials and Light

In three.js a visible object is a **mesh**: a **geometry** (the shape, a list of triangles) plus a **material** (how the surface responds to light). The scene is a tree (a robot contains an arm, which contains a claw), so moving a parent moves its children, exactly as the DOM tree moves nested boxes. A **light** lights the scene, a **camera** defines the view, and a **renderer** draws the result onto a canvas.

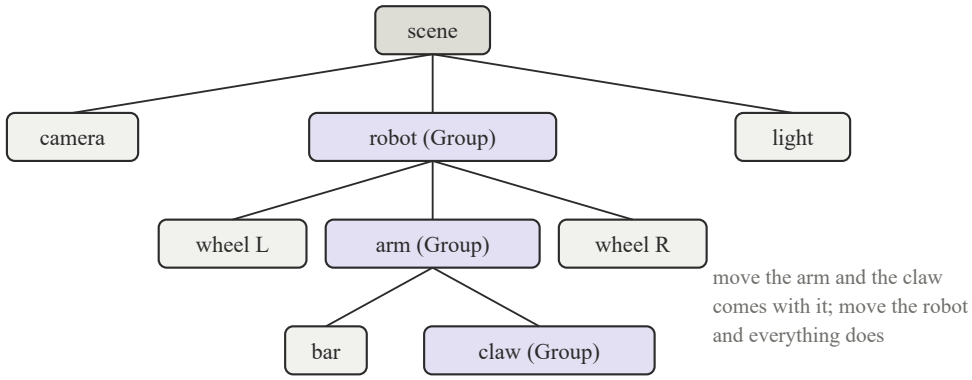


Figure 7.5. A scene graph. Each node's position and rotation are relative to its parent, so swinging the arm group carries the claw with it, and driving the robot group carries everything.

How a surface is lit

The simplest model of light on a matt surface is **Lambert's law**: brightness is proportional to the cosine of the angle between the surface's **normal** (the unit vector sticking straight out of it) and the direction towards the light. For unit vectors n and l the cosine is their dot product, and a surface facing away gets nothing.

brightness = max(0, n · l)

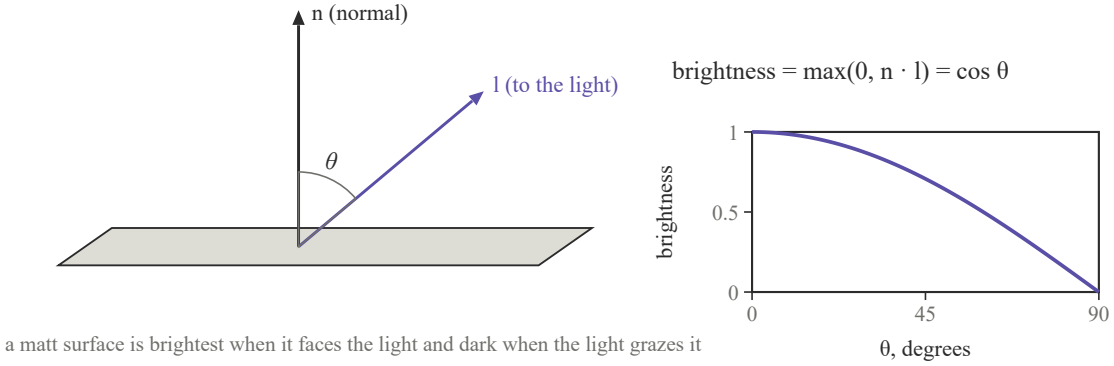


Figure 7.6. Lambert's law. The brightness of a matt surface is the cosine of the angle between its normal and the direction to the light: full when the light is overhead, zero when it grazes the surface, and clamped at zero when the light is behind it.

Worked Example: The brightness of a surface

A flat tabletop has the normal n = (0, 1, 0). A lamp lies in the direction l = (0.6, 0.8, 0) from the surface. Find the brightness, and the brightness if the lamp is moved to the direction (1, 1, 0) after normalising.

Step 1. Check that l is a unit vector. $0.6^2 + 0.8^2 = 0.36 + 0.64 = 1$, so it is already unit length.

Step 2. Dot product. $n \cdot l = 0 \times 0.6 + 1 \times 0.8 + 0 \times 0 = 0.8$, so the brightness is 0.8.

Step 3. The second lamp. The vector (1, 1, 0) has length $\sqrt{2} = 1.4142$, so the unit vector is (0.7071, 0.7071, 0). The dot product is 0.7071, so the brightness is 0.707.

Step 4. Interpret. A lamp directly overhead ($l = (0, 1, 0)$) would give the maximum of 1, and a lamp level with the surface would give 0. The tabletop looks dimmer as the lamp lowers, which is Lambert's law.

Modern renderers use **physically based rendering** (PBR), which describes a material by measurable properties: **base colour**, how **rough** it is (rough scatters light and looks matte, smooth gives a tight bright reflection), and how **metallic** it is. An **environment map**, a panoramic picture of the surroundings, supplies reflections.

> **WHY DO IT THIS WAY? Building a little photography studio to reflect**
>
> **What it is.** The robot page makes a virtual studio (one big soft light overhead, a strip light at the side, a warm fill) and uses it as the environment map.
>
> **Why it is done this way.** Glossy plastic and polished steel look real because of what they *reflect*.
>
> **Why it matters.** With a good studio, a screw head catches a highlight and a plate shows a soft reflection of a softbox. That is what makes it look like a product photograph.
>
> **What happens if you do not.** Glossy surfaces reflect nothing and look like grey putty.

Shadows, occlusion and tone mapping

Three refinements lift a render from "computer graphics" towards "photograph". **Shadow maps** render the scene from the light's point of view to learn which surfaces it can reach. **Ambient occlusion** darkens creases and contact points where light is blocked by nearby geometry. **Tone mapping** squeezes the huge range of brightness in a lit scene (a lamp's reflection may be a hundred times brighter than a shaded wall) into the narrow range a screen can show, with a curve that stops highlights clipping to flat white.

7.6 Keeping a Scene Fast

A GPU can draw millions of triangles, but each separate object costs a **draw call**, a small package of work the main thread must prepare and send. Draw calls are usually the bottleneck. **Merging** meshes that share a material into one geometry turns many draw calls into one. **Instancing** draws one mesh many times with different positions in a single call.

> **WHY DO IT THIS WAY? Merging tiny parts by material**
>
> **What it is.** The robot is made of hundreds of tiny features (screws, pins, chip legs). Those that share a material are merged into one mesh, so they cost one draw call between them.
>
> **Why it is done this way.** Draw calls are expensive and triangles are cheap.

> **Why it matters.** The detailed robot still runs smoothly on a laptop's built-in graphics.
>
> **What happens if you do not.** Hundreds of separate meshes mean hundreds of draw calls, and the frame rate collapses long before the GPU is busy.

What You Learned

- SVG retains shapes as elements (style them, click them, but not thousands); Canvas 2D forgets what it drew (fast, redraw everything); WebGL hands triangles to the GPU.
- The canvas context is a pen with state. Begin a path, draw, stroke. Wrap transforms and clips in `save` and `restore`.
- Map data to pixels with one `X` and one `Y`, and flip the y axis there.
- A 3D point passes through model, world, camera, clip and screen space. Perspective is division by depth: $x_{ndc} = (f / a) \cdot x / (-z)$.
- Depth precision is spent near the camera; keep `near` and `far` tight.
- A surface's brightness is $n \cdot l$; physically based materials add roughness, metalness and an environment to reflect.
- Draw calls, not triangles, usually limit speed. Merge and instance.

Check Your Knowledge

1. What is the difference between retained-mode and immediate-mode graphics? Which is SVG, and which is canvas?
2. In a canvas, which way does the y axis point? What must a chart do to plot values upwards?
3. What is the purpose of `ctx.save()` and `ctx.restore()`?
4. List the coordinate spaces a 3D point passes through, from model to screen.
5. State the perspective projection formulas for x_{ndc} and y_{ndc}, and explain why distant objects look smaller.
6. What is z-fighting and how do you reduce it?
7. State Lambert's law. What does an environment map add to a physically based material?

Review Problems

1. A camera has a vertical field of view of 60 degrees and a 1280 by 720 canvas. A point is at camera-space position $(-0.2, 0.1, -1.0)$. Find its pixel position.
2. For the same camera, how far does a point at $(0.5, 0, -z)$ have to be from the camera for its x pixel position to be within 10 pixels of the centre? (Hint: find the x_{ndc} that corresponds to 10 pixels.)
3. Rotating a point about the y axis by an angle θ in three.js gives $x' = x \cos\theta + z \sin\theta$ and $z' = -x \sin\theta + z \cos\theta$. Where does the point $(1, 0, 0)$ go after a rotation of 90 degrees? Where does the forward direction of a robot (x axis) point after a yaw of 30 degrees?

4. A scene contains 480 meshes. Each costs 0.02 ms of main-thread time to prepare as a draw call. How long do the draw calls take? If merging by material reduces the scene to 48 meshes, how much time is saved per frame, and what is the saving as a fraction of a 60 hertz frame budget?
5. A surface has the normal (0, 0, 1). Compute its brightness under lights from the directions (0, 0, 1), (0, 0.6, 0.8) and (1, 0, 0). Which of these lights leaves it black?
6. With the near plane at 0.1 and the far plane at 100, the normalised depth is given by z_ndc = ((f + n) / (n − f) · z + 2fn / (n − f)) / (−z) for camera-space z, with n the near plane and f the far plane. Compute z_ndc at distances of 1, 10 and 50. What fraction of the depth range is used up by 10 distance units?

Chapter 8. Quality: Access, Speed, Safety and Tools

A page that works on your machine, for your eyes, over your fast connection, is a draft. Quality means it also works for the visitor who cannot see colour well, for the one who cannot use a mouse, on a phone with a weak processor, over a poor connection, and without handing a stranger a way to harm another visitor. This last chapter of Part I covers four parts of quality that every professional page needs, and then the tools that make them practical to keep.

In This Chapter

- Name the four WCAG principles and apply the habits that satisfy most of them: the right element, text alternatives, keyboard use, contrast, motion.
- Compute a contrast ratio from two colours.
- Read the three Core Web Vitals and say which habits move each one.
- Explain cross-site scripting and prevent it with `textContent` and pinned third-party versions.
- Name the four kinds of tool a web developer keeps at hand.

8.1 Accessibility

Accessibility (often shortened to *a11y*, because there are eleven letters between the a and the y) means making a page usable by people with a wide range of abilities: people who are blind and use a screen reader, people with low vision or colour blindness, people who cannot use a mouse, people with hearing loss, people with motor or cognitive difficulties, and people who are only *temporarily* affected, like someone with a broken arm or someone squinting at a phone in sunlight. There is a plain practical case too: features that help the most-affected visitors help everybody, and an accessible page is usually better organised and better ranked by search engines.

The international standard is the **Web Content Accessibility Guidelines (WCAG)**, built on four principles: content must be **perceivable**, **operable**, **understandable** and **robust**.

Use the right element

As Chapter 3 said, native elements come with behaviour assistive technology understands. Use `<button>` for actions, `<a>` for going somewhere, `<nav>`, `<main>` and `<header>` for regions, headings in order so a screen-reader user can skip through the page by its headings, and a `<label>` for every form control.

Text alternatives

An image that carries meaning needs a text alternative in the `alt` attribute. A decorative image gets an empty `alt=""` so it is skipped. A canvas has nothing a screen reader can read, so a canvas that shows a chart should have an `aria-label` that says what it shows.

```
<canvas id="c-speed" role="img"
        aria-label="Speed against time for a full-throttle start, with and without r
      ubber bands"></canvas>
```

> **WHY DO IT THIS WAY? Labelling canvases**

What it is. A `role="img"` and an `aria-label` on every canvas.

Why it is done this way. A canvas is just pixels. To assistive technology it is a black box unless you describe it.

Why it matters. A blind visitor hears "Speed against time for a full-throttle start" and knows what is there, instead of hearing nothing.

What happens if you do not. A third of the content of some pages is invisible to some of your visitors.

Keyboard use

Everything you can do with a mouse must be possible with a keyboard. Native controls handle most of it. For custom things, make them focusable (`tabindex="0"`) and respond to Enter and Space.

WARNING

Never remove the focus outline (`outline: none`) without replacing it with something equally visible. A keyboard user relies on that outline to know where they are. Taking it away is like removing the cursor from a text editor.

Colour and contrast

Text must stand out from its background. WCAG measures this with the **contrast ratio** of the relative luminances L_1 (the lighter colour) and L_2 (the darker):

contrast = $(L_1 + 0.05) / (L_2 + 0.05)$

Relative luminance comes from the red, green and blue parts. Each part c (between 0 and 1) is first made linear: c_lin = c / 12.92 if c ≤ 0.03928, otherwise $((c + 0.055) / 1.055)^{2.4}$. Then **L = 0.2126 R_lin + 0.7152 G_lin + 0.0722 B_lin**, where the weights reflect how much more sensitive the eye is to green. WCAG level AA asks for at least **4.5 : 1** for normal text, and 3 : 1 for large text (about 18 points, or 14 bold) and for the outlines of controls.

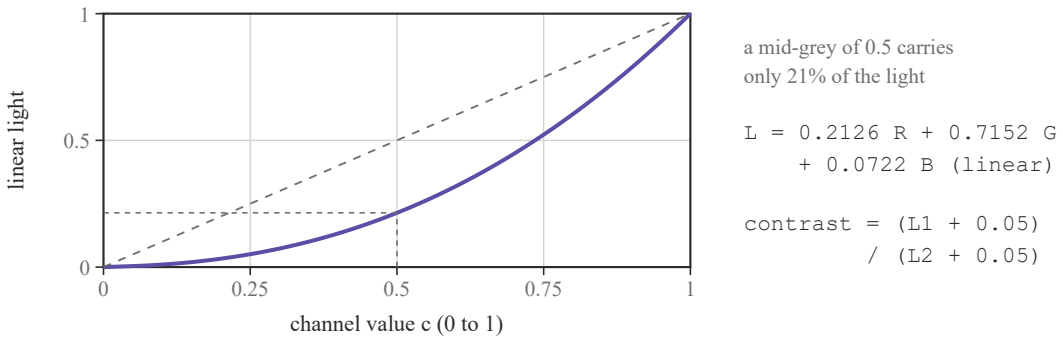


Figure 8.1. Why the formula starts by "making linear". Screens store colour on a curve that matches the eye, so a channel value of 0.5 emits only about a fifth of the light of 1.0. The luminance formula undoes that curve before it adds the channels up.

Worked Example: Is the secondary text readable?

The pages of this website use the page background #070a1a and three text colours: `--text-muted` #9aa0c3, `--accent` #a78bfa and `--text-faint` #5f668f. Compute the contrast ratio of each against the background and say which meet AA for normal text.

Step 1. Luminance of the background. #070a1a has R = 7, G = 10, B = 26. Dividing by 255 gives 0.0275, 0.0392 and 0.1020. The first two are at most 0.03928, so they are divided by 12.92 to give 0.00213 and 0.00303; the third is converted by $((0.1020 + 0.055) / 1.055)^{2.4} = 0.01033$. Then L = 0.2126 × 0.00213 + 0.7152 × 0.00303 + 0.0722 × 0.01033 = 0.00045 + 0.00217 + 0.00075 = 0.00337.

Step 2. Luminance of the text colours (computed the same way): #9aa0c3 gives 0.3595, #a78bfa gives 0.3358 and #5f668f gives 0.1392.

Step 3. Ratios. For #9aa0c3: (0.3595 + 0.05) / (0.00337 + 0.05) = 0.4095 / 0.05337 = 7.67. For #a78bfa: 0.3858 / 0.05337 = 7.23. For #5f668f: 0.1892 / 0.05337 = 3.54.

Step 4. Verdict. The muted and accent colours pass AA comfortably (7.67 and 7.23 against 4.5). The faint colour does not for normal text: 3.54 is below 4.5, although it passes the 3 : 1 limit for large text and for decorative outlines.

What to do. Keep the faint colour for non-essential detail such as axis ticks and decoration, never for text a visitor must read, or lighten it until it clears 4.5. This is a real finding about this website, and it is the sort of thing only measurement reveals: to the eye of a designer looking at a good monitor in a dim room, #5f668f looks perfectly acceptable.

> **WHY DO IT THIS WAY? Measuring contrast instead of judging by eye**
>
> **What it is.** Computing the ratio for every text colour, rather than deciding "that looks fine".
>
> **Why it is done this way.** A designer on a good monitor in a dim room sees things that a visitor on a cheap phone in sunlight cannot.
>
> **Why it matters.** It found a real problem on this very website: the faint grey used for secondary text scores only 3.54, below the 4.5 limit.
>
> **What happens if you do not.** Text that is readable to you is unreadable to people with low vision, older eyes, or bright light, and you never find out.

Worked Example: Fixing the faint grey

The faint text colour `#5f668f` scores 3.54 against the background `#070a1a`. Find a replacement in the same family that passes 4.5 : 1, changing it as little as possible.

Step 1. What the ratio needs. The background's luminance is 0.0034, so the ratio is (L + 0.05) / 0.0534. For 4.5 we need L + 0.05 ≥ 0.240, so L ≥ 0.190. The current colour has L = 0.139.

Step 2. Lighten along the same hue. Add the same amount to each channel so the colour keeps its blue-grey character. Try `#7c84a8`: the channels are 124, 132, 168. Linearised: 0.201, 0.230, 0.391. L = 0.2126 × 0.201 + 0.7152 × 0.230 + 0.0722 × 0.391 = 0.0427 + 0.1645 + 0.0282 = 0.235.

Step 3. The ratio. (0.235 + 0.05) / 0.0534 = 5.36. It passes, with room to spare.

Step 4. Could it be subtler? `#7279a0` sits between the two; the same arithmetic gives 4.65, which also passes, only just. The point of the method is that you do not have to guess: each candidate gives a number, and the first one over 4.5 is the answer. On this website the faint colour is used for hints and decorations, which is why the site's authors left it as it is for decoration and would raise it for any text a visitor must read.

Motion and testing

As Chapter 6 explained, honour `prefers-reduced-motion`, and avoid anything that flashes more than three times a second. Automated checkers (Lighthouse, axe) find perhaps a third of problems. The rest you find by trying: put the mouse aside and use the Tab key; turn on a screen reader and listen; zoom to 200%; look at the page on a small phone.

8.2 Performance

Performance is how fast a page appears and how smoothly it responds. Chapter 1 gave the arithmetic of loading, and Chapter 2 the budget of a frame. Google's **Core Web Vitals** sum up the experience in three numbers.

Metric	Measures	Good
LCP, largest contentful paint	How long until the biggest visible element appears	2.5 s or less
INP, interaction to next paint	How long the page takes to respond to a click or key	200 ms or less
CLS, cumulative layout shift	How much the layout jumps around while loading	0.1 or less

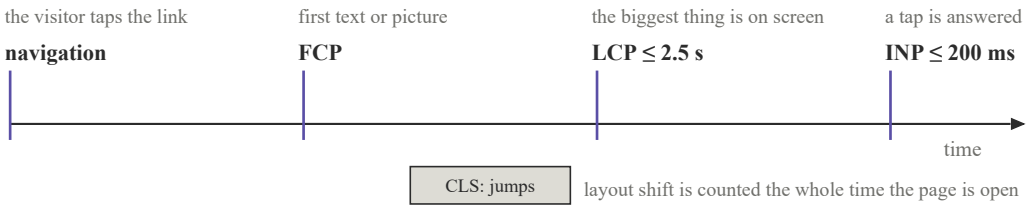


Figure 8.2. The three vitals on the timeline of one visit. LCP is a moment, INP is a delay measured at every tap, and CLS adds up every jump for as long as the page is open.

Most gains come from a short list of habits. Send less: compress text, never send an image bigger than it will be shown. Send it sooner: put critical content first and `defer` scripts. Draw less: avoid animating properties that force layout, and merge and instance in 3D. And degrade gracefully: the robot page watches its own frame rate and steps down its rendering quality when a device cannot keep up.

> **WHY DO IT THIS WAY? Reserving space before things arrive**
>
> **What it is.** Giving images a `width` and `height`, and giving every chart canvas a fixed height in CSS.
>
> **Why it is done this way.** The browser then knows how much room to leave before the content shows up.
>
> **Why it matters.** When a late image or chart pops in and pushes the text down, a visitor may tap the wrong thing. That is layout shift (CLS).
>
> **What happens if you do not.** The page jumps around as it loads, and people lose their place.

8.3 Security

A static page has little to attack, but any page that handles text it did not write must be careful. The main danger is **cross-site scripting (XSS)**: an attacker gets their own JavaScript to run inside your page, where it has all your page's powers. The usual cause is building HTML out of untrusted text.

```
// dangerous: if name is "<img src=x onerror=steal()>", that script runs
box.innerHTML = 'Hello, ' + name;

// safe: the text is only ever treated as text
box.textContent = 'Hello, ' + name;
```

> **WHY DO IT THIS WAY? textContent instead of innerHTML**
>
> **What it is.** `textContent` puts a string in the page as plain text. `innerHTML` parses the string as HTML.
>
> **Why it is done this way.** If the string came from outside your code (a URL parameter, a form field, a file), you cannot know what it contains.
>
> **Why it matters.** With `textContent`, even a malicious string is only displayed. It cannot run.
>
> **What happens if you do not.** Anyone who can get a crafted value into your page can run code as your page.

The same care applies to **third-party scripts**. Every script you load from another site runs with full power over your page. Load only from sources you trust, and pin the exact version.

> **WHY DO IT THIS WAY? Pinning library versions**
>
> **What it is.** Writing `katex@0.16.11` in a URL, not `katex@latest`.
>
> **Why it is done this way.** A pinned URL always returns the same file.
>
> **Why it matters.** A broken or compromised new release cannot reach your page silently, and your site behaves the same next year as today.
>
> **What happens if you do not.** One morning your page breaks, and nothing in your own files has changed.

Finally, serve pages over **HTTPS**, which encrypts the conversation from Chapter 1 and stops anyone on the route from reading or altering what the visitor receives.

8.4 Tools

The habits above are easier with tools, and you will meet four kinds throughout this book.

The browser's developer tools come first: Elements to inspect the tree and styles, Console for errors and experiments, Network for requests, Performance for the frame budget, Lighthouse for automatic audits. Learn them before you learn any library.

A package manager, npm, installs libraries and records which versions a project uses in `package.json`.

A bundler reads a script, follows its `import` statements, and combines everything into one file the browser can load. The robot page is built from several source files plus three.js into one script by a bundler called esbuild, in a fraction of a second. Bundlers can also **minify** the result, stripping whitespace and shortening names, which is why the file in Chapter 1's worked example is 686 kilobytes and not several megabytes.

Version control with Git records every change to every file, lets you return to any earlier state, and makes experimenting safe. A **commit** is a saved snapshot with a message saying why you made the change.

> **WHY DO IT THIS WAY? Writing commit messages that say why**
>
> **What it is.** A short message with each commit explaining the reason for the change.
>
> **Why it is done this way.** The code already shows *what* changed. Only you know *why*.
>
> **Why it matters.** In six months, "Fix the camera jump when the tab is hidden" is a gift to your future self. "update stuff" is a puzzle.
>
> **What happens if you do not.** You cannot tell which of fifty commits introduced a bug.

What You Learned

- Accessibility is perceivable, operable, understandable, robust. Most of it comes free from the right element, a text alternative, keyboard support, enough contrast and respect for motion preferences.
- Contrast = $(L_1 + 0.05) / (L_2 + 0.05)$, with luminance computed from linearised channels. Normal text needs 4.5 : 1.
- LCP (how soon the big thing appears), INP (how fast a tap is answered) and CLS (how much the page jumps) sum up performance. Send less, send it sooner, draw less, degrade gracefully.
- Never build HTML from text you did not write. `textContent`, not `innerHTML`. Pin third-party versions. Serve over HTTPS.
- Developer tools, a package manager, a bundler and version control are the workshop. Commit messages say why.

Check Your Knowledge

1. Why are native elements such as `<button>` better for accessibility than styled `<div>` elements?
2. What does an `alt` attribute do, and what should the `alt` of a decorative image be?
3. State the WCAG AA contrast requirement for normal text.
4. Name the three Core Web Vitals and say what each measures.
5. What is cross-site scripting, and why is `textContent` safer than `innerHTML` for text from a user?
6. Why does pinning a library to an exact version matter for security and reliability?
7. What does a bundler do, and what does minification add?

Review Problems

1. Compute the contrast ratio of white text (#ffffff, luminance 1.0) on the page background (luminance 0.00337). Does it pass AA? Compute also mid-grey text #777777 (luminance 0.1845) on white, and say whether it passes.
2. A designer wants text of colour #5f668f (luminance 0.1392) on a white card (luminance 1.0). Compute the ratio and decide whether it passes AA for normal text.
3. A visitor sets their browser's default font size to 24 px. A heading is sized `2rem` and a caption is sized `12px`. What sizes do they show? Which respects the visitor's setting?
4. A page's largest image arrives after 3.4 s and the page reaches interactivity after 1.1 s. Which Core Web Vital is failing, by how much, and suggest two fixes.
5. The following code builds a greeting from a URL parameter. Explain the vulnerability, give a malicious input, and rewrite the code to be safe.

```
const name = new URLSearchParams(location.search).get('name');
document.getElementById('hello').innerHTML = '<b>Hello, ' + name + '</b>';
```

6. An image is displayed 400 CSS pixels wide on screens with DPR 1, 2 and 3. If you may offer only one file, what width should it be so that it is sharp on all three? What is the cost on the DPR 1 screen, expressed as the ratio of pixels downloaded to pixels needed?

Part II. The Tech Stack

One tool at a time: the words, the theory, worked examples and practice

Chapter 9. HTML: The Skeleton

HTML is the easiest of the three languages to learn and the easiest to do badly. It is easy because there is hardly any logic, only labels. It is easy to do badly because the browser forgives almost anything, so nobody tells you when you are wrong. This chapter teaches you to write HTML that is correct on purpose: a page that reads well as plain text, that a screen reader can navigate, that a search engine can understand, and that CSS and JavaScript can hook into without a fight.

In This Chapter

- Write the skeleton of a page from memory and explain every line of it.
- Predict how the parser treats nesting, whitespace, void elements, entities and mistakes.
- Mark up text, lists, quotations, code, tables, links, images, media and forms with the elements that carry their meaning.
- Organise a page with landmarks and a sensible heading outline.
- Fill the `<head>` so that the page loads fast, looks right when shared, and is found by search.
- Add accessibility where native elements are not enough, and know when ARIA is the wrong tool.
- Validate a page and find mistakes with the Elements panel.

Words You Will Meet

Word	What it means
Element	One building block: a start tag, content and an end tag, such as `<p>Hello</p>`.
Tag	The part in angle brackets that names an element.
Attribute	A name and value inside a start tag, such as `href="page.html"`.
Boolean attribute	An attribute whose presence is the whole message, such as `required` or `disabled`.
Void element	An element with no content and no end tag, such as `<img>` and `<meta>`.
Nesting	Putting one element inside another. Elements must be closed in the reverse order they were opened.
Content model	The rule for what an element may contain. A `<p>` may hold text and inline elements, not another `<p>`.
Semantic	Chosen for its *meaning* (`<nav>`) rather than its appearance.
Landmark	A semantic element marking a region (`<header>`, `<main>`, `<footer>`).
Entity	A code for a special character, such as `&` for `&` and `<` for <.
Document	The whole page, from `<!DOCTYPE html>` to `</html>`.
Metadata	Information *about* the page, kept in the `<head>`: its title, character set, description, icon.
ARIA	Attributes that describe roles and states to assistive technology when no native element does.

9.1 The Skeleton of Every Page

Every HTML page starts the same way. Here is the smallest page you should ever write.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Page</title>
</head>
<body>
    <h1>Hello</h1>
</body>
</html>
```

Let us take it line by line, because every line is there for a reason.

> **WHY DO IT THIS WAY? The doctype**

What it is. `<!DOCTYPE html>` on the very first line.

Why it is done this way. It tells the browser "this page uses modern HTML".

Why it matters. It switches the browser into **standards mode**, where layout follows the modern rules.

What happens if you do not. The browser falls back to **quirks mode**, which imitates 1990s browsers. Boxes are sized differently and nothing quite behaves as this book says. If a page looks mysteriously off, a missing doctype is a classic suspect.

WHY DO IT THIS WAY? The lang attribute

What it is. `lang="en"` on the `<html>` element.

Why it is done this way. It states what human language the page is in.

Why it matters. Screen readers use it to pick the right pronunciation, translation tools use it, and so does hyphenation.

What happens if you do not. A screen reader may read English with the accent and rules of whatever language the visitor's device is set to, and it sounds like nonsense.

WHY DO IT THIS WAY? The charset line

What it is. `<meta charset="UTF-8">` in the head.

Why it is done this way. Files are just bytes. The browser needs to be told which table to use to turn bytes into letters. UTF-8 covers every language and symbol.

Why it matters. It keeps characters like é, ×, θ and → correct.

What happens if you do not. Symbols turn into garbage such as "Â°" and "â†'". Put it first in the head, so the browser knows before it reads anything else.

WHY DO IT THIS WAY? The title element

What it is. `<title>` in the head, which does not appear on the page itself.

Why it is done this way. It names the page for the browser tab, bookmarks, history and search results.

Why it matters. It is the first thing a screen-reader user hears and what a search engine shows as the link.

What happens if you do not. The tab shows the file name, and a visitor with twenty tabs open cannot find your page.

(The viewport line was explained in Chapter 4.)

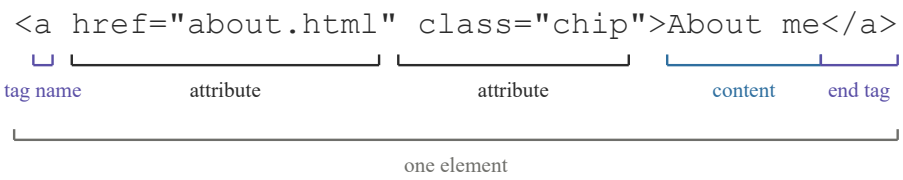


Figure 9.1. The parts of one element. The start tag carries the name and the attributes; the content sits between the tags; the end tag closes it. Everything from `<a` to `</a>` is one element, and one node in the tree.

9.2 How the Parser Thinks

To write HTML that behaves, you need to know what the browser does with your text. Six rules cover almost everything.

1. Tags nest, and close in reverse order. `<p><em>robot</em></p>` is right. `<p><em>robot</p></em>` is wrong: the parser closes `em` for you when it meets `</p>`, then finds a stray `</em>` and throws it away. You get the layout you meant by luck, and the next mistake will not be so lucky.

2. Some elements have no content and no end tag. These are the **void elements**: `<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`, `<source>`, `<track>`, `<wbr>`, `<area>`, `<base>`, `<col>`, `<embed>`. Writing `<br></br>` or `<img ...></img>` is an error that the parser silently repairs.

3. Every element has a content model. A `<p>` may hold text and *phrasing* elements (`<em>`, `<a>`, `<code>`). It may not hold a `<div>`, a list or another `<p>`. If you write `<p><ul>...</ul></p>`, the parser closes the paragraph *before* the list, and the `</p>` at the end becomes an empty paragraph. The Elements panel shows the truth; the source file does not.

4. Whitespace collapses. Any run of spaces, tabs and newlines between words becomes one space, so you may indent freely. The one exception is `<pre>`, which keeps every space and line break, and is how code listings are shown.

5. Five characters are special. `<` starts a tag and `&` starts an entity, so to *show* them you must write `<` and `&`. Inside a double-quoted attribute, `"` must be `"`. Other entities are conveniences: ` ` is a space that will not break a line, `©` is ©, `×` is ×, `→` is →, `°` is °. With UTF-8 declared you can also just type the character.

6. Attributes are name-value pairs, and some are switches. `href="page.html"` has a value. `required`, `disabled`, `checked`, `hidden` and `defer` are **boolean**: their presence is the whole message, and `required="false"` still means required. Always quote values; `class="card wide"` without quotes would be read as a class of `card` and an attribute called `wide`.

Worked Example: Reading what the parser will build

A beginner writes this. Predict the tree the browser builds, then fix the source.

```
<p>Parts: <ul><li>motor<li>wheel</ul>
<img src="a.png">
<p>Done
```

Step 1. The first paragraph. `<p>` opens. The parser meets `<ul>`, which a paragraph cannot contain, so it closes the paragraph: `<p>Parts: </p>`. The list begins.

Step 2. The list items. `<li>motor` opens an item. The next `<li>` closes the previous one automatically, because an item cannot contain another item. `</ul>` closes the open item and the list. So far the tree is a paragraph, then a list with two items. (Omitting `</li>` is actually allowed by the rules, but writing it makes the structure visible.)

Step 3. The image. `<img>` is void; it becomes a node on its own, as a sibling of the paragraph and the list.

Step 4. The last paragraph. `<p>Done` opens and is never closed. The parser closes it when the body ends. It works, and it is sloppy.

Step 5. The fix.

```
<p>Parts:</p>
<ul>
  <li>motor</li>
  <li>wheel</li>
</ul>
<img src="a.png" alt="The assembled chassis" width="400" height="300">
<p>Done</p>
```

Why it matters. Both versions draw almost the same page today. The second says what it means, so that a stylesheet rule like `p + ul` or a script that counts paragraphs does what you expect.

> **TIP**
>
> The Elements panel of the developer tools shows the tree the browser *actually built*, after all its repairs. When the layout is strange, compare it to your source. The difference is the bug.

9.3 Text and Structure

Headings go from `<h1>` (the most important) to `<h6>`. Use them in order, and use one `<h1>` per page. Paragraphs are `<p>`. Lists are `<ul>` (bullets) or `<ol>` (numbers), with each item in an `<li>`, and `<dl>` for a list of terms and definitions (`<dt>` for the term, `<dd>` for its definition), which is exactly what a glossary or a spec sheet is.

```
<dl>
  <dt>Wheelbase</dt><dd>112 mm</dd>
  <dt>Motor</dt><dd>TT gear motor, 1:48</dd>
</dl>
```

> **WHY DO IT THIS WAY? Headings in order**
>
> **What it is.** `<h1>`, then `<h2>` for its sections, then `<h3>` for theirs, never skipping a level.
>
> **Why it is done this way.** Headings form an outline, like a book's contents page.

> **Why it matters.** Screen-reader users jump from heading to heading to scan a page, the way you glance down a page for bold titles.
>
> **What happens if you do not.** Choosing `<h4>` because it "looks the right size" makes the outline jump about, and navigating the page becomes confusing. Choose the *meaning* with HTML, and choose the *size* with CSS.

Inline meaning

Inside a paragraph, a handful of elements say what a run of text *is*. Learn these; they are the difference between markup and typing.

Element	Meaning	Default look
`<em>`	Stress, as in speech: "it is *not* a toy"	italic
`<strong>`	Importance or urgency: "**Warning:** hot"	bold
`<code>`	A fragment of code or a file name	monospace
`<kbd>`	A key or command the reader types: `<kbd>Ctrl</kbd>`	monospace
`<abbr title="...">`	An abbreviation, with the full form in `title`	dotted underline
`<time datetime="2026-11-10">`	A date or time, machine-readable in `datetime`	plain
`<mark>`	Highlighted for relevance, like a search hit	yellow background
`<q>` and `<cite>`	A short quotation, and the title of a work	quotes; italic
`<sup>`, `<sub>`	Superscript and subscript: m², CO₂	raised; lowered
`<small>`	Side comment or small print	smaller
`<span>`	No meaning at all: a hook for CSS or script	none

Use `<em>` and `<strong>`, *not* `<i>` and `<b>`, when you mean stress and importance. (`<i>` and `<b>` still exist for things that are conventionally italic or bold without emphasis, such as a ship's name or a keyword in a summary, but you will rarely need them.) A `<blockquote>` holds a quoted paragraph; a `<pre>` keeps whitespace; `<hr>` marks a change of topic; `<br>` breaks a line *inside* a paragraph, as in an address, and should never be used to make vertical space.

> **WHY DO IT THIS WAY? Marking up code with code, and keys with kbd**
>
> **What it is.** `Press <kbd>F12</kbd> and type <code>document.title</code>.`
>
> **Why it is done this way.** Each element says what the text is, and the stylesheet gives each its own look.

Why it matters. A reader can tell what to type from what to press. A script could find every code fragment on the page. The monospace font is a *consequence*.

What happens if you do not. You write `<span class="mono">` everywhere, the meaning is lost, and the next person cannot tell which spans are code and which are just styled.

9.4 Landmarks and Sectioning

HTML has elements whose names carry meaning for whole regions: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` and `<footer>`. Assistive technology calls the main ones **landmarks** and lets a visitor jump between them.

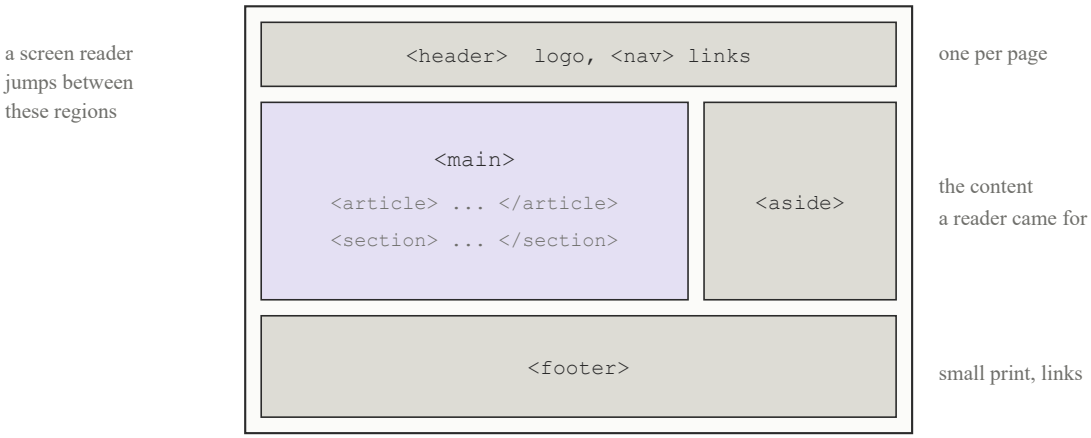


Figure 9.2. The landmarks of a typical page. A screen-reader user can jump straight to `<main>`, or list the navigation regions, the way a sighted visitor's eye finds the menu and the content without reading everything in between.

- `<header>` is the introduction of the page (or of an article): the logo, the title, the navigation.
- `<nav>` holds a set of links for getting around. A page may have more than one (the main menu, a table of contents), and a `aria-label` tells them apart.
- `<main>` holds the content the visitor came for. Exactly one per page.
- `<article>` is a self-contained piece that would make sense on its own: a blog post, a product, one project in a portfolio.
- `<section>` is a thematic grouping with a heading. If you cannot think of a heading for it, it is probably a `<div>`.
- `<aside>` is tangential content: a side note, related links.
- `<footer>` holds the small print, contact details, copyright.
- `<figure>` with `<figcaption>` wraps an image, a chart, a code listing or a table together with its caption, so the two cannot be separated.

WHY DO IT THIS WAY? Using a landmark instead of a div with a class

What it is. `<nav>` rather than `<div class="nav">`.

Why it is done this way. The browser and assistive technology know what `<nav>` is. A class is a private word that only your stylesheet understands.

Why it matters. A blind visitor can press one key to reach the navigation. A search engine can tell the menu from the content. And the element still works if your CSS never loads.

What happens if you do not. The page is a pile of anonymous boxes. It looks the same to a sighted mouse user and is a maze to everyone else.

REMEMBER

`<div>` and `<span>` are not bad. They are the elements for "a box I need for layout" and "a run of text I need to style", when no meaningful element fits. The mistake is reaching for them *first*.

9.5 Links, Images and Media

Links

```
<a href="about.html">About me</a>
<a href="../index.html">Home</a>
<a href="#routh">Jump to the Routh section</a>
<a href="https://example.com" target="_blank" rel="noopener">Another site</a>
<a href="report.pdf" download>Download the report</a>
<a href="mailto:you@example.com">Email me</a>
```

A link's `href` is a URL (Chapter 1), and most of yours will be **relative**: resolved from the folder of the current page. `about.html` is in the same folder; `../index.html` is one folder up; `assets/a.png` is in a subfolder; `/images/a.png` with a leading slash starts from the site's root. A `#fragment` scrolls to the element with that `id`, and works with no script. `download` asks the browser to save the file instead of showing it. `target="_blank"` opens a new tab, and should always come with `rel="noopener"`.

Worked Example: Resolving relative paths

The page `/pages/robots/patrick.html` needs to link to `/pages/index.html`, show the image `/assets/patrick.jpg`, and link to the section with `id="arm"` on itself. Write the three `href` and `src` values.

Step 1. Where am I? The page's folder is `/pages/robots/`. Every relative path starts there.

Step 2. The index. `/pages/index.html` is one folder up: `../index.html`.

Step 3. The image. `/assets/patrick.jpg` is two folders up and then down into `assets`: `../../assets/patrick.jpg`. A root-relative `/assets/patrick.jpg` also works, but only when the site is served from a root; opened from disk as a file, the leading slash means the drive's root, and the image breaks.

Step 4. The section. `#arm`. No file name is needed for a fragment on the same page.

Why it matters. This website is opened from disk during development and served from a sub-folder on its host, so its pages use relative paths throughout, and they work in both places.

> **WHY DO IT THIS WAY? rel="noopener" on links that open a new tab**
>
> **What it is.** `rel="noopener"` next to `target="_blank"`.
>
> **Why it is done this way.** Without it, the new page can reach back and control the page that opened it.
>
> **Why it matters.** It closes a small but real security hole.
>
> **What happens if you do not.** A page you link to could redirect your page to something nasty, and your visitor would not notice. (Modern browsers now add this behaviour by default, but writing it is still good manners and works in older ones.)

Images

```
<img src="robot.jpg" alt="A yellow two-wheeled robot with a claw" width="640" height
      ="480">
```

> **WHY DO IT THIS WAY? alt, width and height on images**
>
> **What it is.** A text description (`alt`) and the real pixel size (`width` and `height`) on every `<img>`.
>
> **Why it is done this way.** The `alt` describes the picture to someone who cannot see it. The size lets the browser reserve space before the image arrives.
>
> **Why it matters.** Blind visitors get the meaning of the picture. And the page does not jump around when the image loads (Chapter 8's layout shift).
>
> **What happens if you do not.** Screen readers read out the file name ("IMG underscore 4032"), and the text jumps down when the picture appears.

Writing good `alt` text is a skill. Describe what the picture *means* in this place, not what it contains: `alt="The claw closing on an egg"` on a page about the gripper, not `alt="robot"`. A purely decorative image gets `alt=""` (empty, but present) so that it is skipped. An image that is also a link needs `alt` text that says where the link goes.

Two more attributes matter on a page with many pictures. `loading="lazy"` tells the browser not to fetch an image until it is near the screen, which is right for everything below the first screenful and wrong for the hero image. `srcset` and `sizes` offer the browser several copies of the same picture so it can fetch the one that fits.

```
<img src="wing-800.jpg"
     srcset="wing-400.jpg 400w, wing-800.jpg 800w, wing-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw, 600px"
     alt="The finished wing on the bench" width="800" height="500" loading="lazy">
```

Worked Example: Which copy of the image is fetched?

With the `srcset` above, which file does a phone 375 CSS pixels wide with DPR 2 fetch? And a desktop browser 1 400 pixels wide with DPR 1?

Step 1. Read sizes. On a viewport up to 600 pixels wide the image is shown at `100vw`, the full width. Wider than that, it is shown at 600 CSS pixels.

Step 2. The phone. 375 CSS pixels at DPR 2 means 750 device pixels are needed. The browser picks the smallest candidate at least that wide: `wing-800.jpg`.

Step 3. The desktop. 600 CSS pixels at DPR 1 means 600 device pixels. Again `wing-800.jpg` is the smallest that will do.

Step 4. A sharper desktop. The same 1 400 pixel window on a DPR 2 laptop needs 1 200 device pixels, so it fetches `wing-1600.jpg`.

Why it matters. The phone never downloads the 1 600 pixel file, which might be ten times the size of the one it needs. The `src` is the fallback for browsers that do not understand `srcset`.

The `<picture>` element goes one step further and lets you offer *different* pictures (a cropped version for phones, a modern format with a fallback):

```
<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
  <img src="hero.jpg" alt="..." width="1200" height="600">
</picture>
```

The browser takes the first `<source>` it can decode, and the `<img>` inside is both the fallback and the place the `alt` lives.

Figures, video and audio

```
<figure>
  <img src="bode.png" alt="Bode plot of the pitch loop" width="800" height="500">
  <figcaption>Figure 3. Gain and phase of the pitch loop; the phase margin is 61 deg
      rees.</figcaption>
</figure>

<video controls width="640" height="360" poster="run.jpg" preload="metadata">
  <source src="run.mp4" type="video/mp4">
  <track kind="captions" src="run.vtt" srclang="en" label="English">
  Your browser cannot play this video. <a href="run.mp4">Download it.</a>
</video>
```

`controls` shows the browser's own play bar. `poster` is the still shown before play. `preload="metadata"` fetches only the length and first frame, not the whole file. A `<track>` adds captions, which are not optional for a video with speech. `autoplay` works only with `muted`, and even then is bad manners on a page the visitor did not come to for a video.

9.6 Tables

A table is for **data with rows and columns**, never for layout. Its parts give the data meaning: a `<caption>` says what the table is, `<thead>` and `<tbody>` separate headings from data, and `<th>` marks a heading cell, with `scope` saying whether it heads a column or a row.

```
<table>
  <caption>Motor readings at three duty cycles</caption>
  <thead>
    <tr><th scope="col">Duty</th><th scope="col">Speed (rpm)</th><th scope="col">Cur
      rent (A)</th></tr>
  </thead>
  <tbody>
```

```
    <tr><th scope="row">50%</th><td>96</td><td>0.14</td></tr>
    <tr><th scope="row">70%</th><td>138</td><td>0.19</td></tr>
    <tr><th scope="row">100%</th><td>199</td><td>0.26</td></tr>
  </tbody>
</table>
```

> **WHY DO IT THIS WAY? th with scope instead of bold td**
>
> **What it is.** Heading cells are `<th>` with `scope="col"` or `scope="row"`.
>
> **Why it is done this way.** The browser then knows which heading describes each data cell.
>
> **Why it matters.** A screen reader can say "Speed, 138 rpm" when the visitor reaches a cell, instead of just "138". A sighted reader gets the same from the position; a blind reader gets it only from the markup.
>
> **What happens if you do not.** A table of numbers is read aloud as a stream of numbers with no idea what each one is.

9.7 Forms

A form collects input. Each control should have a **label**, and the label and control are connected by matching `for` and `id` (or by wrapping the control inside the label).

```
<form action="/subscribe" method="post">
    <label for="email">Your email</label>
    <input id="email" name="email" type="email" required autocomplete="email">
    <button type="submit">Subscribe</button>
</form>
```

> **WHY DO IT THIS WAY? Connecting a label to its input**
>
> **What it is.** `<label for="email">` paired with `<input id="email">`.
>
> **Why it is done this way.** The pair tells the browser which text belongs to which box.
>
> **Why it matters.** Clicking the label focuses the box, which makes a bigger target on a phone. A screen reader announces "Your email" when the box is focused.
>
> **What happens if you do not.** The visitor hears "edit text, blank" and has no idea what to type. The `type="email"` and `required` do some checking for free, and show the right keyboard on a phone.

The controls

Control	What it gives you
`<input type="text">`	One line of text. Also `search`, `tel`, `url`, `password`.
`type="email"`, `type="number"`, `type="date"`	Text with checking, and the right keyboard or picker on a phone.
`type="range" min max step`	A slider; its `value` is a string, so convert with + in script.
`type="checkbox"`, `type="radio"`	Yes/no, and one-of-many (radios share a `name`).
`type="file"`	A file picker; the form then needs `enctype="multipart/form-data"`.
`type="hidden"`	A value sent with the form but not shown.
`<select>` with `<option>`	A drop-down list.
`<textarea rows="4">`	Several lines of text.
`<button type="submit">`, `type="button"`, `type="reset"`	Submits the form; does nothing by itself (for scripts); clears it.
`<fieldset>` with `<legend>`	A group of related controls with a caption, such as a set of radios.

Checking attributes save you script: `required`, `min`, `max`, `step`, `minlength`, `maxlength`, `pattern="[0-9]{4}"`. The browser refuses to submit until they are satisfied and shows its own messages. `autocomplete` tells the browser what the field is for (`name`, `email`, `username`, `current-password`, `one-time-code`) so it can fill it in.

> **WHY DO IT THIS WAY? A button's default type is submit**
>
> **What it is.** `<button>` inside a form submits the form unless you write `type="button"`.
>
> **Why it is done this way.** The common case, one button that sends the form, needs no attribute.
>
> **Why it matters.** A "Clear" or "Preview" button inside a form that forgets `type="button"` submits the form and reloads the page, and the visitor loses what they typed.
>
> **What happens if you do not.** One of the most common form bugs. Give every button in a form an explicit `type`.

What a form actually sends

When the form is submitted, the browser collects every control that has a `name` and sends `name=value` pairs. With `method="get"` they go in the URL's query string: `/search?q=egg&sort=new`. With `method="post"` they go in the request body, encoded the same way, and do not appear in the address bar or the history. Use `get` for searches and filters that a visitor might bookmark; use `post` for anything that changes something or carries private data.

Worked Example: What arrives at the server

A form with `method="get"` and `action="/find"` has a text field `name="q"` containing `egg & salt`, a checked checkbox `name="fresh" value="1"`, and an unchecked checkbox

`name="frozen" value="1"`. What URL does the browser request?

Step 1. Which controls are sent? Every named control with a value, except unchecked checkboxes and radios, which are left out entirely.

Step 2. Encode the values. Spaces become `+` (or `%20`) and `&` becomes `%26`, because a bare `&` would separate two pairs. `egg & salt` becomes `egg+%26+salt`.

Step 3. Assemble. `/find?q=egg+%26+salt&fresh=1`. The `frozen` checkbox does not appear at all.

Why it matters. When a server "does not receive" a checkbox, it is because it was unchecked, not because of a bug. And every `%26` you see in a URL is a character that would otherwise have broken it. This is why Chapter 8 said never to build URLs from text by hand.

Native disclosure, without a script

`<details>` and `<summary>` give you an open-and-close panel that works with the keyboard, a screen reader and no JavaScript.

```
<details>
  <summary>Show the full specification</summary>
  <dl> ... </dl>
</details>
```

9.8 The Head: Metadata That Matters

The `<head>` holds nothing the visitor sees in the page, and several things they see elsewhere.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flight Dynamics | Gopolang Makgothi</title>
  <meta name="description" content="A spiral climb you can fly, and a full stability
      analysis with poles, Routh arrays and a PID loop.">
  <link rel="icon" href="favicon.svg" type="image/svg+xml">
  <meta property="og:title" content="Flight Dynamics">
  <meta property="og:description" content="Fly a spiral climb and watch the stabilit
      y analysis update live.">
  <meta property="og:image" content="https://example.com/preview.jpg">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="stylesheet" href="style.css">
  <script defer src="app.js"></script>
</head>
```

- The **description** is the paragraph a search engine shows under your link. Keep it under about 155 characters and make it a sentence a person would click.
- The **icon** is the little picture in the tab. An SVG icon stays sharp at every size.
- The **Open Graph** tags (`og:title`, `og:description`, `og:image`) are what chat apps and social networks show when someone pastes your link. Without them, the preview is a bare address. The image must be an absolute URL.
- **Order matters.** Charset first, then viewport and title, then stylesheets (they block rendering, so start them early), then deferred scripts.

> **WHY DO IT THIS WAY? Putting the stylesheet link before the script**

What it is. `<link rel="stylesheet">` above `<script defer>` in the head.

Why it is done this way. The browser starts fetching things in the order it meets them. The stylesheet is needed before the first paint; the deferred script is not.

Why it matters. The page appears styled as early as possible.

What happens if you do not. A script listed first takes its turn on the connection first, and the styles arrive a little later. On a slow connection, that is a flash of unstyled text.

9.9 Accessibility: When Native Elements Are Not Enough

Most accessibility comes free from the right element. Four tools cover the rest.

`tabindex`**.** Every link, button and form control can be reached with the Tab key. A custom control made from a `<div>` cannot, until you add `tabindex="0"`, and then you must also handle Enter and Space in script. `tabindex="-1"` makes an element focusable by script but not by Tab, which is right for a panel you move focus into when it opens. Never use positive values; they scramble the tab order.

`aria-label` **and** `aria-labelledby`**.** A name for something that has no visible text: an icon button (`<button aria-label="Close">×</button>`), a chart canvas, a second `<nav>`.

State attributes. `aria-expanded="true"` on a button that opens a panel, `aria-pressed` on a toggle, `aria-current="page"` on the menu link for the page you are on. Scripts update them when the state changes, exactly as they update a class.

`aria-live`**.** A region whose changes should be announced: `aria-live="polite"` on the readout that shows a computed result, so a screen-reader user hears "turn radius 1 240 feet" when the slider moves.

```
<a class="skip" href="#main">Skip to content</a>      <!-- first thing in the body -
      ->
<button aria-expanded="false" aria-controls="spec">Specification</button>
<div id="spec" hidden> ... </div>
<p aria-live="polite">Turn radius: <output id="radius">1 240 ft</output></p>
```

WHY DO IT THIS WAY? A skip link as the first element in the body

What it is. A link to `#main`, usually hidden until it receives focus.

Why it is done this way. A keyboard user otherwise tabs through every navigation link on every page before reaching the content.

Why it matters. One press of Tab and Enter gets them to the content.

What happens if you do not. A visitor with a switch device or a screen reader works through forty links to read one paragraph, on every page of your site.

WARNING

> The first rule of ARIA is *do not use ARIA* when a native element does the job. `<button>` already has a role, keyboard handling and a pressed state. `<div role="button" tabindex="0" onclick=...>` has to rebuild all three, and most people forget one. Reach for ARIA only to describe something the native elements cannot.

The `hidden` attribute removes an element from the page and from assistive technology, like `display: none`. `aria-hidden="true"` hides it from assistive technology *only*, which is right for a decorative icon next to a text label, and wrong for anything a person needs.

9.10 Hooks for CSS and Script

HTML gives the other two languages three ways to find an element. An `id` is a unique name for one element: the target of a fragment link, a label's `for`, and `aria-controls`. A `class` is a reusable label for styling and for finding groups of elements. A **data attribute**, `data-anything="value"`, carries a small piece of information for a script, readable as `element.dataset.anything`.

```
<button class="chip" data-chart="speed">Speed</button>
<button class="chip" data-chart="power">Power</button>
```

> **WHY DO IT THIS WAY? Data attributes instead of parsing the id**
>
> **What it is.** `data-chart="speed"` rather than `id="chip-speed"` and then chopping the string in script.
>
> **Why it is done this way.** The attribute says exactly what the script needs, with a name that explains itself.
>
> **Why it matters.** `button.dataset.chart` is the value, with no string surgery. Several buttons can share a class and differ by data.
>
> **What happens if you do not.** Scripts fill up with `id.split('-')[1]`, and the first time someone renames an id, three handlers break.

9.11 Validation and Debugging

The W3C validator (search "HTML validator") reads your page and lists every mistake: unclosed elements, misplaced tags, missing `alt`, duplicate `id`s. Run it on every page once before you publish. Then use the Elements panel for anything the validator cannot see: it shows the tree as built, the computed styles, and, in the Accessibility pane, the name and role every element presents to assistive technology.

The mistakes you will make most often, in order: a missing `alt`; a `<div>` where a `<button>` or `<a>` belongs; headings chosen for size; a `<p>` wrapped around a block element; two elements with the same `id`; an unquoted attribute value; a form button without a `type`; a link that opens a new tab without `rel="noopener"`.

Worked Example: A Small Page, Built Properly

Worked Example: A recipe card

Build a page with a title, a heading, a picture, a list of ingredients, and a link back to the home page. Use landmarks.

Step 1. Skeleton. Start with the skeleton from 9.1 and set the title to "Pancakes".

Step 2. Landmarks. Put the home link inside `<nav>`, the recipe inside `<main>`, and a copyright line inside `<footer>`. A screen-reader user can now jump straight to the main content.

Step 3. The content.

```
<body>
  <nav><a href="index.html">Home</a></nav>
  <main>
    <h1>Pancakes</h1>
    <img src="pancakes.jpg" alt="A stack of pancakes with syrup" width="600" height=
      "400">
    <h2>Ingredients</h2>
    <ul>
      <li>1 cup flour</li>
      <li>1 egg</li>
      <li>1 cup milk</li>
    </ul>
    <h2>Method</h2>
    <ol>
      <li>Whisk everything together.</li>
      <li>Fry for <time datetime="PT2M">two minutes</time> a side.</li>
    </ol>
  </main>
  <footer>&copy; 2026</footer>
</body>
```

Step 4. Check. Open the browser developer tools, look at the Elements panel, and confirm the tree matches the nesting you meant. Then press Tab: the only stop should be the Home link. Then run the validator.

Why this is "properly". No `<div>` is needed anywhere. Each element says what it is, so the page works for screen readers, search engines and keyboards without extra effort. The method is an ordered list because order matters; the ingredients are unordered because it does not.

Worked Example: A Specification Table and a Form

Worked Example: A part's spec sheet with a question box

A page describes a motor. It needs a table of its ratings, a disclosure with the full datasheet notes, and a short form where a visitor can ask a question. Mark it up.

Step 1. The table. The ratings are data in rows and columns: one row per quantity, columns for the value and the conditions.

```
<table>
  <caption>TT gear motor ratings</caption>
```

```
  <thead><tr><th scope="col">Quantity</th><th scope="col">Value</th><th scope="col">
      Condition</th></tr></thead>
  <tbody>
    <tr><th scope="row">No-load speed</th><td>200 rpm</td><td>6 V</td></tr>
    <tr><th scope="row">Stall torque</th><td>0.08 N·m</td><td>6 V</td></tr>
    <tr><th scope="row">Gear ratio</th><td>1:48</td><td></td></tr>
  </tbody>
</table>
```

Step 2. The notes. They are long and optional, so they go in a `<details>` that starts closed.

Step 3. The form. Two fields and a button. Each field has a label, the email field has the right type, and the message is a `<textarea>`.

```
<form action="/ask" method="post">
  <label for="from">Your email</label>
  <input id="from" name="from" type="email" required autocomplete="email">
  <label for="msg">Your question</label>
  <textarea id="msg" name="msg" rows="4" required maxlength="500"></textarea>
  <button type="submit">Send</button>
</form>
```

Step 4. Check with a keyboard. Tab moves from the summary, to the email box, to the message, to the button, in that order. Press Enter on the summary and the notes open. Submit the empty form and the browser itself asks for the email.

Why it matters. Everything here works before a single line of CSS or JavaScript is written. The styling and the script that sends the question without a page reload are improvements, not requirements. That is progressive enhancement (Chapter 3), and it is the reason the page will still work on the day the script has a bug.

Practice Problems A: The Skeleton and the Parser

Review Problems

1. List the six lines you would put in the `<head>` of a new page, and say what each does.
2. This markup is wrong in three ways. Find them: `<p>Welcome to <b>my site</p></b>` and `<img src="a.jpg">`.
3. Which of these are void elements: `<br>`, `<p>`, `<img>`, `<input>`, `<span>`, `<meta>`, `<li>`?
4. What tree does the browser build from `<p>One<p>Two`? How many paragraphs, and what is in each?
5. Write the text `if (a < b && c > 0)` so that it appears literally inside a `<code>` element.
6. A page shows "Â°" where it should show "°". What is missing, and where must it go?
7. What does `<input type="checkbox" required="false">` require?

Practice Problems B: Content, Links, Images, Tables and Forms

Review Problems

1. Write a navigation bar containing three links using a list, and wrap it in the right landmark element.
2. A page has these headings in order: `<h1>`, `<h3>`, `<h2>`, `<h5>`. Explain the problem and fix it.
3. Which element would you use for each: (a) text that is more important than its neighbours, (b) a clickable thing that submits a form, (c) the list of steps in a recipe where order matters, (d) a glossary of terms and definitions, (e) a keyboard shortcut the reader should press, (f) a date a script might read.
4. The page `/docs/guide/intro.html` must link to `/docs/index.html` and show `/img/cover.png`. Write the relative `href` and `src`.
5. An image is shown at 300 CSS pixels wide on viewports under 700 pixels and at 700 pixels otherwise. Write its `sizes` attribute. With candidates of 300w, 600w, 900w and 1400w, which is fetched on a 390-pixel DPR 3 phone?
6. Write a two-column table of three robot parts and their masses with a caption and proper heading cells.
7. A form has `method="get"`, `action="/filter"`, a select `name="tag"` set to `robots`, and a text field `name="q"` containing `claw 2`. What URL is requested?
8. Write an HTML form for a login with a username and a password, each with a proper label and `autocomplete`.
9. Why does a "Clear" button inside a form reload the page, and what is the one-word fix?

Practice Problems C: The Head, Accessibility and Hooks

Review Problems

1. Write the four `<meta>` and `<link>` lines that make a page show a proper preview when its link is pasted into a chat app, and say which one must hold an absolute URL.
2. An icon-only button shows a "×". What attribute makes it usable with a screen reader, and what should its value be?
3. A custom toggle is built from a `<div>`. Name the three things you must add in HTML and script to make it work like a button, then say what you should have done instead.
4. When is `aria-hidden="true"` correct, and when is it harmful?
5. What is the difference between `hidden` and `aria-hidden`?
6. Three chip buttons share a class and each names a chart. Write the markup so that a script can read which chart a clicked chip means without parsing an id.
7. Name five mistakes the validator would catch in a page, and two it would not.

Mini Project: A Documentation Page for a Robot

Mini Project: Write the page before you style it

Build `patrick.html`: a single page that documents the robot, using nothing but HTML. It must contain, in this order:

1. A `<header>` with the site name and a `<nav>` of three links (Home, Robots, Downloads), one of which has `aria-current="page"`.
2. A `<main>` with an `<h1>`, an introduction paragraph, and three `<section>`s with `<h2>`s: *Parts*, *How it moves*, *Specification*.
3. In *Parts*, a `<figure>` with a photo (`alt`, `width`, `height`, `loading="lazy"`) and a caption, followed by a `<dl>` of five parts and what each does.
4. In *How it moves*, an `<ol>` of the steps of the breakfast run, with the sensor reading written using `<code>` and the trigger distance using `<data value="0.108">10.8 cm</data>`.
5. In *Specification*, a `<table>` with a caption, column headings and row headings, and a `<details>` holding the notes.
6. A `<form>` with a labelled email field and a labelled question `<textarea>`, and one `type="submit"` button.
7. A `<footer>` with a `<small>` copyright line that uses an entity for ©.

Checks: the validator reports no errors; Tab visits the links, the summary, the fields and the button in document order; the Elements panel shows the tree you intended; with CSS disabled the page still reads from top to bottom as a sensible document. Keep this file. Project 1 in Part III styles it, and the JavaScript chapter wires its form.

What You Learned

- Every page starts with the doctype, `lang`, the charset, the viewport and a title. Each line prevents a specific failure.
- The parser repairs mistakes silently. Know its rules: nesting, void elements, content models, whitespace collapsing, entities and boolean attributes.
- Mark up meaning: headings in order, lists, definition lists, quotes, code, keys, dates. `<div>` and `<span>` are the last resort, not the first.
- Landmarks let people jump; `<figure>`, `<table>` with `<th scope>`, and labelled form controls carry meaning that scripts and assistive technology can use.
- Images need `alt`, `width` and `height`; `srcset` and `sizes` let the browser fetch the right copy; `<picture>` offers alternatives.
- Forms send `name=value` pairs; `get` puts them in the URL, `post` in the body. Give every form button a `type`.
- The head carries the description, icon and Open Graph tags; stylesheets before deferred scripts.
- Use ARIA only where no native element will do; `tabindex="0"`, `aria-label`, state attributes and `aria-live` cover most cases.

- Validate every page, then check the tree and the accessibility pane in the developer tools.

Chapter 10. CSS: The Look

If HTML is the skeleton, CSS is everything that makes the skeleton look like a person: skin, clothes, posture. CSS has a reputation for being confusing, and the reputation is deserved, but only because people try to memorise it instead of understanding a handful of ideas: the cascade, the box, formatting contexts, and the few layout systems. You met the first two in Part I. This chapter turns them into working habits and then goes further: positioning, stacking, colour, typography, transforms, transitions, and the way a large stylesheet is organised so that it stays sane.

In This Chapter

- Read a rule, score a selector, and predict which declaration wins.
- Use the box model, margins (and their collapsing), `display`, and overflow on purpose.
- Lay out anything with normal flow, Flexbox or Grid, and do the arithmetic of each.
- Position and stack elements with `position` and `z-index` without guessing.
- Choose units, colours and type with a system, and keep them in custom properties.
- Animate with transforms and transitions, cheaply.
- Make a design respond to width, pointer, and motion preference, and organise a stylesheet that will still make sense next year.

Words You Will Meet

Word	What it means
Selector	The part of a rule that says which elements it applies to.
Property	A thing you can change, such as `color`.
Value	The setting you give it, such as `red`.
Declaration	A property and value together, ending in a semicolon.
Rule	A selector with its declaration block in braces.
Pseudo-class	A state added to a selector, such as `:hover` or `:focus`.
Pseudo-element	A part of an element, such as `::before`.
Combinator	The symbol between two simple selectors: a space (descendant), > (child), + (next sibling), ~ (later sibling).
Cascade	The procedure that settles conflicts between rules.
Specificity	The score that decides which selector wins.
Inheritance	A property taking its value from the parent when none is set.
Computed value	What a property finally works out to, after the cascade, inheritance and unit conversion.
Formatting context	The set of layout rules in force inside a container: block, inline, flex or grid.
Stacking context	A group of elements whose `z-index` values are compared only with each other.
Custom property	A variable, such as `--accent`, read with `var()`.
Media query	A rule that applies only if a condition (like screen width) is true.

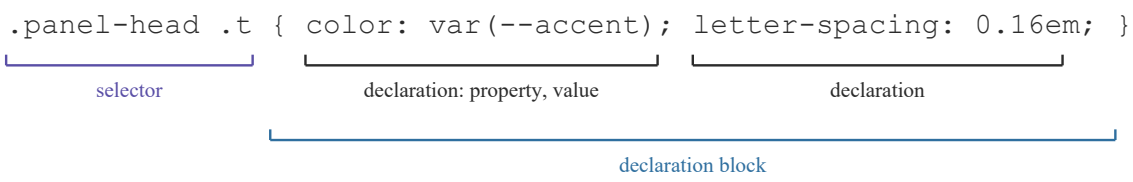


Figure 10.1. The parts of one rule. The selector picks elements; each declaration sets one property; the braces hold the declaration block.

10.1 How the Browser Reads Your Rules

For every element, the browser collects every rule that matches it, throws away the losers (using the cascade from Chapter 3), and is left with one value per property. Then it fills in anything still missing by **inheritance** (from the parent) or the property's **default**. Finally it converts what is left into a **computed value**: `1.5rem` becomes `24px`, `inherit` becomes the parent's number, a keyword like `bold` becomes `700`. Understanding this order is the whole game. When a style "does not work", it is almost always because another rule won, or because the selector did not match.

> **TIP**
> Right-click the element, choose Inspect, and look at the Styles panel. It lists the matching rules from strongest to weakest and **crosses out the ones that lost**. That crossed-out line is usually the answer. The Computed tab shows the final numbers.

Which properties inherit

Text properties inherit: `color`, `font-*`, `line-height`, `letter-spacing`, `text-align`, `visibility`. Box properties do not: `margin`, `padding`, `border`, `background`, `width`, `display`, `position`. This is why you set the typeface once on `body` and never on a box. Any property can be forced to inherit with the value `inherit`, reset with `initial`, or set to whichever of the two applies with `unset`.

10.2 Writing Selectors Well

The most useful selectors are simple: a **class** (`.card`), an **element** (`p`), a **descendant** (`.card p`), a **child** (`.card > p`), and states (`.button:hover`). Here is the full toolkit, with the specificity each piece adds.

Selector	Matches	Adds to (a, b, c)
`p`	every <p>	(0, 0, 1)
`.card`	every element with class `card`	(0, 1, 0)
`#main`	the one element with id `main`	(1, 0, 0)
`[type="range"]`	elements with that attribute value	(0, 1, 0)
`a:hover`, `li:first-child`, `:not(.on)`	states and positions	(0, 1, 0) each, plus the inside of `:not()`
`p::first-line`, `.card::before`	a part of an element	(0, 0, 1)
`.card p`	a <p> anywhere inside `.card`	the sum of the parts
`.card > p`	a <p> that is a direct child of `.card`	the sum
`h2 + p`	a <p> immediately after an <h2>	the sum
`h2 ~ p`	every <p> after an <h2> among its siblings	the sum

Selector	Matches	Adds to (a, b, c)
`.a, .b`	either (two rules written once)	each scored on its own
`*`	everything	(0, 0, 0)
`:is(.a, .b) p`	a <p> inside either	the highest of the arguments
`:where(.a, .b) p`	the same, but `:where` adds nothing	(0, 0, 1)

Structural pseudo-classes let you style by position without extra classes: `:first-child`, `:last-child`, `:nth-child(2n)` (even), `:nth-child(3n + 1)`, `:only-child`, `:empty`. The form states `:checked`, `:disabled`, `:focus-visible`, `:invalid` and `:placeholder-shown` let a stylesheet react to a control's state with no script at all.

Worked Example: Reading a selector you have never seen

What does `.panel:not(.lite) > .row:nth-child(2n) label + input[type="range"]:focus-visible` match, and what is its specificity?

Step 1. Read it right to left, which is how the browser does it. A range input that currently has keyboard focus; that comes immediately after a label; inside a `.row` that is an even-numbered child; which is a direct child of a `.panel` that does not have the class `lite`.

Step 2. Score it. `.panel` (0,1,0), `:not(.lite)` takes the score of its argument (0,1,0), `.row` (0,1,0), `:nth-child(2n)` (0,1,0), `label` (0,0,1), `input` (0,0,1), `[type="range"]` (0,1,0), `:focus-visible` (0,1,0). Total: (0, 6, 2).

Step 3. Judge it. It works, and it is a maintenance trap: six class-level pieces mean nothing simpler can override it, and it breaks if a row is reordered or a label moves. The same intent as a flat class, `.range-even:focus-visible` (0, 2, 0), is both weaker and sturdier.

> **WHY DO IT THIS WAY? Styling with classes, not IDs or deep chains**
>
> **What it is.** Giving elements a class (`class="card"`) and writing `.card { ... }`, instead of `#main-card` or `div.wrapper > div > ul > li:nth-child(3)`.
>
> **Why it is done this way.** A class has middling specificity (0,1,0), and it can be reused anywhere.
>
> **Why it matters.** Rules stay easy to override, and the same class can be on many elements.
>
> **What happens if you do not.** An ID (specificity 1,0,0) beats nearly everything, so you need an even more specific rule to override it, and then another. Long chains also break the moment the HTML structure changes.

10.3 The Box, Margins and Display

Everything is a box (Chapter 4). Start every stylesheet with `* { box-sizing: border-box; }`. Three further facts about boxes explain most layout surprises.

Vertical margins collapse. When two block boxes stack, the gap between them is the *larger* of the two margins, not the sum. Two paragraphs with `margin: 24px 0` are 24 pixels apart, not 48. A child's top margin can even escape through its parent's top edge if the parent has no padding or border there. Horizontal margins never collapse, and margins inside Flexbox and Grid never collapse.

`display` **sets the kind of box.** `block` boxes stack and take the full width; `inline` boxes sit in a line of text and ignore `width` and vertical margins; `inline-block` sits in a line but accepts a width and height; `flex` and `grid` make the element a container for its children; `none` removes it from layout entirely. (`visibility: hidden` hides it but keeps its space; `opacity: 0` keeps it in the page and still clickable.)

Overflow is your choice. When content is bigger than its box, `overflow: visible` (the default) lets it spill out, `hidden` clips it, `auto` adds a scrollbar only when needed, and `scroll` always shows one. `overflow-x: auto` on a code block or an equation is the standard cure for a page that scrolls sideways on a phone.

Worked Example: Margins that do not add up

Three stacked `<p>` elements have `margin: 20px 0`. The first sits at the top of a `<section>` with `padding: 0` and `margin-top: 50px`. What is the gap between the paragraphs, and how far is the first paragraph from the element above the section?

Step 1. Between paragraphs. The 20 px bottom margin of one and the 20 px top margin of the next collapse to the larger: 20 px, not 40.

Step 2. Through the section. The section has no top padding or border, so the first paragraph's 20 px top margin collapses *with* the section's 50 px top margin. The result is a single 50 px gap, and the paragraph sits flush with the top of the section.

Step 3. Stop it collapsing. Give the section `padding-top: 1px`, or make it a flex or grid container, or set `display: flow-root`. Then the gap is 50 + 20 = 70 px, which is probably what the author expected.

Why it matters. "Why does my margin do nothing?" is almost always collapsing. Knowing the rule turns a mystery into a one-line fix.

10.4 Layout: Flow, Flex and Grid

Use **normal flow** for text, **Flexbox** for one-dimensional arrangements (a row of buttons), and **Grid** for two-dimensional ones (a gallery, a page). Chapter 4 showed the arithmetic of `flex-grow` and `auto-fit`; here are the properties you will reach for, in the order you will need them.

```
.row     { display: flex; gap: 1rem; align-items: center; justify-content: space-bet
      ween; flex-wrap: wrap; }
.row > *  { flex: 1 1 12rem; }         /* grow, shrink, basis: at least 12rem, share
      the rest */
.gallery { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)
      ); gap: 1rem; }
.page    { display: grid; grid-template-columns: 220px 1fr; grid-template-rows: auto
      1fr auto; min-height: 100vh; }
```

```
.page > header { grid-column: 1 / -1; }   /* span every column */
```

Flexbox vocabulary. `flex-direction` picks the main axis (`row` or `column`). `justify-content` arranges items along it (`flex-start`, `center`, `space-between`, `space-around`). `align-items` arranges them across it (`stretch`, `center`, `flex-start`, `baseline`). `flex-wrap: wrap` lets items drop to a new line. On an item, `flex: grow shrink basis` and `align-self` override the container. `gap` spaces them.

Grid vocabulary. `grid-template-columns` and `-rows` define the tracks, in any mix of lengths, `fr` shares, `auto`, `minmax()` and `repeat()`. `gap` spaces the tracks. An item is placed with `grid-column: 1 / 3` (from line 1 to line 3, so two tracks wide) or `grid-column: span 2`; `-1` means the last line. `grid-template-areas` lets you draw the layout with words. `align-items` and `justify-items` align within cells; `place-items: center` does both.

Worked Example: Fixed, flexible and doubly flexible columns

A grid is 800 px wide with `grid-template-columns: 200px 1fr 2fr` and `gap: 20px`. How wide is each column?

Step 1. Take out the fixed parts. The 200 px column and two gaps of 20 px: 800 − 200 − 40 = 560 px remain.

Step 2. Share by fr. Three shares in total (1 + 2), so one share is 560 / 3 = 186.67 px. The columns are 200, 186.67 and 373.33 px.

Step 3. Check. 200 + 186.67 + 373.33 + 40 = 800. √

Step 4. What if the content is wider? `1fr` is really `minmax(auto, 1fr)`, so a column will not shrink below its longest unbreakable word or widest image. If a 500 px image lands in the middle column, that column grows and the grid overflows. Write `minmax(0, 1fr)` when you want the share to win over the content.

> **WHY DO IT THIS WAY? gap instead of margins between items**
>
> **What it is.** Using `gap` to space children in a flex or grid container.
>
> **Why it is done this way.** `gap` puts space *between* items only, not before the first or after the last.
>
> **Why it matters.** There is no stray margin at the end, no special case for the last item, and no need to remember to remove it.
>
> **What happens if you do not.** You put `margin-right` on every item and then spend time removing it from the last one, and you forget when the layout wraps.

Worked Example: Choosing the layout tool

For each of these, say which layout system fits and why: (a) a navigation bar with a logo on the left and links on the right; (b) a page with a sidebar and a main column; (c) a paragraph of text with an image floated beside it; (d) a set of cards that should wrap to as many columns as fit; (e) centring one box in the middle of the screen.

(a) **Flexbox.** One row; `justify-content: space-between` puts the logo and links at the ends.

(b) **Grid.** Two columns (`220px 1fr`) and the rows of the page; the header can span both with `grid-column: 1 / -1`.

(c) **Normal flow with** `float: left` on the image. Floats are the one place text wraps around a box; Flexbox and Grid cannot do that.

(d) **Grid** with `repeat(auto-fit, minmax(…, 1fr))`. Flexbox with `flex-wrap` also works, but the last row's items would stretch to fill it; Grid keeps every card the same width.

(e) **Either.** `display: grid; place-items: center; min-height: 100vh` on the parent is the shortest. Flexbox needs `justify-content: center; align-items: center`.

10.5 Position and Stacking

`position` takes an element out of the normal rules of flow, and `z-index` decides what is drawn on top. These two are where most "it is in the wrong place" bugs live.

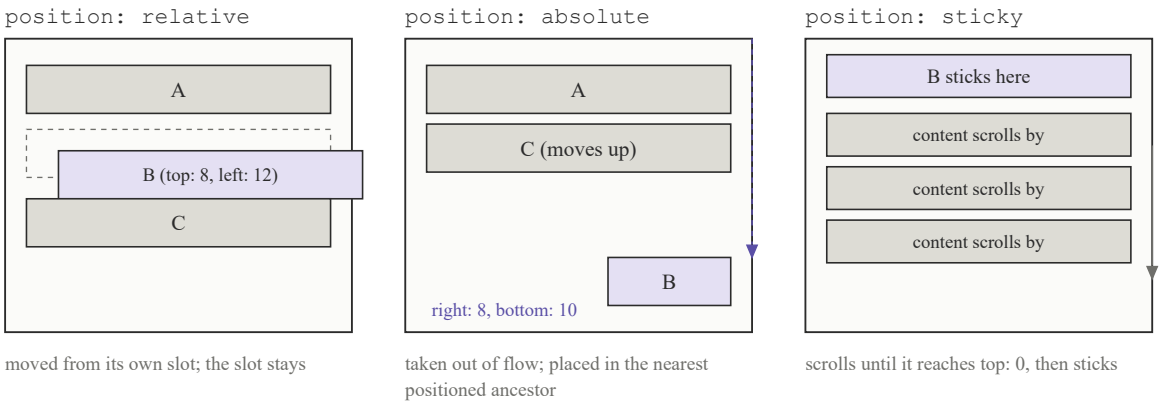


Figure 10.2. The three positioning schemes you will use. Relative moves a box but leaves its slot; absolute removes the box from flow and places it against its nearest positioned ancestor; sticky scrolls with the page until it reaches its offset, then holds.

- `position: static` is the default: the box is where flow puts it, and `top`, `left` and `z-index` do nothing.
- `position: relative` keeps the box in flow and then nudges it by `top`/`left`. Its main use is subtler: it makes the box the **reference** for any absolutely positioned descendant.
- `position: absolute` takes the box out of flow (the others close up) and places it against the nearest ancestor that has a `position` other than static. `inset: 0` stretches it to fill that ancestor.
- `position: fixed` is like absolute, but against the viewport, so it stays put while the page scrolls. A floating "back to top" button, a progress bar.
- `position: sticky` is relative until the element reaches its `top` offset while scrolling, then it sticks there until its parent scrolls away. A table header or a section label.

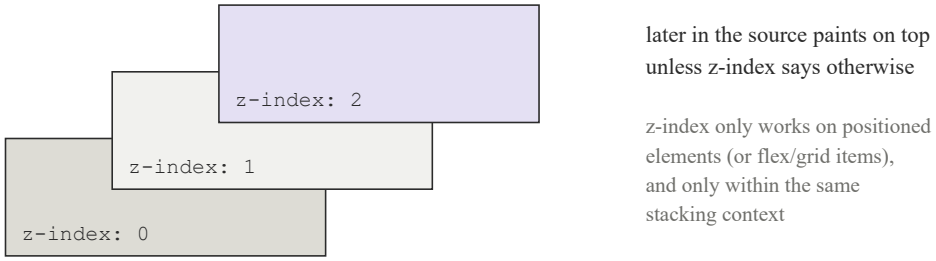


Figure 10.3. Later elements paint over earlier ones; `z-index` reorders positioned elements, but only within the same stacking context.

Stacking. Without `z-index`, boxes are painted in source order, later over earlier, with positioned boxes over non-positioned ones. `z-index` changes the order, with two conditions: it only works on positioned elements (and on flex or grid items), and it only compares elements inside the same **stacking context**. A new context is started by a positioned element with a `z-index`, by `opacity` less than 1, by a `transform`, a `filter`, and a few others. Inside a context, the children's `z-index` values are private: a child with `z-index: 9999` cannot escape a parent that is itself under something else.

Worked Example: Why the menu is behind the hero

A drop-down menu has `position: absolute; z-index: 1000` but still appears *behind* a hero section that comes later in the page and has `transform: translateY(0)`. Why, and what is the fix?

Step 1. Find the contexts. The `transform` on the hero creates a stacking context for the hero. The menu's parent, the header, has `position: relative; z-index: 1`? Suppose it does: that creates a stacking context for the header too.

Step 2. Compare at the right level. The menu's 1000 only competes with other things *inside the header's context*. At the page level, the header (z-index 1) competes with the hero (a transformed element, which counts as z-index 0, but comes later in source order). 1 beats 0, so the header should win. If the header had no `z-index`, it would be at the same level as the hero and the later hero would paint on top.

Step 3. The fix. Give the header a `z-index` higher than the hero's, at the level where they compete, or remove the needless `z-index` on the header so the menu's 1000 competes at page level. The menu's own number never mattered.

Why it matters. Raising `z-index` to larger and larger numbers never fixes a stacking-context problem. Finding which two ancestors are actually being compared does.

10.6 Colour, Type and Space

Choose a small palette and keep it in custom properties. Choose one or two typefaces. Choose a spacing scale (for example 0.25, 0.5, 1, 2, 4 rem) and stick to it.

```
:root {
```

```
    --bg: #070a1a;
    --text: #e6e8f5;
    --accent: #a78bfa;
    --space: 1rem;
    --radius: 10px;
}
body { background: var(--bg); color: var(--text); font-family: 'Inter', system-ui, s
      ans-serif; line-height: 1.7; }
```

Colour

A colour can be written five ways, and they are all the same colour to the browser: a name (`rebeccapurple`), a hex code (`#a78bfa`, or `#a78bfacc` with alpha), `rgb(167 139 250)`, `hsl(255 92% 76%)`, or the newer `oklch()`. HSL is the one to *think* in: hue is the position on the colour wheel in degrees, saturation is how vivid, lightness is how light. The site's accent `#a78bfa` is `hsl(255 92% 76%)`; a darker version for a hover state is the same hue and saturation with lower lightness, `hsl(255 92% 66%)`, which you could never guess from the hex code. Transparency comes from a fourth value: `hsl(255 92% 76% / 0.5)`. The `color-mix()` function blends two colours: `color-mix(in oklab, var(--accent) 30%, transparent)` is a tint of the accent for a faint background.

> **WHY DO IT THIS WAY? Keeping the palette in custom properties and deriving the rest**
>
> **What it is.** Five or six named colours at the top, and every other colour made from them with lightness changes, alpha or `color-mix()`.
>
> **Why it is done this way.** The design then has a small number of decisions, and the browser does the arithmetic.
>
> **Why it matters.** Change `--accent` and every hover tint, border and highlight follows. A dark-to-light theme is a second `:root` block, not a rewrite.
>
> **What happens if you do not.** Fifty near-identical purples live in the file, and a palette change is a week of search and replace that still misses three.

Type

Typography is three numbers and one choice. The choice is the typeface; the numbers are size, line height and measure (line length). Set the size in `rem` so it respects the visitor's settings; set the line height as a unitless number, about 1.5 to 1.7 for body text and 1.1 to 1.2 for headings; keep the measure between 45 and 75 characters with `max-width: 65ch`. Build the sizes as a scale, so headings relate to each other: 1, 1.125, 1.25, 1.5, 2, 2.5 rem is a common one.

Worked Example: Line heights that work out

The root font size is 16 px. `h1 { font-size: 2.5rem; line-height: 1.2 }` and `p { font-size: 1.125rem; line-height: 1.6 }`. What are the computed values, and what would go wrong with `line-height: 28px` on the body instead?

Step 1. The heading. 2.5 × 16 = 40 px, and the line height is 1.2 × 40 = 48 px.

Step 2. The paragraph. 1.125 × 16 = 18 px, with a line height of 1.6 × 18 = 28.8 px.

Step 3. The trap. A fixed `line-height: 28px` on the body is inherited as *28 px* by the heading, whose 40 px glyphs then overlap on wrapped lines. A unitless `1.6` is inherited as a *ratio*, so the heading would get 64 px, which is roomy but at least readable, and then its own rule sets 1.2.

Why it matters. This is the single most common typography bug, and it is invisible until a heading wraps.

> **WHY DO IT THIS WAY? A font stack with system-ui at the end**
>
> **What it is.** `font-family: 'Inter', system-ui, sans-serif` is a list, not a single choice.
>
> **Why it is done this way.** The browser tries each in turn and uses the first one it has. If the chosen web font has not loaded yet, or fails, it falls back.
>
> **Why it matters.** The page is readable immediately in a sensible font instead of showing blank text or a clumsy default.
>
> **What happens if you do not.** If the one font you named fails to load, the browser picks its own default (often Times), and your design collapses.

Web fonts are loaded with `@font-face` or a link to a font service. Add `font-display: swap` so text shows in the fallback font until the real one arrives, and load only the weights you use: each weight is a separate file of 20 to 100 kilobytes.

Space

Spacing is the thing most beginners do by eye and most professionals do by rule. Pick a base (`1rem`) and a few multiples, name them (`--s1: 0.5rem; --s2: 1rem; --s3: 2rem; --s4: 4rem`), and use nothing else. Put space *between* things with `gap` and `margin-block-start` on the second of two things, not on every side of everything.

10.7 Pseudo-elements and Pseudo-classes

`::before` and `::after` insert a generated box at the start or end of an element's content. They need a `content` property, even if it is empty, and they are how decorations are drawn without extra HTML: a diamond before a heading, a corner mark on a card, a fading edge on a scrolling strip.

```
.proj-tag::before { content: ''; display: inline-block; width: 6px; height: 6px;
                    background: var(--accent); transform: rotate(45deg); margin-righ
      t: 0.5em; }
```

A **transition** animates a property when it changes. Combine it with `:hover` and `:focus-visible` for friendly controls.

```
.chip { background: transparent; transition: background 150ms ease; }
.chip:hover { background: var(--accent); }
.chip:focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; }
```

> **WHY DO IT THIS WAY? :focus-visible**
>
> **What it is.** A pseudo-class that matches focus only when the browser thinks the user is using the keyboard.

> **Why it is done this way.** Mouse users do not want a ring after every click. Keyboard users need one.
>
> **Why it matters.** Both are served: the ring appears for Tab, not for clicks.
>
> **What happens if you do not.** Developers who dislike the ring hide it for everyone, and keyboard users get lost (Chapter 8).

Two newer selectors remove whole categories of script. `:has()` styles a parent by its children: `.card:has(img)` is a card that contains an image; `label:has(input:checked)` is the label of a ticked box. `@media (hover: hover)` applies hover styles only on devices that can hover, so a phone does not get stuck in a hover state after a tap.

10.8 Transforms, Transitions and Animations

`transform` moves, rotates, scales or skews an element *without affecting layout*: the box's slot stays where it was and only the picture moves. That is why it is cheap (Chapter 2) and why it is the right tool for motion.

```
.card:hover { transform: translateY(-3px) scale(1.02); }
.needle     { transform: rotate(var(--angle)); transform-origin: 50% 90%; }
```

The functions apply **right to left** as written, or equivalently, each one happens in the coordinate system left by the previous ones. `translate(100px) rotate(45deg)` moves the box 100 px right and then spins it in place. `rotate(45deg) translate(100px)` spins the axes first and then moves 100 px along the rotated x axis, which is 70.7 px right and 70.7 px down. `transform-origin` sets the point things rotate and scale about; a gauge needle wants its origin at the pivot, not the centre.

A **transition** interpolates a property between its old and new value whenever it changes: `transition: property duration timing-function delay`. Only properties with numeric values can be transitioned; `display` cannot, which is why panels are faded with `opacity` and moved with `transform` and only then hidden. The timing function is one of Chapter 6's easing curves: `ease` (the default), `linear`, `ease-in`, `ease-out`, `ease-in-out`, or a `cubic-bezier()`.

An **animation** runs a `@keyframes` sequence, with a duration, an iteration count and a direction, independently of any property change. It is right for things that move on their own: a blinking dot, a loading bar, dashes flowing along a wire.

```
@keyframes flow { to { stroke-dashoffset: -24; } }
.wire { stroke-dasharray: 6 6; animation: flow 1.2s linear infinite; }
```

Worked Example: What the eye sees at a quarter of the way through

A panel slides in with `transition: transform 400ms cubic-bezier(0.16, 1, 0.3, 1)` from `translateY(40px)` to `translateY(0)`. Where is it after 100 ms? Compare with `ease-out`.

Step 1. The progress. 100 ms of 400 ms is t = 0.25 of the time.

Step 2. The curve. For `cubic-bezier(0.16, 1, 0.3, 1)` the progress of motion at t = 0.25 is 0.826. For `ease-out`, which is `cubic-bezier(0, 0, 0.58, 1)`, it is 0.378.

Step 3. The position. The panel has 40 px to travel. With the first curve it has covered 0.826 × 40 = 33 px and sits 7 px from its final place; with `ease-out` it has covered 15 px and still has 25 px to go.

What the answer tells you. The first curve is the "fast arrival, long settle" of Chapter 6: most of the motion is over in the first quarter, and the remaining 300 ms are a gentle landing. That is why interface designers love it for things that appear.

> **WHY DO IT THIS WAY? Animating transform and opacity, not top and height**
>
> **What it is.** Motion written as `transform` and `opacity`, never as `left`, `top`, `width`, `height` or `margin`.
>
> **Why it is done this way.** Transform and opacity can be handled by the compositor without layout or paint (Chapter 2).
>
> **Why it matters.** Sixty frames a second on a phone.
>
> **What happens if you do not.** Every frame of a `height` animation re-lays-out the page below it. Smooth on your desktop, a slideshow on a mid-range phone.

10.9 Responsive Design in Practice

Chapter 4 gave the three tools: the viewport tag, media queries and fluid layout. In practice a responsive stylesheet follows a few habits.

Mobile first. Write the narrow layout as the default and add complexity with `@media (min-width: ...)` queries. The narrow layout is usually simpler, so the default is simpler, and a phone never downloads rules it then has to undo.

Few breakpoints, chosen by the content. Drag the window narrower and add a query where the design *breaks*, not at the width of any particular phone. Two or three breakpoints are usually enough.

Fluid first. Reach for `clamp()`, `minmax()`, `auto-fit`, `flex-wrap` and percentages before you reach for a query. A layout that flows needs fewer special cases than one that jumps.

Ask about more than width. `(hover: none)` and `(pointer: coarse)` for touch, `(prefers-reduced-motion: reduce)` for motion, `(prefers-color-scheme: dark)` for the visitor's theme, `(orientation: landscape)` for a phone turned sideways.

```
.controls { display: grid; gap: 0.75rem; }                              /* phones:
      stacked */
@media (min-width: 720px) { .controls { grid-template-columns: 1fr 1fr; } }   /* wid
      er: two columns */
@media (hover: hover) { .card:hover { transform: translateY(-3px); } }      /* only
      where hover exists */
@media (prefers-reduced-motion: reduce) { * { transition: none !important; animation
      : none !important; } }
```

> **WHY DO IT THIS WAY? Mobile first**
>
> **What it is.** The default rules describe the phone layout; queries add the desktop layout.

> **Why it is done this way.** The phone layout is the simplest. Building up is easier than tearing down.
>
> **Why it matters.** The stylesheet is shorter, and the most constrained device gets the least work.
>
> **What happens if you do not.** The desktop-first stylesheet sets three columns, then a query sets one; then a second query for tablets sets two; and each query undoes rules that came before it, until nobody can tell what a phone actually gets.

10.10 Organising a Stylesheet

A stylesheet for one page can be written in any order. A stylesheet for a site needs a system, and the system is more important than which one you choose. Here is a simple one that scales.

1. **Tokens first.** The `:root` block with colours, spacing, radii, type sizes. Nothing else may contain a raw colour or a magic number.
2. **Reset and base.** `box-sizing`, margins, `body` type, link colours, `img { max-width: 100%; height: auto; }`.
3. **Layout.** The page shell, the grid, the containers.
4. **Components.** One block of rules per reusable thing, each under one class: `.panel`, `.chip`, `.readout`. Inside, variants are modifier classes (`.chip.on`, `.panel--wide`) and children are descendants with short names (`.panel .t`).
5. **Utilities.** A few single-purpose classes you will use everywhere: `.visually-hidden`, `.mono`, `.center`.
6. **Media queries last**, or next to the component they change.

Keep specificity flat: one class, sometimes two. Never style by id. Use `!important` only in utilities that *must* win (like `.visually-hidden`), and never to settle a fight you could settle with order. Name things by role, not appearance (`.text-muted`, not `.grey`). Comment the *why*, not the what.

> **WHY DO IT THIS WAY? Naming by role**
>
> **What it is.** `--text-muted`, `.panel-head`, `.chip.on`, not `--grey`, `.rounded-box`, `.chip-purple`.
>
> **Why it is done this way.** The role stays true when the design changes.
>
> **Why it matters.** When the muted text becomes blue in a redesign, `--text-muted` is still the right name and the right variable.
>
> **What happens if you do not.** `.chip-purple` turns teal and the code lies to everyone who reads it, including you.

Worked Example: A Responsive Card Row

Worked Example: Three cards that wrap on their own

Make a row of three cards that sits side by side on a wide screen and stacks on a phone, without writing a media query.

Step 1. HTML. A container with three children.

```
<div class="cards">
  <article class="card">One</article>
  <article class="card">Two</article>
  <article class="card">Three</article>
</div>
```

Step 2. CSS.

```
* { box-sizing: border-box; }
.cards { display: grid; gap: 1rem; grid-template-columns: repeat(auto-fit, minmax(20
      0px, 1fr)); }
.card  { padding: 1rem; border: 1px solid #ccc; border-radius: 10px; }
```

Step 3. Predict. In a 700-pixel container: how many columns fit? Each needs at least 200 plus gaps of 16: 3 columns need 3 × 200 + 2 × 16 = 632, which is at most 700, so three. In a 400-pixel container: 2 columns need 2 × 200 + 16 = 416 which exceeds 400, so only one column fits, and the cards stack.

Step 4. Check. Drag the window narrower and watch the cards drop into a stack at exactly the width you predicted. There is no query anywhere in the code. The layout is a *rule*, and the browser applied it.

Worked Example: A Panel From the Website, Explained

Worked Example: Reading a real component

The instrument panels of the flight dynamics page use rules like these. Say what each declaration does and why it is there.

```
.panel { position: relative; overflow: hidden; border-radius: 14px;
         border: 1px solid var(--border-accent);
         background: linear-gradient(180deg, #131a3f, var(--bg-raised)); }
.panel::after { content: ''; position: absolute; inset: 0; pointer-events: none;
                background: radial-gradient(60% 50% at 50% 0%, rgba(167,139,250,0.12
      ), transparent); }
.panel-head { display: flex; justify-content: space-between; align-items: baseline;
      gap: 1rem; }
.panel-head .t { color: var(--accent); letter-spacing: 0.16em; text-transform: upper
      case; font-size: 0.75rem; }
```

`position: relative` makes the panel the reference box for the absolutely positioned `::after`. `overflow: hidden` clips the canvas inside to the rounded corners. `border-radius` and the border give the card its shape. The **gradient background** lifts the top edge slightly, the way light falls on a real panel.

The `::after` draws a soft violet glow at the top centre, a pure decoration, with no extra HTML. `inset: 0` stretches it over the whole panel. `pointer-events: none` lets clicks pass through it to the

controls underneath, which would otherwise be unclickable.

The **header** is a flex row with the title and a status word at opposite ends, aligned on their text baseline. The **title** takes the accent colour, wide letter spacing and capitals, a small label in the style of an instrument.

Why it matters. Every declaration has a job, and none of them is there "because it looked nice". When you can read a stylesheet like this, you can also delete from it without fear.

Practice Problems A: Selectors, the Cascade and the Box

Review Problems

1. Rank by specificity, least to most: `p`, `.a .b`, `#x`, `p.a`, `a:hover`.
2. Which element does `.nav > li + li a` match, in words?
3. Two rules apply to a link: `nav a { color: blue }` and `.menu-link { color: red }`, in that order. What colour is a link with `class="menu-link"` inside `<nav>`, and why?
4. Give the computed `font-size` of a `<p>` set to `1.25em` inside a `<section>` set to `1.2rem`, with a 16 px root.
5. A paragraph is 300 px wide with `padding: 20px; border: 2px solid`. With `border-box` what is its content width? With `content-box` what is its total width?
6. Two stacked blocks have `margin-bottom: 30px` and `margin-top: 50px`. How far apart are they? And if they were flex items in a column?
7. Which of these inherit: `color`, `padding`, `font-family`, `border`, `line-height`, `background`?
8. Rewrite this using custom properties so the two colours live in one place: `h1 { color: #e11d48; } a { border-color: #e11d48; background: #fff1f2; }`.

Practice Problems B: Layout, Position and Stacking

Review Problems

1. With `grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 10px;` in a container 500 px wide, how many columns, and how wide is each?
2. A grid has columns `100px 1fr 1fr 2fr` and `gap: 16px` in 900 px. Give all four widths.
3. A flex row of four items has `gap: 12px` and each item `flex: 0 0 200px` in a 700 px container with `flex-wrap: wrap`. How many items sit on the first line?
4. An absolutely positioned tooltip appears at the top-left of the *page* instead of next to its button. What is missing, and where?
5. A table header should stay visible while the rows scroll under it. Which `position` value, and what one property must accompany it?
6. A modal has `z-index: 10000` but a sidebar with `z-index: 2` covers it. The modal is inside a `<main>` that has `opacity: 0.99`. Explain.

7. Write a rule that centres one box both ways inside a full-height parent, using Grid.
8. Which layout tool lets text wrap around an image, and why can Flexbox not do it?

Practice Problems C: Colour, Type, Motion and Organisation

Review Problems

1. Write the site's blue `#60a5fa` as `hsl()` (hue 213, saturation 94%, lightness 68%), then write a 20 percent lighter hover version and a half-transparent version.
2. Why is `line-height: 1.6` better than `line-height: 24px` on a body whose headings are different sizes?
3. A heading is `clamp(1.5rem, 4vw, 3rem)`. What size is it at viewport widths of 300, 800 and 1 400 px, with a 16 px root?
4. Where does a box end up after `transform: rotate(90deg) translate(50px)`? After `translate(50px) rotate(90deg)`?
5. Why can `display: none` not be transitioned, and what do you animate instead?
6. Write a `@media` rule that turns off a hover lift on touch devices, and one that removes all animation for visitors who asked for less motion.
7. Rewrite `#sidebar div.widget ul li a.active { color: red !important }` as a flat class, and explain what was wrong with the original.
8. Put these in the order a well-organised stylesheet lists them: components, reset, tokens, utilities, layout, media queries.

Mini Project: Style the Robot Page

Mini Project: From document to design

Take the `patrick.html` you wrote in Chapter 9 and give it a stylesheet, `patrick.css`, with these requirements:

1. A `:root` block with at least six tokens: two backgrounds, two text colours, an accent, a radius, and a spacing scale. No raw colour anywhere else.
2. A reset with `border-box`, an `img` rule that prevents overflow, and body type in `rem` with a unitless line height and a `max-width` in `ch` on the text column.
3. The header as a flex row; the main area as a grid that becomes two columns (content and the parts list) at 800 px and above.
4. The figure's caption styled with `::before` to carry a small accent mark, and the table with zebra rows using `:nth-child(even)`, a sticky header row, and right-aligned numbers.
5. A hover and focus-visible style on the nav links that uses a transition on `transform` and `color`, disabled under `(hover: none)` and under reduced motion.

6. The `<details>` given a small open-and-close transition on its content using `opacity` and `transform`.
7. A print stylesheet (`@media print`) that hides the nav and the form and sets the text to black on white.

Checks: the Styles panel shows no crossed-out declarations in your own rules (no fights); the page never scrolls sideways at 320 px; every number in the file is either a token or has a comment saying where it comes from; Lighthouse's accessibility score does not drop from the unstyled page.

What You Learned

- The cascade picks one value per property: importance, then specificity (a, b, c), then order. Inheritance fills the gaps, then units are computed. The Styles panel shows every loser crossed out.
- Keep selectors flat and classes reusable; read compound selectors right to left; `:is()`, `:where()`, `:has()` and structural pseudo-classes replace a lot of markup and script.
- Boxes: `border-box`, collapsing vertical margins, `display` as the kind of box, overflow as a deliberate choice.
- Normal flow for text, Flexbox for one axis, Grid for two. Free space is shared by `flex-grow` or `fr`; `auto-fit` with `minmax()` makes columns without queries.
- `position` takes a box out of the flow's rules; `z-index` works only on positioned boxes and only within a stacking context.
- Colour in HSL, type in `rem` with unitless line height and a `ch` measure, spacing from a scale, all stored as tokens named by role.
- Motion is `transform` and `opacity`, shaped by a timing curve; keyframes for things that move on their own.
- Mobile first, few breakpoints chosen by the content, fluid before queries, and queries for hover, motion and colour scheme too.
- Organise: tokens, reset, layout, components, utilities, queries. Name by role. Comment the why.

Chapter 11. JavaScript: The Behaviour

JavaScript is a real programming language, and the only one that browsers run. This chapter is a complete course in it, from a first value to modules, classes, promises and tests, written for someone who has never programmed and paced so that someone who has can skim. Every program in it lives in the folder `Guide/code/js` and was run with Node.js to produce the output you see. Run them yourself; change them; break them. That is how the language goes from the page into your hands.

In This Chapter

- Run JavaScript in the browser console and in Node.js, and read its errors.
- Use every basic type correctly, including the floating-point and conversion traps.
- Declare variables with the right scope, write functions, closures and pure functions.
- Work with arrays and objects fluently: map, filter, reduce, destructuring, spread, Map and Set.
- Write classes, understand the prototype chain and `this`, and handle errors.
- Change the page through the DOM, and respond to events.
- Write asynchronous code with promises and `async`/`await`, and predict its order.
- Split code into modules, work with JSON and CSV, and test what you write.

Words You Will Meet

Word	What it means
Value / type	A piece of data and its kind: number, string, boolean, `null`, `undefined`, object, function.
Variable	A named box holding a value. `const` boxes cannot be re-pointed; `let` boxes can.
Scope	The region of code where a name can be seen.
Expression / statement	Something that produces a value (`a + 1`) / something that does a thing (`if`, `for`).
Function	A named, reusable piece of code that takes inputs (parameters) and may return an output.
Argument	The actual value you pass when calling a function.
Closure	A function that remembers the variables around where it was created.
Pure function	One whose result depends only on its inputs and which changes nothing outside itself.
Callback	A function you pass to something else to be called later.
Array / object	An ordered list of values / a collection of named values (properties).
Reference	The way objects are shared: two names can point at the same object.
Prototype	The object a given object inherits properties from.
Promise	A value that will arrive later, or an error that will.
Module	A file that shares things with `export` and receives them with `import`.
DOM	The browser's tree of the page, which scripts read and change (Chapter 2).

11.1 Running JavaScript

You can run JavaScript in three places, and you will use all three.

The browser console. Press F12, choose Console, type `2 + 2` and press Enter. The console prints `4`. It is a calculator with the whole page attached: type `document.title` and you get the page's title. Every snippet in this chapter that does not touch files can be pasted there.

A script in a page. `<script src="app.js" defer></script>` in the head (Chapter 2). `console.log(...)` prints to the console. This is where your pages' behaviour lives.

Node.js. The same language, outside the browser, with files and a command line instead of a page. Install it from its website, then in a terminal: `node s02_types.js`. There is no `document` in Node, but everything else in this chapter works, which is why the examples are run there: the output can be copied into the book exactly.

> **WHY DO IT THIS WAY? Learning with Node as well as the browser**
>
> **What it is.** Running the examples as files from the command line, not only in the console.
>
> **Why it is done this way.** A file can be saved, re-run, and tested. The console forgets.
>
> **Why it matters.** The habits of the whole chapter, small files, printed checks, a test harness, only make sense with files.
>
> **What happens if you do not.** You learn the language as a series of one-liners and never build anything that is bigger than the console's input box.

11.2 Values and Types

JavaScript has seven kinds of value you will meet daily: **number**, **string**, **boolean**, `undefined`, `null`, **object** (which includes arrays and dates) and **function**. (There are also `bigint` and `symbol`, which you can leave for later.) The `typeof` operator tells you which, with one historic wrinkle.

```
console.log(typeof 42, typeof 'hi', typeof true, typeof undefined, typeof null, type
      of {}, typeof [], typeof (() => 1));
// number string boolean undefined object object object function
```

`typeof null` is `'object'`, a mistake from 1995 that can never be fixed. Test for null with `=== null`, and for an array with `Array.isArray(x)`.

Numbers

There is one kind of number: a 64-bit floating-point value. It holds integers exactly up to 9 007 199 254 740 991 (`Number.MAX_SAFE_INTEGER`), and decimals *approximately*. This is not a JavaScript quirk; it is how every language that uses binary floating point behaves, and C++'s `double` is the same number.

```
console.log(0.1 + 0.2, 0.1 + 0.2 === 0.3, Math.abs(0.1 + 0.2 - 0.3) < 1e-9);
// 0.30000000000000004 false true
console.log(22 * 1.15, (22 * 1.15).toFixed(2));
// 25.299999999999997 25.30
console.log(1 / 0, -1 / 0, 0 / 0, Number.isNaN(0 / 0));
// Infinity -Infinity NaN true
console.log(Number.MAX_SAFE_INTEGER, 2 ** 53 + 1);
// 9007199254740991 9007199254740992
```

> **WARNING**
>
> Never compare decimals with ===. Compare the *difference* with a small tolerance, as in the third expression above. Round numbers for display with `toFixed`, and only for display: `toFixed` returns a *string*. Dividing by zero does not throw; it gives `Infinity`, and `0 / 0` gives `NaN`, "not a number", which is the only value in the language that is not equal to itself.

Strings, booleans and the two empties

A string is text in single quotes, double quotes, or backticks (which allow `${...}` interpolation and line breaks, see 11.4). A boolean is `true` or `false`. `undefined` is the value of a variable that has been declared but not set, of a missing property, and of a function that returns nothing. `null` is the value *you* put somewhere to say "nothing here, on purpose". The distinction is a convention, but it is a useful one: `undefined` means "nobody set this", `null` means "somebody set this to nothing".

Conversion, and why === exists

JavaScript converts between types on its own when an operator mixes them, by rules that are consistent and surprising.

```
console.log('5' + 3, '5' - 3, '5' * '2', true + 1, [] + {});
// 53 2 10 2 [object Object]
console.log(0 == '', 0 === '', null == undefined, null === undefined, '1' == 1, NaN
      == NaN);
// true false true false true false
```

`+` with a string on either side joins strings; every other arithmetic operator converts to numbers. The loose equality `==` converts before comparing, so `0 == ''` is true. The strict equality `===` does not convert, so it says what you mean.

> **WHY DO IT THIS WAY? Using === instead of ==**
>
> **What it is.** `===` compares without converting types. `==` converts first.
>
> **Why it is done this way.** Conversion rules are intricate and surprising.
>
> **Why it matters.** With `==`, `0 == ''` is `true` and `null == undefined` is `true`. With `===`, `0 === ''` is `false`.
>
> **What happens if you do not.** You get bugs that depend on what type the data happened to be, and they appear only for some inputs. The one accepted use of `==` is `x == null`, which is true for both `null` and `undefined` and nothing else.

Truthiness. In an `if`, exactly six values count as false: `false`, `0`, `''`, `null`, `undefined`, `NaN`. Everything else counts as true, *including* an empty array and an empty object.

```
0:F "":F null:F undefined:F null:F false:F []:T {}:T "0":T -1:T
```

(The second `null` in that line is how `JSON.stringify` prints `NaN`.) The string `'0'` is true, because it is a non-empty string. Test an array for emptiness with `arr.length === 0`, never with `if (arr)`.

11.3 Variables and Scope

```
const speed = 0.3;          // cannot be reassigned
let count = 0;              // can be
count = count + 1;
```

> **WHY DO IT THIS WAY? const by default, let when you must**
>
> **What it is.** `const` makes a name that cannot be pointed at something else. `let` makes one that can. (There is also `var`, an older keyword you should leave alone.)

> **Why it is done this way.** Most values in a program never need to change. Saying so up front is information.
>
> **Why it matters.** When you read `const`, you know the name always means the same thing, so there is less to keep in your head. The language also stops you from changing it by accident.
>
> **What happens if you do not.** With `var`, names leak out of blocks and can be re-declared silently, and bugs hide in the leaks. Note that `const` stops *reassigning*, not *changing*: a `const` array can still have items pushed onto it.

`let` and `const` are **block scoped**: a name declared inside `{ ... }` exists only there. `var` is **function scoped** and leaks out of blocks. `var` declarations are also **hoisted**: the name exists from the top of the function with the value `undefined` until its line runs. `let` and `const` are hoisted too, but using them before their line is an error (the "temporal dead zone"), which is far better than a silent `undefined`. The program `s03_scope.js` shows every case:

```
1: TypeError - Assignment to constant variable.
2: undefined string           <- inner (let) is gone after the block; leaky (var) i
      s not
3: undefined                  <- var hoisted: the name exists, the value does not y
      et
4: ReferenceError - Cannot access 'tdz' before initialization
5: [ 3, 3, 3 ] [ 0, 1, 2 ]    <- closures over var share one i; over let, each gets
      its own
6: [ 'motor', 'wheel' ]       <- const stops reassignment, not change
```

Line 5 is the classic. Three functions made in a `var` loop all see the final `i`, because there is one `i`. Three made in a `let` loop each see their own, because `let` makes a fresh binding every time round. The Advanced JavaScript chapter in Part IV returns to this.

11.4 Operators and Expressions

Arithmetic is what you expect, with three things to know. `%` is the remainder and keeps the sign of the left operand, so `-7 % 3` is `-1`. `**` is power. `/` always gives a decimal; use `Math.floor` or `Math.trunc` for integer division (they differ for negatives).

```
1: 1 -1 1024 3.5 3 -3         <- 7 % 3, -7 % 3, 2 ** 10, 7 / 2, floor(7/2), trunc(-
      7/2)
2: 5 7 7                      <- a = n++ gives the old value; b = ++n gives the new
      one
```

Template literals build strings from values without +: `` duty ${duty * 100}% -> ${(duty * 4.6).toFixed(2)} V `` gives duty 70% -> 3.22 V. Anything inside `${}` is an expression.

Three short operators replace a lot of `if`s. The **ternary** `a ? b : c` is an `if` that produces a value. The **nullish** operator `??` gives a default only when the left side is `null` or `undefined`, where `||` gives the default for *any* false value, including `0` and `''`. **Optional chaining** `?.` stops at the first missing link instead of throwing.

```
const settings = { speed: 0, name: '' };
settings.speed || 0.5   // 0.5  (wrong: 0 was a real setting)
settings.speed ?? 0.5   // 0    (right)
settings.name || 'anon' // 'anon'
settings.name ?? 'anon' // ''   (the empty string was set on purpose)
```

```
robot.arm?.claw?.jaw           // 0.8
robot.sensor?.range            // undefined, no error
robot.sensor?.range ?? 'none'  // 'none'
```

> **WHY DO IT THIS WAY? ?? instead of || for defaults**
>
> **What it is.** `value ?? fallback`.
>
> **Why it is done this way.** `||` treats `0`, `''` and `false` as missing. They are often real values.
>
> **Why it matters.** A duty cycle of `0`, an empty label, a switched-off flag all survive.
>
> **What happens if you do not.** A slider set to zero mysteriously jumps to its default, and the bug only appears at zero.

`&&` and `||` **short-circuit**: `a && b` stops at `a` if `a` is false and gives `a`; `a || b` stops at `a` if it is true. They return one of their operands, not necessarily a boolean, which is why `0 || 'fallback'` is `'fallback'`.

11.5 Control Flow

`if`/`else if`/`else`, `switch`, `for`, `for...of`, `while` and `do...while` are all here, and they behave as in C++ (Chapter 17), with one twist each. The program `s05_flow.js` uses them to walk a robot through its phases:

```
const readings = [0.32, 0.21, 0.108, 0.09, 0.095];
let phase = 'APPROACH';
for (const r of readings) {
    if (phase === 'APPROACH' && r <= 0.108) phase = 'BRAKE';
    console.log(`reading ${r.toFixed(3)} m -> ${phase}`);
}
```

```
reading 0.320 m -> APPROACH
reading 0.210 m -> APPROACH
reading 0.108 m -> BRAKE
reading 0.090 m -> BRAKE
reading 0.095 m -> BRAKE
```

- `for...of` walks the *values* of anything iterable: arrays, strings, Maps, Sets. Prefer it to the index loop whenever you do not need the index.
- `for...in` walks the *keys* of an object (as strings). Do not use it on arrays; it walks their indexes as strings and any extra properties too. `Object.entries(obj)` with `for...of` is usually clearer.
- `switch` compares with `===` and falls through from case to case unless you `return` or `break`. The trick `switch (true)` with `case code >= 500:` turns it into a readable chain of ranges.
- `break` leaves the loop, `continue` skips to the next turn, and a **label** (`outer:`) lets either apply to an outer loop, as in the prime sieve below.
- `while` runs while a condition holds; `do...while` runs at least once.

```
const primes = [];
outer: for (let n = 2; primes.length < 8; n++) {
    for (const p of primes) if (n % p === 0) continue outer;
    primes.push(n);
}
```

```
// [ 2, 3, 5, 7, 11, 13, 17, 19 ]
```

Worked Example: A crude simulation in a while loop

A cart starts at rest with an acceleration of 2 m/s². Using steps of 0.1 s, how long until it has travelled 1 m, and how fast is it then?

Step 1. The loop. Each turn adds `2 × 0.1` to the speed and `v × 0.1` to the position, then adds 0.1 to the clock, until x ≥ 1.

```
let t = 0, v = 0, x = 0;
while (x < 1) { v += 2 * 0.1; x += v * 0.1; t += 0.1; }
// reached 1 m after 1.0 s at 2.00 m/s
```

Step 2. Check against physics. x = ½at² gives t = √(2 × 1 / 2) = 1.0 s, and v = at = 2.0 m/s. √

Step 3. Why it is "crude". The loop updates the speed first and then uses the *new* speed for the whole step, so it overshoots the position a little every step. At t = 1.0 the computed x is actually 1.1 m, ten percent too far; the loop reports 1.0 s only because that is the first step at which x has passed 1 m. Chapter 17's section on simulation numerics explains the proper remedy (smaller steps, or a better integrator); the point here is that a `while` loop with a clock *is* a simulation, and that you should always check one number against the physics.

11.6 Functions

A function packages up a job so you can reuse it. There are three ways to write one, and they are nearly interchangeable.

```
function clamp(x, lo = 0, hi = 1) { return Math.min(hi, Math.max(lo, x)); }   // dec
      laration, with defaults
const double = n => n * 2;                                                    // arr
      ow function, short form
const area = function (r) { return Math.PI * r * r; };                        // fun
      ction expression
```

Defaults fill in missing arguments. A **rest** parameter `...nums` collects any number of arguments into an array, and **spread** `...arr` does the reverse at the call. A **declaration** is hoisted (usable before its line); an expression or arrow is not.

```
1: 1 -1 0.4 42                <- clamp(1.7), clamp(-3, -1, 1), clamp(0.4), double(2
      1)
2: 0 6 15                     <- sum(), sum(1, 2, 3), sum(...[4, 5, 6])
7: function undefined         <- a declaration is hoisted; a var holding a function
      is not yet set
```

Closures

A function remembers the variables where it was born. That is a **closure**, and it is how JavaScript keeps private state without classes.

```
function makeCounter() { let n = 0; return () => ++n; }
const c1 = makeCounter(), c2 = makeCounter();
console.log(c1(), c1(), c2());     // 1 2 1: each counter has its own n
```

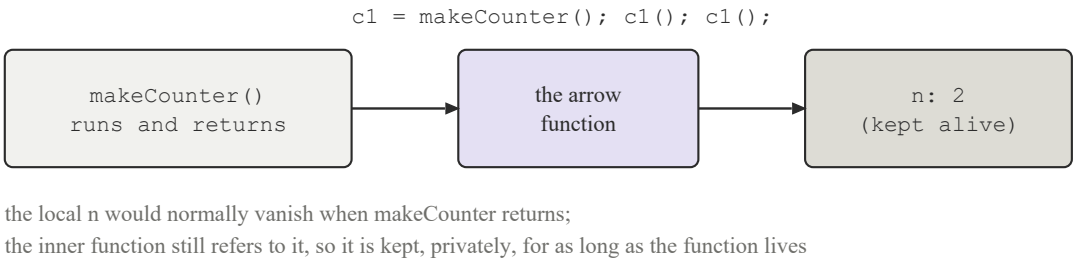


Figure 11.1. The inner function keeps `n` alive after `makeCounter` has returned. Each call to `makeCounter` makes a new `n`, so `c1` and `c2` count separately.

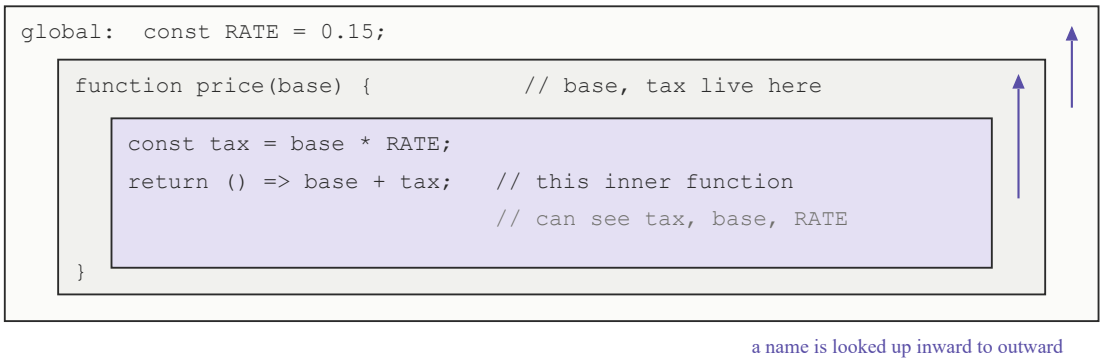


Figure 11.2. Scope is lexical: a name is looked up in the block where the code was written, then in the enclosing function, then outward to the module or global scope. The inner arrow function can see `tax`, `base` and `RATE`; nothing outside can see `tax`.

A PID controller is a closure in disguise: it has to remember the integral and the last error between calls, and a closure is the lightest way to do that.

```
const pid = (kp, ki, kd) => {
    let integral = 0, last = 0;
    return (err, dt) => { integral += err * dt; const d = (err - last) / dt; last =
      err;
                          return kp * err + ki * integral + kd * d; };
};
const ctrl = pid(2, 0.5, 0.1);
[1, 0.5, 0.2].map(e => ctrl(e, 0.1).toFixed(3));   // [ '3.050', '0.575', '0.185' ]
```

Check the first value by hand: integral = 0.1, derivative = (1 − 0) / 0.1 = 10, so 2 × 1 + 0.5 × 0.1 + 0.1 × 10 = 3.05. √

Pure functions and higher-order functions

A **pure** function depends only on its inputs and changes nothing outside itself. `addPure(0, 2)` is 2 every time; `addImpure(2)` gave 2 the first time and 4 the second, because it reads and writes a variable outside.

```
6: 2 4 2 2
```

> **WHY DO IT THIS WAY? Small functions with one job**
>
> **What it is.** `clamp` does one thing and returns a value. It does not touch anything outside itself.

> **Why it is done this way.** A pure function always gives the same answer for the same inputs.
>
> **Why it matters.** You can test it by itself, reuse it anywhere, and understand it without reading the rest of the program. On this website, helpers like `clamp` and the chart `fit` are used in dozens of places.
>
> **What happens if you do not.** A function that reads and changes global variables behaves differently depending on the history of the program, and that is the hardest kind of bug to find.

A **higher-order** function takes or returns a function. `twice(f, x)` applies `f` two times: `twice(double, 3)` is 12. **Recursion** is a function calling itself with a smaller problem: `factorial(5)` is 120. Both are everyday tools in JavaScript, and the next section is built on the first.

11.7 Arrays

An array is a list: `const prices = [4, 9, 15, 22];`. Three methods do most of the work, and each takes a function.

```
prices.map(p => p * 1.15);               // a new array: each item transformed
prices.filter(p => p > 10);              // a new array: only the items that pass
prices.reduce((sum, p) => sum + p, 0);   // one value: all the items combined
// [ 4.6, 10.35, 17.25, 25.299999999999997 ] [ 15, 22 ] 50
```

> **WHY DO IT THIS WAY? map, filter and reduce instead of loops**
>
> **What it is.** Describing *what* you want (transform every item, keep some, combine all) rather than *how* (a counter, an index, a temporary array).
>
> **Why it is done this way.** The intention is visible in the method's name.
>
> **Why it matters.** There is less code, no index to get wrong, and the original array is left untouched.
>
> **What happens if you do not.** A hand-written loop with an index is where "off by one" bugs live. (Use a loop when you genuinely need one, for example to stop early.)

The rest of the toolkit, from `s07_arrays.js`:

Call	Gives	Changes the array?
`find(f)`, `findIndex(f)`	the first item (or index) that passes: `15`, `2`	no
`some(f)`, `every(f)`, `includes(v)`	`true`/`false`	no
`slice(1, 3)`	a copy of part: `[2, 3]`	no
`splice(1, 2)`	removes items in place and returns them	**yes**
`sort()`	sorted in place, **as strings** by default	**yes**
`toSorted(f)`, `toReversed()`	sorted copies (new in 2023)	no
`flat()`, `flat(Infinity)`, `flatMap(f)`	nested arrays unpacked	no

Call	Gives	Changes the array?
`at(-1)`	the last item	no
`Array.from({length: n}, (_, i) => ...)`	an array built from a rule	makes a new one

```
3: [ 1, 10, 100, 9 ] [ 1, 9, 10, 100 ]     <- sort() compares as text; give it a com
      parator
4: [ 2, 3 ] [ 1, 4, 5 ] [ 2, 3 ] [ 1, 4, 5 ] <- slice copies; splice cuts
```

> **WARNING**
>
> `[10, 9, 1, 100].sort()` gives `[1, 10, 100, 9]`, because the default sort compares strings. Always pass a comparator for numbers: `sort((a, b) => a - b)`. And `sort`, `splice`, `push`, `pop`, `shift`, `unshift` and `reverse` all change the array in place; the others return a new one.

Destructuring pulls values out, and **spread** copies:

```
const [first, , third = 'none', ...rest] = ['a', 'b', undefined, 'd', 'e'];   // a n
      one [ 'd', 'e' ]
const copy = [...prices, 30], same = prices;
copy.push(99); same.push(99);
// prices is [ 4, 9, 15, 22, 99 ]; copy is [ 4, 9, 15, 22, 30, 99 ]; copy === prices
      false; same === prices true
```

> **WHY DO IT THIS WAY? Copying with spread instead of changing in place**
>
> **What it is.** Making a *new* array or object instead of altering the existing one.
>
> **Why it is done this way.** Objects and arrays are shared by reference: two names can point at the same thing.
>
> **Why it matters.** Changing `o1.n` also changes `o2.n` if `o2 = o1`. Copying avoids surprises, and it is the foundation of how React notices changes (Chapter 12).
>
> **What happens if you do not.** A change made in one corner of the program silently shows up in another.

Worked Example: Grouping with reduce

Given `people = [{name:'Ana',age:31},{name:'Ben',age:17},{name:'Cy',age:45}]`, group the names by decade of age.

Step 1. What is the result? An object whose keys are decades and whose values are arrays of names: `{ 10: ['Ben'], 30: ['Ana'], 40: ['Cy'] }`. One value from many items means `reduce`.

Step 2. The accumulator. Start with `{}`. For each person, find the decade with `Math.floor(age / 10) * 10`, make sure that key holds an array, and push the name.

```
people.reduce((acc, p) => { const d = Math.floor(p.age / 10) * 10; (acc[d] ??= []).p
      ush(p.name); return acc; }, {});
// { '10': [ 'Ben' ], '30': [ 'Ana' ], '40': [ 'Cy' ] }
```

Step 3. The idiom. `acc[d] ??= []` means "if `acc[d]` is missing, set it to an empty array", then the result is pushed onto. Object keys are always strings, which is why the output shows `'10'`.

Why it matters. Grouping, counting, indexing by id, and summing by category are all this one pattern with a different accumulator.

11.8 Objects, Map and Set

An object is a bundle of named values. Properties are read with a dot or with brackets (which allow any string, and computed names). A function stored in an object is a **method**, and inside it `this` is the object it was called on.

```
const motor = { name: 'TT', rpm: 200, 'stall torque': 0.08, spin() { return `${this.
      name} spins at ${this.rpm} rpm`; } };
motor.rpm, motor['stall torque'], motor.spin(), 'rpm' in motor, motor.volts
// 200 0.08 TT spins at 200 rpm true undefined
```

Destructuring with renaming and defaults, **spread** to copy-and-change, and the `Object` helpers:

```
const { name, rpm: speed, volts = 6 } = motor;          // TT 200 6
const faster = { ...motor, rpm: 260 };                   // a new object; motor.rpm
      is still 200
Object.keys(motor)      // [ 'name', 'rpm', 'stall torque', 'spin' ]
Object.entries({a:1,b:2})   // [ [ 'a', 1 ], [ 'b', 2 ] ]
Object.fromEntries(pairs)   // the reverse
```

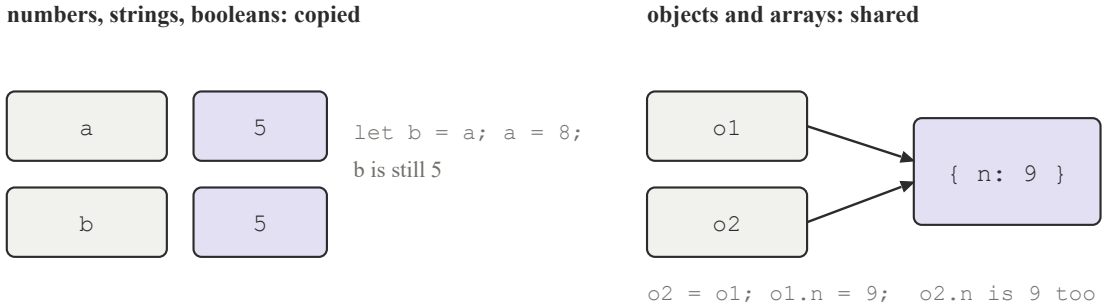


Figure 11.3. Numbers, strings and booleans are copied when assigned; objects and arrays are shared. `o2 = o1` makes two names for one object, and a change through either name is seen through both.

```
const o1 = { n: 1 }, o2 = o1, o3 = { ...o1 };
o1.n = 9;
// o2.n is 9, o3.n is 1, o1 === o2 true, o1 === o3 false
```

Spread makes a **shallow** copy: one level deep. Nested objects are still shared. `structuredClone` makes a deep copy:

```
const nested = { arm: { len: 0.24 } }, shallow = { ...nested }, deep = structuredClo
      ne(nested);
nested.arm.len = 0.3;
// shallow.arm.len is 0.3 (shared), deep.arm.len is 0.24 (copied)
```

JSON is the text form of objects, used for files, storage and the network. `JSON.stringify` drops `undefined` values and functions; `JSON.parse` brings the text back.

```
JSON.stringify({ id: 3, tags: ['aero'], when: undefined, f: () => 1 })   // {"id":3,
```

```
      "tags":["aero"]}
```

Map is an object whose keys can be anything (not only strings), kept in insertion order, with `size`, `get`, `set`, `has`, `delete`. **Set** is a collection of unique values. Both are iterable.

```
const m = new Map([['speed', 0.3]]); m.set('power', 1.2).set(motor, 'an object as a
      key');
m.get('speed'), m.has('power'), m.size, m.get(motor)       // 0.3 true 3 an object a
      s a key
[...new Set([1, 2, 2, 3, 3, 3])], [...new Set('mississippi')].join('')   // [ 1, 2,
      3 ] misp
```

`Object.freeze(obj)` stops an object changing (silently outside strict mode, with an error inside it).

> **WHY DO IT THIS WAY? A Map when the keys are not known in advance**
>
> **What it is.** `new Map()` for a lookup table whose keys come from data; a plain object for a fixed record with known field names.
>
> **Why it is done this way.** Object keys are always strings and an object has inherited properties (`toString` and friends) that can collide with data keys. A Map has neither problem and keeps order.
>
> **Why it matters.** `map.get(key)` is never confused by a key called `constructor`, and `map.size` is free.
>
> **What happens if you do not.** A lookup table built as an object works until a key happens to be `"toString"`, and then it returns a function.

11.9 Strings and Numbers in Practice

Strings are immutable: every method returns a new string. The ones you will use, from `s09_strings_numbers.js`:

```
const s = 'Patrick the Robot';
s.length, s[0], s.at(-1), s.toUpperCase(), s.slice(8), s.indexOf('Robot'), s.include
      s('the')
// 17 P t PATRICK THE ROBOT the Robot 12 true
s.split(' '), s.replace('Robot', 'Cook'), s.replaceAll('t', 'T'), '  pad  '.trim(),
      'ab'.repeat(3)
// [ 'Patrick', 'the', 'Robot' ] Patrick the Cook PaTrick The RoboT pad ababab
'7'.padStart(3, '0'), 'x'.padEnd(4, '.') + '|'      // 007 x...|
```

Turning text into numbers has four tools with different strictness:

```
Number('42'), Number('12px'), parseFloat('12.5px'), parseInt('0x1f', 16), +'3.5', Nu
      mber(''), Number(null), Number(undefined)
// 42 NaN 12.5 31 3.5 0 0 NaN
```

`Number()` and unary `+` want the whole string to be a number; `parseFloat` reads as far as it can. `Number('')` is `0`, which is a trap for empty form fields: check for emptiness first.

Formatting for display:

```
(3.14159).toFixed(2), (0.000123).toExponential(2), (1234.5678).toPrecision(6), Math.
      round(2.5), Math.round(-2.5), Math.round(3.456 * 100) / 100
// 3.14 1.23e-4 1234.57 3 -2 3.46
(1234567.891).toLocaleString('en-US')                                          // 1,
```

```
      234,567.891
new Intl.NumberFormat('en-US', { style: 'unit', unit: 'meter', maximumFractionDigits
      : 1 }).format(0.1083)   // 0.1 m
```

`Math.round` rounds halves *up* (towards +∞), so `-2.5` becomes `-2`. To round to two decimals, multiply, round, divide. `Math.max(...arr)` spreads an array into arguments; `Math.hypot(3, 4)` is 5; `Math.atan2(y, x)` gives an angle with the right quadrant.

Comparing strings compares character codes, so capitals sort before lowercase and `'10' < '9'` is true. For human order use `localeCompare`: `['b', 'a', 'C'].sort((x, y) => x.localeCompare(y))` gives `[ 'a', 'b', 'C' ]`.

Worked Example: Parsing one line of a log

A line of a CSV log reads `0.100,0.321,APPROACH`. Pull out the time plus one millisecond, whether the range is under the brake threshold, and the phase in lowercase.

Step 1. Split. `const [t, range, phase] = csvLine.split(',')` gives three *strings*.

Step 2. Convert where arithmetic is needed. `+t + 0.001` is `0.101`; `Number(range) < 0.108` is `false` (0.321 is not under).

Step 3. Leave text as text. `phase.toLowerCase()` is `'approach'`.

The trap. `t + 0.001` without the `+` would be `'0.1000.001'`, a string. Every value that comes from text, a form field, a URL or a file is a string until you convert it, and the conversion is where you check it.

11.10 Classes and Prototypes

A **class** bundles data and the functions that work on it. The syntax will look familiar from C++ (Chapter 17); the machinery underneath is different, and worth knowing.

```
class Motor {
    static count = 0;
    #rpm = 0;                                   // private field
    constructor(name, maxRpm = 200) { this.name = name; this.maxRpm = maxRpm; Motor.
      count++; }
    set duty(d) { this.#rpm = Math.min(1, Math.max(0, d)) * this.maxRpm; }
    get rpm() { return this.#rpm; }
    describe() { return `${this.name}: ${this.rpm.toFixed(0)} rpm`; }
    static fromSpec(spec) { return new Motor(spec.name, spec.max); }
}
class GearedMotor extends Motor {
    constructor(name, ratio) { super(name, 200 / ratio * 48); this.ratio = ratio; }
    describe() { return super.describe() + ` through 1:${this.ratio}`; }
}
```

- The **constructor** runs on `new`. **Fields** declared in the body are per-instance; `static` ones belong to the class.
- A **getter** (`get rpm()`) is read like a property; a **setter** (`set duty(d)`) is assigned like one. Here `m.duty = 0.7` clamps and converts, and `m.rpm` reads the result.

- A `#private` field is invisible outside the class, even to `JSON.stringify`. Reading `g.#rpm` from outside is a *syntax error*, caught before the program runs.
- `extends` makes a subclass; `super(...)` must be called in its constructor before `this` is used; `super.describe()` calls the parent's version.

```
1: left: 140 rpm | arm: 100 rpm through 1:96 | 2
2: true true true true
3: [ 'name', 'maxRpm', 'ratio' ] function false true
4: {"name":"arm","maxRpm":100,"ratio":96} (private fields are not visible, even to J
      SON)
5: spare: 0 rpm
6: plain: 12 rpm
7: TypeError - this is undefined when a method is called loose
8: left: 140 rpm [ 'left: 140 rpm', 'arm: 100 rpm through 1:96' ]
```

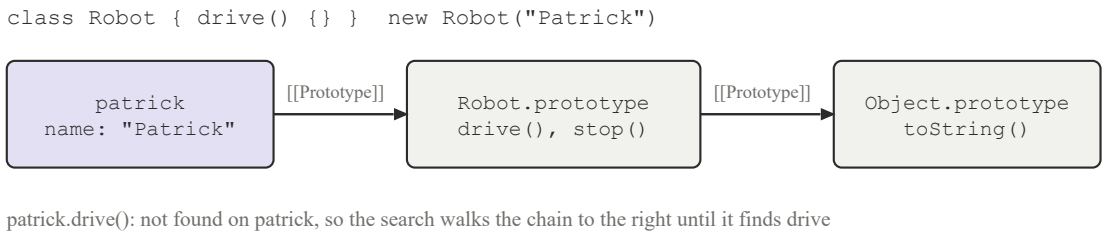


Figure 11.4. Methods are not copied into each object. They live on the class's prototype, and a property lookup that fails on the object walks up the chain until it finds the name or runs out.

Line 3 shows this: `Object.keys(g)` lists only the instance's own data; `describe` is a function reachable through `g` (`'describe' in g` is true) but not its own property. Line 6 shows the other side: a plain object given the same function works, because `this` is simply whatever the method was called on. And line 7 shows the price: a method pulled off its object and called on its own has no `this`, and throws. `bind` fixes a function to an object; an arrow function wrapper (`x => x.describe()`) is often simpler.

> **WHY DO IT THIS WAY? Classes for things with identity and behaviour, plain objects for data**
>
> **What it is.** `class Motor` for something that has a name, state that changes, and methods. `{ t: 0.1, range: 0.32 }` for a record read from a file.
>
> **Why it is done this way.** A class is worth its ceremony when there is behaviour to attach and invariants to protect (the clamp in the setter). A record needs neither.
>
> **Why it matters.** Data stays easy to copy, compare, serialise and send; behaviour stays in one place.
>
> **What happens if you do not.** Either every record becomes a class instance that cannot be `JSON.stringify`d without losing its methods, or every object with behaviour is a bag of fields that any code can corrupt.

11.11 Errors and Debugging

When something goes wrong, JavaScript **throws** an error, which unwinds the call stack until a `try`/`catch` catches it or the program dies with the message in the console. You throw your own with `throw new Error('...')`, and you can define kinds of error by extending `Error`.

```
class RangeOutOfBounds extends Error {
    constructor(value) { super(`sonar reading ${value} m is outside 0.02 to 4 m`); t
      his.name = 'RangeOutOfBounds'; this.value = value; }
}
function readSonar(v) {
    if (typeof v !== 'number' || Number.isNaN(v)) throw new TypeError(`expected a nu
      mber, got ${typeof v}`);
    if (v < 0.02 || v > 4) throw new RangeOutOfBounds(v);
    return v;
}
for (const input of [0.3, 'far', 7]) {
    try { console.log('ok', readSonar(input)); }
    catch (e) {
        if (e instanceof RangeOutOfBounds) console.log('range:', e.message, '(value'
      , e.value + ')');
        else if (e instanceof TypeError) console.log('type:', e.message);
        else throw e;                                    // not ours: let it go up
    } finally { console.log('  (finally runs every time)'); }
}

ok 0.3
  (finally runs every time)
type: expected a number, got string
  (finally runs every time)
range: sonar reading 7 m is outside 0.02 to 4 m (value 7)
  (finally runs every time)
TypeError: Cannot read properties of null (reading 'length')
ReferenceError: undefinedFunction is not defined
SyntaxError: Expected property name or '}' in JSON at
Error: where am I? | true
Assertion failed: this assertion fails and prints, but does not stop the program
still running
```

The three built-in messages above are the ones you will see most. `Cannot read properties of null` means you called a method on something that was not there, almost always a `querySelector` that matched nothing. `is not defined` is a misspelled name. `SyntaxError` from `JSON.parse` means the text was not JSON (often an HTML error page where JSON was expected).

> **WHY DO IT THIS WAY? Catching only the errors you expect and rethrowing the rest**
>
> **What it is.** `if (e instanceof RangeOutOfBounds) ... else throw e.`
>
> **Why it is done this way.** A `catch` that swallows everything also swallows typos, null references and bugs you have not met yet.
>
> **Why it matters.** The errors you planned for are handled; the ones you did not are still loud.
>
> **What happens if you do not.** `try { ... } catch (e) {}` is the most dangerous line in the language. The program carries on in a broken state and fails somewhere far from the cause.

Every error has a `stack`: the chain of calls that led to it, with file names and line numbers. Read it top down; the first line of *your* code in it is usually where to look. Beyond `console.log`, the console has

`console.table(array)` for rows, `console.error` for red, `console.time('x')`/`console.timeEnd('x')` for timing, and `console.assert(condition, message)`, which prints only when the condition is false. The `debugger;` statement pauses the browser at that line with the developer tools open, where you can step through code and inspect every variable; it is faster than any number of logs.

11.12 The DOM in Practice

Chapters 3 and 5 introduced the DOM; here is the working vocabulary. These need a page, so run them in the console on any site.

```
const title = document.querySelector('h1');               // the first match, or nul
      l
const chips = document.querySelectorAll('.chip');          // a static NodeList: ite
      rate with for...of or forEach
title.textContent = 'Meet Patrick';                        // text, safely (Chapter
      8)
title.classList.add('on'); title.classList.toggle('big'); title.classList.contains('
      on');
title.dataset.chart                                         // the data-chart attrib
      ute
title.setAttribute('aria-expanded', 'true'); title.getAttribute('id'); title.hidden
      = true;
title.style.setProperty('--progress', 0.6);                 // a custom property, fo
      r CSS to use (Chapter 10)
const card = document.createElement('article');
card.className = 'card'; card.textContent = 'One';
grid.append(card);  grid.prepend(card);  card.remove();  grid.replaceChildren(...car
      ds);
title.closest('.panel');  title.parentElement;  title.nextElementSibling;  grid.chil
      dren;
```

Reading a control's value: `input.value` is always a *string* (so `+input.value` for numbers), `checkbox.checked` is a boolean, `select.value` is the chosen option's value. Reading geometry: `el.getBoundingClientRect()` gives its position and size in CSS pixels relative to the viewport; `el.clientWidth` and `el.offsetHeight` give sizes; reading any of them forces layout (Chapter 2), so read in batches.

Events, the short version of Chapter 5: `el.addEventListener('click', handler)`; the handler gets an `event` with `target`, `currentTarget`, `clientX`, `key`; `event.preventDefault()` stops the browser's own action (a form submitting, a link navigating); delegation puts one listener on a parent and uses `event.target.closest('.chip')`.

Worked Example: Wiring the question form from Chapter 9

The robot page's form should send its question without reloading the page, and show a thank-you line. Write the script.

Step 1. Find the form and listen for submit, not for the button's click, so that pressing Enter in a field also works.

Step 2. Stop the reload, read the fields, and build the request body with `FormData`, which collects every named control exactly as the browser would.

Step 3. Send it and report.

```
const form = document.querySelector('form');
const note = document.querySelector('#note');          // <p id="note" aria-live="po
      lite"></p>
form.addEventListener('submit', async (event) => {
    event.preventDefault();
    const body = new FormData(form);
    note.textContent = 'Sending...';
    try {
        const res = await fetch(form.action, { method: 'post', body });
        if (!res.ok) throw new Error(`server said ${res.status}`);
        note.textContent = 'Thank you. I will reply by email.';
        form.reset();
    } catch (e) {
        note.textContent = `Could not send (${e.message}). Your text is still here.`
      ;
    }
});
```

Step 4. Check the layers. With the script removed, the form still submits the old-fashioned way to `form.action`. With it, the page stays put and the `aria-live` paragraph is read aloud to screen-reader users when it changes. That is progressive enhancement, done.

11.13 Asynchronous Code: Promises and async/await

Some things take time: loading a file, waiting for the network, waiting for a timer. JavaScript never waits by stopping (Chapter 5). Instead it hands you a **promise**: an object that stands for a result that has not arrived yet.

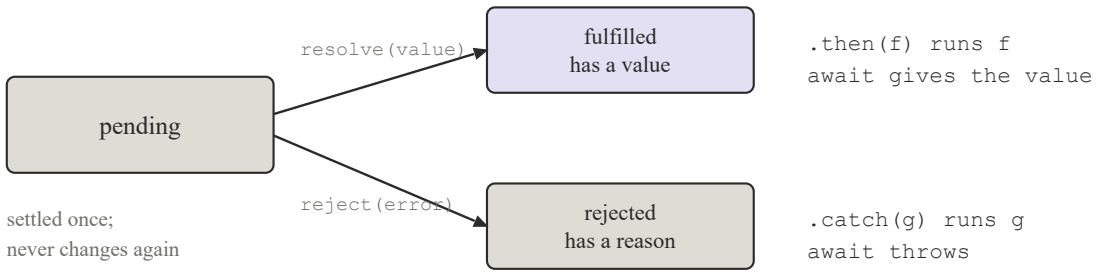


Figure 11.5. A promise starts pending and settles exactly once, either fulfilled with a value or rejected with an error. `then`, `catch` and `await` are ways to be told which.

```
const wait = (ms, value) => new Promise(resolve => setTimeout(() => resolve(value),
      ms));
wait(5, 'done').then(v => console.log('then:', v)).finally(() => console.log('finall
      y'));
```

`new Promise` takes a function with two callbacks: call `resolve(value)` when the work succeeds, `reject(error)` when it fails. `.then(f)` runs `f` with the value when the promise fulfils and returns a new promise for `f`'s result, so `then`s chain. `.catch(g)` handles a rejection anywhere earlier in the chain. `.finally(h)` runs either way.

`async` **and** `await` let you write the waiting as if it were a plain top-to-bottom story. An `async` function always returns a promise; inside it, `await p` pauses *this function* (not the page) until `p` settles, and gives its value, or throws its error, which `try`/`catch` can catch like any other.

```
async function load(url) {
    try { const data = await fakeFetch(url); return `${url}: ${data.items.length} it
      ems`; }
    catch (e) { return `${url}: failed (${e.message})`; }
}
```

> **WHY DO IT THIS WAY? await instead of nested callbacks**
>
> **What it is.** `await` pauses *this function* (not the whole page) until a promise settles.
>
> **Why it is done this way.** Without it you nest a callback inside a callback inside a callback, and error handling gets tangled.
>
> **Why it matters.** The code reads in the order things happen. The page stays responsive, because (Chapter 5) the event loop runs other things during the pause.
>
> **What happens if you do not.** You write "callback hell", and one forgotten error case makes a request fail without a trace.

The order things happen

Chapter 5's rule, applied: the current code finishes, then every microtask (promise follow-ups, the code after an `await`), then the next task (a timer). The first part of `s13_async.js` prints its lines in the order of their numbers:

```
console.log('1 start');
setTimeout(() => console.log('5 timer (task)'), 0);
Promise.resolve().then(() => console.log('3 then (microtask)'));
(async () => { console.log('2 inside async, before await'); await null; console.log(
      '4 after await (microtask)'); })();
console.log('1 end of script');
```

An `async` function runs *synchronously* until its first `await`; that is why line 2 prints before "end of script". Everything after the `await` is a microtask, so it runs before the timer, even a zero-millisecond one.

Several at once

`await` one after another is **sequential**: the second starts when the first finishes. `Promise.all([...])` starts them all and waits for all, which is **parallel** and is what you want for independent requests. It rejects as soon as any one rejects; `Promise.allSettled` waits for all and reports each; `Promise.race` gives the first to settle.

```
6 /a.json: 3 items | /b.json: 3 items | /c.json: 3 items | /d.json: 3 items
7 sequential took longer than parallel: true (69 ms vs 31 ms)
8 /bad.json: failed (404 for /bad.json)
9 [ 'fulfilled', 'rejected' ]
10 fast
11 the promise is pending here: true
12 then: done
13 finally
```

(The milliseconds on line 7 vary from run to run; the comparison does not.) Two awaited 30 ms fetches take about 60 ms; two in `Promise.all` take about 30.

fetch

`fetch(url)` returns a promise of a **response**. The response's body is read with a second promise: `await res.json()` for JSON, `await res.text()` for text. A 404 does *not* reject the promise; only a network failure does. So check `res.ok` yourself.

```
async function getJSON(url) {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`${res.status} ${res.statusText} for ${url}`);
    return res.json();
}
```

> **WHY DO IT THIS WAY? Checking res.ok before reading the body**
>
> **What it is.** An `if (!res.ok) throw` between `fetch` and `res.json()`.
>
> **Why it is done this way.** `fetch` considers any answer from the server a success, even "404 Not Found".
>
> **Why it matters.** Your error handling sees a clear message instead of a `SyntaxError` from trying to parse an HTML error page as JSON.
>
> **What happens if you do not.** The failure shows up as "Unexpected token < in JSON", three functions away from the request that caused it.

11.14 Timers, Animation Frames and Rate Limiting

`setTimeout(fn, ms)` calls `fn` once after at least `ms` milliseconds; `setInterval(fn, ms)` calls it repeatedly; both return an id for `clearTimeout`/`clearInterval`. `requestAnimationFrame(fn)` calls `fn` before the next paint and is the only right way to animate (Chapter 6). An `AbortController` lets you cancel something that is waiting: pass `controller.signal` to `fetch`, or listen for its `abort` event in your own promise.

Two patterns tame events that fire too often. **Debounce** waits until the events stop for a while, then runs once: right for resize, for search-as-you-type, for saving a draft. **Throttle** runs at most once per interval while the events keep coming: right for scroll position and pointer moves.

```
function debounce(fn, ms) { let id = null; return (...args) => { clearTimeout(id); i
      d = setTimeout(() => fn(...args), ms); }; }
function throttle(fn, ms) { let last = -Infinity; return (...args) => { const now =
      Date.now(); if (now - last >= ms) { last = now; fn(...args); } }; }
```

In `s14_timers.js` a burst of six events, 10 ms apart, hits both:

```
throttled 100
throttled 400
debounced 60
debounced ran once, after the burst; throttled ran at most once per 40 ms during it
```

The throttled handler ran on the first event and again 40 ms later; the debounced handler ran once, 40 ms after the *last* event, with the last value. Both are closures from 11.6.

11.15 Modules

A **module** is a file that exports some names and imports others. In the browser, `<script type="module">` enables it; in Node, the `.mjs` extension does; a bundler (Chapter 16) joins modules into one file for a page.

```
// s15_maths.mjs
export const clamp = (x, lo, hi) => Math.min(hi, Math.max(lo, x));
export function smoothstep(t) { t = clamp(t, 0, 1); return t * t * (3 - 2 * t); }
const secret = 'not exported';                        // module scope: invisible outs
      ide
export default function describe() { return `maths module (${secret.length} chars of
      private state)`; }

// s15_main.mjs
import describe, { clamp, lerp, smoothstep as ease } from './s15_maths.mjs';
import * as maths from './s15_maths.mjs';
const { lerp: lerp2 } = await import('./s15_maths.mjs');   // on demand; returns a p
      romise

1: 1 50 0.1563 maths module (12 chars of private state)
2: [ 'clamp', 'default', 'lerp', 'smoothstep' ] undefined
3: true same module instance
```

A module has its own scope: `secret` is invisible outside, and `maths.secret` is `undefined`. There is one **default** export per file and any number of **named** ones; `as` renames on import. A module is evaluated once, however many files import it, so `lerp2 === lerp`. Dynamic `import()` loads a module when the line runs, which is how a page loads a heavy viewer only when a visitor opens it (Part IV, Advanced JavaScript).

> **WHY DO IT THIS WAY? Splitting code into modules**
>
> **What it is.** Each file does one job and says what it shares.
>
> **Why it is done this way.** A thousand-line file is hard to navigate and impossible to reuse.
>
> **Why it matters.** Names do not collide, you can see what depends on what, and a bundler can leave out code you never import.
>
> **What happens if you do not.** Everything lives in one global space, and two scripts that both define `fit` overwrite each other. The Patrick page keeps its physics (`model.js`), its geometry (`robot.js`) and its page logic (`main.js`) apart for exactly this reason.

11.16 Working with Data

Most of a page's script is moving data about: a CSV log into an array of records, a settings object into `localStorage`, a state object into a new state object. `s16_data.js` does all three.

```
const [header, ...rows] = csv.trim().split('\n');
const cols = header.split(',');
const records = rows.map(line => Object.fromEntries(line.split(',').map((v, i) => [c
      ols[i], Number.isNaN(+v) ? v : +v])));
// records[2] is { t: 0.2, range: 0.104, phase: 'BRAKE' }
```

Split into lines, split each line into cells, pair each cell with its column name, and convert the ones that look like numbers. The reverse, `records.map(r => [...].join(',')).join('\n')`, gives back

the input exactly (line 4 of the output prints `true`).

Immutable updates. To change one field deep inside a state object without touching the old one, copy each level you pass through with spread:

```
const next = { ...state, items: [...state.items, 'salt'], pos: { ...state.pos, x: 0.
      15 } };
// state.items [ 'egg' ]; next.items [ 'egg', 'salt' ]; state.pos.x 0.1; next.pos.x
      0.15; state.pos === next.pos false
```

Storage. `localStorage.setItem(key, string)` and `getItem(key)` keep strings in the browser between visits. Store objects as JSON, and give `JSON.parse` a default for a missing key: `JSON.parse(localStorage.getItem('settings') ?? '{}')`.

```
7: 0.7 undefined        <- a saved setting, and a missing one that falls back to {}
```

11.17 Writing Code That Lasts

Everything so far is about making code work. This is about making it *keep* working, for you in six months and for anyone else.

Strict mode. Put `'use strict';` at the top of a script (modules and classes are strict automatically). It turns silent mistakes into errors: assigning to an undeclared name, writing to a frozen object, a loose `this`.

Names. `camelCase` for variables and functions, `PascalCase` for classes, `UPPER_SNAKE` for true constants. A name should say what the thing *is*; a function name should say what it *does*, usually with a verb.

Shape. Functions short enough to read without scrolling. Pure functions for calculation, a few impure ones at the edges for the DOM, the network and storage. Data flows in as arguments and out as return values.

Comments. Say why, not what. `// clamp dt so a hidden tab does not fling everything` is a comment worth having. `// increment i` is not.

A linter. ESLint reads your code and flags unused variables, missing `await`s, ==, and a hundred other things, in the editor as you type. Set it up once per project.

Tests. A test is a function that calls your code with known inputs and checks the output. You do not need a framework to start; `s17_tests.js` is a complete harness in fifteen lines:

```
let passed = 0, failed = 0;
function test(name, fn) {
    try { fn(); passed++; console.log('  ok   ', name); }
    catch (e) { failed++; console.log('  FAIL ', name, '-', e.message); }
}
const assertEq = (got, want, tol = 0) => { if (Math.abs(got - want) > tol) throw new
      Error(`got ${got}, wanted ${want}`); };

test('clamp keeps values inside', () => { assertEq(clamp(5, 0, 1), 1); assertEq(clam
      p(-2, 0, 1), 0); });
test('friction never exceeds the peak', () => { for (let s = 0; s <= 1; s += 0.01) i
      f (mu(s, 0.8) > 0.8 + 1e-12) throw new Error(`exceeded at ${s}`); });
test('a deliberate failure, to show the output', () => assertEq(0.1 + 0.2, 0.3));
test('the same test with a tolerance', () => assertEq(0.1 + 0.2, 0.3, 1e-9));
```

```
  ok    clamp keeps values inside
  ok    friction is zero with no slip
  ok    friction peaks near 17% slip
  ok    friction never exceeds the peak
  ok    ticks are round numbers
  FAIL  a deliberate failure, to show the output - got 0.30000000000000004, wanted 0
      .3
  ok    the same test with a tolerance
6 passed, 1 failed
```

> **WHY DO IT THIS WAY? Testing the pure functions first**
>
> **What it is.** Tests for `clamp`, `mu`, `niceTicks`: the functions that take numbers and return numbers.
>
> **Why it is done this way.** They need no page, no network and no setup, so the tests are fast, and they are the functions the charts and simulations depend on.
>
> **Why it matters.** When the friction curve is wrong, the test says so in a line, instead of a chart looking slightly odd.
>
> **What happens if you do not.** You find out that `niceTicks` returns 0.005 instead of 0.002 (as the author of this chapter did, from the test above) by staring at a chart with strange gridlines, weeks later.

Practice Problems A: Values, Variables and Flow

Review Problems

1. Give the result and the type of each: `typeof null`, `typeof []`, `'3' * '4'`, `'3' + 4`, `0.1 * 3`, `Number('')`, `Number('1e3')`, `[] + []`.
2. Which of these are false in an `if`: `'false'`, `0`, `[]`, `''`, `NaN`, `-0`, `{}`, `null`?
3. What are `a` and `b` after `let a = 5; const b = a; a = 8;`? What is `o2.n` after `const o1 = {n: 1}; const o2 = o1; o1.n = 9;`? Explain the difference.
4. Predict the output of `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));` and of the same loop with `let`.
5. Write `grade(score)` as one expression with ternaries: 75 and up is `'distinction'`, 50 and up `'pass'`, otherwise `'fail'`.
6. Explain why `settings.volume || 10` is wrong when `volume` can be `0`, and fix it.
7. Rewrite this `if` chain as a `switch (true)`: codes 500 and up are server errors, 400 and up client errors, 300 and up redirects, 200 and up successes.
8. Write a `while` loop that doubles `x` starting from 1 until it exceeds 1 000, counting the doublings. What is the count?

Practice Problems B: Functions, Arrays and Objects

Review Problems

1. Given `const prices = [4, 9, 15, 22]`, what do `prices.map(p => p * 1.15)`, `prices.filter(p => p > 10)` and `prices.reduce((s, p) => s + p, 0)` return? Why is one of the numbers in the first answer ugly?
2. What does `console.log(c1(), c1(), c2())` print, given `makeCounter` from 11.6?
3. Write `range(a, b)` that returns the integers from a to b inclusive using `Array.from`. Then use `map` and `filter` to get the squares of the odd numbers from 1 to 9.
4. What does `[10, 9, 1, 100].sort()` give, and why? Fix it.
5. Using `reduce`, count how many times each word appears in `['a', 'b', 'a', 'c', 'a']`.
6. `const p = { name: 'Ana', tags: ['x'] }; const q = { ...p }; q.tags.push('y');` What is `p.tags` now, and why? How would you avoid it?
7. Write a function `pick(obj, keys)` that returns a new object with only the named keys, using `Object.fromEntries`.
8. Write a closure `makeAccumulator(start)` that returns a function which adds its argument to a running total and returns the total. Show its output for `acc(5); acc(10)` with `start = 100`.
9. Why does `people.sort((a, b) => a.age - b.age)` change `people`, and what would you call instead to keep the original?

Practice Problems C: Classes, Errors, the DOM and Async

Review Problems

1. Add a `stop()` method to the `Motor` class that sets the duty to 0, and a getter `isRunning` that is true when `rpm > 0`. Why must `stop` go through the setter rather than writing `this.#rpm = 0` directly?
2. `const f = m.describe; f();` throws. Explain why, and give two fixes.
3. What is wrong with `try { risky(); } catch (e) {}`? Rewrite it so that it handles `RangeOutOfBounds` and lets everything else propagate.
4. A script does `document.querySelector('#chart').getContext('2d')` and throws `Cannot read properties of null`. Name the two most likely causes.
5. Predict the printed order: `console.log('1'); (async () => { console.log('2'); await null; console.log('3'); })(); console.log('4');`
6. Two fetches are independent. Rewrite `const a = await getJSON(u1); const b = await getJSON(u2);` so they run at the same time, and say what happens if the second fails.
7. Why does `fetch` not throw on a 404, and what one line fixes the problem?
8. Write a debounced search: the function `search(q)` must run 300 ms after the visitor stops typing in `#q`.

Practice Problems D: Modules, Data and Tests

Review Problems

1. A module exports `export default function fit() {}` and `export const COL = {}`. Write the import line that brings both in, naming the default `fitCanvas`.
2. Why does `import` at the top of a file not work in a plain `<script src="main.js">`, and what are the two fixes?
3. Parse `'name,age\nAna,31\nBen,17'` into an array of objects with numeric ages, in one expression chain.
4. `localStorage.setItem('n', 5); const n = localStorage.getItem('n') + 1;` What is `n`, and why? Fix it.
5. Write a test, in the style of 11.17, that checks `lerp(0, 200, 0.25)` is 50 and `smoothstep(0.5)` is 0.5.
6. The state is `{ chart: 'speed', paused: false, filters: { tag: 'all' } }`. Write the expression that produces a new state with `filters.tag` set to `'aero'` and nothing else changed, without mutating the old state.
7. Give three things a linter catches that a test does not, and one thing a test catches that a linter does not.

Mini Projects

Four programs, each in `Guide/code/js`, each short enough to type in an evening and complete enough to be useful. Run them, then change them.

Mini Project: 1. A unit converter on the command line

`node mp1_convert.js 250 kt` prints 250 knots in every other speed unit; `node mp1_convert.js 250 kt m/s` prints one. The whole program is a table of units (`TO_SI`), a table of kinds, a four-line `convert` function that refuses to convert a length to a speed, and a command-line parser that prints usage when the arguments are wrong. Output:

```
250 kt = 128.611 m/s
250 kt = 463 km/h
250 kt = 25,317.148 ft/min
error: cannot convert length to speed          <- from: node mp1_convert.js 3 ft kt
usage: node mp1_convert.js <value> <unit> [unit]   e.g. 250 kt m/s
```

Check one by hand: 250 knots × 1 852 m / 3 600 s = 128.61 m/s. √ Add temperature (which needs an offset, not just a factor) and see what has to change.

Mini Project: 2. A sonar log analyser

`mp2_sonar.js` writes a deterministic 301-line log of a sensor approaching a wall at 15 cm/s, with a dropout (a reading of 4.0 m, the sensor's maximum) every 37th sample, then analyses it: counts dropouts, computes the mean and median of the valid readings, filters the signal with a five-sample

median, and finds when a brake set at 0.108 m would trigger.

```
301 readings, 9 dropouts (3.0%)
mean of valid readings 0.275 m, median 0.274 m
raw minimum 0.050 m, raw maximum 4.000 m
brake would trigger at t = 2.62 s on the filtered signal
on the raw signal it would trigger at t = 2.62 s (same here, because the dropouts re
      ad far, not near)
```

Check: $0.5 - 0.15 \times 2.62 = 0.107$ m, just under the threshold. √ Change the dropouts to read *near* (0.02 m) instead of far and watch the raw trigger time collapse while the filtered one holds. That is why the robot's sonar reading is filtered before the controller sees it.

Mini Project: 3. A damped oscillation, computed and checked

`mp3_oscillation.js` evaluates $x(t) = e^{-\zeta\omega_n t}\cos(\omega_d t)$ with $\zeta = 0.5$ and $\omega_n = 2$, finds the peaks numerically, and compares every number with the theory from Project 4 of Part III:

```
damped frequency 1.732 rad/s, period 3.628 s
x at one period: 0.0266 (prediction e^-T = 0.0266)
peaks (t, x): (3.326, 0.0311) (6.954, 0.0008) (10.581, 0.0000)
ratio of successive peaks 0.0266, theory e^(-2*pi*zeta/sqrt(1-zeta^2)) = 0.0266
envelope reaches 2% at t = 3.91 s (theory 3.91 s)
```

Notice that the first *peak* is at 3.326 s, not at one period (3.628 s): the decaying envelope pulls each maximum slightly earlier than the cosine's own peak. The program then prints a small text chart. Make it take ζ and ω_n from the command line, as in project 1.

Mini Project: 4. A robot's phases as a state machine

`mp4_phases.js` is the pattern that runs the whole Patrick page: one variable holds the phase; each 1 ms tick does only what that phase says and checks one condition to move on. The robot approaches a wall 50 cm away at 25 cm/s, brakes at 2 m/s² when the sonar reads 10.8 cm, lowers the arm, closes the claw, lifts.

```
0.000 s: IDLE -> APPROACH
1.570 s: APPROACH -> BRAKE
1.695 s: BRAKE -> REACH
2.313 s: REACH -> GRIP
2.713 s: GRIP -> LIFT
3.020 s: LIFT -> DONE
finished at 3.021 s, 40.8 cm from the start, 9.2 cm from the wall
braking distance v^2 / 2a = 1.56 cm, so it stops at 10.8 - 1.56 = 9.24 cm
```

Check: it must cover $50 - 10.8 = 39.2$ cm at 25 cm/s before the sonar trips: 1.568 s. √ Braking from 0.25 m/s at 2 m/s² takes 0.125 s, so the brake ends at 1.695 s. √ Add a `PLACE` phase and a second item, and you have written the skeleton of Chapter 42.

What You Learned

- Seven types; `typeof null` lies; numbers are floating point, so compare with a tolerance and round only for display; `===` always, `??` for defaults.

- `const` by default, `let` when you must, never `var`; block scope; closures remember their birthplace.
- Write small pure functions and pass them to `map`, `filter`, `reduce`; copy with spread instead of mutating; know which array methods mutate.
- Objects are shared by reference; spread copies one level, `structuredClone` copies all; Map and Set for lookups and uniqueness; JSON for text.
- Classes bundle state and behaviour; methods live on the prototype; `this` is whatever the method was called on, so bind or wrap when you pass one.
- Throw real errors, catch only what you expect, read the stack, use `debugger`.
- The DOM: `querySelector`, `textContent`, `classList`, `dataset`, `append`, events with delegation; `FormData` for forms.
- Promises settle once; `async`/`await` reads top to bottom; microtasks run before timers; `Promise.all` for independent work; check `res.ok`.
- Debounce for "after they stop", throttle for "at most this often"; `requestAnimationFrame` for motion.
- Modules give private scope and explicit dependencies; strict mode, a linter and a fifteen-line test harness keep code honest.

Chapter 12. React: Interfaces from Components

Chapter 5 ended with a promise: *screen = render(state)*. React is a library that keeps that promise for a whole page. You describe what the page should look like for the current state, and React works out the smallest set of changes needed to make the real page match. It is not magic, and it is not necessary for every page (this website's live pages do not use it), but it is the most common way professional teams build interfaces, so you should know how it thinks. Every example in this chapter was rendered for real with React 18.3 in Node (the file `Guide/code/react/examples.jsx`), and the outputs are quoted from that run.

In This Chapter

- Write components with JSX, pass props down and send events up.
- Hold state with `useState`, update it without mutation, and explain batching.
- Render lists with stable keys and know what goes wrong without them.
- Run side effects with `useEffect`, declare dependencies honestly, and clean up.
- Decide what is state, what is derived, and where state should live.
- Build controlled forms, use refs for things outside React, and read the render cycle.
- Get React into a real page with a build step.

Words You Will Meet

Word	What it means
Component	A function that returns a description of part of the screen.
JSX	A syntax that lets you write HTML-like tags inside JavaScript.
Element	What JSX produces: a plain object describing a tag or component and its props.
Props	Inputs passed *down* to a component, like function arguments.
State	Data a component remembers between renders.
Hook	A function whose name starts with `use` that adds a feature to a component (`useState`, `useEffect`, `useRef`).
Render	Calling your component to get its description of the screen.
Reconciliation	Comparing the new description with the old one to find the changes.
Effect	Code that talks to the world outside React (timers, the network, the DOM) after rendering.
Key	A stable label React uses to tell list items apart.
Controlled input	A form control whose value is held in state and written back on every change.
Lifting state up	Moving state to the nearest component that all its users share.

12.1 What React Is, and Is Not

React is a JavaScript library with one job: keep the page in step with your data. You write functions that take data and return a description of the screen; React turns the description into real DOM nodes, and when the data changes, updates only the nodes that need it. It does not fetch data, route between pages, or style anything; other libraries do those, and most React projects use several.

React earns its place when a page has a lot of state that many parts share: a dashboard, an editor, a shop. For a page that is mostly content with a few instruments, like this website's, plain JavaScript with Chapter 5's render-from-state habit does the same job with no build step and no library to download. Part V shows the one React file in this project and why it is not wired in. The ideas, though, transfer completely: a React component is Chapter 5's `render(state)` with the bookkeeping done for you.

12.2 Components and JSX

A component is a function whose name starts with a capital letter and which returns JSX.

```
function Greeting({ name }) {
    return <h1 className="title">Hello, {name}!</h1>;
}
// used like an HTML tag:  <Greeting name="Patrick" />
// renders:  <h1 class="title">Hello, Patrick!</h1>
```

JSX is not HTML. It is JavaScript in disguise, and a build step turns each tag into a function call that produces a plain object (an **element**). The rules that follow from that:

- **Curly braces drop back into JavaScript.** `{name}`, `{count + 1}`, `{items.map(...)}`. Anything that is an expression. Statements (`if`, `for`) are not allowed inside braces, which is why you see ternaries and `&&` there.
- **A few attribute names differ**, because they are JavaScript words: `className` not `class`, `htmlFor` not `for`. Event handlers are `onClick`, `onChange`, `onSubmit`, written in camelCase, and take a *function*, not a string.
- **A component returns one root.** Wrap siblings in a `<div>` or, when you do not want an extra element, in a fragment `<>...</>`.
- **Components start with a capital letter**, so that `<greeting>` (an unknown HTML tag) and `<Greeting>` (your function) can be told apart.
- `style` **takes an object** (`style={{ color: 'red' }}`), but you should rarely use it; classes and a stylesheet are still the right tool (Chapter 3).
- **Conditional rendering** is `cond ? <A /> : <B />` or `cond && <A />`. Lists are `array.map(item => <li key={item.id}>...</li>)`.

```
function Status({ phase, items }) {
    return <div>{phase === 'DONE' ? <p>Finished</p> : <p>Working: {phase}</p>}
                {items.length > 0 && <ul>{items.map(i => <li key={i.id}>{i.label}</l
      i>)}</ul>}</div>;
}
// <Status phase="CARRY" items={[{id: 1, label: 'egg'}]} />  ->  <div><p>Working: CA
      RRY</p><ul><li>egg</li></ul></div>
// <Status phase="DONE" items={[]} />                        ->  <div><p>Finished</p
      ></div>
```

> **WHY DO IT THIS WAY? Why JSX needs a build step**
>
> **What it is.** Browsers cannot read JSX. A tool such as esbuild or Vite converts it into plain JavaScript before the page loads.
>
> **Why it is done this way.** JSX makes the structure of an interface readable at a glance, and compiling it costs nothing at run time.
>
> **Why it matters.** You write `<h1>Hello</h1>` and get exactly what it says.
>
> **What happens if you do not use a build step.** The browser reports a syntax error on the first `<`. (You can compile JSX in the browser with a library called Babel for experiments, but it is slow and not meant for real sites.)

> **WARNING**
>
> `items.length && <ul>...</ul>` renders the number `0` on the page when the list is empty, because `0 && x` is `0` and React prints numbers. Write `items.length > 0 && ...` so the left side is a real boolean.

12.3 Props Down, Events Up

Data flows in one direction: from parents to children, as **props**. A child never changes its props. When something must change, the child calls a function the parent gave it.

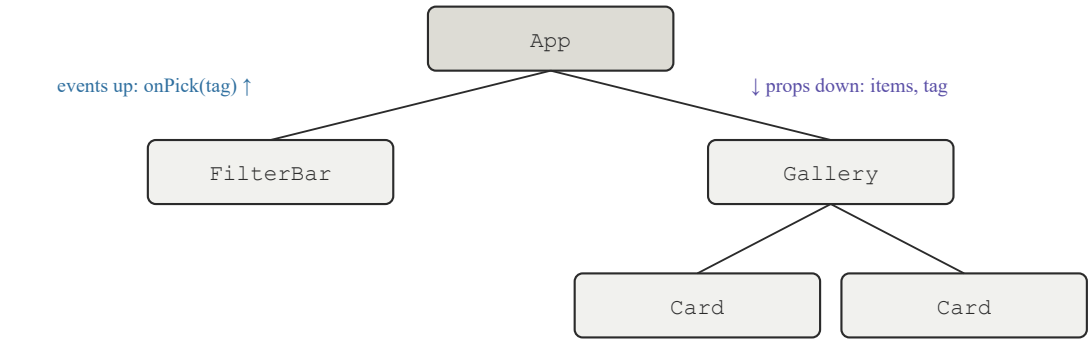


Figure 12.1. The state that two branches share lives in their nearest common parent. It flows down as props; changes flow up as function calls. No component ever reaches sideways.

```
function App() {
    const [tag, setTag] = useState('all');
    return <>
        <FilterBar tag={tag} onPick={setTag} />
        <Gallery items={ITEMS.filter(i => tag === 'all' || i.tag === tag)} />
    </>;
}
function FilterBar({ tag, onPick }) {
    return <div>{['all', 'aero', 'robots'].map(t =>
        <button key={t} className={t === tag ? 'on' : ''} onClick={() => onPick(t)}>
      {t}</button>)}</div>;
}
```

`FilterBar` does not know what happens when a chip is picked. It reports the fact to whoever gave it `onPick`. `Gallery` does not know how the items were chosen. That ignorance is the point: each component can be understood, tested and reused alone.

> **WHY DO IT THIS WAY? One-way data flow**
>
> **What it is.** Information moves down through props. Changes move up through functions that were passed down.
>
> **Why it is done this way.** If any child could change data anywhere, you could never be sure where a value came from.

Why it matters. When something looks wrong, you only have to follow the data *upward*, to the one place that owns it. This is Chapter 5's "keep each fact once" rule, enforced by the library.

What happens if you do not. The same data ends up copied in several components and drifting apart, which is exactly the bug that React exists to prevent.

12.4 State

`useState(initial)` returns the current value and a function to change it. Calling that function schedules a new render of the component; the next time the function runs, `useState` returns the new value.

```
function Counter() {
    const [count, setCount] = useState(0);
    return (
        <div>
            <p>You clicked {count} times.</p>
            <button onClick={() => setCount(count + 1)}>Click me</button>
        </div>
    );
}
```

Two rules make state behave.

Replace, never mutate. React decides whether anything changed by comparing the old and new value with ===. Pushing onto the same array leaves the same reference, so it looks unchanged.

```
const [items, setItems] = useState(['a', 'b']);
items.push('c');                 // WRONG: same array, React sees no change
setItems([...items, 'c']);       // RIGHT: a new array, so React notices
```

Updates are batched, and the value is a snapshot. Inside one event, `count` is the value from *this render*. Three calls to `setCount(count + 1)` all compute `0 + 1`. The function form `setCount(c => c + 1)` is given the latest value each time.

```
2: three set calls in one click, plain form -> You clicked 1 times.
2: three set calls in one click, functional form -> You clicked 3 times.
```

WHY DO IT THIS WAY? Replace state, do not mutate it

What it is. Calling the setter with a *new* array or object, never altering the old one.

Why it is done this way. React decides whether to re-render by comparing the old and new value with ===. Pushing onto the same array leaves the same reference, so it looks unchanged.

Why it matters. The screen updates when the data does. This is why Chapter 11 taught the spread operator and immutable updates.

What happens if you do not. The data changes and the screen does not, and you spend an hour wondering why a button "does nothing".

TIP

> When the new state depends on the old state, pass a function: `setCount(c => c + 1)`. If several updates are batched in one event, this form gets the latest value every time. For an object, spread and override: `setForm(f => ({ ...f, email: value }))`.

12.5 Rendering Lists and Keys

To show a list, map an array to elements and give each a `key`.

```
<ul>
    {items.map(item => <li key={item.id}>{item.label}</li>)}
</ul>
```

Leave the key out and React tells you so, in the console, every time:

```
3: Warning: Each child in a list should have a unique "key" prop.
```

> **WHY DO IT THIS WAY? Keys on list items**
>
> **What it is.** A unique, stable `key` on each element made in a list.
>
> **Why it is done this way.** When the list changes, React needs to know which old item corresponds to which new item, so it can move, add or remove the right ones.
>
> **Why it matters.** Without keys, React guesses by position, and reordering a list can leave typed text or focus on the wrong row.
>
> **What happens if you do not.** You get the warning above, and subtle bugs: edit the third item, delete the first, and the text you typed jumps to a different row. Do not use the array index as a key if the list can be reordered or filtered.

12.6 Effects: Talking to the Outside World

A component's render should be a pure calculation. Anything else (starting a timer, subscribing to an event, fetching data, touching the DOM directly) belongs in an **effect**, which runs *after* rendering.

```
useEffect(() => {
    const id = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(id);            // cleanup
}, []);                                         // run once, when the component appe
      ars
```

The function returned from an effect is its **cleanup**: React runs it when the component is removed, and before the effect runs again. The array at the end is the **dependency list**: the values from the component that the effect reads. An empty array means "run once"; leaving the array out means "run after every render", which is almost never what you want.

```
5: effect: start timer | cleanup: stop timer
```

> **WHY DO IT THIS WAY? The cleanup function and the dependency list**
>
> **What it is.** The function returned from an effect runs when the component disappears (or before the effect runs again). The array at the end lists the values the effect depends on.

Why it is done this way. Effects start things (a timer, a listener). Something has to stop them.

Why it matters. Without cleanup, the timer keeps running after the component is gone. Without the right dependency list, the effect uses stale values or runs far too often.

What happens if you do not. You leak timers and event listeners, so a page that has been open for an hour is slow and uses more and more memory.

WARNING

In development, inside `<StrictMode>`, React deliberately runs every effect twice on first mount (effect, cleanup, effect) to check that your cleanup really undoes your setup. If something happens twice "for no reason", that is why, and the fix is a correct cleanup, not removing StrictMode.

Fetching data in an effect

```
useEffect(() => {
    let cancelled = false;
    fetch(`/api/parts/${id}`).then(r => r.json()).then(data => { if (!cancelled) set
      Part(data); });
    return () => { cancelled = true; };
}, [id]);
```

The effect depends on `id`, so it runs again when the id changes. The `cancelled` flag stops a slow reply for an old id overwriting the data for the new one, which is the classic race in any interface that loads things.

12.7 Thinking in React: What Is State?

The hardest part of React is not the syntax. It is deciding what to store. Chapter 5's rule applies with full force: keep each fact once, and derive everything else.

Ask three questions of every piece of data. Is it passed in from a parent? Then it is a prop, not state. Does it stay the same over time? Then it is a constant. Can it be computed from other state or props? Then it is **derived**, and you compute it in the render, every time. What is left is the state.

Worked Example: Type to filter

Build a component with a text box and a list of fruits, showing only the fruits that contain the typed text.

Step 1. What is the state? Only one thing changes: the text in the box. Call it `query`. The *filtered list is not state*, because it can be worked out from the query.

Step 2. Write it.

```
function FruitFilter({ fruits }) {
    const [query, setQuery] = useState('');
    const shown = fruits.filter(f => f.toLowerCase().includes(query.toLowerCase()));
    return (
        <div>
            <label>Filter <input value={query} onChange={e => setQuery(e.target.valu
      e)} /></label>
```

```
            <ul>{shown.map(f => <li key={f}>{f}</li>)}</ul>
        </div>
    );
}
```

Step 3. Run it. With `fruits = ['Apple', 'Banana', 'Cherry']`:

```
4: [ 'Apple', 'Banana', 'Cherry' ]
   after typing "an": [ 'Banana' ]
   after typing "e":  [ 'Apple', 'Cherry' ]
```

Step 4. Why it works. The box shows `query` and edits it, so state and screen cannot disagree. The list is *derived*: every render recomputes it. There is no "update the list" code at all.

Lifting state up. When two components need the same fact, the fact moves to their nearest common parent and comes down to both as props. The chart chips and the chart in Chapter 5's worked example are one state (`chart`) in a parent, a `Chips` child that shows it and asks to change it, and a `Chart` child that draws it.

12.8 Forms and Controlled Inputs

A **controlled** input has its value in state and writes every change back: `<input value={query} onChange={e => setQuery(e.target.value)} />`. The state is the one truth and the box merely shows it, so you can validate, transform or reject keystrokes as they happen. A checkbox uses `checked` and `e.target.checked`; a select uses `value` on the `<select>`. A whole form is an object in state, updated with spread:

```
const [form, setForm] = useState({ email: '', msg: '' });
const edit = e => setForm(f => ({ ...f, [e.target.name]: e.target.value }));
<form onSubmit={e => { e.preventDefault(); send(form); }}>
    <input name="email" type="email" value={form.email} onChange={edit} required />
    <textarea name="msg" value={form.msg} onChange={edit} />
    <button type="submit">Send</button>
</form>
```

The computed key `[e.target.name]` lets one handler serve every field, which is Chapter 9's `name` attribute doing double duty.

12.9 Refs: Things Outside React

`useRef` holds a value that survives renders *without* causing one. It has two uses: a handle on a real DOM node (a canvas to draw on, an input to focus), and a mutable box for things like a timer id or a render count that are not part of the picture.

```
function RenderCounter() {
    const renders = useRef(0); renders.current++;
    const [n, setN] = useState(0);
    return <div><p>render {renders.current}, state {n}</p><button onClick={() => set
      N(n + 1)} /></div>;
}
// after two clicks:  6: render 3, state 2
```

Changing `renders.current` did not cause a render; the two state changes did. For a DOM node, `<canvas ref={canvasRef} />` and then `canvasRef.current` inside an effect, because the node does not exist until after the first render. The Part IV chapter shows a three.js renderer living in a ref.

12.10 The Render Cycle

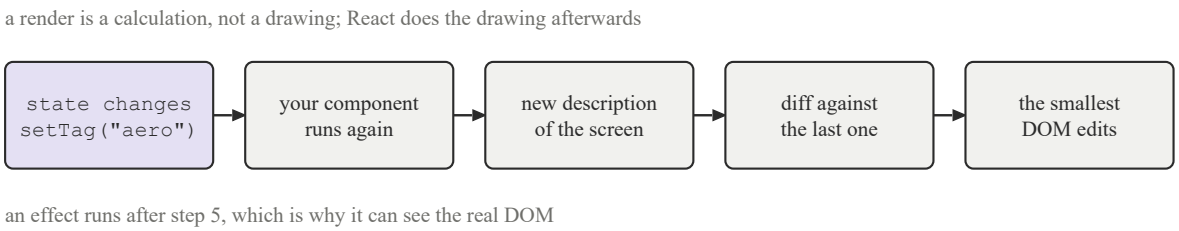


Figure 12.2. What happens when state changes. Your function runs again and returns a new description; React compares it with the previous one and edits only the DOM nodes that differ; effects run after the edits.

Three consequences of this picture explain most React puzzles.

A render is a function call. Everything in the body runs again: variables are recreated, derived values recomputed, inner functions remade. That is fine and usually cheap. State and refs are the only things that persist.

Reconciliation is by position and type, then by key. If the same component type is in the same place, React keeps its DOM node and state and updates the props. If the type changes (`<p>` to `<div>`), the old node is thrown away and a new one made, state included. In a list, keys tell React which items are "the same" across renders.

Effects run after the DOM is updated. That is why an effect can measure an element or draw on a canvas, and why doing those in the render body is a bug.

> **WHY DO IT THIS WAY? Measuring before memoising**
>
> **What it is.** `React.memo`, `useMemo` and `useCallback` can skip work between renders. Reach for them only after you have *seen* a slow render in the profiler.
>
> **Why it is done this way.** Memoising has a cost of its own, and most renders are cheap.
>
> **Why it matters.** The code stays simple where it can.
>
> **What happens if you do not.** You scatter `useMemo` everywhere "just in case", and it makes the code harder to read without making it faster (Chapter 2's rule: measure, do not guess). The Part IV chapter shows the three tools with their outputs.

12.11 Getting React Into a Page

A React page needs three things: the library, a build step for JSX, and a mount point.

```
npm install react@18.3.1 react-dom@18.3.1
npx esbuild app.jsx --bundle --minify --jsx=automatic --outfile=app.js

<!-- index.html -->
<div id="root"></div>
<script defer src="app.js"></script>

// app.jsx
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
createRoot(document.getElementById('root')).render(<App />);
```

That is the whole of it. `createRoot` claims the `<div>`, and `render` draws the component tree into it and keeps it in step from then on. The book's `run_jsx.js` does the same transformation with esbuild in Node, which is how every output in this chapter was produced.

> **WHY DO IT THIS WAY? Keeping the mount point small**
>
> **What it is.** One `<div id="root">`, inside a page that is otherwise normal HTML.
>
> **Why it is done this way.** React does not have to own the whole page. A content page can stay plain HTML and hand one interactive region to React.
>
> **Why it matters.** The rest of the page loads and reads without JavaScript (Chapter 3), and the React bundle is only as big as the interactive part needs.
>
> **What happens if you do not.** The entire page becomes a JavaScript application: blank until the bundle runs, invisible to anything that does not run scripts, and heavier than it needed to be.

Worked Example: A Stopwatch With a Clean Effect

Worked Example: Start, stop, and no leaks

Build a stopwatch that shows elapsed seconds, with Start and Stop buttons, and make sure nothing keeps running when it is removed.

Step 1. The state. Two facts: `running` (boolean) and `elapsed` (seconds). The button labels and the displayed time are derived.

Step 2. The effect depends on `running`. When it becomes true, start an interval; the cleanup stops it. When it becomes false, the cleanup from the previous run has already stopped the interval, and the new effect does nothing.

```
function Stopwatch() {
    const [running, setRunning] = useState(false);
    const [elapsed, setElapsed] = useState(0);
    useEffect(() => {
        if (!running) return;
        const t0 = Date.now() - elapsed * 1000;
        const id = setInterval(() => setElapsed((Date.now() - t0) / 1000), 100);
        return () => clearInterval(id);
    }, [running]);
```

```
    return <div>
        <p>{elapsed.toFixed(1)} s</p>
        <button onClick={() => setRunning(r => !r)}>{running ? 'Stop' : 'Start'}</bu
      tton>
        <button onClick={() => { setRunning(false); setElapsed(0); }}>Reset</button>
    </div>;
}
```

Step 3. Why `Date.now()` and not `elapsed + 0.1`. Timers drift (Chapter 6). Computing the time from the clock keeps it honest, and resuming from the stored `elapsed` makes Stop and Start continue rather than restart.

Step 4. Check. Stop, start, remove the component: the Ticker example in `examples.jsx` shows the pattern's log, `effect: start timer | cleanup: stop timer`, and this component behaves the same way.

Practice Problems A: Components, Props and State

Review Problems

1. A component has `const [n, setN] = useState(0)` and a button that runs `setN(n + 1); setN(n + 1); setN(n + 1);`. After one click, what is `n`? How would you make it 3?
2. Why does `setItems(items.push('c'))` fail? (Think about what `push` returns.)
3. For a to-do list with items `{id, text, done}`, write the code to toggle one item's `done` without mutating the array.
4. What does `{count && <Badge n={count} />}` show when `count` is 0, and what is the fix?
5. Which of these are state, which are props, and which are derived, in a product page: the product's price (from the server), the quantity the visitor chose, the total, whether the "Add" button is enabled, the currency symbol?
6. Convert this HTML to JSX: `<label for="q" class="big">Search <input id="q" onchange="go()"></label>`.
7. Two sibling components both need to know which part is selected. Where does the state go, and how does each get it?

Practice Problems B: Lists, Effects, Forms and Refs

Review Problems

1. A list of 3 rows each has an input box. You delete the first row and the second row's typed text moves to the third. What is likely wrong with the keys, and what is the fix?
2. Write an effect that adds a `resize` listener to `window` that stores the width in state, and removes it on cleanup.
3. An effect fetches `/api/part/${id}` but has `[]` as its dependency list. What goes wrong when `id` changes, and what else is needed to make it safe when `id` changes quickly?

4. State what is wrong with storing both `query` and `shown` in state in the filterable-list worked example.
5. In StrictMode a component's effect logs "subscribed" twice on load. Is this a bug? What should the cleanup do?
6. Write a controlled `<select>` for choosing a chart (`speed`, `power`, `sound`) that keeps its value in state.
7. A `useRef(0)` counter is incremented in a click handler but the number on screen never changes. Why, and when is a ref the right tool anyway?
8. Give the two commands and the three lines of code that put a `<App />` into `<div id="root">`.

Mini Project: A Part Picker

Mini Project: The robot's parts panel, in React

Build the part panel of the Patrick page as a small React application: a search box, a list of parts, and a detail panel for the selected one.

1. **Data.** A constant array `PARTS` of at least eight objects: `{ id, name, tags, desc, mass }`. Load it from a JSON file with `fetch` in an effect, with a cancelled flag, and show "Loading..." until it arrives.
2. **State, and only state.** `query` (the search text), `selectedId`, and the loaded `parts`. Everything else is derived: the filtered list, the selected part object, the total mass of the filtered parts.
3. **Components.** `App` owns the state. `Search` is a controlled input. `PartList` renders the filtered parts with stable keys and highlights the selected one; clicking calls `onSelect(id)`. `Detail` shows the selected part or a prompt to pick one.
4. **Keyboard.** Up and Down arrows in the list move the selection; Enter in the search box selects the first match. (A `keydown` listener on the list, through an effect with cleanup.)
5. **A canvas inside React.** `Detail` includes a `<canvas ref>` on which an effect draws a bar of the part's mass against the heaviest part, redrawn when the selection changes.
6. **Build it** with esbuild and open it. Then open the React DevTools browser extension, change `query` from the Components panel, and watch the list follow.

Checks: no key warnings in the console; the filtered list is never stored in state; removing and re-adding the list in StrictMode leaves exactly one keyboard listener; the canvas redraws without flicker when you change selection.

What You Learned

- A component is a function from props to a description of the screen; JSX is that description written as tags, compiled by a build step.
- Props flow down and never change in the child; events flow up as function calls; shared state lives in the nearest common parent.

- `useState` gives a snapshot and a setter. Replace, never mutate; use the function form when the new value depends on the old; updates in one event are batched.
- Lists need stable keys, or React matches by position and state jumps between rows.
- Effects run after render, declare what they read, and return a cleanup. StrictMode runs them twice on purpose.
- Keep only the facts; derive the rest in the render. Controlled inputs keep the form in state. Refs hold what the picture does not depend on.
- A render is a function call; reconciliation keeps nodes of the same type and position; effects see the updated DOM.
- Two commands and three lines put React into one `<div>` of an otherwise ordinary page.

Chapter 13. Canvas and SVG: Drawing in the Browser

Chapter 7 described these two tools in outline. This chapter is the practical companion: the words, the commands you will really use, and the habits that make drawings sharp, fast and correct. Almost every instrument on this website is a canvas, and the block diagram is an SVG, so these are the tools that did most of the work. By the end you will be able to draw a chart with proper axes, a gauge with a needle, an animated diagram, and anything else that is a picture made from numbers.

In This Chapter

- Draw paths, arcs, curves and text on a canvas, and style them.
- Map data to pixels with one pair of functions, and lay out an axis with round ticks.
- Use transforms with `save` and `restore`, and clip to a plot area.
- Cache static drawing on an offscreen canvas and animate only what moves.
- Hit-test points and shapes under the pointer, and build hover and drag.
- Write SVG by hand and from script: shapes, paths, `viewBox`, groups, text, CSS styling and animation.
- Choose between canvas and SVG for a given picture, and mix them.

Words You Will Meet

Word	What it means
Context	The object you draw with (`canvas.getContext('2d')`).
Path	A shape described by a series of moves and lines or curves.
Stroke / fill	Drawing the outline of a path / painting its inside.
Transform	A change of how coordinates are read: translate, rotate, scale.
Gradient	A smooth blend between colours.
Clip	A region outside of which nothing is drawn.
Offscreen canvas	A canvas that is not in the page, used as a cache.
viewBox	SVG's private coordinate system.
Hit testing	Working out what shape is under the pointer.
Device pixel ratio	The ratio of physical pixels to CSS pixels (Chapter 4).
Namespace	The XML name that tells the browser an element is SVG, needed when creating SVG from script.

13.1 The Drawing Recipe

Almost every canvas drawing follows the same recipe: get the context, set the state, begin a path, add to it, then stroke or fill.

```
const { ctx, w, h } = fit(canvas);          // Chapter 4: sharp on every screen
ctx.strokeStyle = '#60a5fa';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(40, h - 40);
for (let x = 0; x <= 100; x++) {
    ctx.lineTo(40 + x * 3, h - 40 - Math.sin(x / 8) * 30);
}
ctx.stroke();
```

> **WHY DO IT THIS WAY? beginPath before each new shape**
>
> **What it is.** `ctx.beginPath()` starts a fresh path.
>
> **Why it is done this way.** The context keeps *one* current path, and adds to it every time you call `lineTo`. `stroke()` draws the whole thing.
>
> **Why it matters.** Starting fresh means each shape is drawn once, with its own colour.
>
> **What happens if you do not.** The second line you draw also re-draws the first one, in the new colour, and long animations slow down as the path grows every frame. Forgetting `beginPath` is one of the most common canvas bugs.

The state you will set most: `fillStyle` and `strokeStyle` (any CSS colour, or a gradient), `lineWidth`, `lineCap` (`butt`, `round`, `square`), `lineJoin`, `setLineDash([6, 4])` with `lineDashOffset`, `globalAlpha`, `font` (a CSS font string such as `'12px JetBrains Mono'`), `textAlign` and `textBaseline`. Every one of these stays in force until you change it or `restore()` an earlier state.

13.2 Paths: Lines, Arcs and Curves

A path is built from these commands, all of which add to the current path:

Command	Draws
`moveTo(x, y)`	nothing; lifts the pen and puts it here
`lineTo(x, y)`	a straight segment from the pen to here
`arc(cx, cy, r, a0, a1, anticlockwise)`	part of a circle, angles in radians, 0 at three o'clock, increasing **clockwise** (because y points down)
`arcTo(x1, y1, x2, y2, r)`	a rounded corner between two lines (how rounded rectangles are made)
`quadraticCurveTo(cx, cy, x, y)`	a curve pulled towards one control point
`bezierCurveTo(c1x, c1y, c2x, c2y, x, y)`	a curve with two control points (the same curve as Chapter 6's easing)
`rect(x, y, w, h)`, `roundRect(x, y, w, h, r)`	a rectangle, as a path
`closePath()`	a line back to the start

Then `stroke()`, `fill()`, or both. The shortcuts `fillRect`, `strokeRect` and `clearRect` draw rectangles immediately without touching the current path.

Worked Example: A gauge with a needle

Draw a quarter-circle gauge: an arc from the lower left over the top to the lower right, sweeping 270 degrees, and a needle pointing at a value v between 0 and 1. Centre (150, 150), radius 100.

Step 1. The arc's angles. In canvas angles, straight up is −90° (or 270°). The gauge starts 135° anticlockwise from straight up, at 135°, and ends 135° clockwise from it, at 405° (which is 45°). Sweeping clockwise from 135° to 405° covers 270°.

```
ctx.beginPath();
ctx.arc(150, 150, 100, 135 * Math.PI / 180, 405 * Math.PI / 180);
ctx.lineWidth = 10; ctx.lineCap = 'round'; ctx.strokeStyle = '#2b2b2b'; ctx.stroke()
      ;
```

Step 2. The needle. Draw it pointing straight up from the centre, then rotate. At v = 0 it should point along the start of the arc, which is 135° anticlockwise of up, so the rotation is −135° + 270° × v.

```
ctx.save();
ctx.translate(150, 150);
ctx.rotate((-135 + 270 * v) * Math.PI / 180);
ctx.beginPath(); ctx.moveTo(0, 12); ctx.lineTo(0, -90);
ctx.lineWidth = 3; ctx.strokeStyle = '#a78bfa'; ctx.stroke();
ctx.restore();
```

Step 3. Check a value. For v = 0.3 the rotation is −135 + 81 = −54°, so the needle leans up and to the left, a little past a third of the way round. For v = 0.5 it is 0°, straight up. For v = 1 it is +135°, the lower right. √

Step 4. The arc of the value. Draw a second arc over the first, from 135° to 135° + 270° × v, in the accent colour, to fill in the gauge up to the needle. Both arcs and the needle are driven by the one number v, which is Chapter 5's rule in a drawing.

13.3 Mapping Data to Pixels, and Ticks

A chart needs a function that maps your data range to pixels (see Chapter 7). Write it once and use it for everything: axes, ticks, curves and labels.

```
function makeMap(x0, x1, y0, y1, rect) {
    return {
        X: v => rect.x + (v - x0) / (x1 - x0) * rect.w,
        Y: v => rect.y + rect.h - (v - y0) / (y1 - y0) * rect.h,
    };
}
```

> **WHY DO IT THIS WAY? One mapping used everywhere**
>
> **What it is.** `X` and `Y` are the only places that convert data to pixels.
>
> **Why it is done this way.** If curves, gridlines and markers each do their own arithmetic, they will disagree by a few pixels.
>
> **Why it matters.** Changing an axis range changes one place, and every mark moves together.
>
> **What happens if you do not.** A marker sits slightly off its curve, and you cannot tell which piece of code is wrong.

Round ticks

An axis from 0 to 97 should not be labelled 0, 19.4, 38.8. The standard trick: divide the range by the number of ticks you want, round that step to 1, 2 or 5 times a power of ten, then start from a multiple of it.

```
function niceTicks(lo, hi, n) {
    const raw = (hi - lo) / n, p = 10 ** Math.floor(Math.log10(raw)), f = raw / p;
    return (f < 1.5 ? 1 : f < 3 ? 2 : f < 7 ? 5 : 10) * p;
}
// niceTicks(0, 1, 5) -> 0.2;   niceTicks(0, 97, 5) -> 20;   niceTicks(0, 0.0123, 4)
      -> 0.005
```

With a step of 20 on 0 to 97, the ticks are 0, 20, 40, 60, 80, 100, and the axis top becomes `Math.ceil(97 / 20) * 20 = 100`. These three results are checked by the test harness in Chapter 11 (`s17_tests.js`), and the same function, under the same name, draws every axis on this website.

Worked Example: Laying out a line chart

Draw the speed of a robot over 10 seconds, values between 0 and 0.37 m/s, on a canvas 400 by 240 CSS pixels, with a margin of 40 on the left and 24 elsewhere.

Step 1. The plot area. x from 40 to 376 (width 336), y from 24 to 216 (height 192).

Step 2. The y axis. `niceTicks(0, 0.37, 4)`: raw 0.0925, p = 0.01, f = 9.25, so the step is 0.1. Ticks 0, 0.1, 0.2, 0.3, 0.4; top = 0.4.

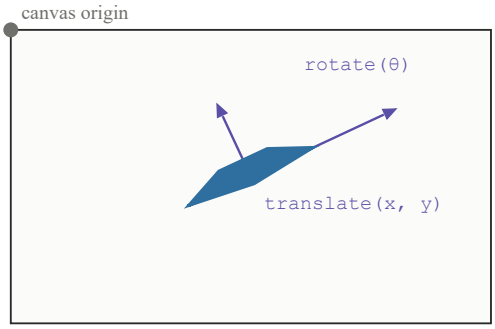
Step 3. The x axis. `niceTicks(0, 10, 5)` gives 2: ticks 0, 2, 4, 6, 8, 10.

Step 4. The mapping. `makeMap(0, 10, 0, 0.4, { x: 40, y: 24, w: 336, h: 192 })`. The point (6 s, 0.3 m/s) lands at X = 40 + 0.6 × 336 = 241.6 and Y = 24 + 192 − 0.75 × 192 = 72.

Step 5. Draw in order. Gridlines and tick labels first (faint), then the curve clipped to the plot, then any marker, then axis titles. Each layer uses `X` and `Y`, so they cannot disagree.

13.4 Transforms

The **transform** is part of the state: `translate`, `rotate` and `scale` change how later coordinates are read. The pattern for anything drawn at an angle: move the origin there, turn, draw the thing upright around (0, 0), put everything back.




```
ctx.save();
ctx.translate(x, y);
ctx.rotate(theta);
drawSymbol(ctx);   // drawn upright at (0, 0)
ctx.restore();
```

order matters: translate then rotate turns the symbol about its own centre; rotate then translate swings it around the canvas origin

Figure 13.1. To draw a symbol at a position and angle, translate to the position, rotate, and draw the symbol around the origin. The order matters: rotate then translate would swing the symbol around the canvas origin instead.

```
ctx.save();
ctx.translate(x, y);          // move the origin to the aircraft
ctx.rotate(-pitch);           // turn the axes (negative: y points down on screen)
drawAircraft(ctx);            // drawn upright around the origin
ctx.restore();                // everything back as it was
```

> **WHY DO IT THIS WAY? save and restore around a transform**
>
> **What it is.** Calling `ctx.save()` before changing the transform and `ctx.restore()` after.
>
> **Why it is done this way.** The transform is *sticky*. It stays in force until you undo it.
>
> **Why it matters.** Everything you draw afterwards is unaffected.
>
> **What happens if you do not.** The axes of your chart end up rotated and shifted because you drew an aircraft symbol earlier, and it is very hard to tell why.

`scale(1, -1)` flips the y axis, which tempts people who want maths-style coordinates. Resist it: text drawn under a flipped transform comes out upside down. Use the `Y` mapping function instead.

13.5 Clipping, Gradients, Text and Images

`ctx.clip()` restricts drawing to the current path, which is how a chart stops a curve from spilling out of its plot area.

```
ctx.save();
ctx.beginPath(); ctx.rect(plot.x, plot.y, plot.w, plot.h); ctx.clip();   // keep cur
```

```
      ves inside the box
drawCurves();
ctx.restore();                                                    // clippin
      g removed
```

> **WHY DO IT THIS WAY? Clipping inside save and restore**
>
> **What it is.** A clip stays in force until you `restore()`.
>
> **Why it is done this way.** There is no "unclip" command. The only way back is `save()` and `restore()`.
>
> **Why it matters.** Labels drawn after the curves are not clipped away.
>
> **What happens if you do not.** Everything you draw after the clip is cut off outside the plot box, including your axis labels.

Gradients are fill or stroke styles: `const g = ctx.createLinearGradient(x0, y0, x1, y1); g.addColorStop(0, '#a78bfa'); g.addColorStop(1, 'transparent'); ctx.fillStyle = g;`. A radial gradient is `createRadialGradient(cx, cy, r0, cx, cy, r1)`. The area under a curve on this website's charts is a linear gradient from the accent colour to transparent, clipped to the plot.

Text is `ctx.fillText(text, x, y)` with `font`, `textAlign` (`left`, `center`, `right`) and `textBaseline` (`top`, `middle`, `alphabetic`, `bottom`). `ctx.measureText(text).width` tells you how wide it will be, which is how a label is kept inside the canvas or a tooltip box is sized to its text.

Images are `ctx.drawImage(image, x, y, w, h)`, where the image is an `<img>` that has finished loading, another canvas, or a video frame. The second form, with source and destination rectangles, crops. Drawing another canvas is the basis of the next section.

13.6 Offscreen Canvases and Layers

A chart often has a **static** part (axes, grid, labels) and a **moving** part (curves, markers). Redrawing the static part every frame wastes time. Draw it once to an offscreen canvas, then copy it in with `drawImage`.

```
const bg = document.createElement('canvas');     // an offscreen canvas, not in the
      page
bg.width = w * dpr; bg.height = h * dpr;
drawAxes(bg.getContext('2d'));                   // done once, or when the size chan
      ges

function frame() {
    ctx.drawImage(bg, 0, 0, w, h);               // stamp the axes in one fast call
    drawCurves(ctx);                             // only the moving part is redrawn
}
```

> **WHY DO IT THIS WAY? Caching the static drawing**
>
> **What it is.** The grid and axes are drawn once to a hidden canvas and stamped onto the visible one each frame.

> **Why it is done this way.** Copying a finished bitmap is much cheaper than recomputing hundreds of lines and labels.
>
> **Why it matters.** The cylinder-flow panels on this website do exactly this: the colour field is built once per Mach number and only the moving particles are redrawn each frame.
>
> **What happens if you do not.** You recompute things that have not changed, 60 times a second, and the frame budget disappears.

Two canvases stacked in the page with CSS (`position: absolute` in a `relative` parent) do the same job without `drawImage`: the bottom one holds the static picture, the top one is cleared and redrawn each frame. The `OffscreenCanvas` class goes further and lets a Web Worker draw without touching the main thread.

Crisp lines

A 1-pixel line centred on a whole-number coordinate is drawn across *two* device pixels at half brightness, and looks blurry. Offset by 0.5 (`moveTo(10.5, 20.5)`) to land it on one pixel. On a DPR 2 screen the issue halves; the `fit` function's transform keeps the arithmetic in CSS pixels either way.

13.7 Animation on a Canvas

Chapter 6 gave the loop; the canvas version adds two habits. Clear and redraw everything from state each frame (Chapter 5), and draw only while the canvas is on screen.

```
let visible = false, last = null;
new IntersectionObserver(([e]) => { visible = e.isIntersecting; if (visible) request
      AnimationFrame(frame); }).observe(canvas);
function frame(now) {
    const dt = Math.min((now - (last ?? now)) / 1000, 0.1); last = now;
    update(dt);
    const { ctx, w, h } = fit(canvas);           // fit clears the canvas
    ctx.drawImage(bg, 0, 0, w, h);
    draw(ctx, w, h);
    if (visible) requestAnimationFrame(frame);
}
```

A moving dot along a curve is `X(t), Y(f(t))` with `t` advanced by `dt`. A trace that fades is the last N points drawn with `globalAlpha` falling from 1 to 0. A scrolling oscilloscope keeps a ring buffer of samples and maps the newest to the right edge.

13.8 Interaction: Hit Testing, Hover and Drag

A canvas does not know about its shapes, so you must do hit testing yourself: compare the pointer position with your own data. Convert the event's coordinates to canvas coordinates first.

```
canvas.addEventListener('pointerdown', e => {
    const r = canvas.getBoundingClientRect();
    const x = e.clientX - r.left, y = e.clientY - r.top;     // CSS pixels inside th
      e canvas
    const hit = points.find(p => Math.hypot(p.x - x, p.y - y) < 8);
});
```

> **WHY DO IT THIS WAY? getBoundingClientRect when reading the pointer**
>
> **What it is.** Subtracting the canvas's on-screen position from the pointer's position.
>
> **Why it is done this way.** `clientX` is measured from the top-left of the *window*, not from the canvas.
>
> **Why it matters.** Your coordinates then match the ones you drew with.
>
> **What happens if you do not.** Clicks land far from where you pointed, and the error changes as the page scrolls.

For a **hover readout** on a chart, invert the x mapping: `const t = x0 + (x - rect.x) / rect.w * (x1 - x0)`, find the nearest sample, and draw a vertical line and a label at it on the next frame. Store the hovered index in state and let `draw` show it; do not draw inside the event handler. For **dragging** a point, `setPointerCapture` on `pointerdown` (Chapter 5), update the point's data on `pointermove`, release on `pointerup`. For rectangles, hit testing is four comparisons; for circles, `Math.hypot`; for anything else, `ctx.isPointInPath(path, x, y)` with a `Path2D` object does the geometry for you.

13.9 SVG: The Practical Bits

An SVG is a set of elements in the page, and everything you know about HTML applies: they are in the DOM, they take classes and ids, CSS styles them, scripts change them, and a click lands on the element under the pointer with no hit testing at all.

```
<svg viewBox="0 0 100 40" width="300" role="img" aria-label="An arrow">
  <path d="M5 20 H80" stroke="currentColor" stroke-width="2" fill="none"/>
  <path d="M80 12 L95 20 L80 28 Z" fill="currentColor"/>
</svg>
```

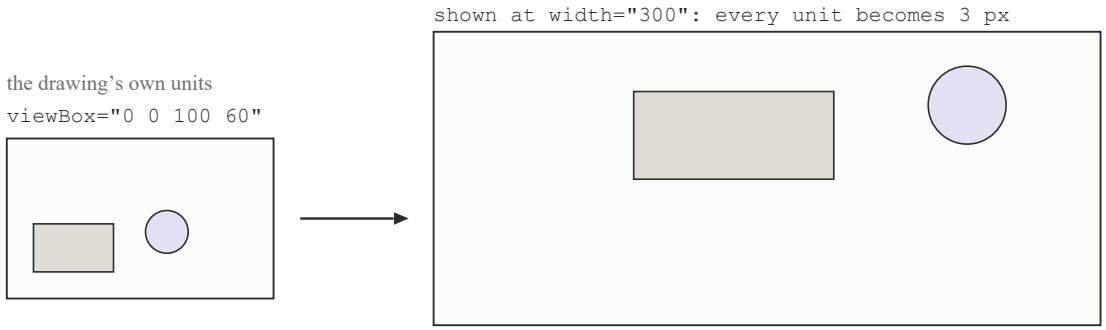


Figure 13.2. The `viewBox` is the drawing's own coordinate system. However the SVG is sized on the page, the browser scales those units to fit, which is what makes an SVG sharp at any size.

The elements

`<rect x y width height rx>`, `<circle cx cy r>`, `<ellipse>`, `<line x1 y1 x2 y2>`, `<polyline points="...">`, `<polygon>`, `<path d="...">`, `<text x y>`, and `<g>` to group (with `transform="translate(10 20) rotate(30)"` applying to everything inside). `<defs>` holds things used elsewhere: gradients, markers (arrowheads), clip paths, and symbols reused with `<use href="#id">`.

Path data

In `d`, uppercase commands are absolute and lowercase are relative to the current point: `M` move, `L` line, `H` and `V` horizontal and vertical lines, `C` a cubic Bézier with two control points, `S` a smooth continuation, `Q` a quadratic with one control point, `A` an elliptical arc (`A rx ry rotation large-arc sweep x y`), `Z` close. A rounded-corner path for a block diagram's wire is `M 10 20 H 60 Q 70 20 70 30 V 50`: along, a quarter-turn, down.

Styling and animating with CSS

`fill`, `stroke`, `stroke-width`, `stroke-dasharray`, `stroke-dashoffset`, `opacity` and `transform` are all CSS properties, so a stylesheet rule or a class toggled by a script changes them, and a transition animates them. The control-system diagram on the flight dynamics page is an SVG animated purely with CSS: the dashed signal lines have their `stroke-dashoffset` changed by a keyframe animation, so the dashes seem to flow along the wires.

```
.wire { stroke-dasharray: 6 6; animation: flow 1.2s linear infinite; }
@keyframes flow { to { stroke-dashoffset: -12; } }
```

> **WHY DO IT THIS WAY? Animating the diagram with CSS only**
>
> **What it is.** A `@keyframes` rule that shifts the dash pattern, with no JavaScript.
>
> **Why it is done this way.** It is a decoration that never changes in response to data.
>
> **Why it matters.** It costs nothing on the main thread and keeps working even if a script fails.
>
> **What happens if you do not.** A script that animates a decoration is one more thing to break and one more thing to slow the page.

> **WHY DO IT THIS WAY? Using currentColor in SVG**
>
> **What it is.** `stroke="currentColor"` uses the CSS `color` of the surrounding text.
>
> **Why it is done this way.** The icon takes whatever colour its context says.
>
> **Why it matters.** The same SVG can be violet in one place and white in another, and it changes with hover, without editing the SVG.
>
> **What happens if you do not.** You hard-code a colour in the graphic, and need a separate copy for every colour you want.

Making SVG from script

SVG elements live in their own XML namespace, so `document.createElement('circle')` makes a useless HTML element called circle. Use the namespaced version, and set attributes with `setAttribute`:

```
const NS = 'http://www.w3.org/2000/svg';
const dot = document.createElementNS(NS, 'circle');
dot.setAttribute('cx', 80); dot.setAttribute('cy', 45); dot.setAttribute('r', 4);
svg.append(dot);
```

That is how a diagram can be generated from data (one `<rect>` per block, one `<path>` per wire) and still be styled and clicked like hand-written markup. The isometric turbofan on the propulsion page is

thousands of SVG elements written by a script, exactly this way (Part V).

13.10 Choosing, and Mixing

If the picture...	Use	Because
redraws many times a second with hundreds of marks	canvas	immediate mode, no per-shape cost
has a few dozen shapes you want to style, hover and click	SVG	the shapes are elements
must be read by a screen reader as parts	SVG with `<title>` and `aria`	canvas is one opaque image
is a diagram that scales with the page	SVG	vector, `viewBox`
is a heat map or particle field	canvas	pixels are the point
needs 3D	WebGL (Chapter 15)	the GPU

They mix well. A canvas chart under an SVG overlay for the hover line and labels gets fast curves and crisp, styleable annotations. A canvas has `role="img"` and an `aria-label`; for a chart that matters, add a visually hidden table of the data beside it.

Worked Example: A Bar Chart on a Canvas

Worked Example: Three bars

Draw bars for the values [30, 55, 20] on a canvas shown at 300 by 160 CSS pixels, with a 20-pixel margin and y axis from 0 to 60.

Step 1. Plot area. Margins of 20 leave a plot 260 wide and 120 tall, starting at (20, 20).

Step 2. Mapping. Value v maps to a bar top at y = 20 + 120 − (v / 60) × 120. For 30: 140 − 60 = 80. For 55: 140 − 110 = 30. For 20: 140 − 40 = 100. All bars end at the baseline y = 140.

Step 3. Bar width and positions. Three bars with equal slots of 260 / 3 = 86.7 px. Make each bar 60 wide, centred in its slot, so the left edges are at 20 + 13.3 = 33.3, 120, and 206.7.

Step 4. Code.

```
const vals = [30, 55, 20];
const slot = 260 / vals.length;
vals.forEach((v, i) => {
    const top = 20 + 120 - (v / 60) * 120;
    ctx.fillRect(20 + i * slot + (slot - 60) / 2, top, 60, 140 - top);
});
```

Step 5. Check. The first bar's height is 140 − 80 = 60 px, which is half of the 120 px plot, and 30 is half of 60. Yes.

Worked Example: A Block Diagram in SVG

Worked Example: Two blocks and a feedback wire

Draw a controller block, a plant block, a wire between them, and a feedback wire back, in SVG, with the dashes flowing.

Step 1. The coordinate system. `viewBox="0 0 300 120"`. Blocks are 80 by 40.

Step 2. The markup.

```
<svg viewBox="0 0 300 120" class="diagram" role="img" aria-label="A controller drivi
      ng a plant with feedback">
  <defs><marker id="arrow" viewBox="0 0 10 10" refX="9" refY="5" markerWidth="6" mar
      kerHeight="6" orient="auto">
    <path d="M0 0 L10 5 L0 10 Z" fill="currentColor"/></marker></defs>
  <g class="block"><rect x="40" y="30" width="80" height="40" rx="6"/><text x="80" y
      ="55" text-anchor="middle">PID</text></g>
  <g class="block"><rect x="180" y="30" width="80" height="40" rx="6"/><text x="220"
      y="55" text-anchor="middle">plant</text></g>
  <path class="wire" d="M120 50 H180" marker-end="url(#arrow)"/>
  <path class="wire" d="M260 50 H285 V100 H15 V50 H40" marker-end="url(#arrow)"/>
</svg>
```

Step 3. The CSS. `.block rect { fill: var(--bg-raised); stroke: var(--accent); } .block text { fill: var(--text); font: 12px Inter; } .wire { fill: none; stroke: currentColor; stroke-dasharray: 6 6; animation: flow 1.2s linear infinite; }`

Step 4. Why SVG here. Eight elements, each styleable, with a text label that a screen reader can read and that scales with the page. A canvas would need hand-drawn arrowheads, text positioning arithmetic and a script to animate the dashes; here the browser does all three.

Practice Problems A: Canvas

Review Problems

1. A canvas is shown at 400 by 250 CSS pixels on a DPR 2 screen. What `width` and `height` should its surface have? Where would a line drawn from (10, 10) to (110, 10) land on the surface?
2. A chart's x axis runs from 0 to 10 over a plot starting at pixel x = 50 and 400 px wide. What pixel x is the data value 2.5? What data value is at pixel 450?
3. A chart's y axis runs from −1 to 1 over a plot starting at pixel y = 20 and 200 px tall. What pixel y is the value 0? The value 1? Why is the value 1 *smaller* in pixels than the value −1?
4. What step does `niceTicks(0, 0.37, 4)` return, and what are the ticks? And for `niceTicks(-12, 48, 6)`?
5. Write the `arc` call for a full circle of radius 10 at (50, 50), and for the top half only.
6. A drawing function forgets `beginPath()` and draws a red line, then a blue line, each frame. What does the screen show after two frames?

7. A pointer event at `clientX = 320`, `clientY = 180` happens over a canvas whose top-left corner is at (100, 60) on the window. Where is the pointer in canvas CSS coordinates?
8. A gauge needle is drawn with `translate(cx, cy); rotate(a)`. The author wrote `rotate(a); translate(cx, cy)` by mistake. Describe what the needle does as `a` changes.
9. Why should a hover tooltip be drawn in the next frame from a stored index, and not inside the `pointermove` handler?

Practice Problems B: SVG and Choosing

Review Problems

1. Write the path data for a triangle with corners at (0, 0), (10, 0), (5, 8) in an SVG.
2. An SVG has `viewBox="0 0 200 100"` and is shown at `width="600"`. How big is one unit, and what is the displayed height?
3. Write a `<g>` that moves everything inside it to (40, 20) and rotates it 30 degrees.
4. Why does `document.createElement('rect')` not draw anything, and what is the fix?
5. Give the CSS that makes every `.wire` path's dashes flow, and say why it belongs in CSS rather than a script.
6. For each, canvas or SVG, with a reason: (a) 50 000 particles in a flow field; (b) a map with 300 clickable regions; (c) an icon that must change colour on hover; (d) a scrolling oscilloscope trace; (e) a pie chart with six slices and tooltips.
7. A chart drawn on a canvas must be usable by a screen-reader user. Name two things to add.

Mini Project: A Live Oscilloscope

Mini Project: Four hundred samples a second, drawn at sixty frames

Build a canvas instrument that shows a signal scrolling from right to left, like an oscilloscope, with a hover readout.

1. **The signal.** A function `sample(t)` returning, say, a damped oscillation (Chapter 11's mini project 3) plus a little noise. A ring buffer of the last 400 samples, filled at 400 samples per second from the animation loop using `dt` (several samples per frame, however long the frame took).
2. **The static layer.** An offscreen canvas with the grid, the y ticks from `niceTicks`, and the axis labels, rebuilt only when the canvas is resized.
3. **The moving layer.** Each frame: stamp the static layer, clip to the plot, draw the buffer as one path with the newest sample at the right edge, draw a fading trace with `globalAlpha` for the last 40 samples, restore.
4. **Hover.** On `pointermove`, store the x position; in the frame, convert it to a sample index, draw a vertical line and a label with the value using `measureText` to size a box that stays inside the

canvas.

5. **Controls.** A Pause button that stops filling the buffer but keeps drawing (so hover still works), and a slider for the signal's frequency, read from state each frame.
6. **Manners.** Stop the loop when the canvas scrolls out of view; show one still frame and no motion under `prefers-reduced-motion`.

Checks: the trace scrolls at the same speed on a 60 Hz and a 144 Hz screen (count seconds across the width); the Performance panel shows one short task per frame; resizing the window keeps the lines sharp; the console has no errors.

What You Learned

- Canvas: set state, `beginPath`, add commands, `stroke` or `fill`. Angles are radians, clockwise from three o'clock.
- Map data to pixels with one `x` and one `y`; round the ticks with the 1-2-5 rule; the axis top is a multiple of the step.
- Translate, rotate, draw upright, restore. Clip inside `save`/`restore`. Gradients and text are state like any other.
- Cache the static picture on an offscreen canvas; redraw only what moves; draw only while visible; offset hairlines by 0.5.
- Hit-test with your own data after converting pointer coordinates; store hover and drag in state and draw in the frame.
- SVG shapes are elements: styled by CSS, animated by CSS, clickable, readable, scaled by `viewBox`; create them with `createElementNS`.
- Canvas for many marks redrawn often; SVG for few shapes you want to style and click; both together when you want both.

Chapter 14. GSAP: Animation Without the Headaches

Chapter 6 taught you how animation works from first principles: a loop, delta time, easing. You can build everything that way, and understanding it is valuable. But there is a point where writing your own timeline, your own easing library, and your own scroll detection stops being educational and becomes plain slow. **GSAP** (the GreenSock Animation Platform) is a library that has already solved all of that, and solved it well. This chapter shows you how to use it, and every timeline in it was run through GSAP itself in Node (`Guide/code/gsap/timeline.js`) to check the arithmetic. A note on honesty: this website does not use GSAP. It does its animation by hand, for reasons Part V discusses. You are learning GSAP because it is one of the most widely used tools in the field, and because you will be asked about it.

In This Chapter

- Load GSAP and its plugins, and write tweens with `to`, `from`, `fromTo` and `set`.
- Choose an ease and say what it does to the motion.
- Stagger a group, and build timelines with the position parameter, labels and defaults.
- Control a timeline: play, pause, reverse, seek, scrub, speed.
- Trigger and scrub animations from scrolling with ScrollTrigger, including pinning.
- Respect reduced motion and clean up, and decide when GSAP earns its place.

Note

GSAP's licence has changed more than once over the years. Before using it in a commercial project, read its current licence terms on the GSAP website.

Words You Will Meet

Word	What it means
Tween	A single animation of one or more properties from one value to another.
Target	What a tween animates: an element, a selector string, or a plain object.
Timeline	A container that sequences tweens and lets you control them as a group.
Ease	The curve that shapes the speed of a tween (Chapter 6).
Stagger	Starting a group of tweens one after another with a small delay.
Position parameter	The extra argument that says *where on the timeline* a tween goes.
Label	A named point on a timeline.
ScrollTrigger	A plugin that starts or scrubs animations based on scroll position.
Scrub	Linking an animation's progress directly to the scroll position.
Pin	Holding an element in place while the page scrolls past.
Plugin	An add-on that must be registered before it is used.

14.1 Loading It

For learning, load GSAP from a CDN with a pinned version (Chapter 8's advice). In a bundled project, `npm install gsap` and `import { gsap } from 'gsap'`.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></scrip
      t>
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/ScrollTrigger.min.js
      "></script>
<script>gsap.registerPlugin(ScrollTrigger);</script>
```

> **WHY DO IT THIS WAY? Registering the plugin**
>
> **What it is.** `gsap.registerPlugin(ScrollTrigger)` tells GSAP's core that the plugin exists.
>
> **Why it is done this way.** The core is kept small. Plugins are optional, so each must announce itself.
>
> **Why it matters.** It stops the bundler from throwing away the plugin as "unused", and makes GSAP aware of it.
>
> **What happens if you do not.** ScrollTrigger options are silently ignored, and your scroll animations simply never fire. There is no error to tell you.

14.2 Tweens

The three tween methods are `gsap.to()` (animate from the current value *to* new ones), `gsap.from()` (animate *from* given values to the current ones) and `gsap.fromTo()` (state both). `gsap.set()` applies values immediately, with no animation.

```
gsap.to('.box', { x: 200, rotation: 90, duration: 1, ease: 'power2.out' });
gsap.from('.title', { opacity: 0, y: 30, duration: 0.6 });
gsap.fromTo('.bar', { scaleX: 0 }, { scaleX: 1, duration: 0.8, transformOrigin: 'lef
      t center' });
gsap.set('.panel', { opacity: 0 });
```

The first argument is the **target**: a selector string (every match is animated), an element, an array, or a plain object (`gsap.to(state, { t: 1 })` animates a number in your own state, which is how a canvas instrument can be driven by GSAP). The second is the **vars** object: properties to animate, plus special settings such as `duration`, `ease`, `delay`, `repeat` (−1 for ever), `yoyo`, `stagger`, `onUpdate`, `onComplete`.

Properties. `x`, `y`, `scale`, `scaleX`, `rotation`, `skewX` are applied as CSS transforms. `opacity`, `backgroundColor`, `width` and any CSS property work by name (camelCased). Numbers default to pixels (or degrees for rotation); strings carry units (`'50%'`, `'2rem'`). A **relative** value `'+=100'` moves from wherever the target is now. A tween returns an object with `.pause()`, `.play()`, `.reverse()`, `.progress(p)` and `.kill()`.

> **WHY DO IT THIS WAY? Animating x and y rather than left and top**
>
> **What it is.** GSAP's `x`, `y`, `scale` and `rotation` are applied as CSS **transforms**.
>
> **Why it is done this way.** As Chapter 2 explained, transforms and opacity can be handled by the compositor without layout.
>
> **Why it matters.** The animation stays smooth even when the page is busy.
>
> **What happens if you do not.** Animating `left` forces layout on every frame, and the animation judders on weaker devices.

14.3 Easing

An **ease** shapes the motion. The names are families with three variants each.

Ease	Feel
`none`	linear, mechanical; right for scrubbing and for steady motion
`power1` to `power4` `.out`	fast start, settling stop; stronger numbers settle longer
`power2.in`	slow start, fast finish; for things leaving
`power2.inOut`	both; for things moving between two places on screen
`back.out(1.7)`	overshoots slightly and settles; a playful arrival
`elastic.out(1, 0.3)`, `bounce.out`	springs and bounces; use sparingly
`expo.out`, `circ.out`	very fast start, very long settle
`steps(5)`	jumps in five equal steps; for sprite sheets and counters

In numbers: at a quarter of the way through a one-second tween from 0 to 100, `none` is at 25.0 and `power2.out` is already at 57.8 (checked in `timeline.js`). That is the "arrive fast, settle slowly" of Chapter 6. As a rule, ease things *out* as they arrive and *in* as they leave.

14.4 Staggers

```
gsap.from('.card', { opacity: 0, y: 20, duration: 0.5, stagger: 0.1 });
gsap.from('.cell', { scale: 0, duration: 0.4, stagger: { each: 0.05, from: 'center',
      grid: 'auto' } });
```

A number staggers each target that long after the previous one. The object form adds `from` (`'start'`, `'center'`, `'end'`, `'random'`, or an index) and `grid` for a two-dimensional ripple. Five boxes with `stagger: 0.2` and `duration: 1` take 1.8 seconds in all: the last box starts at 0.8 and ends at 1.8.

```
stagger: total 1.80 s; the last box starts at 0.80
```

> **WHY DO IT THIS WAY? Stagger instead of writing a delay for each item**
>
> **What it is.** One setting that starts each target 0.1 seconds after the last.
>
> **Why it is done this way.** The pattern "one after another" is so common it deserves its own word.
>
> **Why it matters.** A row of cards ripples in rather than appearing as a block, and adding a fifth card needs no extra code.
>
> **What happens if you do not.** You write a separate delay for each card, and every time the number of cards changes you must redo the arithmetic.

14.5 Timelines

A **timeline** is a sequence you can play, pause, reverse and scrub as one thing.

```
const tl = gsap.timeline({ defaults: { duration: 0.5, ease: 'power2.out' } });
tl.from('.logo', { opacity: 0, y: -20 })
  .from('.nav a', { opacity: 0, stagger: 0.1 })
  .from('.hero', { opacity: 0, y: 30 }, '-=0.2');
```

By default each tween starts when the previous one ends. The **position parameter** (the extra argument at the end) changes that.

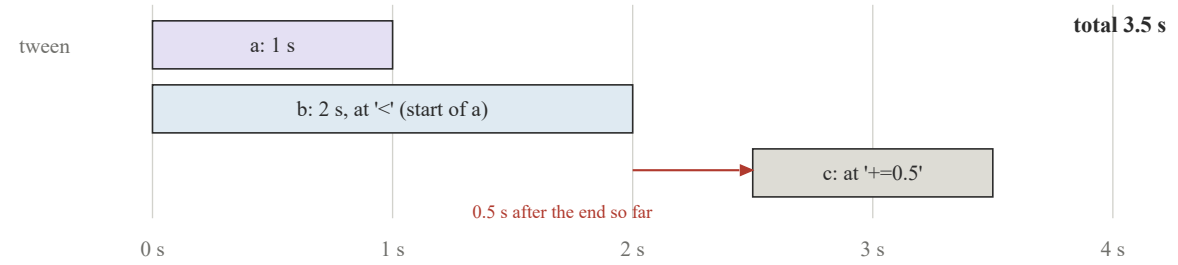


Figure 14.1. The worked example's three tweens on the timeline. `'<'` puts b at the start of a; `'+=0.5'` puts c half a second after the *latest end so far*, which is b's, not a's.

Position	Means
(none)	at the end of the timeline so far
`2`	at 2 seconds, absolute
`'+=0.5'` / `'-=0.5'`	half a second after / before the end so far
`'<'`	at the start of the previous tween
`'>'`	at the end of the previous tween
`'<0.3'`	0.3 s after the start of the previous tween
`'mid'`	at the label `mid` (added with `tl.addLabel('mid')`)
`'mid+=0.5'`	half a second after the label

GSAP computed these for the examples in this chapter:

```
nav sequence: logo: 0.00..0.50  nav: 0.50..1.20  hero: 1.00..1.50  total 1.50 s
labels:       a: 0.00..1.00  b: 1.00..2.00  c: 1.00..1.50  total 2.00 s
```

In the nav sequence the three links start at 0.5, 0.6 and 0.7 and each lasts 0.5, so the stagger ends at 1.2; the hero starts 0.2 before that. In the labelled one, `c` is placed at `'mid'`, which was added after `a`, so it overlaps `b`.

Controls. `tl.play()`, `pause()`, `resume()`, `reverse()`, `restart()`, `seek(1.2)` or `seek('mid')`, `progress(0.5)`, `timeScale(2)` (twice the speed), `duration()`, `kill()`. A timeline made with `{ paused: true }` waits to be told. Every one of these works on a tween too.

> **WHY DO IT THIS WAY? Using a timeline rather than a chain of delays**
>
> **What it is.** Tweens added to one timeline, positioned relative to each other.
>
> **Why it is done this way.** If you give each tween a fixed `delay`, they all depend on each other's durations, and you must recalculate when one changes.

> **Why it matters.** Change the first tween's duration and everything after it shifts automatically. You can also pause, reverse or jump to any point, just as the tour of the robot page uses a single clock that can be paused and restarted.
>
> **What happens if you do not.** A hand-timed sequence is brittle: one changed number means redoing the whole schedule.

Worked Example: When does each tween start and end?

Given the following, find the total duration of the timeline.

```
tl.to('.a', { x: 100, duration: 1 })
  .to('.b', { x: 100, duration: 2 }, '<')
  .to('.c', { x: 100, duration: 1 }, '+=0.5');
```

Step 1. Tween a. It is first, so it starts at 0 and ends at 1.

Step 2. Tween b. The position `'<'` means "at the start of the previous tween", which started at 0. So b runs 0 to 2.

Step 3. Tween c. `'+=0.5'` is measured from the **end of the timeline so far**. The timeline's end is the latest end among the tweens, which is 2 (from b), not 1. So c starts at 2.5 and ends at 3.5.

Step 4. Total. The timeline lasts 3.5 seconds. GSAP agrees:

```
worked example: a: 0.00..1.00  b: 0.00..2.00  c: 2.50..3.50  total 3.50 s
```

The trap. Many people expect c to start at 1.5 (after a, which was "the previous tween"). The `+=` is relative to the timeline's end, and the timeline's end is set by the longest tween so far.

14.6 ScrollTrigger

ScrollTrigger runs an animation when an element reaches a point in the viewport, or ties it to scrolling.

```
gsap.from('.panel', {
    opacity: 0, y: 40, duration: 0.8,
    scrollTrigger: { trigger: '.panel', start: 'top 80%', toggleActions: 'play none
      none reverse' }
});
```

`start: 'top 80%'` reads as "when the **top of the trigger** reaches **80% of the way down the viewport**". `end` has the same shape (`'bottom 20%'`). `toggleActions` lists what happens on enter, leave, enter back and leave back: `'play none none reverse'` plays on the way down and reverses on the way back up; `once: true` fires only the first time. `markers: true` draws the start and end lines on the page while you tune them.

With `scrub: true` the animation's progress follows the scrollbar instead of playing by itself; `scrub: 0.5` adds half a second of smoothing so it does not feel glued to the wheel. With `pin: true` the trigger element is held in place while the rest scrolls, for the length between `start` and `end`, which is how "scroll to play the sequence" sections are made.

```
gsap.timeline({ scrollTrigger: { trigger: '#story', start: 'top top', end: '+=2000',
      scrub: 0.5, pin: true } })
```

```
    .from('.frame-1', { opacity: 0 })
    .from('.frame-2', { opacity: 0, x: 100 })
    .to('.frame-1', { opacity: 0 }, '<');
```

> **WHY DO IT THIS WAY? Starting at 80% rather than at the very bottom**
>
> **What it is.** The animation starts when the element is a little way up the screen, not the moment its top peeks in.
>
> **Why it is done this way.** An animation that plays while the element is barely visible is wasted, because nobody sees it.
>
> **Why it matters.** The reader sees the element animate in once it is properly in view.
>
> **What happens if you do not.** Things finish animating before you scroll to them, and the effect is lost.

After images load or layout changes, call `ScrollTrigger.refresh()` so the start and end positions are measured again.

14.7 Reduced Motion, Responsiveness and Cleaning Up

GSAP does not read the visitor's motion preference for you. Do it yourself, with `gsap.matchMedia()`, which also handles breakpoints and reverts everything when a query stops matching.

```
const mm = gsap.matchMedia();
mm.add('(prefers-reduced-motion: no-preference) and (min-width: 800px)', () => {
    gsap.from('.panel', { opacity: 0, y: 40, scrollTrigger: '.panel' });
    return () => { /* optional extra cleanup */ };
});
mm.add('(prefers-reduced-motion: reduce)', () => { gsap.set('.panel', { opacity: 1 }
      ); });
```

> **WHY DO IT THIS WAY? matchMedia for the animation**
>
> **What it is.** The animation is created only when the media query matches, and cleaned up when it stops matching.
>
> **Why it is done this way.** The visitor's preference can change while the page is open, and so can the window width.
>
> **Why it matters.** Someone who asks for less motion gets none, a phone gets the simpler version, and a page that was animated reverts cleanly.
>
> **What happens if you do not.** You animate for everyone, including people it makes ill (Chapter 6), and a pinned desktop sequence traps phone visitors.

Animations hold references to elements. When a component or a page section goes away, kill them: `tl.kill()`, `ScrollTrigger.getAll().forEach(t => t.kill())`, or wrap the setup in `gsap.context(() => { ... }, scope)` and call `ctx.revert()` to undo everything at once. In React, the context goes in an effect and `revert` in its cleanup (Chapter 12).

14.8 When GSAP Earns Its Place

GSAP is about 70 kilobytes with ScrollTrigger, and it does the awkward parts well: resizing, refreshing, reversing, overlapping sequences, scrubbing with inertia. For four fading sections, Chapter 6's twenty lines of CSS and an `IntersectionObserver` are lighter (Project 3 builds both and compares). For a choreographed scroll story with pins, labels and a dozen overlapping tweens, GSAP saves days. The decision is engineering: weigh the download and the dependency against the code you would otherwise write and maintain. Part V shows the one place on this website where GSAP would fit and why the hand-written version was kept.

Practice Problems A: Tweens and Timelines

Review Problems

1. Five boxes use `gsap.to('.box', { x: 100, duration: 1, stagger: 0.2 })`. When does the last box start and finish, and what is the total time?
2. In a timeline, `tl.to(a, {duration: 2}).to(b, {duration: 1}, '-=0.5').to(c, {duration: 1}, 0)`. Give the start and end time of each tween and the total length.
3. Explain why animating `x: 200` is smoother than animating `left: 200px`.
4. Write a timeline in which a heading fades in over 0.6 s, then three list items fade in with a 0.15 s stagger, with the list starting 0.2 s before the heading finishes.
5. A tween with `ease: 'power2.out'` runs from 0 to 100 over one second. At 0.25 s it is at about 58. Where is it with `ease: 'none'`, and roughly where would you expect it with `power2.in`?
6. What is the difference between `'<'`, `'>'` and `'+=0'` as position parameters?
7. A timeline is built once and a menu button should open it and close it. Which two calls, and why is that better than two timelines?

Practice Problems B: Scroll, Manners and Judgement

Review Problems

1. A viewport is 900 px tall. Where, in pixels from the top of the viewport, does `start: 'top 80%'` fire? What about `start: 'top center'` and `start: 'bottom bottom'`?
2. What does `scrub: 0.5` do that `scrub: true` does not?
3. A pinned scroll section works on desktop and traps visitors on phones. What is the fix, in one call?
4. A visitor has `prefers-reduced-motion: reduce` set. Show how to wrap an animation so that it does not run for them, and what they should see instead.
5. Images load after ScrollTrigger measured the page and the triggers now fire in the wrong places. What one call fixes it?
6. Name the three things that must be cleaned up when a React component that created a timeline with ScrollTrigger unmounts, and the one GSAP call that does all three.

7. For each, GSAP or by hand: (a) four sections fading in on scroll; (b) a pinned product story with twelve overlapping tweens scrubbed by scroll; (c) a hover lift on a card; (d) an 88-frame image sequence scrubbed by scroll.

Mini Project: A Scroll Story, Twice

Mini Project: The same page by hand and with GSAP

Build a page of five full-height sections that tell the story of one breakfast run: approach, grip, lift, carry, place. Each section has a heading, a paragraph and a simple SVG of the robot's arm in the right pose.

1. **By hand first.** Hidden and shown states in CSS, an `IntersectionObserver` that adds the class once, and a `prefers-reduced-motion` block that shows everything. Measure: how many lines, how many kilobytes.
2. **Then with GSAP.** Delete the observer. A `gsap.matchMedia()` with a no-preference query; inside it, a `gsap.from` per section with `scrollTrigger: { start: 'top 80%', once: true }`, and a timeline per section that staggers the heading, the paragraph and the SVG with the SVG's arm rotating into pose (`rotation` on a `<g>` with `transformOrigin` at the pivot).
3. **A pinned finale.** The last section pins for 1 500 px of scrolling while a timeline, scrubbed with `scrub: 0.5`, moves the arm through the whole motion and counts a readout from 0 to 100 percent (animate a number on a plain object and write it into the page in `onUpdate`).
4. **Manners.** The pin and scrub exist only for `(min-width: 800px)` and `(prefers-reduced-motion: no-preference)`; phones get the once-only reveals; reduced motion gets the final states.
5. **Cleanup.** Wrap everything in `gsap.context` and prove you can `revert()` from the console and be left with a plain page.

Checks: `markers: true` shows every start line where you intended; the Performance panel shows no long tasks while scrolling; the hand-made version is lighter and the GSAP version is shorter, and you can say which you would ship, and why.

What You Learned

- Load the core and register each plugin. Tweens are `to`, `from`, `fromTo`, `set` on selectors, elements or plain objects; animate transforms and opacity.
- Eases shape speed: `.out` to arrive, `.in` to leave, `.inOut` to travel; `none` for scrubbing.
- Stagger with a number or an object with `each`, `from` and `grid`.
- Timelines sequence tweens; the position parameter (`'<'`, `'>'`, `'+=0.5'`, labels, absolute times) places them, and `+=` is relative to the *latest end so far*.
- Control with play, pause, reverse, seek, progress, timeScale; one timeline reversed is the cleanest open-and-close.

- ScrollTrigger: `start`/`end` in "trigger viewport" pairs, `toggleActions`, `once`, `scrub` (with smoothing), `pin`, `markers`, `refresh()`.
- `gsap.matchMedia()` for reduced motion and breakpoints; `gsap.context()` and `revert()` to clean up.
- Weigh the library against the code you would write. For a few fades, do it by hand; for choreography, GSAP.

Chapter 15. three.js: A 3D World in Your Page

three.js is a JavaScript library that makes WebGL (the browser's 3D drawing system) bearable. Raw WebGL needs hundreds of lines before it draws one triangle. three.js needs about twenty to draw a spinning cube with proper lighting. This website uses it on the Patrick and drone pages, so this chapter has a particular goal: by the end, you should be able to read those pages' code and understand every part, and to build a 3D instrument of your own. Every number in the worked examples was checked by three.js itself, release 186, in Node (`Guide/code/three/check.mjs`).

In This Chapter

- Set up a renderer, scene, camera, light and loop, and resize them correctly.
- Explain the camera's parameters and position it by orbit angles or to fit an object.
- Build geometry from primitives and from raw vertex data, and merge it.
- Choose materials by what they do with light, and light a scene with shadows and an environment.
- Use the scene tree for pivots and assemblies, and pose a model from state.
- Pick objects with a raycaster, load models, and keep a scene fast on weak hardware.

Words You Will Meet

Word	What it means
Scene	The container for everything you want to draw.
Camera	The viewpoint (usually a `PerspectiveCamera`).
Renderer	The thing that draws the scene, as seen by the camera, onto a canvas.
Mesh	A visible object: a geometry plus a material.
Geometry	The shape: vertices and triangles, held in typed arrays (a `BufferGeometry`).
Material	How the surface looks: colour, roughness, metalness, transparency.
Light	A source of illumination (ambient, hemisphere, directional, point, spot).
Object3D / Group	A node in the scene tree that can hold children and be moved as one.
Local / world space	Coordinates relative to a node's parent / to the scene.
Raycaster	A line shot through the scene to find what it hits (used for picking).
Texture	An image applied to a surface.
Environment map	A panorama used for reflections and ambient light.
Draw call	One command sent to the GPU to draw something.
Instancing	Drawing one geometry many times with different transforms in one draw call.

15.1 The Smallest Scene

```
import * as THREE from 'three';

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(36, innerWidth / innerHeight, 0.05, 50);
camera.position.set(2, 1.5, 3);
camera.lookAt(0, 0, 0);

const cube = new THREE.Mesh(
    new THREE.BoxGeometry(1, 1, 1),
    new THREE.MeshStandardMaterial({ color: '#a78bfa', roughness: 0.4 })
);
scene.add(cube);
scene.add(new THREE.DirectionalLight('#ffffff', 3).translateX(2).translateY(3));
scene.add(new THREE.AmbientLight('#8899cc', 0.5));

const clock = new THREE.Clock();
function frame() {
    const dt = Math.min(clock.getDelta(), 0.1);
    cube.rotation.y += 0.8 * dt;                 // radians per second, not per fram
      e (Chapter 6)
    renderer.render(scene, camera);
    requestAnimationFrame(frame);
}
frame();
```

> **WHY DO IT THIS WAY? The five ingredients**
>
> **What it is.** Every three.js program has a renderer, a scene, a camera, something to look at, and a loop.
>
> **Why it is done this way.** It mirrors a photographer's studio: a set (the scene), something in it (the mesh), a light, a camera, and a shutter (the render call).
>
> **Why it matters.** When nothing appears, one of the five is missing, and you can check them in order. The most common culprits are no light (a standard material is black without light), the camera inside the object, or no render call.
>
> **What happens if you do not.** You stare at a black rectangle and guess.

Units are yours to choose, but choose metres and stick to them: the lights' physical falloff, the camera's near and far planes and any model you load all assume one consistent unit. The Patrick robot is 0.3 m tall in the scene because it is 0.3 m tall on the desk.

15.2 The Renderer: Size, Pixel Ratio, Colour and Tone

```
const renderer = new THREE.WebGLRenderer({ canvas, antialias: true, alpha: false });
function resize() {
    const w = canvas.clientWidth, h = canvas.clientHeight;
    renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
    renderer.setSize(w, h, false);              // false: do not touch the CSS size
    camera.aspect = w / h;
    camera.updateProjectionMatrix();
}
addEventListener('resize', resize); resize();
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 1.0;
```

> **WHY DO IT THIS WAY? updateProjectionMatrix and a capped pixel ratio**
>
> **What it is.** After changing the camera's `aspect`, you must call `camera.updateProjectionMatrix()`. And the pixel ratio is capped at 2.
>
> **Why it is done this way.** The projection matrix (Chapter 7) is calculated once and stored. Changing `aspect` does not recalculate it by itself. The cap stops a 3x-density phone from rendering nine pixels for every one it needs.
>
> **Why it matters.** The picture keeps the right proportions, and rendering cost stays reasonable.
>
> **What happens if you do not.** After resizing the window, the scene looks stretched, because the old matrix is still in use. Without the cap, a high-density phone renders far more pixels than the eye can tell apart, and runs slowly and hot.

three.js checked this for the book: with a 36° camera, the first element of the projection matrix is 3.0777; after setting `aspect = 2` it is *still* 3.0777, and only after `updateProjectionMatrix()` does it become 1.5388.

Colour and brightness are handled for you if you leave the defaults: the renderer works in linear light and writes sRGB to the screen (`outputColorSpace`), and **tone mapping** squeezes the wide range of a lit scene into what a screen can show. ACES Filmic is the usual choice; `toneMappingExposure` is the one knob to turn when the whole picture is too dark or too bright.

15.3 Cameras

`new THREE.PerspectiveCamera(fov, aspect, near, far)`: the vertical field of view in degrees, the width-to-height ratio of the canvas, and the nearest and farthest distances that are drawn.

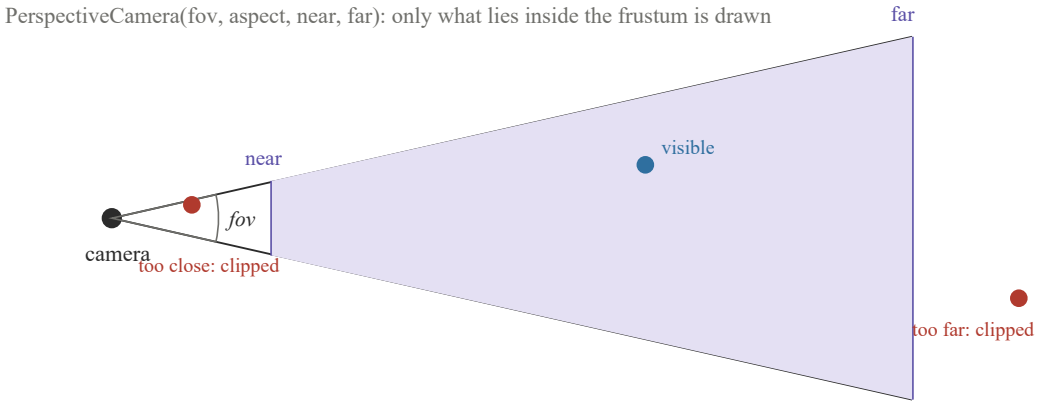


Figure 15.1. The camera sees a truncated pyramid, the frustum. Anything closer than `near` or farther than `far` is clipped away; the depth buffer's precision is spread between those two planes, so keep them tight (Chapter 7).

A camera is positioned like any object (`position.set`) and aimed with `lookAt(x, y, z)`. An **orbit** camera, the kind on every viewer page of this website, keeps a target and three numbers: azimuth, elevation, distance.

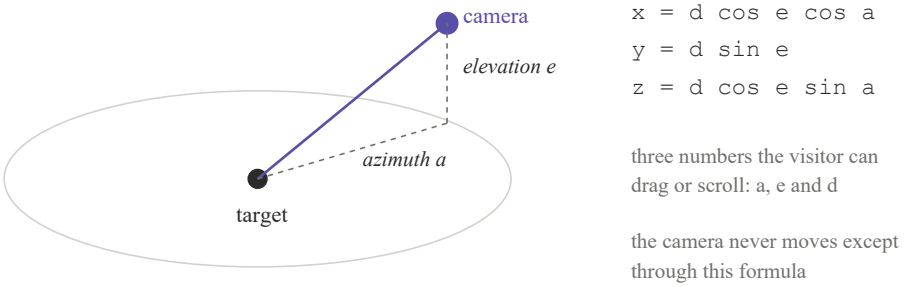


Figure 15.2. An orbit camera is three numbers around a target. Dragging changes the azimuth and elevation; the wheel changes the distance; the position is always computed from the formula, never set by hand.

```
camera.position.set(target.x + d * Math.cos(el) * Math.cos(az), target.y + d * Math.
      sin(el), target.z + d * Math.cos(el) * Math.sin(az));
camera.lookAt(target);
```

For a quick viewer, `OrbitControls` from `three/examples/jsm/controls/OrbitControls.js` does the dragging for you (`new OrbitControls(camera, canvas)`, then `controls.update()` each frame). The website writes its own so that the camera can ease toward wanted values (Chapter 6) and be driven by a tour.

Worked Example: How far back should the camera be?

An object is 0.30 m tall. The camera has a vertical field of view of 36 degrees. How far should the camera be to just fit the object vertically, and where is it if it orbits at azimuth −50 degrees, elevation

22 degrees and distance 0.78 m (the Patrick page's starting view)?

Step 1. The geometry. The visible height at distance d is 2 d tan(θ/2). To fit height H, d = (H/2) / tan(θ/2).

Step 2. Numbers. (0.15) / tan(18°) = 0.15 / 0.32492 = 0.462 m.

Step 3. Orbit position. A camera orbiting a target at distance d, azimuth a and elevation e sits at (d cos e cos a, d sin e, d cos e sin a) from the target.

Step 4. Evaluate. With d = 0.78, e = 22°, a = −50°: x = 0.78 × 0.9272 × 0.6428 = 0.465; y = 0.78 × 0.3746 = 0.292; z = 0.78 × 0.9272 × (−0.7660) = −0.554.

Step 5. Check. The distance from the target should be 0.78: √(0.465² + 0.292² + 0.554²) = √(0.2162 + 0.0853 + 0.3069) = √0.6084 = 0.78. √ three.js gives `[0.465, 0.292, -0.554]`, length 0.780, and a fit distance of 0.462 m.

Why the orbit camera is built this way. One angle for left and right (azimuth), one for up and down (elevation), and one distance are the three things a visitor can drag or scroll. The code never touches `camera.position` directly except through this formula.

To frame any object automatically: `new THREE.Box3().setFromObject(obj)` gives its bounds, `getSize` and `getCenter` give the size and centre, and the fit formula above (with the larger of height and width ÷ aspect) gives the distance. The robot's bounding box, from `check.mjs`, is 0.2 by 0.3 by 0.25 m around its centre.

15.4 Geometry

Built-in geometries cover most instruments: `BoxGeometry`, `SphereGeometry`, `CylinderGeometry` (also cones), `TorusGeometry`, `PlaneGeometry`, `CircleGeometry`, `RingGeometry`, `LatheGeometry` (a profile spun about an axis: a shaft, a nozzle, a wheel), `ExtrudeGeometry` (a 2D shape pulled into 3D: a bracket, a plate with holes), `TubeGeometry` (a tube along a curve: a wire, a pushrod). Each is a `BufferGeometry` underneath: typed arrays of vertex positions, normals and texture coordinates, plus an index that says which three vertices make each triangle.

You can build one directly, which is how a chart or a terrain becomes 3D:

```
const g = new THREE.BufferGeometry();
const positions = new Float32Array([0, 0, 0,  1, 0, 0,  0, 1, 0]);     // three vert
      ices, one triangle
g.setAttribute('position', new THREE.BufferAttribute(positions, 3));
g.computeVertexNormals();                                              // so lighting
      works
```

Geometry can be transformed once at build time (`geometry.translate(x, y, z)`, `.rotateX(a)`, `.scale(...)`), which is different from moving the mesh: it bakes the change into the vertices. That is what makes **merging** possible: transform each part into place, then `mergeGeometries([...])` from `three/examples/jsm/utils/BufferGeometryUtils.js` joins them into one. three.js merged 300 small boxes for the book into a single geometry of 7 200 vertices (300 × 24), which draws in one call instead of 300.

> **WHY DO IT THIS WAY? Merging tiny parts by material**
>
> **What it is.** The robot is made of hundreds of tiny features (screws, pins, chip legs). Those that share a material are merged into one mesh, so they cost one draw call between them.
>
> **Why it is done this way.** Draw calls are expensive and triangles are cheap (Chapter 7).
>
> **Why it matters.** The detailed robot still runs smoothly on a laptop's built-in graphics.
>
> **What happens if you do not.** Hundreds of separate meshes mean hundreds of draw calls, and the frame rate collapses long before the GPU is busy.

15.5 Materials

A material decides what a surface does with light. Choose by that, not by appearance.

Material	Lighting	Use it for
`MeshBasicMaterial`	none: flat colour or texture	wireframes, unlit overlays, debug
`MeshLambertMaterial`	matte (Chapter 7's n · l)	cheap, soft surfaces; many objects
`MeshStandardMaterial`	physically based: `roughness`, `metalness`	nearly everything
`MeshPhysicalMaterial`	Standard plus `clearcoat`, `transmission`, `sheen`, `ior`	lacquered plastic, glass, fabric
`LineBasicMaterial`, `PointsMaterial`	lines and points	axes, traces, particle fields

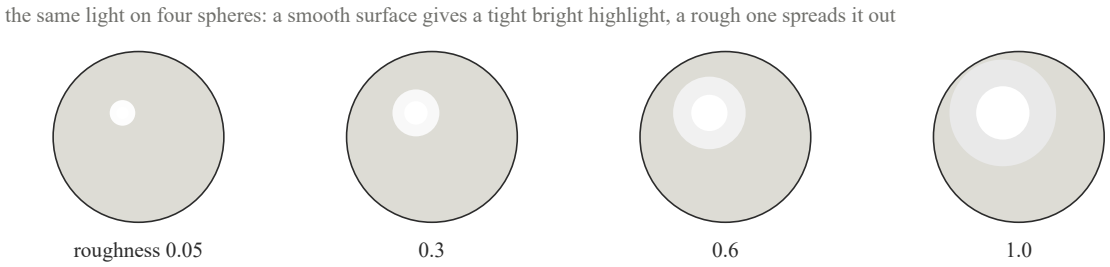


Figure 15.3. Roughness is the one slider that changes a material most. Near zero, the highlight is a tight mirror of the light; near one, it spreads over the whole surface. Metalness decides whether the base colour tints the reflection (metal) or sits under it (plastic).

```
new THREE.MeshStandardMaterial({ color: '#f2b400', roughness: 0.45, metalness: 0 });
      // yellow plastic
new THREE.MeshStandardMaterial({ color: '#cfd3d8', roughness: 0.3, metalness: 1 });
      // brushed steel
new THREE.MeshPhysicalMaterial({ color: '#f2b400', roughness: 0.3, clearcoat: 1, cle
      arcoatRoughness: 0.1 });   // glossy coat
new THREE.MeshPhysicalMaterial({ color: '#ffffff', transmission: 1, roughness: 0.05,
      thickness: 0.01 });        // acrylic
```

Other properties you will set: `transparent: true` with `opacity`, `side: THREE.DoubleSide` for thin single-sided shapes, `wireframe: true`, `emissive` and `emissiveIntensity` for things that glow (an LED), `map` for a colour texture, `bumpMap` or `normalMap` for fine surface detail,

`envMapIntensity` for how strongly the environment reflects. A material is shared by every mesh that uses it, so one object is enough per look; change `material.color.set('#...')` and every mesh with it changes.

> **WHY DO IT THIS WAY? Giving glossy things something to reflect**
>
> **What it is.** `scene.environment = texture`, here made from a small virtual studio with a few bright panels.
>
> **Why it is done this way.** A smooth surface shows a mirror image of its surroundings. If there is nothing in the surroundings, there is nothing to show.
>
> **Why it matters.** Plastics get a highlight, metals get depth, and the model looks like a photograph.
>
> **What happens if you do not.** Metals look flat grey, because a metal's colour comes almost entirely from reflections.

15.6 Lights, Shadows and the Environment

Light	What it does	Cost
`AmbientLight(color, intensity)`	lights everything equally; no shading	nothing
`HemisphereLight(sky, ground, intensity)`	sky colour from above, ground colour from below; a cheap "outdoors"	nothing
`DirectionalLight`	parallel rays from a direction, like the sun; casts shadows	low
`PointLight`	rays from a point in all directions; falls off with distance	medium
`SpotLight`	a cone from a point	medium
`RectAreaLight`	a soft rectangular panel, like a softbox	higher

Since release 155, intensities are in physical units and the defaults are physically correct: a directional light of 3 and an ambient of 0.5 are sensible starting values, and point lights fall off with the square of distance, so put them close or make them strong.

Shadows need three switches and cost a second render from the light's point of view:

```
renderer.shadowMap.enabled = true;
sun.castShadow = true; sun.shadow.mapSize.set(1024, 1024);
mesh.castShadow = true; floor.receiveShadow = true;
```

The **environment** is a panorama that supplies reflections and soft ambient light at once. The easiest good one is the room provided with three.js:

```
import { RoomEnvironment } from 'three/examples/jsm/environments/RoomEnvironment.js'
      ;
const pmrem = new THREE.PMREMGenerator(renderer);
scene.environment = pmrem.fromScene(new RoomEnvironment()).texture;
```

The Patrick page builds its own small studio (an overhead softbox, a side strip, a warm fill) and runs it through the same `PMREMGenerator`, which is what gives its plastics and steel their highlights (Chapter 7).

Worked Example: The brightness of a tabletop

A flat table has normal n = (0, 1, 0). Find its Lambert brightness under a light in the direction (0.6, 0.8, 0), and then from (1, 1, 0).

Step 1. The first light. Its direction is already a unit vector (0.36 + 0.64 = 1). $n \cdot l = 0 \times 0.6 + 1 \times 0.8 + 0 \times 0 = 0.8$.

Step 2. The second light. Normalise (1, 1, 0): length $\sqrt{2}$, so l = (0.707, 0.707, 0). $n \cdot l = 0.707$.

Step 3. Check. three.js's `Vector3.dot` gives 0.800 and 0.707. √ A `MeshLambertMaterial` under a directional light does exactly this sum per vertex; the Standard material adds the roughness and environment terms on top.

15.7 The Scene Tree

A `Group` holds children. Moving, rotating or scaling the group moves all its children. This is how the robot is built: a root group contains the wheels, the arm, the claw. And the claw is a child of the arm's end, so lifting the arm carries it along.

```
const arm = new THREE.Group();
const bar = new THREE.Mesh(new THREE.BoxGeometry(0.24, 0.012, 0.004), material);
bar.position.x = 0.12;              // bar's centre is 0.12 from the pivot
arm.add(bar);
arm.rotation.z = Math.PI / 6;       // swing the whole arm about its pivot
```

> **WHY DO IT THIS WAY? Putting the pivot at the group's origin**
>
> **What it is.** The bar is offset from the group's origin by half its length, so the group's origin is the pivot point.
>
> **Why it is done this way.** A rotation always happens around the object's own origin.
>
> **Why it matters.** Rotating the group now swings the bar around its end, like a real arm. Rotating the bar alone would spin it around its middle.
>
> **What happens if you do not.** The arm spins about its centre, and the model looks broken. This is the single most common newcomer mistake in 3D.

three.js confirms the arithmetic: after the 30° swing, the bar's centre is at (0.1039, 0.06, 0), which is (0.12 cos 30°, 0.12 sin 30°). And a child at (1, 0, 0) inside a group turned 90° about y ends up at (0, 0, −1) in the world.

Every node has a **local** position (relative to its parent) and a **world** position (`obj.getWorldPosition(v)`). `obj.lookAt(point)` aims a node at a world point. Rotations are Euler angles in radians (`rotation.set(x, y, z, order)`), or a quaternion for anything that must avoid gimbal lock, such as a free-flying drone. Posing a model from state is a function: given the servo

angle, the wheel angles and the jaw opening, set each node's rotation, every frame, from scratch (Chapter 5 and Part V).

15.8 Animation

The loop is Chapter 6's: a `Clock`, a clamped `dt`, update then render. Three patterns cover most needs. **Pose from state**: compute every rotation and position from a few numbers each frame (the robot). **Keyframes**: an `AnimationMixer` plays clips that come with a loaded model (a walking character). **Procedural**: a small sway from `Math.sin(t)` on the drone's rotation, a slow orbit of the camera when the visitor is idle.

```
function frame() {
    const dt = Math.min(clock.getDelta(), 0.1);
    sim.step(dt);                                  // physics in its own module
    pose(robot, sim.state);                        // the picture is a function of t
      he state
    renderer.render(scene, camera);
    if (visible) requestAnimationFrame(frame);
}
```

Render only while the canvas is visible (an `IntersectionObserver`), and give a still frame under `prefers-reduced-motion`.

15.9 Picking: What Did the Visitor Click?

A **raycaster** shoots a line from the camera through the pointer and tells you what it hit.

```
const ray = new THREE.Raycaster(), ndc = new THREE.Vector2();
function pick(clientX, clientY) {
    const r = canvas.getBoundingClientRect();
    ndc.set(((clientX - r.left) / r.width) * 2 - 1, -((clientY - r.top) / r.height)
      * 2 + 1);
    ray.setFromCamera(ndc, camera);
    const hits = ray.intersectObjects(meshes, false);
    return hits.length ? hits[0].object : null;
}
```

> **WHY DO IT THIS WAY? The two conversions in the pointer position**
>
> **What it is.** The pointer position is converted to **normalised device coordinates** (−1 to 1 on each axis) with the y axis flipped.
>
> **Why it is done this way.** The camera works in that coordinate space (Chapter 7), where y is *up*, and the screen has y *down*.
>
> **Why it matters.** The ray goes where the visitor pointed.
>
> **What happens if you do not.** Clicks pick things in the wrong place, mirrored top to bottom.

A hit carries `object`, `point` (where in world space), `distance` and `face`. For the book, a ray from the screen centre (NDC 0, 0) with the camera at z = 2 hit a unit cube at a distance of 1.500: the near face is at z = 0.5. Put the part's record in `mesh.userData` so a hit tells you what it is (Project 5). The raycaster ignores the `visible` flag, so filter hidden meshes out before testing (Part V shows why that

matters on the Patrick page).

15.10 Textures and Models

A **texture** is an image on a surface. `new THREE.TextureLoader().load('grain.png')` gives one; set `texture.colorSpace = THREE.SRGBColorSpace` for colour maps (not for bump, normal or roughness maps, which hold data, not colour), and `wrapS`/`wrapT` with `repeat` to tile. A `CanvasTexture` turns a canvas you drew (a label, a gauge face, the fine grain of moulded plastic) into a texture, which is how the robot's bump maps are made without any image files.

Models come from modelling tools as **glTF** (`.glb`), the format three.js reads best:

```
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js';
new GLTFLoader().load('arm.glb', gltf => { scene.add(gltf.scene); });
```

A loaded model may use a different up axis or unit (Chapter 7); rotate and scale its root group once, at the boundary, and leave the rest alone. Both pages of this website build their geometry from code instead of loading files, which keeps each page under a megabyte and makes every dimension a named number.

15.11 Keeping It Fast

The budget is Chapter 2's frame, and the usual limit is draw calls, not triangles. `renderer.info.render.calls` tells you how many you are making; aim for under a hundred on a page.

- **Merge** static parts that share a material (15.4). **Instance** repeated parts: `new THREE.InstancedMesh(geometry, material, count)`, then `setMatrixAt(i, matrix)` for each copy, draws them all in one call. Screws, LEDs on a strip, a field of particles.
- **Cap the pixel ratio** (15.2) and **size the canvas** no larger than it is shown.
- **Share materials**; a new material per mesh means a new shader program per mesh.
- **Keep near and far tight** for depth precision, and **frustum culling** (on by default) skips what the camera cannot see.
- **Quality tiers.** Decide at start-up from the device (cores, memory, touch) which effects to enable, and measure the frame time; if it stays long, step down a tier: smaller shadow maps, no ambient occlusion, a lower pixel ratio. The Patrick page has three tiers and switches between them without reloading.
- **Warm up shaders** with `await renderer.compileAsync(scene, camera)` before the first frame, so the first appearance of each material does not stall.
- **Dispose** of what you stop using: `geometry.dispose()`, `material.dispose()`, `texture.dispose()`. GPU memory is invisible to the garbage collector.

> **WHY DO IT THIS WAY? Calling dispose**
>
> **What it is.** An explicit "I am done with this" call.
>
> **Why it is done this way.** GPU memory is outside JavaScript's control.

> **Why it matters.** A page that creates and discards objects (a game, a viewer that swaps models) stays healthy.
>
> **What happens if you do not.** Memory climbs until the browser slows or the tab crashes.

15.12 Bundling a three.js Page

three.js is a package, so a page that uses it needs a build (Chapter 16): `npm install three`, write `main.js` with `import * as THREE from 'three'`, and bundle with `npx esbuild main.js --bundle --minify --format=iife --outfile=app.js`. The result is one file of about 600 kilobytes for the core, more with the examples modules you import. Import only what you use (`three/examples/jsm/...` paths, never the whole examples folder) so the bundler can leave the rest out.

Worked Example: A Needle Gauge in 3D

Worked Example: A dial that reads a number

Build a round gauge whose needle turns to show a value v between 0 and 1, with the pivot at the dial's centre and a 270-degree sweep, and check the pose.

Step 1. The dial. A `CylinderGeometry(0.1, 0.1, 0.01, 64)` rotated so its flat face points at the camera (`geometry.rotateX(Math.PI / 2)`), with a matte dark material.

Step 2. The needle, with its pivot at the origin. A thin box `BoxGeometry(0.004, 0.08, 0.004)` translated up by 0.035 so that it extends from just below the pivot to near the rim: `geometry.translate(0, 0.035, 0)`. Put it in a `Group` called `needle` at the dial's centre, slightly in front of the face.

Step 3. The pose. At v = 0 the needle points to the lower left, 135° anticlockwise of straight up; at v = 1 to the lower right. A rotation about the z axis (towards the camera) by `THREE.MathUtils.degToRad(135 - 270 * v)` does it: positive z rotation is anticlockwise when seen from the front. For v = 0.3 that is 135 − 81 = 54°, up and to the left. For v = 0.5 it is 0°, straight up. √

Step 4. Drive it from state. Each frame, `needle.rotation.z = degToRad(135 - 270 * state.v)`. A slider, a simulation or a tour can all set `state.v`, and the needle follows. Add a `CanvasTexture` of tick marks drawn with Chapter 13's arc code for the dial face, and you have a 3D instrument built from the same ideas as the 2D one.

Practice Problems A: Scene, Camera and Tree

Review Problems

1. A camera has a vertical field of view of 50 degrees. What height of the world is visible at a distance of 2 m? What field of view would show exactly 1.2 m at a distance of 3 m?
2. A group is at position (0, 0, 0) with a child at (1, 0, 0). The group is rotated by 90 degrees about the y axis. Where is the child in world space? (Use $x' = x\cos\theta + z\sin\theta$, $z' = -x\sin\theta + z\cos\theta$.)
3. You write `camera.aspect = w / h` in a resize handler, and the scene looks stretched after resizing. What did you forget?
4. An arm bar is 0.24 m long and must swing about one end. Give the two lines that place it in a group so that the group's rotation is the swing, and the world position of its centre after a 30° swing.
5. The orbit camera is at azimuth 90°, elevation 0°, distance 1 from the origin. Where is it? And at azimuth 0°, elevation 90°?
6. Nothing appears on the canvas. List the five things to check, in order.
7. Why is `cube.rotation.y += 0.01` wrong in a frame loop, and what replaces it?

Practice Problems B: Materials, Lights, Picking and Speed

Review Problems

1. A metal sphere looks flat grey and lifeless. Give two possible causes and fixes.
2. Which material for each: a wireframe debug overlay; a thousand matte pebbles; the yellow plastic of a robot's shell with a glossy coat; a sheet of acrylic?
3. A directional light at (3, 4, 0) lights a floor with normal (0, 1, 0). Find the Lambert brightness. (Normalise first.)
4. A scene has 300 meshes with the same material. Roughly what does merging them into one geometry change, and what can you no longer do individually? When would you instance instead?
5. A click picks a part that is hidden (`visible = false`) in front of the one the visitor meant. Why, and what is the fix?
6. A page creates 50 new textures every minute and never disposes of them. Predict what happens over an hour and say what to add.
7. The page runs at 60 fps on a desktop and 12 fps on a laptop. Name four things to try, in the order you would try them.
8. A colour texture looks washed out. What property was not set?

Mini Project: An Instrument You Can Take Apart

Mini Project: A gearbox, posed, picked and exploded

Build a small gearbox from primitives and make it an instrument, using everything in this chapter.

1. **Parts from code.** A housing (a box with `ExtrudeGeometry` or merged boxes), two gears (`CylinderGeometry` with teeth made by instancing a small box around the rim with `InstancedMesh`), two shafts (`CylinderGeometry`), and four screws (instanced). Every dimension a named constant in metres.
2. **The tree.** Each gear in its own `Group` with the pivot on the shaft axis. One number, `angle`, poses both gears: the second turns the opposite way at the ratio of the tooth counts.
3. **Materials and light.** Standard materials for the housing and gears, a metallic one for the shafts, a `RoomEnvironment`, one directional light with shadows and a hemisphere fill. Tone mapping on.
4. **The loop.** A `Clock`, a clamped `dt`, `angle += speed * dt`, pose from `angle`, render only while visible. A slider sets `speed`.
5. **Explode.** A second number, `explode` from 0 to 1, moves each part along its own offset (Project 5's pattern). A slider drives it; the tour eases it.
6. **Picking.** A raycaster that ignores hidden parts, highlights the hit part with an emissive colour, and writes its name and dimension into a panel from `userData`.
7. **Speed.** Print `renderer.info.render.calls` in the panel and get it under 20. Cap the pixel ratio at 2; add a "lite" tier that drops shadows when the frame time stays over 25 ms for a second.
8. **Bundle** with esbuild and open it on the weakest device you can find.

Checks: the gears mesh without passing through each other (adjust a number, never the camera); the exploded view returns exactly to the assembled one; disposing the scene from the console frees memory (watch the GPU memory in the task manager); the page still shows a still frame under reduced motion.

What You Learned

- Renderer, scene, camera, light, loop: check them in that order when nothing appears. One unit (metres) everywhere.
- Resize sets the pixel ratio (capped), the size and the camera's aspect, then `updateProjectionMatrix()`. Leave colour management to the defaults; tune `toneMappingExposure`.
- The camera sees a frustum; keep near and far tight; an orbit camera is three numbers and a formula; fit with d = (H/2) / tan(fov/2).
- Geometry is typed arrays; transform at build time to bake parts into place and merge what shares a material.
- Pick materials by what they do with light; roughness and metalness, with an environment to reflect, do most of the work.
- Lights have physical intensities; shadows need three switches; `RoomEnvironment` through `PMREMGenerator` is the quick good environment.
- Groups give pivots and assemblies; pose everything from state each frame.
- Raycast through NDC with the y flipped; filter hidden meshes; put records in `userData`.
- Draw calls limit speed: merge, instance, share materials, cap the pixel ratio, use quality tiers, warm up shaders, dispose.

Chapter 16. Build Tools and Git: The Workshop

Every craft has a workshop: the benches, the vices, the labelled drawers. Web development has one too, and beginners often find it more intimidating than the languages themselves. It does not have to be. There are only four tools you need to understand: Node.js, npm, a bundler, and Git. Between them, they let you install libraries, combine your code into something a browser can load, and keep a complete, safe history of everything you do. Every command in this chapter was run for real; the outputs are what came back.

In This Chapter

- Install Node.js, start a project with npm, and read `package.json` and the lock file.
- Read a version number and a version range, and say what each allows.
- Bundle, minify and watch with esbuild, and know what tree shaking and source maps do.
- Record work with Git: stage, commit, log, diff; branch, merge, and undo safely.
- Work with a remote on GitHub, write a `.gitignore`, and publish a static site.
- Set up an editor with a formatter and a linter, and debug a build that fails.

Words You Will Meet

Word	What it means
Node.js	A program that runs JavaScript outside the browser, on your computer.
npm	The package manager that comes with Node.js.
Package	A bundle of code someone published for you to reuse (three.js is one).
package.json	The file that lists a project's packages, versions and scripts.
Lock file	`package-lock.json`: the exact versions that were installed.
node_modules	The folder npm fills with the packages you installed.
Semantic version	A version number in the form major.minor.patch, such as 0.186.1.
Bundler	A tool that follows `import` statements and joins everything into one file.
Minify	Shrink a file by removing whitespace and shortening names.
Tree shaking	Leaving out exported code that nothing imports.
Source map	A file that maps the bundled code back to your original lines, for debugging.
Repository (repo)	A project folder whose history Git records.
Commit	A saved snapshot of the project with a message.
Branch	A separate line of work; a name that points at a commit.
Merge	Bringing one branch's commits into another.
Remote	A copy of the repository on a server (such as GitHub).
Push / pull	Send your commits to / fetch commits from a remote.

16.1 Node.js and npm

Download Node.js from its website and install it. It comes with `npm`. Check it worked by typing `node -v` in a terminal (this book's examples ran on Node 24). To start a project:

```
npm init -y                  # creates package.json
npm install three            # downloads three.js into node_modules and records it
```

Now your code can say `import * as THREE from 'three'`. `npm install` with no package name installs everything `package.json` lists, which is what you run after cloning a project.

> **WHY DO IT THIS WAY? A package.json file**
>
> **What it is.** A small file listing what your project needs.

> **Why it is done this way.** The packages themselves are big, and they belong to other people. Your project only needs to record *which ones, and which versions*.
>
> **Why it matters.** Anyone (including you, on another computer next month) can type `npm install` and get exactly the same libraries.
>
> **What happens if you do not.** You copy giant folders around by hand, and nobody remembers which version of what a project was built with.

Scripts

The `scripts` section of `package.json` names commands so nobody has to remember flags:

```
"scripts": {
    "build": "esbuild main.js --bundle --minify --format=iife --outfile=app.js",
    "watch": "esbuild main.js --bundle --format=iife --outfile=app.js --watch",
    "test":  "node s17_tests.js"
}
```

Then `npm run build`, `npm run watch`, `npm test`. The Patrick and drone pages each have a `build.mjs` that does the same job as a small script, so the command is `node build.mjs`.

16.2 Version Numbers

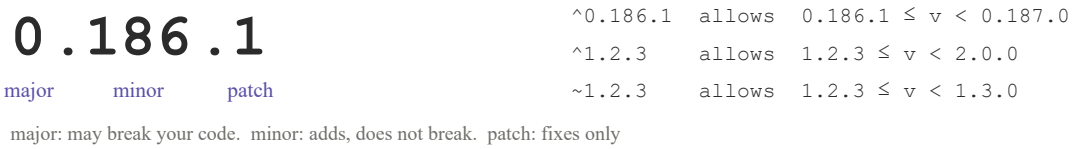


Figure 16.1. A semantic version is three numbers with three meanings. The caret and tilde ranges in `package.json` say how far forward npm may go.

Versions look like `0.186.1`: **major.minor.patch**. A patch bump fixes bugs. A minor bump adds features without breaking things. A major bump may break things. In `package.json` you will see ranges.

Written	Allows
`"1.2.3"`	Exactly 1.2.3.
`"^1.2.3"`	1.2.3 up to, but not including, 2.0.0.
`"~1.2.3"`	1.2.3 up to, but not including, 1.3.0.
`"^0.186.1"`	0.186.1 up to, but not including, 0.187.0 (for a 0.x version, the caret only allows patches).

> **WHY DO IT THIS WAY? Locking versions**
>
> **What it is.** Besides `package.json`, npm writes a `package-lock.json` recording the *exact* version of everything it installed, including the packages your packages depend on.
>
> **Why it is done this way.** A range like `^1.2.3` could install a different version next week.

Why it matters. Your build is repeatable.

What happens if you do not. One morning the project fails to build because a package published a new release overnight, and nothing in your code changed. (This is the same reason pages pin their CDN versions, as in Chapter 8.)

WARNING

Never edit the files inside `node_modules`, and never put the folder in Git. It is huge, it is generated from `package.json`, and it can always be recreated with `npm install`.

16.3 The Bundler

A browser, loading a page, does not know where to find the word `'three'` in `import ... from 'three'`. A **bundler** does: it reads your entry file, follows each `import` to the file or package it names, and writes everything into one file.

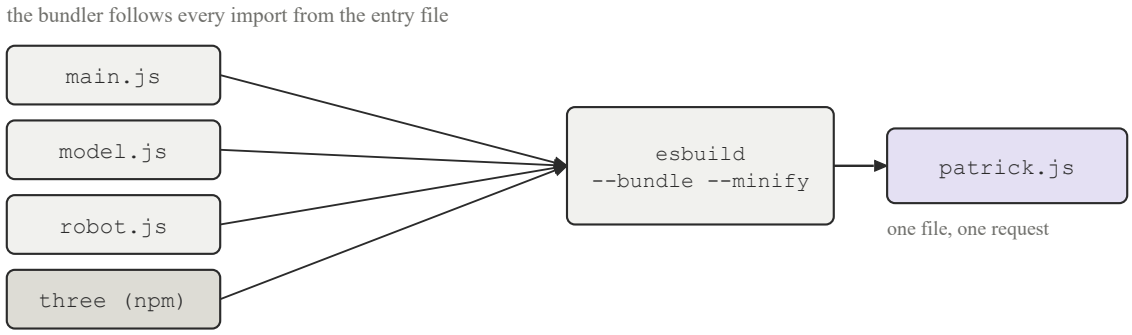


Figure 16.2. The bundler starts at the entry file and follows every import, into your own modules and into packages, and writes one file the page can load with a single request.

```
npx esbuild main.js --bundle --minify --format=iife --outfile=app.js
```

Read the flags as a sentence. *Bundle* all the imports into one file, *minify* the result, wrap it in an immediately-invoked function (`iife`) so its names do not leak into the page, and write it to `app.js`. Here is what esbuild 0.28 made of the two-file project from the worked example at the end of this chapter, first without and then with `--minify`:

```
(() => {
  // math.js
  var cbrt = (x) => Math.cbrt(x);

  // main.js
  console.log("cube root of 27 is", cbrt(27));
})();
```

```
(()=>{var o=t=>Math.cbrt(t);console.log("cube root of 27 is",o(27));})();
```

The sizes were 123 bytes and 74 bytes, and the builds took 56 and 15 milliseconds. Look closely at the first version: `math.js` also exported a function called `unused`, and it is *not* in the bundle. That is **tree shaking**: the bundler keeps only what is reachable from the entry file. It is why importing one helper

from `three/examples/jsm/...` costs you that helper and not the whole examples folder.

Three more flags you will want. `--sourcemap` writes `app.js.map`, so the browser's debugger shows your original files and line numbers inside the minified bundle. `--watch` keeps esbuild running and rebuilds the instant you save a file. `--target=es2020` tells it which browsers to write for.

> **WHY DO IT THIS WAY? Bundling, minifying and wrapping in an iife**
>
> **What it is.** Three flags, three jobs.
>
> **Why it is done this way.** Bundling means one request instead of hundreds (Chapter 1). Minifying shrinks the download. The iife keeps your variable names private.
>
> **Why it matters.** The robot page is one script of about 686 kilobytes. Without these steps, it would be hundreds of separate files and several times larger.
>
> **What happens if you do not.** Many small requests slow the page. Unwrapped scripts can clash with the page's other scripts: a flight dynamics page and a bundled library that both define `fit` would overwrite each other, which is exactly why the stability script on that page is wrapped.

> **TIP**
>
> Re-run the bundle every time you change a source file, or leave `--watch` running. Put the command in `build.mjs` or in the `scripts` section of `package.json`, so you can type `npm run build` and never mistype the flags.

16.4 Git: A Time Machine for Your Project

Git records snapshots of your project, called **commits**. Between commits, it notices what you changed. Start a repository, then repeat the cycle: change, stage, commit.

```
git init                          # start recording this folder
git status                        # what has changed?
git add index.html style.css      # stage: choose what goes in the next snapshot
git commit -m "Add the landing page"   # save the snapshot with a message
git log --oneline                 # the history, one line per commit
```

After editing `main.js` in the demo project, this is what Git reported:

```
$ git status --short
 M main.js
?? app.js
?? app.min.js

$ git diff
diff --git a/main.js b/main.js
--- a/main.js
+++ b/main.js
@@ -1,2 +1,4 @@
 import { cbrt } from './math.js';
 console.log('cube root of 27 is', cbrt(27));
+
+// a change
```

`M` means modified and tracked; `??` means Git has never been told about the file. The diff shows the exact lines added (+) and removed (−), with a little context. Read `git status` and `git diff` before every commit; it is how you catch the debugging line you forgot to delete.

> **WHY DO IT THIS WAY? Staging before committing**
>
> **What it is.** `git add` chooses *which* changes go into the next commit.
>
> **Why it is done this way.** You often have several unrelated changes in your folder.
>
> **Why it matters.** You can commit them as separate, clearly labelled snapshots. "Fix the camera jump" and "Add the new chart" stay separate, so either can be undone alone.
>
> **What happens if you do not.** One enormous commit called "stuff", which is impossible to review or to undo cleanly.

What to ignore

A `.gitignore` file lists what Git should never record: generated and private things.

```
node_modules/
*.log
.env
.DS_Store
```

Whether to ignore *built* files (`app.js`) is a choice. This website commits its bundles, because the pages load them directly and the host has no build step. A project with a build server would ignore them.

16.5 Branches and Merging

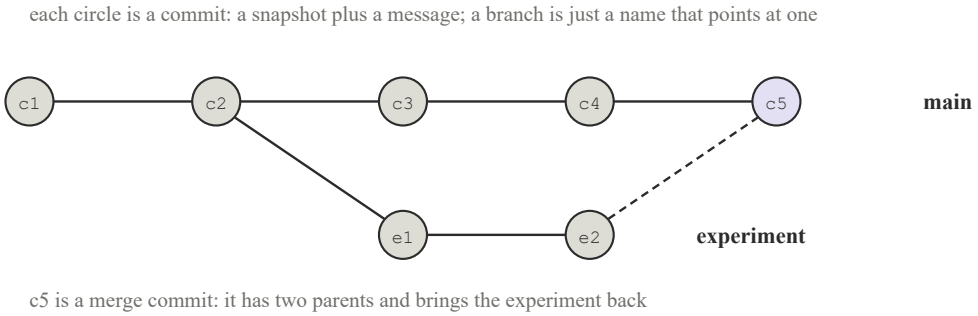


Figure 16.3. Commits form a chain; a branch is a name pointing at the newest one; a merge commit joins two chains back together and has two parents.

A branch is a separate line of work: `git switch -c experiment` starts one from where you are. Commits made there do not touch `main`. When the experiment works, go back and merge it; when it does not, delete it.

```
git switch -c experiment         # make and move to a new branch
...edit, add, commit...
git switch main                  # back to the main line
git merge experiment             # bring the experiment's commits in
```

```
git branch -d experiment         # delete the name (the commits are now on main)
```

The demo project's history after a branch, a commit on each side, and a merge:

```
$ git log --oneline --graph
*   417ccc5 Merge experiment
|\
| * 918dc2b Add sqrt to the maths module
* | 1c8dea7 Add the page that loads the bundle
|/
* be51bac Add a script that prints a cube root
```

The seven-character codes are the first characters of each commit's **hash**, a fingerprint of its contents that is the same on every computer. When two branches changed the *same lines*, the merge stops and marks the file with <<<<<<< and >>>>>>>; you edit it to what it should be, `git add` it, and commit. A conflict is not an error. It is Git asking a question only you can answer.

> **WHY DO IT THIS WAY? Branching before experimenting**
>
> **What it is.** Creating a branch before a risky change.
>
> **Why it is done this way.** The branch is a private copy of the history.
>
> **Why it matters.** If the experiment fails, you switch back to your main branch and delete it. The main line was never touched.
>
> **What happens if you do not.** You break the working version and discover you did not save a copy.

16.6 Undoing Things

Git's reputation for danger comes from a few commands that rewrite history. The safe ones first.

I want to...	Command	Safe?
throw away my edits to one file since the last commit	`git restore file.js`	loses those edits, nothing else
un-stage a file I added by mistake	`git restore --staged file.js`	yes
see an old version of a file	`git show abc1234:path/file.js`	yes
undo a commit *that is already shared*, keeping history honest	`git revert abc1234`	yes: adds a new commit that reverses it
undo my last commit but keep the changes to recommit	`git reset --soft HEAD~1`	only on commits you have not pushed
put my half-done work aside and get it back later	`git stash`, then `git stash pop`	yes
go back to a commit to look around	`git switch --detach abc1234`, then `git switch main`	yes

> **WARNING**
> Do not rewrite history that others already have. Commands like `git push --force` and `git reset --hard` can erase their work, or yours. If you are not sure what a Git command will do, read what it says before you press Enter, and ask. `git revert` is the undo for anything that has left your machine.

16.7 Remotes and GitHub

A **remote** is a copy of the repository on a server. GitHub is the usual one. Create an empty repository there, then connect and push:

```
git remote add origin https://github.com/you/your-site.git
git push -u origin main          # the first push; -u remembers where main goes
git push                         # every later push
git pull                         # fetch and merge what changed on the server
git clone https://github.com/you/your-site.git    # get a whole repository on anothe
      r machine
```

On a team, work goes through **pull requests**: push a branch, open a request on GitHub, a colleague reads the diff and comments, and the branch is merged on the server. Alone, the same habit (a branch per piece of work, merged when done) keeps `main` always publishable.

> **WHY DO IT THIS WAY? Writing commit messages that say why**
> **What it is.** A short message with each commit explaining the reason for the change.
> **Why it is done this way.** The code already shows *what* changed. Only you know *why*.
> **Why it matters.** In six months, "Fix the camera jump when the tab is hidden" is a gift to your future self. "update stuff" is a puzzle.
> **What happens if you do not.** You cannot tell which of fifty commits introduced a bug.

16.8 Publishing a Static Site

A static site is files, so publishing is putting the files where a server can reach them (Chapter 1). With the repository on GitHub, **GitHub Pages** will serve it: in the repository's Settings, under Pages, choose the branch (`main`) and the folder (the root), and the site appears at `https://you.github.io/your-site/` within a minute of every push. This website is published this way, from its repository, which is why Part V's publishing step is `git push`.

Two things to check on the first publish. Links must be relative (Chapter 9), because the site lives in a sub-folder, not at the domain's root. And the bundles must be committed, because Pages runs no build.

16.9 The Editor, the Formatter and the Linter

Visual Studio Code is free and is what most people use. Three additions pay for themselves in the first week. A **formatter** (Prettier) rewrites your code in one consistent style every time you save, so you never argue with yourself about spaces. A **linter** (ESLint) underlines mistakes as you type: an unused variable, a missing `await`, `==`, a variable used before it is declared. **Live Server** (an extension) serves your folder over HTTP and reloads the page when you save, which modules and `fetch` need, since both refuse to work from a `file://` address.

Set them up once per project (`npm install --save-dev prettier eslint`, plus a config file for each), commit the config files, and everyone who clones the project gets the same rules.

16.10 When the Build Breaks

A failing build prints an error with a file and a line. Read it from the top. The usual causes, in order of likelihood: a typo in an import path (case matters on the server even if it does not on your machine); a package that was never installed (`npm install` fixes it); a syntax error in the file named; a package that needs a flag (`--jsx=automatic` for React, a loader for `.cpp` or `.glsl` text files); and a stale `node_modules` after switching branches (delete it and `npm install`). When a bundle builds but the page is blank, open the Console: a bundle that throws on its first line prints why, and a source map points at your original file.

Worked Example: From Nothing to a Bundled Page

Worked Example: Install, bundle, commit

Create a folder with a script that logs the cube root of 27 using a helper module, bundle it, and commit it.

Step 1. Files. In an empty folder, make `math.js` and `main.js`.

```
// math.js
export const cbrt = (x) => Math.cbrt(x);
export const unused = () => 'never imported';

// main.js
import { cbrt } from './math.js';
console.log('cube root of 27 is', cbrt(27));
```

Step 2. Bundle. Run `npx esbuild main.js --bundle --format=iife --outfile=app.js`. This writes `app.js` with both modules joined (and `unused` shaken out). The output is the 123-byte file shown in 16.3.

Step 3. Test. `node app.js` prints `cube root of 27 is 3`. Make `index.html` with `<script src="app.js"></script>`, open it, and the Console prints the same.

Step 4. Ignore the generated folder. Create a file `.gitignore` containing `node_modules`, so Git never records it.

Step 5. Commit.

```
git init
git add main.js math.js .gitignore
git commit -m "Add a script that prints a cube root"
```

Step 6. Reflect. `app.js` is generated from the sources. Some teams commit it, some do not. This website commits its built bundles, because the pages load them directly and a visitor's browser has no build step.

Practice Problems A: Packages and Builds

Review Problems

1. A package.json lists `"lodash": "^4.17.2"`. Which of these versions would be accepted: 4.17.2, 4.18.0, 4.99.9, 5.0.0?
2. And for `"~4.17.2"`: which of 4.17.9, 4.18.0, 5.0.0? And for `"^0.186.1"`: which of 0.186.9, 0.187.0?
3. A script reads `import { add } from './add.js'` and is opened directly with `<script src="main.js">`, failing with an error. Name two ways to fix it.
4. `math.js` exports six functions and `main.js` imports two. Roughly how many end up in the bundle, and what is the name of the feature?
5. The minified bundle throws an error at "line 1, column 48 212". What flag makes the browser show you the real file and line?
6. Write the `scripts` entry that bundles `main.js` to `app.js` with minification, and the command that runs it.
7. Explain in two sentences why `node_modules` is not stored in Git but `package-lock.json` is.

Practice Problems B: Git

Review Problems

1. You have changed three files: two belong to a bug fix and one is an unrelated new feature. Show the commands to commit them as two separate commits.
2. You started editing on the main branch and then realise you wanted an experiment branch. Say how to move the current work to a new branch (without losing it), using a command from this chapter.
3. Read this graph and say what happened, in order: `* 7a1 Merge experiment`, `|\`, `| * 3c9 Try a wider track`, `* | b20 Fix the typo`, `|/`, `* 9e4 First version`.
4. A merge stops with `CONFLICT (content): Merge conflict in style.css`. What are the three steps to finish it?
5. You committed a debugging `console.log` and pushed it an hour ago. Which undo command is right, and why not `reset`?

6. You want to discard every uncommitted change to `main.js` but keep your changes to `style.css`. Which command?
7. Your site is on GitHub Pages at `https://you.github.io/site/` and every image is broken, though they worked locally. What is the most likely cause?
8. Write a commit message for a change that caps the pixel ratio at 2 because a phone was rendering nine times too many pixels. Then write a bad one, and say what makes it bad.

Mini Project: Set Up the Robot Page for Real

Mini Project: A project folder that builds, tests and publishes

Take the `patrick.html`, `patrick.css` and script from the earlier mini projects and turn them into a proper project.

1. `npm init -y`; install `three` (for the gauge from Chapter 15) and, as dev dependencies, `esbuild`, `eslint` and `prettier`. Commit `package.json` and the lock file, with a `.gitignore` for `node_modules`.
2. Move the script into `src/main.js` importing `src/gauge.js` and `src/data.js`. Add `build`, `watch` and `test` scripts; the test runs a `tests.js` in the style of Chapter 11.
3. Build with `--sourcemap`; open the page, put a `debugger;` in `gauge.js`, and confirm the browser stops in the original file.
4. Make the history honest: one commit for the project setup, one for the build scripts, one for the gauge, each with a message that says why. Try a change on a branch, merge it, and read the graph.
5. Push to a new GitHub repository and enable Pages. Fix whatever the first publish breaks (it will be a path).
6. Break the build on purpose (misspell an import) and read the error; then fix it. Break the page on purpose (throw in `main.js`) and use the source map to find the line.

Checks: `npm run build` succeeds from a fresh clone after `npm install`; `npm test` passes; `git status` is clean after a build (built files are either committed or ignored, deliberately); the published page works with the browser's cache cleared.

What You Learned

- Node runs JavaScript on your machine; npm installs packages and records them in `package.json`; the lock file pins the exact versions; `scripts` name your commands.
- major.minor.patch; `^` allows minor and patch, `~` allows patch, and for 0.x versions the caret allows only patches.
- The bundler follows imports into one file: `--bundle --minify --format=iife`, plus `--sourcemap` for debugging and `--watch` for development. Unused exports are shaken out.
- Git: change, `status`, `diff`, `add`, `commit` with a message that says why. `log --oneline --graph` to read history.

- Branch before experimenting; merge when it works; a conflict is a question, not an error.
- Undo safely: `restore` for edits, `revert` for shared commits, `stash` for half-done work; never rewrite shared history.
- A remote on GitHub, `push` and `pull`, pull requests for review, Pages to publish; relative paths and committed bundles for a static site.
- An editor with a formatter, a linter and a live server; read build errors from the top.

Chapter 17. C++: From Your First Program to Your Own Robot

Everything so far has run inside a web browser. Patrick's controller, the program that decides when to drive, brake, reach and grip, is different. On the real robot it is a small program running on a microcontroller, and microcontrollers do not speak JavaScript. They speak C++ (or its older cousin, C). That is why the robot's whole motion also exists as a C++ file, and why this book has a long chapter on it.

This chapter is a complete C++ course, from "what is a compiler?" up to writing a controller for a robot of your own. It is written for someone who has never programmed in C++, but who has read Chapter 11 (JavaScript) and so already knows what a variable, a function and a loop are. If you have not, nothing here will break, but the first sections will move quickly.

How the chapter is organised. Sections 17.1 to 17.18 teach the language, from the first program to templates. Each group of sections ends with **Practice Problems** and line-by-line answers. Section 17.19 gives **five mini projects**, each a complete, working program. Then, in Part V, **Chapter 42** is the grand finale: the guided project in which we build Patrick's whole program from an empty file, one runnable stage at a time.

The code is real. Every program in this chapter is a file in the folder `Guide/code/cpp`, and every output shown was produced by compiling and running that file. Type them in yourself if you can; typing teaches more than reading. If a listing and your screen ever disagree, trust the screen and look for a typing slip.

> **REMEMBER**
> JavaScript is read and run by the browser as it goes. C++ is first **compiled**: a program called the compiler reads all your code, checks it, and turns it into a file the computer can run directly. The checking is the point. Many mistakes that JavaScript only reveals while running, C++ catches before it runs at all.

Words You Will Meet

Word	What it means
Source file	The text file you write, such as `robot.cpp`.
Compiler	The program (here, `g++`) that turns source into a runnable program.
Linker	The last step of compiling: it joins your pieces and the libraries into one program.
Header	A file of declarations you pull in with `#include`, such as `<cstdio>` for printing.
`main`	The function where every C++ program starts.
Type	What kind of value a variable holds: `int`, `double`, `bool`, `char`.
Variable	A named box that holds a value of one type.
`const`	A promise that a value never changes after it is set.
Function	A named piece of code that takes **parameters** and **returns** a value.
Overloading	Several functions sharing one name, told apart by their parameter types.
Reference (`&`)	A second name for an existing variable.
Pointer (`*`)	A variable holding the *address* of another.
Array / `std::vector`	A numbered row of values of one type; a vector can grow.
`struct` / `class`	A bundle of related data (and the functions that use it) under one name.
Member / member function	A variable or function that belongs to a struct or class.
Constructor	The function that sets up a new object.
`enum`	A list of named choices, such as `RED, GREEN, AMBER`.
Lambda	A small function written in the middle of other code.
Template	A function or class written once that works for many types.
Namespace	A name prefix that keeps names apart; `std::` is the standard library's.
Time step (`dt`)	How much simulated time passes in one pass of the loop.

17.1 Your First Program

Create a file called `hello.cpp` and type this:

```
1  #include <cstdio>
2
3  int main() {
4    std::printf("Hello, Patrick!\n");
```

```
5     return 0;
6   }
```

Listing from `Guide/code/cpp/s01_hello.cpp`*, lines 1 to 6.*

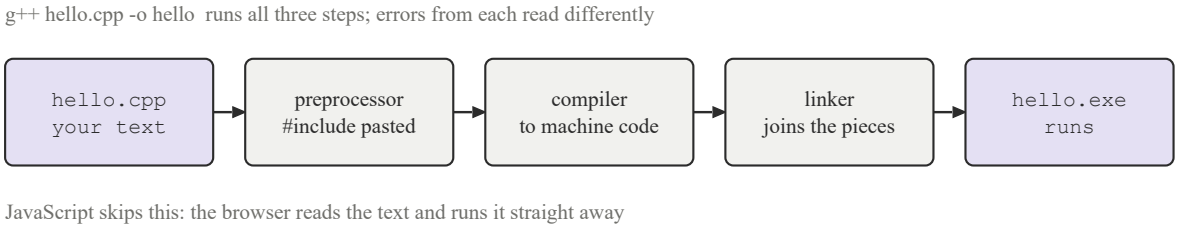


Figure 17.1. What `g++` does with the file. The preprocessor pastes in the headers, the compiler turns the text into machine code, and the linker joins it with the standard library into a program. Each stage has its own kind of error message, which is why "undefined reference" (the linker) reads so differently from "expected ';'" (the compiler).

Line 1 pulls in the header for printing. **Line 3** is `main`, where the program starts; the `int` in front says it hands back a whole number when it finishes. **Line 4** prints a line of text: `std::printf` is the function, the text is in quotes, and the two characters `\n` inside the quotes mean "start a new line". **Line 5** returns 0, which by tradition means "everything went well". Every statement ends with a semicolon.

Now compile and run it, in a terminal:

```
g++ -std=c++17 -Wall -Wextra -O2 hello.cpp -o hello
./hello
```

Read the first line as a sentence. Use the 2017 version of the language (`-std=c++17`). Warn me about suspicious code (`-Wall -Wextra`). Optimise for speed (`-O2`). Compile `hello.cpp` and call the result `hello`. Running it prints `Hello, Patrick!`. (On Windows the program is `hello.exe`, and you run it by typing `hello` or `.\hello`.)

> **WHY DO IT THIS WAY? Always compiling with -Wall -Wextra**
>
> **What it is.** Two flags that make the compiler tell you about code that is legal but probably a mistake.
>
> **Why it is done this way.** C++ will happily accept a lot of nonsense. A warning is the compiler tapping you on the shoulder.
>
> **Why it matters.** When the robot's code was written, the compiler flagged one line in the controller where `if (...) go(...); break;` sat on a single line and looked as though the `break` belonged to the `if`. It did not, and the line was reformatted so that nobody reading it later would be fooled.
>
> **What happens if you do not.** You find the same mistakes later, by watching a robot do the wrong thing, which is slower and more confusing than reading a warning.

C++ has two ways to print. The older `printf` comes with C. The newer `std::cout` comes with C++ and uses << to chain pieces together:

```
1  #include <iostream>
2
3  int main() {
4    std::cout << "Hello, Patrick!" << std::endl;
5    return 0;
6  }
```

Listing from `Guide/code/cpp/s01_hello_cout.cpp`*, lines 1 to 6.*

Both work, and you will see both in the wild. This chapter mostly uses `printf`, because it lets you control the number of decimal places precisely (`%.3f` means "a decimal number with three places"), which matters when you are comparing numbers from two programs. The codes you need are `%d` for a whole number, `%f` for a decimal, `%c` for one character, `%s` for text, and `%%` for a real percent sign.

Reading an error message. Delete the semicolon at the end of line 4 (this is the file `s01_broken.cpp`) and compile. The compiler answers:

```
s01_broken.cpp: In function 'int main()':
s01_broken.cpp:4:35: error: expected ';' before 'return'
```

Read it from the left: the file, then the line number (4), then the column (35), then the problem. The compiler reports where it *noticed* the trouble, which is often the line after the real mistake: here the missing semicolon is on line 4, but the compiler only realised on line 5, at `return`. When you cannot see the error on the line named, look at the line above.

> **TIP**
>
> Fix the first error first. One missing bracket can make the compiler print twenty errors, and nineteen of them vanish when you fix the first.

17.2 Numbers, Types and Arithmetic

In JavaScript every number is one kind. In C++ you choose, and the choice has consequences. Here is a tour:

```
1  #include <cstdio>
2  #include <cmath>
3
4  int main() {
5    int cars = 3;                  // whole number
6    double speed = 0.45;           // decimal number
7    bool paused = false;           // true or false
8    char grade = 'A';              // one character
9    std::printf("sizes in bytes: int %zu, double %zu, bool %zu, char %zu\n", sizeo
   f(int), sizeof(double), sizeof(bool), sizeof(char));
10   std::printf("7 / 2   = %d\n", 7 / 2);              // integer division
11   std::printf("7 / 2.0 = %.1f\n", 7 / 2.0);          // decimal division
12   std::printf("7 %% 2   = %d\n", 7 % 2);             // remainder
13   int whole = static_cast<int>(3.99);                // cast: the decimals are c
   ut off, not rounded
14   std::printf("static_cast<int>(3.99) = %d, round(3.99) = %.0f\n", whole, std::r
   ound(3.99));
15   auto k = 2 * 3.5;                                  // auto: the compiler works
```

```
      out the type (double)
16    std::printf("auto k = 2 * 3.5 -> %.1f\n", k);
17    std::printf("%d %.2f %d %c\n", cars, speed, paused, grade);
18    return 0;
19  }
```

Listing from `Guide/code/cpp/s02_types.cpp`, *lines 1 to 19.*

Running it prints:

```
sizes in bytes: int 4, double 8, bool 1, char 1
7 / 2   = 3
7 / 2.0 = 3.5
7 % 2   = 1
static_cast<int>(3.99) = 3, round(3.99) = 4
auto k = 2 * 3.5 -> 7.0
3 0.45 0 A
```

Four things to take from it. **First**, `int` holds whole numbers (4 bytes: about plus or minus two billion), `double` holds decimals (8 bytes, about 15 significant digits), `bool` holds true or false, and `char` holds one character. **Second**, dividing two `int` values gives an `int` and throws the remainder away, so `7 / 2` is `3`; make one side a `double` (`7 / 2.0`) to get `3.5`. The `%` operator gives the remainder. **Third**, `static_cast<int>(3.99)` *cuts off* the decimals (giving 3); it does not round. Use `std::round` for rounding. **Fourth**, `auto` lets the compiler work out the type from the value, which saves typing but should not hide what the type is.

> **WARNING**
> Dividing two whole numbers is the most common beginner bug in C++, and it never gives an error. `1 / 2` is `0`. If a result is mysteriously zero, check for it. Patrick's code writes `9.0 / 5.0` and `1.0` on purpose.

> **WHY DO IT THIS WAY? Putting units in names and constants at the top**
>
> **What it is.** Names like `wheelR` (metres) and `STOP_AT` (metres), declared once near the top, each with a comment giving its unit.
>
> **Why it is done this way.** A bare `0.108` in the middle of code means nothing. A named constant means something, and has one place to change.
>
> **Why it matters.** The distance at which Patrick brakes was tuned several times. Each time it was a one-line change, and every use of it updated.
>
> **What happens if you do not.** The number appears in five places, you change four of them, and the robot brakes at two different distances depending on which code path ran.

Maths functions live in `<cmath>`: `std::sqrt`, `std::pow`, `std::sin` (angles in radians), `std::atan2`, `std::fabs` (absolute value of a decimal), `std::hypot`, `std::min` and `std::max` (in `<algorithm>`). A constant that never changes is declared `const`, so that the compiler refuses any attempt to change it.

Worked Example: Wheel speed from rpm

How fast does Patrick move if his 65 mm wheels turn at 25, 50, 75 and 100 rpm?

Step 1. The physics. A wheel turning at *rpm* revolutions per minute turns at *omega = rpm x 2 pi / 60* radians per second. A point on its rim, and so the robot, moves at *v = omega x r*.

Step 2. The program.

```
 1  #include <cmath>
 2  #include <cstdio>
 3
 4  const double PI = 3.14159265358979323846;
 5
 6  // Robot speed in m/s from wheel rpm and wheel radius in metres.
 7  double speedFromRpm(double rpm, double radius) {
 8    double omega = rpm * 2.0 * PI / 60.0;       // rad/s
 9    return omega * radius;                       // v = omega r
10  }
11
12  int main() {
13    const double wheelR = 0.0325;               // Patrick's 65 mm wheels
14    for (int rpm = 0; rpm <= 100; rpm += 25)
15      std::printf("%3d rpm -> %.3f m/s\n", rpm, speedFromRpm(rpm, wheelR));
16    return 0;
17  }
```

Listing from `Guide/code/cpp/s02_wheel_speed.cpp`*, lines 1 to 17.*

Step 3. Predict, then run. 100 rpm is 10.47 rad/s, and times 0.0325 m that is 0.340 m/s. Running the program prints:

```
  0 rpm -> 0.000 m/s
 25 rpm -> 0.085 m/s
 50 rpm -> 0.170 m/s
 75 rpm -> 0.255 m/s
100 rpm -> 0.340 m/s
```

Step 4. Reflect. Doubling the rpm doubles the speed, as the formula says. The `for` loop counts in steps of 25 (`rpm += 25`). The function does one job and is easy to test. The `int` loop variable is passed to a `double` parameter and C++ converts it for you.

17.3 Reading Input

So far the programs talk and nobody answers. To read what the user types, use `std::cin`:

```
1  #include <iostream>
2
3  int main() {
4    double c;
5    std::cout << "Temperature in C: ";
6    std::cin >> c;
7    std::cout << c << " C = " << c * 9.0 / 5.0 + 32.0 << " F" << std::endl;
8    return 0;
9  }
```

Listing from `Guide/code/cpp/s03_input.cpp`*, lines 1 to 9.*

Compile it and run it. It asks for a temperature, and if you type `25` and press Enter it prints `25 C = 77 F`. (To try it without typing, feed it a value from the terminal: `echo 25 | ./s03_input`.) The `>>`

pulls the next value off the keyboard into the variable `c`, converting the text to a `double` as it goes.

> **WHY DO IT THIS WAY? Reading with cin, and checking for failure in real programs**
>
> **What it is.** `std::cin >> c` reads a value into `c`. If the user types `hello` instead of a number, the read fails and `c` is left unchanged.
>
> **Why it is done this way.** The stream remembers whether the last read worked, so you can ask `if (!(std::cin >> c))` and handle the problem.
>
> **Why it matters.** A robot's input is rarely a keyboard, but the habit is the same: never assume the data you read is what you hoped.
>
> **What happens if you do not.** The program quietly works on garbage and gives wrong answers with no error.

Practice Problems A: First Steps

Review Problems

1. The compiler says `error: expected ';' before 'return'` pointing at line 12. Where should you look first?
2. What does `std::printf("%d", 9 / 4);` print? What does `std::printf("%.2f", 9 / 4.0);` print?
3. After `int x = 7; x += 3; x *= 2;` what is `x`?
4. A robot's wheels have a radius of 0.05 m and turn at 120 rpm. How fast does it move?
5. What do `static_cast<int>(-2.7)` and `std::round(-2.7)` give?
6. Write a program that reads two numbers and prints their average.
7. Why does `printf` need `%%` to print a percent sign?
8. Which type would you choose for: the number of items on a table, a motor voltage, whether the claw is holding something?

17.4 Making Decisions

A program becomes interesting when it can choose. The tools are `if`, `else if`, `else`, the comparison operators (<, <=, >, >=, ==, `!=`) and the logical operators (`&&` for "and", `||` for "or", `!` for "not").

```
 1  #include <cstdio>
 2
 3  // What should the robot do, given the sonar distance in metres?
 4  const char* decide(double d) {
 5    if (d <= 0.108) return "BRAKE";
 6    else if (d <= 0.30) return "SLOW";
 7    else if (d >= 4.0) return "NO ECHO: keep driving";
 8    else return "DRIVE";
 9  }
10
11  int main() {
12    double readings[] = {2.5, 0.8, 0.29, 0.108, 0.05, 4.0};
```

```
13    for (double d : readings) std::printf("%5.3f m -> %s\n", d, decide(d));
14    bool near = 0.2 < 0.3, lowBattery = false;
15    std::printf("near && !lowBattery = %d, near || lowBattery = %d\n", near && !lo
    wBattery, near || lowBattery);
16    int duty = 80;
17    std::printf("ternary: duty is %s\n", duty > 70 ? "high" : "normal");
18    return 0;
19  }
```

Listing from `Guide/code/cpp/s04_decide.cpp`*, lines 1 to 19.*

Running it prints:

```
2.500 m -> DRIVE
0.800 m -> DRIVE
0.290 m -> SLOW
0.108 m -> BRAKE
0.050 m -> BRAKE
4.000 m -> NO ECHO: keep driving
near && !lowBattery = 1, near || lowBattery = 1
ternary: duty is high
```

Notice the **order** of the tests in `decide`. A reading of 0.05 is below 0.108, below 0.30, and so on; the first test that is true wins, and the rest are skipped. The three-way choice `duty > 70 ? "high" : "normal"` is the **ternary operator**: a condition, then the value if true, then the value if false.

> **WARNING**
>
> `==` compares; `=` assigns. Writing `if (x = 5)` assigns 5 to `x` and is then true. The compiler's `-Wall` warns about it. Another trap: never compare two calculated decimals with `==` (see Section 17.15), because `0.1 + 0.2 == 0.3` is false.

> **WHY DO IT THIS WAY? Testing the closest threshold first**
>
> **What it is.** In `decide`, the BRAKE test comes before the SLOW test.
>
> **Why it is done this way.** An `if / else if` chain stops at the first true test. A distance of 0.05 m also satisfies "less than 0.30", so if SLOW came first the robot would never brake.
>
> **Why it matters.** This is the shape of every sensor-driven decision in the robot: the most urgent test first.
>
> **What happens if you do not.** The robot behaves correctly for most distances and fails at exactly the one that matters.

17.5 Loops

Four tools repeat things. A `for` loop counts. A `while` loop repeats while a condition holds. A `do ... while` loop does the same but runs at least once. And inside any of them, `break` leaves the loop and `continue` skips to the next lap.

```
1  #include <cstdio>
2
3  int main() {
4    int sum = 0;
```

```
 5    for (int i = 1; i <= 10; i++) sum += i;              // for: count from 1 to
   10
 6    std::printf("sum 1..10 = %d\n", sum);
 7
 8    double v = 1.0; int steps = 0;
 9    while (v > 0.01) { v *= 0.5; steps++; }              // while: repeat while a
    condition holds
10    std::printf("halving from 1.0 until below 0.01 took %d steps\n", steps);
11
12    int n = 0;
13    do { n++; } while (n * n < 50);                      // do-while: always runs
    at least once
14    std::printf("first n with n*n >= 50 is %d\n", n);
15
16    for (int i = 1; i <= 20; i++) {
17      if (i % 2 == 0) continue;                          // continue: skip to the
    next lap
18      if (i > 9) break;                                  // break: leave the loop
19      std::printf("%d ", i);
20    }
21    std::printf("<- odd numbers up to 9\n");
22
23    for (int r = 1; r <= 3; r++) {                       // loops inside loops
24      for (int c = 1; c <= 4; c++) std::printf("%3d", r * c);
25      std::printf("\n");
26    }
27    return 0;
28  }
```

Listing from `Guide/code/cpp/s05_loops.cpp`, *lines 1 to 28.*

Its output:

```
sum 1..10 = 55
halving from 1.0 until below 0.01 took 7 steps
first n with n*n >= 50 is 8
1 3 5 7 9 <- odd numbers up to 9
  1  2  3  4
  2  4  6  8
  3  6  9 12
```

The last two lines show a loop *inside* a loop: the outer loop picks the row and the inner loop prints the columns. A simulation is just a loop: each lap advances the world by one small time step, which is the idea behind everything in the second half of this chapter.

> **WHY DO IT THIS WAY? A fixed time step, not "however long the frame took"**
>
> **What it is.** In a simulation, `dt` is a constant such as 0.001 s, whatever the computer is doing.
>
> **Why it is done this way.** Physics equations are sensitive to the step size. A fixed step gives the same answer every time, on every computer.
>
> **Why it matters.** This is exactly why the C++ and the JavaScript versions of the robot can be compared tick for tick, and why they agree exactly. If the step varied, two runs would never match.
>
> **What happens if you do not.** The robot behaves slightly differently on every run, and a bug you saw once cannot be repeated to study it.

17.6 Functions

A function takes values in and gives one back. You have already used several. Here are the ideas that matter for writing your own:

```
 1  #include <cstdio>
 2  #include <cmath>
 3  #include <algorithm>
 4
 5  double clampd(double x, double a, double b) { return std::min(b, std::max(a, x))
    ; }
 6  double lerp(double a, double b, double t) { return a + (b - a) * t; }
     // blend from a to b as t goes 0..1
 7  int square(int x) { return x * x; }
 8  double square(double x) { return x * x; }
     // overloading: same name, different types
 9  double power(double base, int exp = 2) { return std::pow(base, exp); }
     // default argument
10
11  void swapValues(double& a, double& b) { double t = a; a = b; b = t; }
     // reference: changes the caller's variables
12  double lengthOf(const double (&v)[3]) { return std::sqrt(v[0]*v[0] + v[1]*v[1] +
     v[2]*v[2]); }   // const reference to an array
13
14  int counter = 0;
     // a global: avoid these
15  void tick() { static int calls = 0; calls++; counter += calls; }
     // static: remembered between calls
16
17  int main() {
18    std::printf("clampd(1.7, 0, 1) = %.1f   clampd(-3, 0, 1) = %.1f\n", clampd(1.7
    , 0, 1), clampd(-3, 0, 1));
19    std::printf("lerp(10, 20, 0.25) = %.2f\n", lerp(10, 20, 0.25));
20    std::printf("square(4) = %d, square(1.5) = %.2f\n", square(4), square(1.5));
21    std::printf("power(3) = %.0f, power(2, 10) = %.0f\n", power(3), power(2, 10));
22    double a = 1, b = 2; swapValues(a, b); std::printf("after swap: a=%.0f b=%.0f\
    n", a, b);
23    double v[3] = {3, 4, 12}; std::printf("length of (3,4,12) = %.0f\n", lengthOf(
    v));
24    tick(); tick(); tick(); std::printf("counter after three ticks = %d\n", counte
    r);
25    return 0;
26  }
```

Listing from `Guide/code/cpp/s06_functions.cpp`*, lines 1 to 26.*

Its output:

```
clampd(1.7, 0, 1) = 1.0   clampd(-3, 0, 1) = 0.0
lerp(10, 20, 0.25) = 12.50
square(4) = 16, square(1.5) = 2.25
power(3) = 9, power(2, 10) = 1024
after swap: a=2 b=1
length of (3,4,12) = 13
counter after three ticks = 6
```

Parameters and return values. `clampd` limits a number to a range; `lerp` (short for linear interpolation) blends from `a` to `b` as `t` goes from 0 to 1. **Overloading** lets two functions share a name when their parameter types differ: `square(4)` calls the `int` version and `square(1.5)` the `double` one. A **default argument** (`int exp = 2`) fills in a value when the caller leaves it out. **References**: normally a function receives *copies*, so changing them changes nothing outside.

`swapValues(double& a, double& b)` takes references, so it really swaps the caller's variables. `const double (&v)[3]` is a reference to an array that the function promises not to change. A **static** local variable (`calls` in `tick`) is remembered between calls, whereas an ordinary local variable is created afresh on each call.

> **WHY DO IT THIS WAY? Small, pure functions**
>
> **What it is.** Functions that depend only on what you pass in and change nothing else.
>
> **Why it is done this way.** They are easy to check one at a time. `mu(0.17, 1.0)` always gives the same answer.
>
> **Why it matters.** Patrick's physics is a dozen of these. Each was tested alone before they were joined, so when a whole run looked wrong, the search for the fault was short.
>
> **What happens if you do not.** One giant function that does everything, in which a mistake anywhere makes everything look wrong, and nothing can be tested alone.

> **TIP**
>
> Prefer passing by `const` reference for anything bigger than a few numbers: `void draw(const Mission& m)` hands over the original without copying it and promises not to change it.

Practice Problems B: Decisions, Loops, Functions

Review Problems

1. With the `decide` function of Section 17.4, what does `decide(0.2)` return?
2. State De Morgan's law: what is `!(a && b)` the same as?
3. What does `for (int i = 10; i > 0; i -= 3) std::printf("%d ", i);` print?
4. `int n = 1; while (n < 100) n *= 3;` How many times does the loop body run, and what is `n` at the end?
5. Write a function `double average(double a, double b)`.
6. A function is `int f(int x) { static int c = 0; c += x; return c; }`. What do `f(2)`, `f(3)`, `f(4)` return in turn?
7. With the overloads in Section 17.6, what do `square(3)` and `square(3.0)` print?
8. `double power(double base, int exp = 2)`: what does `power(5)` return?

17.7 Arrays and Vectors

One number at a time is not enough for a robot: a sensor gives hundreds of readings. A fixed row is an **array**: `double arr[4] = {1, 2, 3, 4};`. A row that can grow is a `std::vector`, from `<vector>`, and it comes with a toolbox of ready-made algorithms in `<algorithm>` and `<numeric>`.

```
1  #include <algorithm>
2  #include <cstdio>
```

```
 3  #include <numeric>
 4  #include <vector>
 5
 6  int main() {
 7    std::vector<double> d = {0.82, 0.79, 0.85, 0.81, 2.40, 0.80, 0.78};      // so
    nar readings in metres: one is a glitch
 8    d.push_back(0.83);                                                       // ve
    ctors grow
 9    std::printf("%zu readings, first %.2f, last %.2f\n", d.size(), d.front(), d.ba
    ck());
10
11    double sum = std::accumulate(d.begin(), d.end(), 0.0);
12    double mean = sum / d.size();
13    std::vector<double> s = d; std::sort(s.begin(), s.end());                // so
    rt a copy
14    double median = (s[s.size() / 2 - 1] + s[s.size() / 2]) / 2;             // ev
    en count: mean of the middle two
15    auto [lo, hi] = std::minmax_element(d.begin(), d.end());
16    std::printf("mean %.3f  median %.3f  min %.2f  max %.2f\n", mean, median, *lo,
     *hi);
17    int big = (int)std::count_if(d.begin(), d.end(), [](double x) { return x > 1.0
    ; });
18    std::printf("readings above 1 m: %d\n", big);
19    for (double x : d) std::printf("%.2f ", x);                              // ra
    nge-for: every element in turn
20    std::printf("\n");
21    double arr[4] = {1, 2, 3, 4}; double tot = 0; for (int i = 0; i < 4; i++) tot
    += arr[i];
22    std::printf("fixed array total = %.0f\n", tot);
23    return 0;
24  }
```

Listing from `Guide/code/cpp/s07_vectors.cpp`*, lines 1 to 24.*

The output:

```
8 readings, first 0.82, last 0.83
mean 1.010  median 0.815  min 0.78  max 2.40
readings above 1 m: 1
0.82 0.79 0.85 0.81 2.40 0.80 0.78 0.83
fixed array total = 10
```

Look at the **mean and median**. One glitch reading, 2.40 m, drags the mean up to 1.010, which is wrong for a robot that is really about 0.8 m from the wall. The median (the middle value once sorted) is 0.815 and barely notices. This is why sonar data is cleaned with median filters, as you will see in mini project 2. The loop `for (double x : d)` is a **range-for**: it visits every element in turn without any counting. Arrays count from zero: `arr[0]` is the first element and `arr[3]` the last of four.

> **WHY DO IT THIS WAY? Using the standard algorithms**
>
> **What it is.** `std::sort`, `std::accumulate`, `std::count_if` and `std::minmax_element` instead of hand-written loops.
>
> **Why it is done this way.** They have been written, tested and optimised by thousands of people.
>
> **Why it matters.** Less code to read means fewer places for a bug to hide, and the names say what you mean: `count_if` is clearer than a loop with a counter.
>
> **What happens if you do not.** You write your own sort at 2 a.m., and it is slightly wrong.

> **WARNING**
> Reading past the end of an array or vector (`v[8]` in a vector of 8) is not an error that the compiler or even the program will always notice. It reads whatever happens to be in memory. Always keep indexes within `0` to `size() - 1`; `v.at(i)` checks for you and stops with a clear message.

17.8 Strings and the Command Line

Text is a `std::string`, from `<string>`. You can join strings with `+`, ask for `size()`, cut pieces with `substr`, and turn a number into text with `std::to_string`. A program can also be given **arguments** when it starts: `main(int argc, char** argv)` receives the count and the words. This is how Patrick's program accepts `--duty 0.9` or `--bands 0`.

```
 1  #include <cstdio>
 2  #include <cstdlib>
 3  #include <cstring>
 4  #include <string>
 5
 6  int main(int argc, char** argv) {
 7    double duty = 0.7; bool bands = true; std::string name = "Patrick";
 8    for (int i = 1; i < argc; i++) {
 9      if (!std::strcmp(argv[i], "--duty") && i + 1 < argc) duty = std::atof(argv[+
   +i]);
10      else if (!std::strcmp(argv[i], "--bands") && i + 1 < argc) bands = std::atoi
   (argv[++i]) != 0;
11      else if (!std::strcmp(argv[i], "--name") && i + 1 < argc) name = argv[++i];
12    }
13    std::string line = "Robot " + name + ": duty " + std::to_string(duty).substr(0
   , 4) + ", bands " + (bands ? "on" : "off");
14    std::printf("%s (%zu characters)\n", line.c_str(), line.size());
15    std::printf("argc was %d\n", argc);
16    return 0;
17  }
```

Listing from `Guide/code/cpp/s08_args.cpp`*, lines 1 to 17.*

Run it plainly and with arguments:

```
$ ./s08_args
Robot Patrick: duty 0.70, bands on (34 characters)
argc was 1
$ ./s08_args --duty 0.9 --bands 0 --name Rex
Robot Rex: duty 0.90, bands off (31 characters)
argc was 7
```

`argv[0]` is the program's own name, so `argc` is always at least 1. `std::strcmp` returns 0 when two C strings are equal (note the `!` before it). `std::atof` and `std::atoi` convert text to a `double` and an `int`. The `i + 1 < argc` test makes sure there *is* a next argument before `++i` reads it.

> **WHY DO IT THIS WAY? Command-line options instead of editing the code**
> **What it is.** Settings such as the duty cycle and whether the rubber bands are on are chosen when the program is run.
> **Why it is done this way.** The same compiled program can run a hundred experiments, from a script, without being rebuilt.

> **Why it matters.** The experiments in Chapter 42 (full throttle on bare wheels, pausing at 10.5 seconds) are each one command line.
>
> **What happens if you do not.** You edit, recompile and rerun for every experiment, and eventually run the wrong build.

17.9 Structs and Classes

A **struct** groups related variables; a **class** does the same and can hide some of them. Giving a type its own functions, called **member functions**, is the beginning of object-oriented programming. A **constructor** is the member function that sets up a new object.

```
 1  #include <cmath>
 2  #include <cstdio>
 3  #include <initializer_list>
 4
 5  class Motor {
 6   public:
 7    Motor(double Vbat, double Rm, double ke, double I0) : Vbat_(Vbat), Rm_(Rm), ke
    _(ke), I0_(I0) {}
 8    double current(double V, double w) const { return (V - ke_ * w) / Rm_; }
     // the motor law
 9    double freeSpeed(double V) const { return (V - I0_ * Rm_) / ke_; }
      // speed with no load, rad/s
10    double stallCurrent(double V) const { return V / Rm_; }
11    double supply(double duty) const { return duty * (Vbat_ - 1.4); }
      // volts after the driver's drops
12   private:
      // hidden: only the member functions touch these
13    double Vbat_, Rm_, ke_, I0_;
14  };
15
16  int main() {
17    Motor m(6.0, 6.0, 0.2326, 0.19);
18    for (double duty : {0.4, 0.7, 1.0}) {
19      double V = m.supply(duty), w0 = m.freeSpeed(V);
20      std::printf("duty %.1f: %.2f V, free speed %.2f rad/s = %.0f rpm, stall curr
    ent %.2f A\n", duty, V, w0, w0 * 60 / (2 * 3.14159265), m.stallCurrent(V));
21    }
22    return 0;
23  }
```

Listing from `Guide/code/cpp/s09_motor_class.cpp`*, lines 1 to 23.*

It prints Patrick's motor at three throttle settings:

```
duty 0.4: 1.84 V, free speed 3.01 rad/s = 29 rpm, stall current 0.31 A
duty 0.7: 3.22 V, free speed 8.94 rad/s = 85 rpm, stall current 0.54 A
duty 1.0: 4.60 V, free speed 14.88 rad/s = 142 rpm, stall current 0.77 A
```

Line 5 declares the class. Everything after `public:` can be used from outside; everything after `private:` (**line 12**) can be used only by the class's own functions. **Line 7** is the constructor: after the colon is an **initialiser list**, which sets the hidden members `Vbat_`, `Rm_`, `ke_` and `I0_` as the object is created (the trailing underscore is a habit that marks members). The functions marked `const` after the brackets promise not to change the motor. In `main` (**line 17**), `Motor m(6.0, 6.0, 0.2326, 0.19);` creates one, and `m.supply(duty)` calls its member function with a dot.

The numbers are Patrick's: at 70 percent duty the motor sees 3.22 V, and with no load it would spin at 8.94 rad/s (about 85 rpm). With a wheel of radius 0.0325 m, that is the robot's top speed of about 0.29 m/s, which is what you see on the page.

> **WHY DO IT THIS WAY? Hiding the data inside a class**
>
> **What it is.** The numbers that describe the motor are private; the outside world can only call `current`, `freeSpeed`, `stallCurrent` and `supply`.
>
> **Why it is done this way.** The class guards its own rules. No other code can accidentally set the resistance to zero.
>
> **Why it matters.** The motor law is written in one place. When the real motor was measured and the resistance corrected, one line changed and every part of the program used the right value.
>
> **What happens if you do not.** Any code anywhere can change any number, and when something is wrong you must search the whole program for who did it.

> **WHY DO IT THIS WAY? One struct holding the whole state**
>
> **What it is.** In Patrick's program, every changing value of the simulation (position, heading, arm angle, which phase) lives inside one `Mission` object.
>
> **Why it is done this way.** A simulation is "a state, and a rule that turns it into the next state". Putting the state in one place makes that literal.
>
> **Why it matters.** You can run two missions side by side, or restart one, by making another object. Nothing is hidden in the program's global variables.
>
> **What happens if you do not.** State scattered across many loose global variables, where restarting means remembering to reset forty of them and missing the forty-first.

Practice Problems C: Collections, Text and Classes

Review Problems

1. A `std::vector<int> v;` receives `v.push_back(4); v.push_back(9); v.push_back(2);`. What are `v.size()`, `v[1]` and `v.back()`?
2. What is the median of {5, 1, 4, 2, 3}? Why does a median beat a mean for glitchy sonar data?
3. What is the mean of the readings 0.82, 0.79, 0.85?
4. A program is run as `./prog --duty 0.9`. What are `argc`, `argv[1]` and `argv[2]`?
5. In Section 17.9, what are the free speed and stall current at 100 percent duty?
6. Why are the data members of a class usually private?
7. A member function does not change the object but is not marked `const`. What goes wrong?
8. After `std::string s = "ab"; s += "cd";`, what are `s` and `s.size()`?

17.10 Choosing Between States: enum and switch

An `enum` is a list of named choices; a `switch` runs the code for the current one. Together they make a **state machine**, which is how Patrick thinks: APPROACH, BRAKE, REACH, GRIP and so on. At any moment the machine is in exactly one state, and each state knows what to do and when to move on.

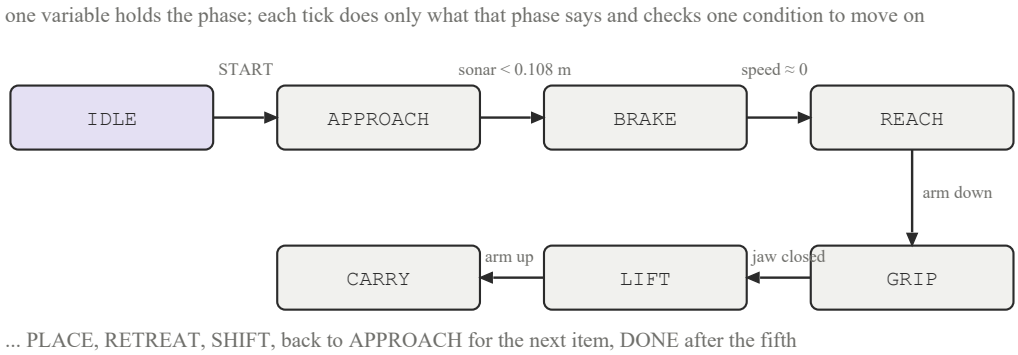


Figure 17.2. The first phases of the breakfast run as a state machine. The arrows carry the condition that moves the robot on; everything else about a phase is what it does while it waits for that condition.

Here is a smaller one, an automatic door:

```
 1  #include <cstdio>
 2
 3  enum class Door { CLOSED, OPENING, OPEN, CLOSING };
 4  const char* name(Door d) {
 5    switch (d) { case Door::CLOSED: return "CLOSED"; case Door::OPENING: return "O
   PENING"; case Door::OPEN: return "OPEN"; case Door::CLOSING: return "CLOSING"; }
 6    return "?";
 7  }
 8
 9  int main() {
10    Door door = Door::CLOSED; double pos = 0, dt = 0.5;                  // positi
   on 0 (shut) to 1 (open)
11    bool buttonAt[] = {false, true, false, false, false, false, false, false, true
   , false, false, false, false, false};
12    for (int i = 0; i < 14; i++) {
13      bool button = buttonAt[i];
14      switch (door) {
15        case Door::CLOSED:  if (button) door = Door::OPENING; break;
16        case Door::OPENING: pos += 0.4 * dt; if (pos >= 1) { pos = 1; door = Door:
   :OPEN; } break;
17        case Door::OPEN:    if (button) door = Door::CLOSING; break;
18        case Door::CLOSING: pos -= 0.4 * dt; if (pos <= 0) { pos = 0; door = Door:
   :CLOSED; } break;
19      }
20      std::printf("t=%4.1f s  button=%d  %-8s position %.1f\n", (i + 1) * dt, butt
   on, name(door), pos);
21    }
22    return 0;
23  }
```

Listing from `Guide/code/cpp/s10_door.cpp`*, lines 1 to 23.*

The door starts closed. A button press at 1.0 s opens it, and another at 4.5 s closes it:

```
t= 0.5 s  button=0  CLOSED   position 0.0
t= 1.0 s  button=1  OPENING  position 0.0
t= 1.5 s  button=0  OPENING  position 0.2
```

```
t= 2.0 s  button=0  OPENING  position 0.4
t= 2.5 s  button=0  OPENING  position 0.6
t= 3.0 s  button=0  OPENING  position 0.8
t= 3.5 s  button=0  OPEN     position 1.0
t= 4.0 s  button=0  OPEN     position 1.0
t= 4.5 s  button=1  CLOSING  position 1.0
t= 5.0 s  button=0  CLOSING  position 0.8
...
t= 7.0 s  button=0  CLOSED   position 0.0
```

(The `...` stands for four more lines of the door closing.) `enum class` (line 3) is the modern form: its names are reached as `Door::OPEN`, so two enums can both have an `OPEN` without clashing. Each `case` ends with `break;`; leave it out and execution *falls through* into the next case, which is rarely wanted. Notice that a button press while the door is OPENING is simply ignored: no `case` mentions it, so nothing happens.

> **WHY DO IT THIS WAY? An enum instead of numbers**
>
> **What it is.** Writing `case Door::CLOSING:` instead of `case 3:`.
>
> **Why it is done this way.** Numbers carry no meaning. Names do.
>
> **Why it matters.** The code reads like the behaviour. The web page's code viewer even finds the line to highlight by searching for the name of the current state.
>
> **What happens if you do not.** `if (phase == 7)` appears and nobody, including you next month, knows what 7 is.

17.11 References, Pointers and Cleaning Up After Yourself

A **reference** is another name for a variable. A **pointer** is a variable that holds the *address* of another, written with `*`; you follow it with `*p`, take an address with `&x`, and a pointer that points at nothing is `nullptr`. In modern C++ you meet raw pointers less than you might fear, because the standard library manages memory for you.

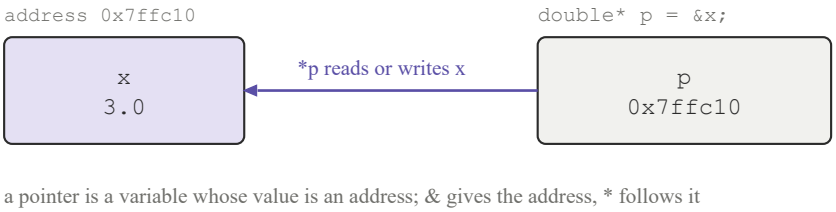


Figure 17.3. A pointer holds an address. `&x` asks where `x` lives; `*p` goes there. Reading `*p` reads `x`; writing `*p = 4` changes `x`.

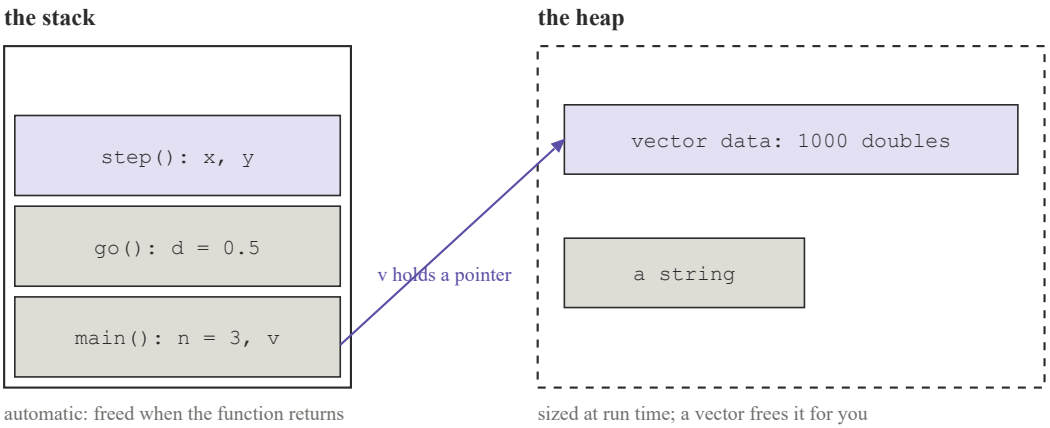


Figure 17.4. Where variables live. Locals sit on the stack in the frame of the function that made them and vanish when it returns. Anything whose size is only known at run time, such as a vector's contents, lives on the heap, and a local vector holds a pointer to it and frees it for you.

```
 1  #include <cstdio>
 2  #include <memory>
 3  #include <vector>
 4
 5  void addOne(int& x) { x += 1; }           // by reference
 6  void addOnePtr(int* x) { *x += 1; }       // by pointer
 7
 8  struct Sensor { double reading = 0; ~Sensor() { std::printf("  sensor destroyed\
    n"); } };
 9
10  int main() {
11    int n = 5; addOne(n); addOnePtr(&n);
12    std::printf("n = %d\n", n);
13    int* p = &n; std::printf("p points at %d; *p + 1 = %d\n", *p, *p + 1);
14    int* nothing = nullptr; std::printf("a null pointer is %s\n", nothing ? "set"
    : "null: do not use it");
15    {
16      std::unique_ptr<Sensor> s = std::make_unique<Sensor>();            // owns i
    ts object and cleans up on its own
17      s->reading = 0.42; std::printf("reading %.2f\n", s->reading);
18    }                                                                    // <- des
    troyed here, automatically
19    std::vector<Sensor> list(2); (void)list;
20    std::printf("end of main\n");
21    return 0;
22  }
```

Listing from `Guide/code/cpp/s11_pointers.cpp`*, lines 1 to 22.*

Its output:

```
n = 7
p points at 7; *p + 1 = 8
a null pointer is null: do not use it
reading 0.42
  sensor destroyed
end of main
  sensor destroyed
  sensor destroyed
```

`addOne(int& x)` and `addOnePtr(int* x)` do the same job by two routes, and `n` goes from 5 to 7. The interesting part is the **destructor** (`~Sensor`), a function that runs automatically when an object's

life ends. The `std::unique_ptr` owns its sensor, and when it leaves its braces the sensor is destroyed at once, printing the first "sensor destroyed" *before* `end of main`. The two vector elements are destroyed at the very end of `main`. This habit, tying a resource's life to a variable's scope, is called **RAII**, and it is why modern C++ rarely needs `new` and `delete`.

> **WHY DO IT THIS WAY? Letting objects clean up after themselves**
>
> **What it is.** A smart pointer (`std::unique_ptr`) or a container (`std::vector`) frees what it owns when it goes out of scope.
>
> **Why it is done this way.** Forgetting to free memory (a leak), or freeing it twice, are among the oldest bugs in programming. Tying cleanup to scope makes them impossible to forget.
>
> **Why it matters.** A robot controller runs for hours. A tiny leak on every control tick would eventually fill the memory and crash it.
>
> **What happens if you do not.** The program works in a five-second test and fails at hour three.

> **WARNING**
>
> Never use a pointer after the object it points at is gone, and never follow a `nullptr`. Both are crashes waiting to happen. When you can, pass references and use `std::vector` and `std::unique_ptr` rather than raw pointers.

17.12 Lambdas and the Standard Algorithms

A **lambda** is a small function written in the middle of other code. Its square brackets say what it may see from the surrounding code: `[]` nothing, `[limit]` a copy of `limit`, `[&calls]` the variable `calls` itself, `[&]` everything by reference. Lambdas are how you tell the standard algorithms what you mean.

```
#include <algorithm>
#include <cstdio>
#include <numeric>
#include <vector>

struct Reading { const char* source; double distance; };

int main() {
  std::vector<Reading> r = {{"front", 0.42}, {"left", 1.30}, {"right", 0.15}, {"rear", 2.20}};
  std::sort(r.begin(), r.end(), [](const Reading& a, const Reading& b) { return a.distance < b.distance; });   // sort nearest first
  for (auto& x : r) std::printf("%-5s %.2f\n", x.source, x.distance);
  double limit = 1.0;
  int near = (int)std::count_if(r.begin(), r.end(), [limit](const Reading& x) { return x.distance < limit; });   // capture by value
  std::printf("closer than %.1f m: %d\n", limit, near);
  double total = std::accumulate(r.begin(), r.end(), 0.0, [](double s, const Reading& x) { return s + x.distance; });
  std::printf("total distance %.2f\n", total);
  int calls = 0; auto count = [&calls]() { calls++; };
                                 // capture by reference
  count(); count(); count(); std::printf("lambda called %d times\n", calls);
  return 0;
}
```

Listing from `Guide/code/cpp/s12_lambdas.cpp`*, lines 1 to 20.*

Its output:

```
right 0.15
front 0.42
left  1.30
rear  2.20
closer than 1.0 m: 2
total distance 4.07
lambda called 3 times
```

The first lambda tells `std::sort` how to order two readings (nearest first). The second, `[limit]`, counts readings under a limit, capturing a *copy* of `limit`. The third adds up the distances. The last captures `calls` by reference, so each call really changes it: the count reaches 3. Compare JavaScript: `const ramp = (T) => clamp(phaseT / T, 0, 1);`. In Patrick's `step()` there are lambdas such as `auto ramp = [&](double T) { return clampd(phaseT / T, 0, 1); };`.

> **WHY DO IT THIS WAY? Capturing with [&]**
>
> **What it is.** The `[&]` lets the lambda see and use `phaseT` and the other variables of the surrounding function.
>
> **Why it is done this way.** These helpers are only useful *here*, right next to the code that needs them, and they need the current `phaseT`.
>
> **Why it matters.** `ramp(0.35)` reads clearly where it is used: "ramp up over 0.35 seconds". No extra parameters need passing.
>
> **What happens if you do not.** Either you pass a long list of variables into each helper, or you promote them all into globals. Both are worse.

17.13 Reading and Writing Files

A simulation is far more useful if it writes its results where you can plot them. `std::ofstream` writes a file, `std::ifstream` reads one, and `std::getline` reads a line at a time. The most useful format is **CSV**: one row per line, values separated by commas, which every spreadsheet understands.

```
 1  #include <cstdio>
 2  #include <fstream>
 3  #include <sstream>
 4  #include <string>
 5  #include <vector>
 6
 7  int main() {
 8    {                                                    // write
 9      std::ofstream out("run.csv");
10      out << "t,x,v\n";
11      double x = 0, v = 0;
12      for (int i = 0; i <= 5; i++) { out << i * 0.5 << "," << x << "," << v << "\n
    "; v += 0.4; x += v * 0.5; }
13    }                                                    // closing brace closes t
    he file
14    std::ifstream in("run.csv");                        // read it back
15    std::string line; std::getline(in, line);           // the header
16    std::printf("header: %s\n", line.c_str());
17    std::vector<double> xs;
```

```
18    while (std::getline(in, line)) {
19      std::stringstream ss(line); std::string cell; std::vector<double> row;
20      while (std::getline(ss, cell, ',')) row.push_back(std::stod(cell));
21      xs.push_back(row[1]);
22    }
23    std::printf("read %zu rows, last x = %.2f\n", xs.size(), xs.back());
24    return 0;
25  }
```

Listing from `Guide/code/cpp/s13_csv.cpp`*, lines 1 to 25.*

It prints:

```
header: t,x,v
read 6 rows, last x = 3.00
```

The first block (lines 8 to 13) writes a header and six rows; the closing brace ends the block, which closes the file. The second block opens the file again, throws away the header, and for each remaining line uses a `std::stringstream` to split it at the commas and `std::stod` to turn each piece into a number. Patrick's whole program is built this way: it writes one row every 10 ms with `fprintf`.

> **WHY DO IT THIS WAY? Writing a table, not drawing a picture**
>
> **What it is.** The program's whole output is rows of numbers in a CSV file.
>
> **Why it is done this way.** A table can be plotted, filtered, compared, and read by any other tool. A picture can only be looked at.
>
> **Why it matters.** The twin test in Chapter 42 only works because both programs write a table you can compare byte for byte.
>
> **What happens if you do not.** You can see that two runs look similar and have no way to prove they are identical.

17.14 Splitting a Program Across Files

A program of any size lives in several files. A **header** (`.h`) declares what exists; a **source file** (`.cpp`) defines it; and any file that wants to use a function includes the header.

```
1  #pragma once                      // include guard: this header is only read onc
   e per file
2  double clampd(double x, double a, double b);
3  double lerp(double a, double b, double t);
```

Listing from `Guide/code/cpp/s14_mymath.h`*, lines 1 to 3.*

```
1  #include "s14_mymath.h"
2  #include <algorithm>
3
4  double clampd(double x, double a, double b) { return std::min(b, std::max(a, x))
   ; }
5  double lerp(double a, double b, double t) { return a + (b - a) * t; }
```

Listing from `Guide/code/cpp/s14_mymath.cpp`*, lines 1 to 5.*

```
1  #include <cstdio>
2  #include "s14_mymath.h"
3
4  int main() {
```

```
5    std::printf("clampd(5, 0, 1) = %.0f, lerp(0, 10, 0.3) = %.1f\n", clampd(5, 0,
   1), lerp(0, 10, 0.3));
6    return 0;
7  }
```

Listing from `Guide/code/cpp/s14_main.cpp`*, lines 1 to 7.*

Compile all the source files together and run the result:

```
g++ -std=c++17 -Wall -Wextra -O2 s14_main.cpp s14_mymath.cpp -o s14
./s14
```

It prints `clampd(5, 0, 1) = 1, lerp(0, 10, 0.3) = 3.0`. The first line of the header, `#pragma once`, is an **include guard**: it makes sure the header is read only once per file, even if several headers include it.

> **WHY DO IT THIS WAY? Declaring in a header and defining in a source file**
>
> **What it is.** The header says "this function exists and here is its shape"; the source file holds the body.
>
> **Why it is done this way.** Each `.cpp` is compiled separately, so each needs to know what the others offer. Splitting also means a change to one file recompiles only that file.
>
> **Why it matters.** On a big project, rebuilding one file takes a second instead of a minute. Patrick's program is one file because it is small; Part V explains when to split it.
>
> **What happens if you do not.** One enormous file, slow rebuilds, and every change risks everything.

> **TIP**
>
> If you forget to list `s14_mymath.cpp` on the command line, the compile succeeds but the **linker** fails with "undefined reference to `clampd`". A compile error is about one file; a link error is about the pieces not fitting together.

Practice Problems D: States, Memory, Lambdas, Files

Review Problems

1. In the door program, what happens if the button is pressed while the door is OPENING? Why?
2. What is the advantage of `enum class` over a plain `enum`?
3. When exactly is the `Sensor` inside the `std::unique_ptr` in Section 17.11 destroyed?
4. What is the difference between the lambda captures `[limit]` and `[&limit]` if `limit` changes later?
5. How many lines does the file `run.csv` from Section 17.13 have, counting the header?
6. Write the command that builds a program from `main.cpp` and `util.cpp`, with warnings and optimisation.

7. A build fails with `undefined reference to 'lerp'`. Is this a compile error or a link error, and what is the likeliest cause?
8. Why does the header in Section 17.14 begin with `#pragma once`?

17.15 Numerical Simulation: How Wrong Can a Step Be?

Floating-point numbers are not exact. A `double` keeps about 15 significant digits, in binary, and some simple decimals such as 0.1 have no exact binary form. In `double`, `0.1 + 0.2` is `0.30000000000000004`, so `0.1 + 0.2 == 0.3` is false.

> **WARNING**
> Never test a calculated decimal for exact equality (`if (x == 10.0)`). Test with >=, <= or a small tolerance such as `std::fabs(x - 10.0) < 1e-9`. A traffic light that was meant to change at 10.0 seconds can change at 10.1 because the running total reaches 9.99999999999998 first.

A simulation turns a differential equation into a loop. A mass on a spring obeys *x″ = -w squared x*, whose exact answer is *x(t) = cos(w t)*. Here are three ways to step it forward, compared with that exact answer:

```
#include <cmath>
#include <cstdio>
#include <initializer_list>

// A mass on a spring: x'' = -w^2 x, with exact answer x(t) = cos(w t). Three ways to step it.
int main() {
  const double w = 2.0, T = 10.0;
  for (double dt : {0.1, 0.01}) {
    double xe = 1, ve = 0;          // explicit Euler
    double xs = 1, vs = 0;          // semi-implicit Euler (what the robot uses)
    double xr = 1, vr = 0;          // RK4
    for (int i = 0; i < (int)(T / dt + 0.5); i++) {
      { double a = -w * w * xe; xe += ve * dt; ve += a * dt; }
      { vs += -w * w * xs * dt; xs += vs * dt; }
      { auto f = [&](double x, double v, double& dx, double& dv) { dx = v; dv = -w * w * x; };
        double k1x, k1v, k2x, k2v, k3x, k3v, k4x, k4v;
        f(xr, vr, k1x, k1v); f(xr + 0.5 * dt * k1x, vr + 0.5 * dt * k1v, k2x, k2v);
        f(xr + 0.5 * dt * k2x, vr + 0.5 * dt * k2v, k3x, k3v); f(xr + dt * k3x, vr + dt * k3v, k4x, k4v);
        xr += dt / 6 * (k1x + 2 * k2x + 2 * k3x + k4x); vr += dt / 6 * (k1v + 2 * k2v + 2 * k3v + k4v); }
    }
    double exact = std::cos(w * T);
    std::printf("dt=%.2f  exact %+.5f | Euler %+.5f (err %+.5f) | semi-implicit %+.5f (err %+.5f) | RK4 %+.7f (err %+.7f)\n",
                dt, exact, xe, xe - exact, xs, xs - exact, xr, xr - exact);
  }
  return 0;
}
```

Listing from `Guide/code/cpp/s15_integrators.cpp`, *lines 1 to 26.*

Its output, for two step sizes (w = 2, time 10 s):

```
dt=0.10  exact +0.40808 | Euler +4.47303 (err +4.06495) | semi-implicit +0.28421 (er
      r -0.12387) | RK4 +0.4083040 (err +0.0002219)
dt=0.01  exact +0.40808 | Euler +0.50138 (err +0.09330) | semi-implicit +0.39865 (er
      r -0.00944) | RK4 +0.4080821 (err +0.0000000)
```

Study the first row. **Explicit Euler** (update the position with the *old* speed, then the speed) has added energy at every step, so the spring that should swing between -1 and +1 has grown to 4.47: the answer is not just inaccurate but *wrong in kind*. **Semi-implicit Euler** (update the speed first, then the position with the *new* speed, which is what Patrick's code does) stays bounded, with a modest error of -0.12. **RK4** (the fourth-order Runge-Kutta method, which samples the slope four times per step) is accurate to 0.0002 with the same step. With a hundred times as many steps, Euler improves to an error of 0.09, semi-implicit to 0.009 and RK4 to nothing visible.

> **WHY DO IT THIS WAY? Semi-implicit Euler for the robot**
>
> **What it is.** Each tick, the speed is updated first, and the position then uses the new speed.
>
> **Why it is done this way.** It costs the same as plain Euler but does not pump energy into oscillating systems, and it is simple enough to write identically in two languages.
>
> **Why it matters.** Wheels, arms and servos all oscillate if you treat them badly. A method that adds energy turns a tidy simulation into one that shakes itself apart.
>
> **What happens if you do not.** The robot works at slow speeds and goes unstable at the moments that matter most: starting and stopping.

> **TECHNICAL STUFF: Technical Stuff**
>
> Even semi-implicit Euler has a limit: for a spring-like system the step size times the natural frequency must stay below 2. That is the reason Patrick's tyre model (an extremely stiff "spring" at low speed) is advanced in four sub-steps per tick, as Chapter 31 and Chapter 42 explain.

17.16 A PID Controller in Code

You met PID control on the drone page. A PID controller computes a command from the *error* (target minus measured): *u = Kp x e + Ki x (integral of e) + Kd x (rate of change of e)*. The **P** term pushes in proportion to the error; the **I** term builds up while an error lingers and removes steady offsets; the **D** term brakes when the error is changing fast.

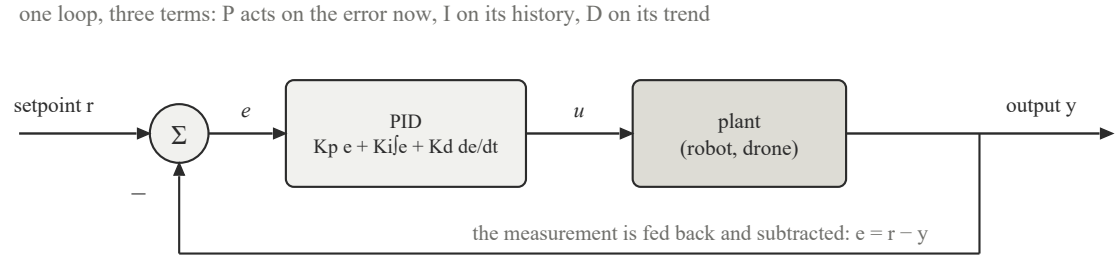


Figure 17.5. The feedback loop. The measurement is subtracted from the setpoint to give the error, the controller turns the error into a command, the plant responds, and the new measurement comes back round. The controller's memory (the integral and the last error) is what the class below keeps between calls.

In code it is a small class with a little memory (the integral and the previous error):

```
#include <cstdio>
#include <algorithm>
#include <cmath>

// A PID controller as a class: u = Kp e + Ki integral(e) + Kd de/dt, with a lim
it on the output and on the integral (anti-windup).
class PID {
 public:
  PID(double kp, double ki, double kd, double lo, double hi) : kp_(kp), ki_(ki),
 kd_(kd), lo_(lo), hi_(hi) {}
  double update(double setpoint, double measured, double dt) {
    double e = setpoint - measured;
    double d = first_ ? 0.0 : (e - prev_) / dt;
    first_ = false; prev_ = e;
    double uNoI = kp_ * e + kd_ * d;
    integral_ += e * dt;
    if (ki_ > 0) integral_ = std::min(hi_ / ki_, std::max(lo_ / ki_, integral_))
;          // anti-windup: the integral may not demand more than the output can
give
    return std::min(hi_, std::max(lo_, uNoI + ki_ * integral_));
  }
 private:
  double kp_, ki_, kd_, lo_, hi_, integral_ = 0, prev_ = 0; bool first_ = true;
};

// A cart on a hill: the motor pushes with force u (N), gravity pulls back with
1.5 N, drag 2 N s/m, mass 0.6 kg.
double run(double kp, double ki, double kd, double target, double& settle) {
  PID pid(kp, ki, kd, 0.0, 8.0);
  double v = 0, dt = 0.001; settle = -1;
  for (int i = 0; i < 20000; i++) {
    double u = pid.update(target, v, dt);
    double a = (u - 1.5 - 2.0 * v) / 0.6;
    v += a * dt;
    if (settle < 0 && std::fabs(v - target) < 0.02 * target && i > 100) settle =
 i * dt;
  }
  return v;
}
int main() {
  double st;
  double v1 = run(10, 0, 0, 0.5, st);   std::printf("P only      : speed after 2
0 s = %.4f m/s (target 0.5)\n", v1);
```

```
37    double v2 = run(10, 8, 0, 0.5, st);   std::printf("PI          : speed after 2
    0 s = %.4f m/s, within 2%% at %.2f s\n", v2, st);
38    double v3 = run(10, 8, 1.0, 0.5, st); std::printf("PID (Kd = 1): speed after 2
    0 s = %.4f m/s, within 2%% at %.2f s\n", v3, st);
39    return 0;
40  }
```

Listing from `Guide/code/cpp/s16_pid.cpp`*, lines 1 to 40.*

The test is a cart climbing a gentle slope: gravity pulls back with 1.5 N, drag adds 2 N per m/s, and the target speed is 0.5 m/s. Running it gives:

```
P only      : speed after 20 s = 0.2917 m/s (target 0.5)
PI          : speed after 20 s = 0.5000 m/s, within 2% at 4.38 s
PID (Kd = 1): speed after 20 s = 0.4979 m/s, within 2% at 1.31 s
```

Look at the first line. With **P only** the cart never reaches the target: it settles at 0.2917 m/s. That is not a bug; it is a law. To hold 0.5 m/s the motor must push with 1.5 + 2 x 0.5 = 2.5 N, but a P controller only pushes when there is an error, so it needs an error to produce its push. Solving 10 x (0.5 - v) = 1.5 + 2v gives v = 0.2917 exactly. Adding the **I** term (second line) removes the error entirely, because the integral keeps growing until the force is enough. Adding **D** as well (third line) makes the response quicker: within 2 percent of the target at 1.31 s instead of 4.38 s.

> **WHY DO IT THIS WAY? Anti-windup: limiting the integral**
>
> **What it is.** The line that clamps `integral_` so that `ki_ * integral_` can never ask for more than the output limits allow.
>
> **Why it is done this way.** If the cart is held back (or the motor is saturated), the error stays large and the integral would grow without limit.
>
> **Why it matters.** When the obstruction clears, a huge stored integral keeps pushing long after the error has gone, and the system overshoots wildly. This is called integrator windup.
>
> **What happens if you do not.** The controller works in the gentle test and misbehaves the first time something blocks it.

17.17 Testing and Debugging

How do you know a function is right? Test it. A test is a call with an answer you worked out by hand. The smallest useful test harness is a macro and a counter:

```
1  #include <algorithm>
2  #include <cmath>
3  #include <cstdio>
4
5  double clampd(double x, double a, double b) { return std::min(b, std::max(a, x))
   ; }
6  double speedFromRpm(double rpm, double r) { return rpm * 2.0 * 3.141592653589793
   23846 / 60.0 * r; }
7
8  int failures = 0;
9  #define CHECK_NEAR(expr, expected, tol) do { double got = (expr); if (std::fabs(
   got - (expected)) > (tol)) { std::printf("FAIL line %d: %s = %g, expected %g\n",
    __LINE__, #expr, got, (double)(expected)); failures++; } else std::printf("ok
    %s\n", #expr); } while (0)
```

```
10
11  int main() {
12    CHECK_NEAR(clampd(5, 0, 1), 1, 1e-12);
13    CHECK_NEAR(clampd(-5, 0, 1), 0, 1e-12);
14    CHECK_NEAR(clampd(0.3, 0, 1), 0.3, 1e-12);
15    CHECK_NEAR(speedFromRpm(60, 1.0), 6.283185, 1e-5);        // 60 rpm on a 1 m r
    adius wheel: 2 pi m/s
16    CHECK_NEAR(speedFromRpm(100, 0.0325), 0.340, 1e-3);
17    CHECK_NEAR(speedFromRpm(0, 0.0325), 1.0, 1e-9);           // a deliberately wr
    ong test, to see a failure
18    std::printf("%d failure(s)\n", failures);
19    return failures ? 1 : 0;
20  }
```

Listing from `Guide/code/cpp/s17_tests.cpp`*, lines 1 to 20.*

Its output:

```
ok   clampd(5, 0, 1)
ok   clampd(-5, 0, 1)
ok   clampd(0.3, 0, 1)
ok   speedFromRpm(60, 1.0)
ok   speedFromRpm(100, 0.0325)
FAIL line 17: speedFromRpm(0, 0.0325) = 0, expected 1
1 failure(s)
```

The last test is deliberately wrong (a robot standing still has speed 0, not 1) so that you can see what a failure looks like: it reports the line, the expression, the value it got and the value expected. Notice the comparison uses a **tolerance** (`std::fabs(got - expected) > tol`), never ==. The program also returns a non-zero exit code when anything fails, which is how build scripts know something is broken.

Beyond tests, the debugging toolbox has four tools. **Print and look** (`printf` of the values around the suspect line) solves most bugs. `assert` from `<cassert>` stops the program with the failing condition if an assumption is false. **Sanitizers** (`g++ -g -fsanitize=address,undefined` on Linux or macOS) make the program stop with a clear message on an out-of-range read or a divide by zero. A **debugger** such as `gdb` lets you stop at a line and look at every variable. And the strongest method of all, used on Patrick's program, is a **twin**: write the same thing a second time, differently, and compare.

> **WHY DO IT THIS WAY? Testing the edges, not just the middle**
>
> **What it is.** Tests for values at the limits: below the range, above the range, and exactly zero.
>
> **Why it is done this way.** Bugs live at the edges: where a decision flips, where a number is zero, where a list is empty.
>
> **Why it matters.** `clampd(0.3, 0, 1)` passing tells you almost nothing. `clampd(-5, 0, 1)` passing tells you the lower limit works.
>
> **What happens if you do not.** The code passes your tests and fails the first time it meets a real sensor that reads zero.

17.18 Templates and Operators: Writing Once, Using Many Times

Sometimes one function should work for several types. A **template** is a function (or class) written once with a placeholder type. And **operator overloading** lets your own types use the usual symbols, so that adding two positions reads as `a + b`.

```
 1  #include <cmath>
 2  #include <cstdio>
 3
 4  struct Vec2 {
 5    double x = 0, z = 0;
 6    Vec2 operator+(const Vec2& o) const { return {x + o.x, z + o.z}; }
     // operator overloading: a + b reads like maths
 7    Vec2 operator*(double k) const { return {x * k, z * k}; }
 8    double length() const { return std::hypot(x, z); }
 9  };
10
11  template <typename T>
      // a template: one function, any numeric type
12  T clampT(T x, T a, T b) { return x < a ? a : (x > b ? b : x); }
13
14  template <typename T>
15  T largest(const T* a, int n) { T m = a[0]; for (int i = 1; i < n; i++) if (a[i]
    > m) m = a[i]; return m; }
16
17  int main() {
18    Vec2 pos{0.0, 0.0}, step{0.3, 0.4};
19    for (int i = 0; i < 3; i++) pos = pos + step * 0.5;
20    std::printf("after three half-steps: (%.2f, %.2f), distance from start %.3f\n"
    , pos.x, pos.z, pos.length());
21    std::printf("clampT<int>(15, 0, 10) = %d, clampT<double>(-0.5, 0, 1) = %.1f\n"
    , clampT(15, 0, 10), clampT(-0.5, 0.0, 1.0));
22    int a[] = {3, 9, 2}; double b[] = {0.5, 0.25};
23    std::printf("largest int %d, largest double %.2f\n", largest(a, 3), largest(b,
     2));
24    return 0;
25  }
```

Listing from `Guide/code/cpp/s18_vec2.cpp`*, lines 1 to 25.*

Its output:

```
after three half-steps: (0.45, 0.60), distance from start 0.750
clampT<int>(15, 0, 10) = 10, clampT<double>(-0.5, 0, 1) = 0.0
largest int 9, largest double 0.50
```

`Vec2` is a two-number position with `+`, `*` and `length()` defined, so the loop `pos = pos + step * 0.5` reads like the maths. `template <typename T>` tells the compiler that `T` stands for whatever type the caller uses; it builds a separate version for `int` and for `double` automatically. Templates are how the standard library's `std::vector<int>` and `std::vector<double>` come from one definition.

> **TECHNICAL STUFF: Technical Stuff**
>
> There is more to C++ than this chapter covers, and you do not need it to build a robot: inheritance and virtual functions (objects that share an interface), move semantics (handing a big object over without copying it), `std::function` (storing a function in a variable), threads (running two things at once) and the compile-time features (`constexpr`). When you meet one of them, you now know enough to read about it.

Practice Problems E and F: Numerics, PID, Tests, Templates

Review Problems

1. In Section 17.15, why is explicit Euler's answer at dt = 0.1 (+4.47) "wrong in kind", not just inaccurate?
2. Why can't a P-only controller hold the cart at 0.5 m/s on the slope? Derive the speed it settles at when Kp = 10.
3. What does anti-windup protect against, and when would you notice its absence?
4. The failing test `speedFromRpm(0, 0.0325)` expected 1.0. What should the expected value be, and why was the test written to fail?
5. Why does the test harness compare with a tolerance instead of ==?
6. Write the call `CHECK_NEAR(...)` that tests `lerp(10, 20, 0.25)` against its correct value 12.5.
7. Using `Vec2` from Section 17.18, what is `Vec2{1, 2} + Vec2{3, 4} * 0.5`?
8. What is `Vec2{3, 4}.length()`, and what does `clampT<int>(15, 0, 10)` return?

17.19 Five Mini Projects

You now know enough C++ to write real programs. Each project below is complete and runs; type it in, run it, then do the "extend it" list. They build up from a plain tool to a small robot controller.

17.19.1 Mini Project 1: A Length Converter

Goal. `./mp1_convert 12 in` prints a length in millimetres, centimetres, metres, inches and feet. **Plan.** Keep a table of units and their size in metres (a struct and an array), find the unit the user named, convert to metres, and then to every unit. **Skills.** Structs, arrays, `argv`, a loop, error messages.

```
1  // Mini project 1: a length converter.  ./mp1 12 in   ->  the length in mm, cm,
   m, in and ft
2  #include <cstdio>
3  #include <cstdlib>
4  #include <cstring>
5
6  struct Unit { const char* name; double metres; };
7  const Unit UNITS[] = {{"mm", 0.001}, {"cm", 0.01}, {"m", 1.0}, {"in", 0.0254}, {
   "ft", 0.3048}};
8  const int N = sizeof(UNITS) / sizeof(UNITS[0]);
9
10 int find(const char* name) {
11   for (int i = 0; i < N; i++) if (!std::strcmp(UNITS[i].name, name)) return i;
12   return -1;
13 }
14
15 int main(int argc, char** argv) {
16   if (argc != 3) { std::printf("usage: %s <value> <mm|cm|m|in|ft>\n", argv[0]);
   return 1; }
17   double value = std::atof(argv[1]);
18   int from = find(argv[2]);
19   if (from < 0) { std::printf("unknown unit '%s'\n", argv[2]); return 1; }
20   double metres = value * UNITS[from].metres;
```

```
  for (int i = 0; i < N; i++) std::printf("%10.4f %s\n", metres / UNITS[i].metres, UNITS[i].name);
  return 0;
}
```

Listing from `Guide/code/cpp/mp1_convert.cpp`*, lines 1 to 23.*

```
$ ./mp1_convert 12 in
  304.8000 mm
   30.4800 cm
    0.3048 m
   12.0000 in
    1.0000 ft
$ ./mp1_convert 5 parsec
unknown unit 'parsec'
```

Notice that the table drives everything: to add a unit you add one line to `UNITS`, and `N`, computed from the array's size, keeps up. **Extend it:** add kilometres and miles; add a `--to` option that prints only one unit; reject negative lengths.

17.19.2 Mini Project 2: A Sonar Log Analyser

Goal. Read a CSV log of sonar distances, report the glitches, clean them and estimate the robot's speed. **Plan.** The program has two modes: `--make` writes a fake log (a robot closing on a wall at 0.1 m/s, with 5 mm of noise and three "no echo" glitches of 4.0 m), and the normal mode reads and analyses it. A **median of three** filter replaces each reading by the middle of itself and its two neighbours, which removes any glitch that lasts a single reading. **Skills.** Files, vectors, lambdas, algorithms.

```
// Mini project 2: a sonar log analyser.   ./mp2 --make   writes sonar.csv;   ./mp2   reads it and cleans it up.
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

double median3(double a, double b, double c) { return std::max(std::min(a, b), std::min(std::max(a, b), c)); }

int main(int argc, char** argv) {
  if (argc > 1 && !std::strcmp(argv[1], "--make")) {                    // a fake drive towards a wall at 10 cm/s, with glitches
    std::ofstream out("sonar.csv"); out << "t,distance\n";
    unsigned s = 12345;
    for (int i = 0; i < 40; i++) {
      s = s * 1103515245u + 12345u; double noise = ((s >> 16) % 100) / 100.0 * 0.01 - 0.005;      // +/- 5 mm
      double d = 1.0 - 0.1 * i * 0.25 + noise;
      if (i == 7 || i == 22 || i == 23) d = 4.0;                         // "no echo": the sensor's maximum
      out << i * 0.25 << "," << d << "\n";
    }
    std::printf("wrote sonar.csv\n"); return 0;
  }
  std::ifstream in("sonar.csv"); std::string line; std::getline(in, line);
  std::vector<double> d;
  while (std::getline(in, line)) { std::stringstream ss(line); std::string t, v;
   std::getline(ss, t, ','); std::getline(ss, v, ','); d.push_back(std::stod(v));
```

```
    }
  27    int bad = (int)std::count_if(d.begin(), d.end(), [](double x) { return x >= 4.
    0; });
  28    std::printf("%zu readings, %d are 'no echo' glitches\n", d.size(), bad);
  29    std::vector<double> clean = d;
  30    for (size_t i = 1; i + 1 < d.size(); i++) clean[i] = median3(d[i - 1], d[i], d
    [i + 1]);       // median of three removes single glitches
  31    int left = (int)std::count_if(clean.begin(), clean.end(), [](double x) { retur
    n x >= 4.0; });
  32    std::printf("after the median-of-3 filter: %d glitches left (the pair at 22-23
     beats a 3-point filter)\n", left);
  33    double speed = (d[0] - d[39]) / (39 * 0.25);
  34    std::printf("distance fell from %.3f to %.3f m: closing speed about %.3f m/s\n
    ", d[0], d[39], speed);
  35    return 0;
  36  }
```

Listing from `Guide/code/cpp/mp2_sonar.cpp`*, lines 1 to 36.*

```
$ ./mp2_sonar --make
wrote sonar.csv
$ ./mp2_sonar
40 readings, 3 are 'no echo' glitches
after the median-of-3 filter: 2 glitches left (the pair at 22-23 beats a 3-point fil
      ter)
distance fell from 0.999 to 0.028 m: closing speed about 0.100 m/s
```

The filter removed the single glitch at reading 7, but the *pair* at readings 22 and 23 survives: with two bad readings in a row, the median of three still contains a bad one. **Extend it:** use a median of five, which removes pairs; print the cleaned log to a new CSV; flag any reading that jumps by more than 0.2 m in one step.

17.19.3 Mini Project 3: A Bouncing Ball

Goal. Simulate a ball dropped from 1 m that loses 20 percent of its speed at each bounce, write its height to a CSV, and check the first bounce against physics. **Plan.** Each tick, gravity changes the speed and the speed changes the position (semi-implicit Euler from Section 17.15). When the height goes below zero, put the ball on the floor and reverse the speed times the bounce factor. **Skills.** The simulation loop, a time step, file output, checking against a formula.

```
   1  // Mini project 3: a bouncing ball.  Writes ball.csv and reports the first bounc
     es.
   2  #include <cmath>
   3  #include <cstdio>
   4  #include <fstream>
   5
   6  int main() {
   7    const double g = 9.80665, e = 0.8, dt = 0.001;      // e: how much speed survi
     ves a bounce
   8    double y = 1.0, v = 0.0, t = 0.0, peak = 1.0;
   9    std::ofstream out("ball.csv"); out << "t,y\n";
  10    int bounces = 0; bool rising = false;
  11    for (int i = 0; i < 6000; i++) {
  12      v -= g * dt; y += v * dt; t += dt;
  13      if (y < 0) { y = 0; v = -v * e; bounces++; std::printf("bounce %d at t = %.3
     f s, then rises to ", bounces, t); peak = 0; rising = true; }
  14      if (rising && v > 0) peak = std::fmax(peak, y);
  15      if (rising && v <= 0) { std::printf("%.3f m\n", peak); rising = false; }
  16      if (i % 10 == 0) out << t << "," << y << "\n";
  17    }
  18    std::printf("exact: first bounce at %.3f s, first rebound height %.3f m\n", st
```

```
    d::sqrt(2 * 1.0 / g), e * e * 1.0);
19    return 0;
20  }
```

Listing from `Guide/code/cpp/mp3_ball.cpp`*, lines 1 to 20.*

```
bounce 1 at t = 0.452 s, then rises to 0.639 m
bounce 2 at t = 1.175 s, then rises to 0.408 m
bounce 3 at t = 1.753 s, then rises to 0.261 m
bounce 4 at t = 2.215 s, then rises to 0.166 m
...
exact: first bounce at 0.452 s, first rebound height 0.640 m
```

The simulation agrees with the formulas: the fall takes the square root of (2 x 1 / g), which is 0.452 s, and the first rebound reaches 0.8 squared times 1 m, which is 0.640 m (the simulation's 0.639 is the time step's tiny error). Open `ball.csv` in a spreadsheet and plot it to see the shrinking arcs. **Extend it:** stop when the bounces become smaller than 1 cm and report how long that took; add air drag proportional to speed; make the bounce factor a command-line option.

17.19.4 Mini Project 4: Turning a Robot with P and PD Control

Goal. A two-wheeled robot must turn on the spot through 90 degrees. Compare a P controller with PD controllers, looking at overshoot and settling time. **Plan.** The wheels follow their commands with a lag of 0.15 s, like real motors. The turning rate is the difference of the wheel speeds over the track width. The command is *Kp x (target - yaw) - Kd x (turning rate)*, limited to the range -1 to +1. **Skills.** A struct returned from a function, the PID idea from Section 17.16 applied to heading, and reading results critically.

```
 1  // Mini project 4: turn a two-wheeled robot to a heading with P and PD control.
 2  #include <cmath>
 3  #include <cstdio>
 4  #include <algorithm>
 5
 6  struct Result { double overshootPct, settle; };
 7
 8  Result turnTo(double target, double kp, double kd) {
 9    const double track = 0.193, dt = 0.001, tau = 0.15, vmax = 0.40;   // wheel sp
   eed follows its command with a lag of 0.15 s
10    double yaw = 0, vl = 0, vr = 0, peak = 0, settle = 0;
11    for (int i = 0; i < 10000; i++) {
12      double om = (vr - vl) / track;                                    // turning
    rate
13      double u = std::min(1.0, std::max(-1.0, kp * (target - yaw) - kd * om));   /
   / command in -1..1
14      vr += (u * vmax - vr) / tau * dt;  vl += (-u * vmax - vl) / tau * dt;     //
    turn on the spot: wheels opposite
15      yaw += om * dt;
16      peak = std::max(peak, yaw);
17      if (std::fabs(yaw - target) > 0.03 * target) settle = i * dt;     // last ti
   me we were more than 3 % away
18    }
19    return {std::max(0.0, (peak - target) / target * 100), settle};
20  }
21
22  int main() {
23    double target = 3.14159265 / 2;                                     // 90 degr
   ees
24    Result a = turnTo(target, 2.0, 0.0), b = turnTo(target, 2.0, 0.5), c = turnTo(
   target, 2.0, 1.0);
25    std::printf("P  (Kp 2, Kd 0)  : overshoot %5.1f %%, settled within 3 %% after
```

```
    %.2f s\n", a.overshootPct, a.settle);
26    std::printf("PD (Kp 2, Kd 0.5): overshoot %5.1f %%, settled within 3 %% after
    %.2f s\n", b.overshootPct, b.settle);
27    std::printf("PD (Kp 2, Kd 1.0): overshoot %5.1f %%, settled within 3 %% after
    %.2f s\n", c.overshootPct, c.settle);
28    return 0;
29  }
```

Listing from `Guide/code/cpp/mp4_heading.cpp`*, lines 1 to 29.*

```
P  (Kp 2, Kd 0)  : overshoot  12.0 %, settled within 3 % after 0.94 s
PD (Kp 2, Kd 0.5): overshoot   0.0 %, settled within 3 % after 1.22 s
PD (Kp 2, Kd 1.0): overshoot   0.0 %, settled within 3 % after 2.13 s
```

P alone is quick but overshoots by 12 percent, because the wheels lag the command and the robot is still turning when it reaches the target. Adding the D term (`- kd * om`) brakes the turn as it approaches: with Kd = 0.5 the overshoot disappears for a small cost in time, and with Kd = 1.0 the robot is over-damped and takes 2.13 s. Choosing the gains is a trade, and this is exactly the trade Patrick's turn controller makes. **Extend it:** search Kd between 0 and 1 for the fastest settling with no overshoot; add the 0.35 minimum command that Patrick uses to beat motor stiction; try a turn of 180 degrees.

17.19.5 Mini Project 5: A Robot That Drives a Square

Goal. Drive a 1 m square (forward, turn 90 degrees, four times) using only *dead reckoning*: the controller counts time and commanded speed, not the real position. Then give the right wheel a 1 percent error and see how far it misses. **Plan.** A two-state machine (FORWARD, TURN) integrates the true pose from the true wheel speeds, while the controller decides when to switch using only what it *commanded*. **Skills.** State machines, differential-drive geometry, the difference between what a robot believes and what happened.

```
 1  // Mini project 5: drive a 1 m square by dead reckoning, and see how a 1 % wheel
     error ruins it.
 2  #include <cmath>
 3  #include <cstdio>
 4  #include <initializer_list>
 5
 6  struct Pose { double x = 0, z = 0, yaw = 0; };
 7
 8  Pose drive(double rightScale) {
 9    const double track = 0.193, v = 0.3, dt = 0.001, L = 1.0, turnRate = 1.0;
10    Pose p; enum {FORWARD, TURN} state = FORWARD; double travelled = 0, turned = 0
    ; int side = 0;
11    for (int i = 0; i < 100000 && side < 4; i++) {
12      double vl, vr;
13      if (state == FORWARD) { vl = v; vr = v * rightScale; } else { vl = -0.5 * tu
    rnRate * track; vr = 0.5 * turnRate * track * rightScale; }
14      double fwd = 0.5 * (vl + vr), om = (vr - vl) / track;
15      p.yaw += om * dt; p.x += fwd * std::cos(p.yaw) * dt; p.z += fwd * std::sin(p
    .yaw) * dt;
16      // the controller only knows what it COMMANDED, not what happened: it counts
     time and speed, like dead reckoning
17      if (state == FORWARD) { travelled += v * dt; if (travelled >= L) { state = T
    URN; travelled = 0; turned = 0; } }
18      else { turned += turnRate * dt; if (turned >= 3.14159265358979 / 2) { state
    = FORWARD; side++; } }
19    }
20    return p;
21  }
```

```
22
23  int main() {
24    for (double s : {1.0, 1.01}) {
25      Pose p = drive(s);
26      std::printf("right wheel x%.2f: ends at (%.3f, %.3f) m, heading %.1f deg, %.
    3f m from the start\n", s, p.x, p.z, p.yaw * 180 / 3.14159265, std::hypot(p.x, p
    .z));
27    }
28    return 0;
29  }
```

Listing from `Guide/code/cpp/mp5_square.cpp`*, lines 1 to 29.*

```
right wheel x1.00: ends at (0.000, -0.000) m, heading 360.0 deg, 0.001 m from the st
      art
right wheel x1.01: ends at (0.129, -0.102) m, heading 373.7 deg, 0.165 m from the st
      art
```

With perfect wheels the robot returns to its start to within a millimetre. With one wheel a mere 1 percent too fast, it ends 16.5 cm away and 13.7 degrees off. Errors in dead reckoning never cancel; they accumulate. That is why Patrick does not trust his wheels to find the pantry: he *looks*, with the sonar, and brakes when the sensor says he is close enough. **Extend it:** add a sonar-like check that corrects the heading at each corner; make the square 2 m; print the pose at every corner.

Review Problems

1. In mini project 1, what is the smallest change that adds miles (1609.344 m) as a unit?
2. In mini project 2, why does a median of three miss the two-reading glitch? What filter would catch it?
3. In mini project 3, the first bounce is at 0.452 s. Show that this follows from y = 1/2 g t squared.
4. In mini project 4, why does the P-only robot overshoot even though the target is reached at the first crossing?
5. In mini project 5, where does the 13.7 degrees of heading error come from?
6. Design (on paper) a sixth project that combines two of the above. What would it do?

What Comes Next

You now have the whole toolbox: types, decisions, loops, functions, collections, classes, state machines, files, numerical simulation, a PID controller, tests and templates. That is enough to write Patrick's program. In Part V, Chapter 42, we do exactly that: starting from an empty file, we build the whole thing in nine stages, each a program that compiles and runs, from one spinning motor to five ingredients laid out for breakfast with a soft stop. Before you read it, you can prepare by solving Project 6 in Part III (the cart that brakes before a wall) and Advanced C++ in Part IV (stable simulation, and proving a program right with a twin).

Part III. Six Guided Projects

From a first page to a 3D model you can take apart

Chapter 18. Project 1: A Landing Page (HTML and CSS)

Level: beginner. **You will use:** Chapters 3, 4, 9 and 10. **You will finish with:** a responsive one-page site with a header, a hero, a row of cards and a footer.

This is the project where you stop reading and start building. Do not copy and paste the code. Type it. Your fingers learn things your eyes skip.

18.1 Step 1: The Skeleton and the Content

Create a folder called `landing` with an `index.html` and a `style.css`. In `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Your Name | Engineer</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header class="top">
        <a class="brand" href="#">Your Name</a>
        <nav><a href="#work">Work</a> <a href="#contact">Contact</a></nav>
    </header>
    <main>
        <section class="hero">
            <h1>I design things that fly.</h1>
            <p>Aerospace engineering student. I like wind tunnels, robots and good c
      offee.</p>
        </section>
        <section id="work" class="cards">
            <article class="card"><h2>Wing design</h2><p>A small wing, a big lesson.
      </p></article>
            <article class="card"><h2>A robot</h2><p>It picks things up. Mostly.</p>
      </article>
            <article class="card"><h2>A simulation</h2><p>Numbers that move.</p></ar
      ticle>
        </section>
    </main>
    <footer id="contact">&copy; 2026 Your Name</footer>
</body>
</html>
```

> **WHY DO IT THIS WAY? Writing the content before any styling**
>
> **What it is.** You write the HTML first and look at it with no CSS at all.
>
> **Why it is done this way.** This is progressive enhancement (Chapter 3). The unstyled page is your first layer.
>
> **Why it matters.** If the page reads sensibly as plain text (a heading, a paragraph, three cards, a footer), the structure is right. Any problem you find now is a structure problem, which is cheap to fix.
>
> **What happens if you do not.** You style a broken structure, and the CSS becomes a pile of workarounds for HTML mistakes.

Open `index.html` in your browser. It will look like a boring document, and that is exactly right.

18.2 Step 2: Reset and Variables

Start `style.css` with the habits from Chapters 3 and 4.

```
* { box-sizing: border-box; margin: 0; padding: 0; }

:root {
    --bg: #0e1230;
    --text: #e6e8f5;
    --muted: #9aa0c3;
    --accent: #a78bfa;
    --radius: 12px;
}

body {
    background: var(--bg);
    color: var(--text);
    font-family: system-ui, sans-serif;
    line-height: 1.7;
}
a { color: var(--accent); }
```

> **WHY DO IT THIS WAY? Removing the default margins with the reset**
>
> **What it is.** `margin: 0; padding: 0` on everything.
>
> **Why it is done this way.** Browsers give headings, paragraphs and the body their own margins, and the amounts differ slightly between browsers.
>
> **Why it matters.** You start from a blank sheet and add spacing deliberately, so the page looks the same everywhere.
>
> **What happens if you do not.** You fight invisible margins, and something is always 8 pixels off.

18.3 Step 3: Layout

```
.top { display: flex; justify-content: space-between; align-items: center; padding:
      1rem 5%; }
.top nav { display: flex; gap: 1.25rem; }
.brand { font-weight: 700; text-decoration: none; color: var(--text); }

.hero { padding: 5rem 5% 3rem; max-width: 900px; margin: 0 auto; }
.hero h1 { font-size: clamp(2.2rem, 6vw, 4rem); line-height: 1.05; }
.hero p { margin-top: 1rem; color: var(--muted); max-width: 52ch; }

.cards { display: grid; gap: 1rem; padding: 2rem 5%;
         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { background: rgba(255,255,255,0.04); border: 1px solid rgba(255,255,255,0.1);
        border-radius: var(--radius); padding: 1.25rem; }
footer { text-align: center; padding: 2rem; color: var(--muted); }
```

> **WHY DO IT THIS WAY? max-width in ch units on the paragraph**
>
> **What it is.** `max-width: 52ch` limits the paragraph to about 52 characters wide. A `ch` is roughly the width of the digit zero in the current font.

Why it is done this way. Long lines of text are tiring to read, because your eye loses its place going back to the start of the next line.

Why it matters. Between 45 and 75 characters per line is comfortable for most people.

What happens if you do not. On a wide monitor the paragraph runs the full width of the screen and becomes a wall of text.

18.4 Step 4: Hover, Focus and Phone

```
.card { transition: transform 150ms ease, border-color 150ms ease; }
.card:hover { transform: translateY(-3px); border-color: var(--accent); }
a:focus-visible { outline: 2px solid var(--accent); outline-offset: 3px; }

@media (max-width: 600px) {
    .top { flex-direction: column; gap: 0.5rem; }
}
```

Now test. Drag the window narrow: the cards should stack by themselves (the grid rule), and the header should stack at 600 pixels. Press Tab: every link should show a ring.

WHY DO IT THIS WAY? Moving the card with transform

What it is. `transform: translateY(-3px)` lifts the card on hover.

Why it is done this way. As Chapter 2 explained, transform needs no layout.

Why it matters. The hover is smooth and cheap.

What happens if you do not. Using `margin-top: -3px` makes the whole page reflow on every hover, and neighbouring cards jump.

Check Yourself: Project 1

1. What would happen to the card layout if you removed `box-sizing: border-box`?
2. Why is the `<meta name="viewport">` line there?
3. Which rule makes the cards stack on a narrow screen, and why does it not need a media query?
4. Why use `rem` and `clamp` for the heading instead of `px`?
5. Which element is the landmark that a screen reader user would jump to for the main content?

Chapter 19. Project 2: A Filterable Gallery (JavaScript and State)

Level: beginner to intermediate. **You will use:** Chapters 5, 11 and 13. **You will finish with:** a gallery where clicking a category chip filters the cards.

The point of this project is not the gallery. It is to feel the idea from Chapter 5 in your hands: **keep the state in one place and draw the screen from it.**

19.1 Step 1: The Data and the State

Rather than writing the cards into HTML, keep them as data.

```
const items = [
    { id: 1, title: 'Wing profile', tag: 'aero' },
    { id: 2, title: 'Delta wing',   tag: 'aero' },
    { id: 3, title: 'Line follower', tag: 'robots' },
    { id: 4, title: 'Claw gripper',  tag: 'robots' },
    { id: 5, title: 'Orbit plot',    tag: 'sim' },
];
const state = { tag: 'all' };
```

> **WHY DO IT THIS WAY? Starting with data**
>
> **What it is.** An array of objects and a `state` object, before any HTML is touched.
>
> **Why it is done this way.** The cards are a *consequence* of the data and the current filter (Chapter 5).
>
> **Why it matters.** Adding a sixth item means adding one line of data. Nothing else changes.
>
> **What happens if you do not.** You hand-write HTML for each card, and then have to remember to update it in two places when something changes.

19.2 Step 2: The Page

```
<div id="chips">
    <button data-tag="all" class="on">All</button>
    <button data-tag="aero">Aero</button>
    <button data-tag="robots">Robots</button>
    <button data-tag="sim">Simulation</button>
</div>
<div id="grid"></div>
```

19.3 Step 3: One Function That Draws Everything

```
const grid = document.querySelector('#grid');
const chips = document.querySelector('#chips');

function render() {
    const shown = items.filter(i => state.tag === 'all' || i.tag === state.tag);
    grid.replaceChildren(...shown.map(i => {
        const card = document.createElement('article');
        card.className = 'card';
        card.textContent = i.title;                 // text, never innerHTML (Chapte
      r 8)
        return card;
    }));
    chips.querySelectorAll('button').forEach(b => {
        b.classList.toggle('on', b.dataset.tag === state.tag);
    });
}
```

> **WHY DO IT THIS WAY? One render function that redraws the cards**
>
> **What it is.** `render()` rebuilds the grid and the chip highlights from `state`, every time.
>
> **Why it is done this way.** Chapter 5's rule: *screen = render(state)*.
>
> **Why it matters.** There is no "when this chip is clicked, hide cards one and two and show card three" code, and so no way for the screen to disagree with the state.
>
> **What happens if you do not.** Filtering "robots" and then "aero" leaves leftovers from the first filter, because your code only handled "hide" and not "restore".

19.4 Step 4: One Listener on the Parent

```
chips.addEventListener('click', e => {
    const button = e.target.closest('button');
    if (!button) return;
    state.tag = button.dataset.tag;
    render();
});
render();
```

This is **event delegation** (Chapter 5): one listener serves all four chips, and would serve a fifth if you added one later.

Check Yourself: Project 2

1. Which of `items`, `state`, and the cards on screen is the "source of truth"?
2. Why does `render()` use `textContent` and not `innerHTML`?
3. After clicking "Robots", the data says the filter is `robots`. How many cards are shown, and which?
4. What is the benefit of one click listener on `#chips` instead of one per button?
5. What would you change to add a "Reset" chip that shows everything?

Chapter 20. Project 3: An Animated Scroll Story (CSS, IntersectionObserver, GSAP)

Level: intermediate. **You will use:** Chapters 6, 10, 11 and 14. **You will finish with:** a page where sections fade up as you scroll to them, built twice: first by hand, then with GSAP.

Building it twice is the point. The by-hand version teaches you what is happening. The GSAP version shows you what the library saves you from writing.

20.1 Step 1: The Hidden State in CSS

Start with a page of four `<section class="reveal">` blocks, each tall enough to need scrolling. Give them a hidden starting look and a "shown" look.

```
.reveal { opacity: 0; transform: translateY(30px);
          transition: opacity 600ms ease, transform 600ms ease; }
.reveal.shown { opacity: 1; transform: none; }

@media (prefers-reduced-motion: reduce) {
    .reveal { opacity: 1; transform: none; transition: none; }
}
```

> **WHY DO IT THIS WAY? Putting the animation in CSS and the trigger in JavaScript**
>
> **What it is.** CSS describes the two looks and the transition between them. JavaScript only adds the class `shown`.
>
> **Why it is done this way.** This is the three-jobs rule from Chapter 3: looks in CSS, behaviour in JavaScript.
>
> **Why it matters.** A designer can change the timing or the distance without touching a line of script.
>
> **What happens if you do not.** If you animate in JavaScript every frame, you take on the work (and the bugs) the browser already knows how to do.

> **WHY DO IT THIS WAY? The reduced-motion block**
>
> **What it is.** When the visitor asks for less motion, the sections are simply visible, with no transition.
>
> **Why it is done this way.** Chapter 6's rule: honour the request.
>
> **Why it matters.** It keeps the content readable for everyone, including people the animation would make ill.
>
> **What happens if you do not.** Those visitors get an animation they asked not to have.

20.2 Step 2: Noticing When a Section Appears

You could listen for scroll events and measure positions, but there is a better tool. An **IntersectionObserver** tells you when an element enters or leaves the viewport.

```
const observer = new IntersectionObserver((entries) => {
    for (const entry of entries) {
        if (entry.isIntersecting) {
            entry.target.classList.add('shown');
            observer.unobserve(entry.target);     // only the first time
        }
    }
}, { threshold: 0.2 });

document.querySelectorAll('.reveal').forEach(el => observer.observe(el));
```

> **WHY DO IT THIS WAY? IntersectionObserver instead of a scroll listener**
>
> **What it is.** The browser watches the elements for you and calls your function only when something changes.
>
> **Why it is done this way.** A scroll listener runs on every scroll tick (dozens of times a second), and measuring positions inside it forces layout (Chapter 2's layout thrashing).
>
> **Why it matters.** The observer does the measuring off your code's critical path. Your function runs a handful of times in total.
>
> **What happens if you do not.** A page with many scroll-watching elements stutters while you scroll, which is the one time smoothness matters most.

> **WHY DO IT THIS WAY? unobserve after the first time**
>
> **What it is.** Once a section is revealed, stop watching it.
>
> **Why it is done this way.** The reveal is a one-time event.
>
> **Why it matters.** Less work for the observer, and sections do not hide again and flicker if you scroll back.
>
> **What happens if you do not.** The animation repeats every time a section crosses the edge of the screen, which most people find distracting.

20.3 Step 3: The Same Page with GSAP

Load GSAP and ScrollTrigger (Chapter 14) and delete the observer. Replace it with:

```
gsap.registerPlugin(ScrollTrigger);
const mm = gsap.matchMedia();
mm.add('(prefers-reduced-motion: no-preference)', () => {
    gsap.utils.toArray('.reveal').forEach(el => {
        gsap.from(el, { opacity: 0, y: 30, duration: 0.6,
                        scrollTrigger: { trigger: el, start: 'top 80%', once: true }
      });
    });
});
```

(Remove the `.reveal` hidden CSS too, or GSAP's start values will fight it.)

> **WHY DO IT THIS WAY? Comparing the two versions**
>
> **What it is.** The hand-made version is about 15 lines of JavaScript and 8 of CSS. The GSAP version is 8 lines of JavaScript.
>
> **Why it is done this way.** The library's value is not that it can do something you cannot, but that it has already handled the awkward cases (mid-scroll refreshes, resizing, reduced motion, cleanup).
>
> **Why it matters.** Knowing both means you can choose. For a handful of simple fades, the by-hand version is lighter. For a long choreographed sequence, GSAP saves a lot of time.
>
> **What happens if you do not.** You either reinvent a timeline engine for a complicated page, or load a whole library to fade in four paragraphs.

Check Yourself: Project 3

1. Which language describes the "hidden" and "shown" looks, and which one adds the class?
2. What is the advantage of `IntersectionObserver` over a scroll listener?
3. Why does the code call `unobserve` after revealing a section?
4. What does `start: 'top 80%'` mean?
5. For four simple fades, which version would you choose, and why?

Chapter 21. Project 4: An Interactive Physics Chart (Canvas 2D)

Level: intermediate to advanced. **You will use:** Chapters 4, 6, 7 and 13. **You will finish with:** a chart of a damped oscillation with sliders, and a dot that traces the curve in real time.

A damped oscillation is the maths behind a car's suspension, a swinging door, and (on this website) the phugoid and short-period motions of an aircraft. The equation is

$$x(t) = e^{-\zeta \omega_n t} \cdot \cos(\omega_d t), \text{ where } \omega_d = \omega_n \sqrt{(1 - \zeta^2)}$$

Here ω_n is the natural frequency (how fast it would swing with no damping) and ζ ("zeta") is the damping ratio (0 means it swings forever, 1 means it just settles with no swing).

21.1 Step 1: Check the Numbers by Hand First

Worked Example: What should the chart show?

Take $\zeta = 0.5$ and $\omega_n = 2$ rad/s. Predict three things before you write any code.

Step 1. The damped frequency. $\omega_d = 2 \times \sqrt{(1 - 0.25)} = 2 \times 0.8660 = 1.732$ rad/s.

Step 2. The period. $T = 2\pi / \omega_d = 6.2832 / 1.732 = 3.628$ s. That is the time between successive peaks.

Step 3. How fast it dies. The envelope is $e^{-\zeta \omega_n t} = e^{-t}$. After one period, the next peak has fallen to $e^{-3.628} = 0.0266$, about 2.7% of the first.

Step 4. Time to settle. The envelope falls to 2% when $e^{-t} = 0.02$, so $t = -\ln 0.02 = 3.91$ s.

Why do this first? When your chart appears, you will compare it with these numbers. If the second peak is not at 3.63 s and about 0.027 high, your code or your understanding is wrong, and you will know straight away which to suspect.

21.2 Step 2: The Page

```
<canvas id="chart" style="width:100%; height:300px"></canvas>
<label>Damping ζ <input id="zeta" type="range" min="0.02" max="1.5" step="0.01" valu
      e="0.5"></label>
<label>Frequency ωn <input id="wn" type="range" min="0.5" max="6" step="0.1" value="
      2"></label>
```

21.3 Step 3: Computing the Curve

```
function x(t, zeta, wn) {
    if (zeta < 1) {
        const wd = wn * Math.sqrt(1 - zeta * zeta);
        return Math.exp(-zeta * wn * t) * Math.cos(wd * t);
    }
    // zeta >= 1: no oscillation, a simple decay (a good enough picture)
    return Math.exp(-wn * t * (zeta - Math.sqrt(zeta * zeta - 1)));
}
```

WHY DO IT THIS WAY? Keeping the maths in its own function

What it is. `x(t, zeta, wn)` takes numbers and returns a number. It touches no canvas.

Why it is done this way. A pure function (Chapter 11) is easy to test and reuse.

Why it matters. You can check `x(3.628, 0.5, 2)` in the Console against your hand prediction (0.0266) *before* drawing anything. The Patrick and flight dynamics pages do the same: the physics lives in one file, the drawing in another.

What happens if you do not. The maths gets tangled with the drawing, and you cannot tell whether a wrong picture is a maths bug or a drawing bug.

WARNING

The `zeta >= 1` branch above is the *overdamped* case, and the formula shown is a simplification of the slower decaying part, good enough to draw a convincing curve. A full treatment gives a sum of two exponentials. Do not use this simplification for engineering calculations.

21.4 Step 4: Drawing It

```
const canvas = document.querySelector('#chart');
const zIn = document.querySelector('#zeta'), wIn = document.querySelector('#wn');
let t0 = performance.now();

function fit() {                                    // Chapter 4
    const dpr = devicePixelRatio || 1, w = canvas.clientWidth, h = canvas.clientHeig
      ht;
    if (canvas.width !== Math.round(w * dpr)) { canvas.width = Math.round(w * dpr);
      canvas.height = Math.round(h * dpr); }
    const ctx = canvas.getContext('2d');
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.clearRect(0, 0, w, h);
    return { ctx, w, h };
}

function draw(now) {
    const { ctx, w, h } = fit();
    const zeta = +zIn.value, wn = +wIn.value, T = 10;           // show 10 seconds
    const X = t => 40 + t / T * (w - 60);
    const Y = v => h / 2 - v * (h / 2 - 20);                     // v in -1..1, y fl
      ipped
    ctx.strokeStyle = 'rgba(255,255,255,0.2)';
    ctx.beginPath(); ctx.moveTo(40, Y(0)); ctx.lineTo(w - 20, Y(0)); ctx.stroke();
      // zero line
    ctx.strokeStyle = '#60a5fa'; ctx.lineWidth = 2; ctx.beginPath();
    for (let i = 0; i <= 400; i++) {
        const t = i / 400 * T;
        i ? ctx.lineTo(X(t), Y(x(t, zeta, wn))) : ctx.moveTo(X(t), Y(x(t, zeta, wn))
      );
    }
    ctx.stroke();
    const tt = ((now - t0) / 1000) % T;                           // a dot that mov
      es along it
    ctx.fillStyle = '#e879f9';
    ctx.beginPath(); ctx.arc(X(tt), Y(x(tt, zeta, wn)), 5, 0, 7); ctx.fill();
    requestAnimationFrame(draw);
}
requestAnimationFrame(draw);
```

Move the damping slider to 0.02 and you should see a long ringing oscillation. At 1.5 you should see a smooth decay with no swing. At 0.5 and $\omega_n = 2$, the second positive peak should land at 3.63 seconds on the time axis and be nearly invisible.

> **WHY DO IT THIS WAY? Redrawing the whole chart every frame**
>
> **What it is.** `draw` recomputes the curve from the sliders on every frame.
>
> **Why it is done this way.** Chapter 5's *screen = render(state)*. The sliders *are* the state.
>
> **Why it matters.** There is no "slider changed" code. Moving a slider just changes a number that the next frame reads.
>
> **What happens if you do not.** You write update code for every control, and one slider is always forgotten.

Check Yourself: Project 4

1. With $\zeta = 0.1$ and $\omega_n = 2$, what is the damped frequency? What is the period?

2. By what factor does each successive peak shrink for $\zeta = 0.1$ and $\omega_n = 2$?
3. Why do the maths and the drawing live in separate functions?
4. Why is the y mapping `h / 2 - v * (h / 2 - 20)`?
5. What does the dot's position depend on, and why is it frame-rate independent?

Chapter 22. Project 5: A 3D Model You Can Take Apart (three.js)

Level: advanced. **You will use:** Chapters 5, 7, 15 and 16. **You will finish with:** a small 3D model made of parts, with a slider that explodes it, and a panel that describes whatever part you click. This is the idea of the Patrick page in miniature.

22.1 Step 1: Plan the Data First

As in Project 2, the model is described by *data*, then drawn from it.

```
const PARTS = [
    { id: 'base',  name: 'Base plate', desc: 'Everything bolts to this.', size: [1.4
      , 0.06, 0.9], at: [0, 0, 0],    out: [0, 0, 0],    color: '#222' },
    { id: 'motor', name: 'Motor',      desc: 'Turns the wheel.',          size: [0.5
      , 0.2, 0.2],  at: [0, 0.15, 0.4], out: [0, 0, 0.7],  color: '#f2b400' },
    { id: 'arm',   name: 'Arm',        desc: 'Reaches out to grip.',      size: [0.1
      2, 0.8, 0.12], at: [-0.5, 0.45, 0], out: [0, 0.9, 0], color: '#f2b400' },
];
```

Each part has a size, a resting position (`at`), and an **explode offset** (`out`), the direction it moves when the model is taken apart.

> **WHY DO IT THIS WAY? Describing the parts as data**
>
> **What it is.** A list of objects with name, description, size, position, and explode direction.
>
> **Why it is done this way.** The name and description are used by the panel, and the size and position by the 3D view. One list serves both.
>
> **Why it matters.** The part you click and the description you read cannot disagree, because they come from the same record. Adding a part is one more line.
>
> **What happens if you do not.** You make meshes in one place and write descriptions in another, and the two lists drift apart.

22.2 Step 2: The Scene

```
import * as THREE from 'three';

const canvas = document.querySelector('#gl');
const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(36, 1, 0.05, 50);
camera.position.set(2.2, 1.6, 2.6); camera.lookAt(0, 0.3, 0);
scene.add(new THREE.HemisphereLight('#ffffff', '#223', 1.2));
const sun = new THREE.DirectionalLight('#ffffff', 2); sun.position.set(3, 4, 2); sce
      ne.add(sun);

const meshes = [];
for (const p of PARTS) {
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(...p.size),
                                new THREE.MeshStandardMaterial({ color: p.color, rou
      ghness: 0.5 }));
    mesh.userData = p;                          // remember which part this mesh is
    mesh.position.set(...p.at);
    scene.add(mesh); meshes.push(mesh);
}
```

> **WHY DO IT THIS WAY? Storing the part record on the mesh**
>
> **What it is.** `mesh.userData = p`.
>
> **Why it is done this way.** When the visitor clicks, the raycaster gives you back a *mesh*. You need to know which part that is.
>
> **Why it matters.** `hit.object.userData.desc` is the description, with no lookup table.
>
> **What happens if you do not.** You search the list by position or colour, which breaks the moment two parts are alike.

22.3 Step 3: Explode and Resize

```
let explode = 0;                                // 0 together, 1 fully apart
function layout() {
    meshes.forEach(m => {
        const p = m.userData;
        m.position.set(p.at[0] + p.out[0] * explode, p.at[1] + p.out[1] * explode, p
      .at[2] + p.out[2] * explode);
    });
}
document.querySelector('#explode').addEventListener('input', e => { explode = +e.tar
      get.value; });

function resize() {
    const w = canvas.clientWidth, h = canvas.clientHeight;
    renderer.setSize(w, h, false); camera.aspect = w / h; camera.updateProjectionMat
      rix();
}
addEventListener('resize', resize); resize();
```

> **WHY DO IT THIS WAY? Computing every position from one number**

What it is. The position of each part is `at + out × explode`, recomputed from the single value `explode`.

Why it is done this way. Chapter 5 again: one source of truth.

Why it matters. The slider, a "Take apart" button and an automatic tour can all just set `explode`, and the model follows. This is exactly how the Patrick page works, with one number driving the whole teardown.

What happens if you do not. Every control has to move each mesh by hand, and a part always gets left behind.

22.4 Step 4: Picking and the Panel

```
const ray = new THREE.Raycaster(), ndc = new THREE.Vector2();
canvas.addEventListener('click', e => {
    const r = canvas.getBoundingClientRect();
    ndc.set(((e.clientX - r.left) / r.width) * 2 - 1, -((e.clientY - r.top) / r.heig
      ht) * 2 + 1);
    ray.setFromCamera(ndc, camera);
    const hit = ray.intersectObjects(meshes)[0];
    const panel = document.querySelector('#panel');
    panel.textContent = hit ? hit.object.userData.name + ': ' + hit.object.userData.
      desc : 'Click a part.';
});

(function loop() { layout(); renderer.render(scene, camera); requestAnimationFrame(l
      oop); })();
```

22.5 Step 5: Bundle and Run

With `three` installed (Chapter 16) run `npx esbuild main.js --bundle --minify --format=iife --outfile=app.js`, add `<script src="app.js">` to a page that has a `<canvas id="gl">`, an `<input id="explode" type="range" min="0" max="1" step="0.01" value="0">` and a `<p id="panel">`. Open it, drag the slider, and click the parts.

TIP

If the screen is black, check in order: is there a light? Is the camera looking at the model? Is the canvas given a size in CSS (a canvas with no height shows nothing)? Is the render call inside the loop?

Check Yourself: Project 5

1. Why does each mesh carry its part record in `userData`?
2. What single value drives the whole teardown, and why is that good?
3. What two conversions does the click handler make before using the raycaster?
4. If the canvas looks stretched after resizing, which line did you forget?
5. How would you add a fourth part?

Chapter 23. Project 6: A Cart That Brakes Before the Wall (C++)

This is the smallest possible version of Patrick: a cart with one motor that drives towards a wall and must stop before it hits. It has the same three ingredients as the real robot: a **motor model** (voltage in, push out), a **time-stepped simulation**, and a **state machine** (DRIVE, BRAKE, DONE). By the end you will also add the soft stop that Patrick's PAUSE button uses. You need the compiler from Chapter 17.

23.1 Step 1: Check the Numbers by Hand First

Before writing code, work out what the answer should roughly be, so you can tell a bug from a result. The cart has mass 0.6 kg and wheels of radius 0.0325 m. The motor has a winding resistance of 6 ohms and a constant of 0.2326 (volts per radian per second, which is also newton-metres per amp). At full duty the motor sees 4.6 volts; at 70 percent duty, 3.22 volts.

At top speed the motor's own back-voltage cancels the supply (almost), so the current is small. Ignoring friction, the top wheel speed is about 3.22 / 0.2326 = 13.8 rad/s, and the cart speed is that times the wheel radius: 13.8 x 0.0325 = 0.45 m/s. So we expect a top speed a little under **0.45 m/s**.

> **WHY DO IT THIS WAY? Estimating the answer before running the code**
>
> **What it is.** A quick hand calculation of the result.
>
> **Why it is done this way.** A program that prints a number gives you no way to know whether the number is right.
>
> **Why it matters.** When the finished program prints 0.450 m/s, you know it agrees with the physics. If it had printed 45 or 0.045, you would know at once something was wrong with the units.
>
> **What happens if you do not.** You trust whatever the program prints, and ship a robot whose numbers are off by a factor of ten.

23.2 Step 2: The Constants and the Cart

```
#include <algorithm>
#include <cmath>
#include <cstdio>

const double M = 0.6;          // mass, kg
const double RW = 0.0325;      // wheel radius, m
const double KE = 0.2326;      // motor constant
const double RM = 6.0;         // winding resistance, ohm
const double ETA = 0.55;       // gearbox efficiency
const double VBAT = 4.6;       // volts reaching the motor at full duty
const double WALL = 1.0;       // the wall is at x = 1 m
const double STOP_AT = 0.20;   // brake when the wall is this close, m

double clampd(double x, double a, double b) { return std::min(b, std::max(a, x)); }

enum State { DRIVE, BRAKE, DONE };

struct Cart {
  double x = 0, v = 0, t = 0;
  State state = DRIVE;
  double step(double duty, double dt) {
    double w = v / RW;                                    // wheel speed, rad/s
    double volts = (state == DRIVE) ? duty * VBAT : 0.0;     // BRAKE: 0 V, so the m
      otor drags
    double current = (volts - KE * w) / RM;
    double force = ETA * KE * current / RW;                  // motor torque as push
      at the ground
    v += force / M * dt;
    x += v * dt;
    t += dt;
    return volts;
  }
};
```

The motor equation is one line: **current = (volts - back-voltage) / resistance**, where the back-voltage is the constant times the wheel speed. A spinning motor is also a generator, and pushes back. That one fact is why the cart reaches a top speed, and why it brakes when the supply is cut.

> **WHY DO IT THIS WAY? Braking by setting the voltage to zero**
>
> **What it is.** In the BRAKE state the code sets `volts` to 0 rather than reversing the motor.
>
> **Why it is done this way.** With 0 V across it, a spinning motor's own back-voltage drives a current through its own winding, and that current produces a torque *against* the motion. This is an electrical brake, and it costs no energy from the battery.
>
> **Why it matters.** It is smooth, gentle on the gearbox, and cannot accidentally drive the cart backwards. On the real robot it is done by shorting the motor terminals through the H-bridge.
>
> **What happens if you do not.** If you simply switched the motor off (disconnected it), nothing would oppose the motion and the cart would coast much further, straight into the wall.

23.3 Step 3: The Loop and the State Machine

```
int main() {
  Cart c;
  const double dt = 0.001, duty = 0.7;
  double vmax = 0;
  for (int i = 0; i < 20000 && c.state != DONE; i++) {
    double ramp = clampd(c.t / 0.35, 0, 1);            // soft start over 0.35 s
    c.step(duty * ramp, dt);
    vmax = std::max(vmax, c.v);
    if (c.state == DRIVE && WALL - c.x <= STOP_AT) {
      c.state = BRAKE;
      std::printf("brake at t=%.2f s, x=%.3f m, v=%.3f m/s\n", c.t, c.x, c.v);
    }
    if (c.state == BRAKE && std::fabs(c.v) < 0.005) c.state = DONE;
    if (i % 500 == 0) std::printf("t=%.2f  x=%.3f  v=%.3f\n", c.t, c.x, c.v);
  }
  std::printf("stopped at x=%.3f m (%.1f cm from the wall) after %.2f s; top speed %
      .3f m/s\n",
              c.x, (WALL - c.x) * 100, c.t, vmax);
  return 0;
}
```

Compile with `g++ -std=c++17 -Wall -Wextra -O2 cart.cpp -o cart` and run it. You should see:

```
t=0.00  x=0.000  v=0.000
t=0.50  x=0.095  v=0.403
t=1.00  x=0.314  v=0.449
t=1.50  x=0.539  v=0.450
t=2.00  x=0.764  v=0.450
brake at t=2.08 s, x=0.800 m, v=0.450 m/s
t=2.50  x=0.855  v=0.017
stopped at x=0.857 m (14.3 cm from the wall) after 2.65 s; top speed 0.450 m/s
```

The top speed, 0.450 m/s, matches the hand estimate from Step 1. The cart braked when the wall was 20 cm away, coasted another 5.7 cm while the electrical brake slowed it, and stopped 14.3 cm short.

> **WHY DO IT THIS WAY? A soft start (the ramp)**
>
> **What it is.** `ramp` rises from 0 to 1 over the first 0.35 seconds and multiplies the duty.
>
> **Why it is done this way.** Full voltage applied at once makes the wheels spin on the table.
>
> **Why it matters.** Patrick's tyres have a limited grip. A gentle start keeps the wheels rolling instead of slipping, and spares the gears a jolt. It is the same idea Patrick's `ramp(0.35)` uses in every driving state.
>
> **What happens if you do not.** The first instant of every drive is a lurch, and the robot's position after each move becomes less repeatable.

23.4 Step 4: Plot It

Make the program write a table, then look at it. Replace the `printf` inside the loop with one that prints `t,x,v` on each line, run `./cart > cart.csv`, and open the file in a spreadsheet. Plot `v` against `t`. You should see the speed rise smoothly to 0.45, stay there, and fall away to zero after the brake point.

> **TIP**
>
> A picture finds bugs that a column of numbers hides. If the speed curve has a sharp corner where you expected a smooth curve, you have found a place where your model is not what you meant.

23.5 Step 5: The Soft Stop (Pause)

Patrick's PAUSE button does not stop him instantly. It slides a number called the **go-gain** from 1 down to 0 over 0.8 seconds, and multiplies every command by it. Add the same to the cart. Near the top of `main`, before the loop, add `double g = 1, goal = 1;`. Inside the loop, before `c.step`, add:

```
if (c.t > 1.0) goal = 0;                                  // pause at t = 1 s
double rate = 1 / 0.8;                                    // 0.8 s from 1 to 0
g += clampd(goal - g, -rate * dt, rate * dt);
double ge = g * g * (3 - 2 * g);                          // eased, so it starts and
      ends gently
c.step(duty * ramp * ge, dt);
```

(and replace the old `c.step(...)` line). Run it and print the speed as it goes. When I ran it, the speed was 0.449 m/s at t = 1.0 s (the moment of the pause), 0.239 at 1.5 s, 0.010 at 2.0 s and 0 after that: a smooth S-shaped fall to zero instead of a drop off a cliff. The cart rolled from x = 0.314 m to a halt at x = 0.551 m, about 24 cm further on, and stayed there.

> **WHY DO IT THIS WAY? Easing the gain, not just ramping it**
>
> **What it is.** `g * g * (3 - 2 * g)` is a smooth S-curve that goes from 0 to 1 (and back) with zero slope at both ends.
>
> **Why it is done this way.** A straight-line ramp has a sharp corner where it begins and ends, and a corner in a command is a jolt in the motion.
>
> **Why it matters.** On the page, the robot visibly eases to a halt when you press PAUSE: its speed fell from 28.5 cm/s to zero over about 0.65 seconds, with no sudden change anywhere.
>
> **What happens if you do not.** The pause looks like the robot hit an invisible wall.

Check Yourself: Project 6

1. Why is the top speed about 0.45 m/s? What stops the motor from speeding up forever?
2. What does setting `volts` to 0 do to a spinning motor, and why is that a brake?
3. Why does the cart stop 14.3 cm from the wall, not exactly at the wall?
4. What two things would you change to make it stop closer to the wall?
5. In Step 5, why is the gain multiplied into the *command* and not applied by changing the speed directly?

Part IV. Advanced Topics

Going deeper into each tool, once the basics feel comfortable

Chapter 24. Advanced CSS

You can build a lot with the CSS of Chapter 10. This chapter covers the tools that make a *large* stylesheet stay manageable, and that make a design respond to more than the width of the screen.

24.1 Cascade Layers

Specificity wars (Chapter 3) get worse as a project grows. **Cascade layers** let you declare, up front, which *groups* of rules beat which, regardless of specificity.

```
@layer reset, base, components, overrides;

@layer reset { * { margin: 0; box-sizing: border-box; } }
@layer base { body { font-family: system-ui, sans-serif; } }
@layer components { .card { padding: 1rem; } }
@layer overrides { .card { padding: 2rem; } }
```

> **WHY DO IT THIS WAY? Declaring the layer order first**
>
> **What it is.** The first line lists the layers from weakest to strongest. A rule in a later layer wins over one in an earlier layer, whatever the specificity.
>
> **Why it is done this way.** You decide who outranks whom once, in one place.
>
> **Why it matters.** A plain `.card` in `overrides` beats an ID selector in `base`. You stop needing `!important` and long selectors.
>
> **What happens if you do not.** As the stylesheet grows, you add ever more specific selectors to win arguments, until nobody dares touch anything.

24.2 Container Queries

A media query asks about the *screen*. A **container query** asks about the *space the component actually has*, which is what a reusable component really cares about.

```
.card-wrap { container-type: inline-size; }
@container (min-width: 420px) {
    .card { display: grid; grid-template-columns: 120px 1fr; gap: 1rem; }
}
```

> **WHY DO IT THIS WAY? Asking the container, not the screen**
>
> **What it is.** The card switches to a side-by-side layout when *its own container* is at least 420 pixels wide.
>
> **Why it is done this way.** The same card might sit in a wide main column in one place and a narrow sidebar in another.
>
> **Why it matters.** The card behaves correctly wherever you put it.
>
> **What happens if you do not.** A screen-width query makes the card go wide in the sidebar just because the screen is wide, and it overflows.

24.3 Logical Properties, :has() and Custom Property Tricks

Logical properties (`margin-inline`, `padding-block`) are written in terms of text direction, not left and right, so layouts flip correctly for right-to-left languages. The `:has()` selector styles a parent based on its children (`.card:has(img)`). Custom properties can be changed from JavaScript, which is a clean bridge between the two languages.

```
panel.style.setProperty('--progress', 0.6);        /* JavaScript sets the number */
.bar { transform: scaleX(var(--progress)); }       /* CSS decides what it looks like
      */
```

> **WHY DO IT THIS WAY? Passing a number through a custom property**
>
> **What it is.** JavaScript sets `--progress`; CSS uses it.
>
> **Why it is done this way.** The script reports the *fact*, the stylesheet owns the looks (Chapter 3).
>
> **Why it matters.** A designer can change what "60 percent" looks like without touching the script.
>
> **What happens if you do not.** The script sets pixel widths directly, so changing the design means editing JavaScript.

Check Yourself: Advanced CSS

1. What does `@layer a, b, c;` establish?
2. When is a container query better than a media query?
3. Why is `margin-inline` preferable to `margin-left` for an international site?

Chapter 25. Advanced JavaScript

25.1 Modules, Bundling and Dynamic Import

Chapter 11 showed `import` and `export`. A big page can also load code *only when it is needed* with **dynamic import**.

```
button.addEventListener('click', async () => {
    const { openViewer } = await import('./viewer.js');     // loaded now, not at pa
      ge start
    openViewer();
});
```

> **WHY DO IT THIS WAY? Loading heavy code on demand**
>
> **What it is.** The 3D viewer's code is downloaded only when the visitor asks for it.
>
> **Why it is done this way.** Most visitors never open the viewer.
>
> **Why it matters.** The first load is smaller and faster (Chapter 1's cost-of-a-request arithmetic).
>
> **What happens if you do not.** Everyone pays for a heavy library that only a few use.

25.2 Closures, `this` and Pitfalls

Closures (Chapter 11) are powerful and have one classic trap: a closure captures the *variable*, not its value at that moment.

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);   // prints 3, 3, 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);   // prints 0, 1, 2
```

> **WHY DO IT THIS WAY? let in loops**
>
> **What it is.** `let` makes a fresh `i` for each trip round the loop. `var` shares a single `i` across all of them.
>
> **Why it is done this way.** `let` has block scope.
>
> **Why it matters.** Each callback closes over its own `i`.
>
> **What happens if you do not.** All three timers see the final value, 3, and the bug is baffling the first time.

25.3 Web Workers

The main thread (Chapter 2) must stay free for drawing. A **Web Worker** runs a script on a *separate* thread, so heavy calculations do not freeze the page.

```
const worker = new Worker('solver.js');
worker.postMessage({ n: 1e7 });
worker.onmessage = e => console.log('result', e.data);
```

> **WHY DO IT THIS WAY? Offloading heavy sums to a worker**

What it is. The expensive calculation happens on another thread. Results come back as messages.

Why it is done this way. A long task on the main thread freezes input and animation (Chapter 2).

Why it matters. The page stays smooth while the worker crunches numbers (a simulation, an image filter).

What happens if you do not. A half-second calculation freezes the page for half a second, every time.

TECHNICAL STUFF: Technical Stuff

Workers cannot touch the DOM. They only exchange copies of data with the page through messages, which is why they are safe: there is no shared variable to corrupt.

Check Yourself: Advanced JavaScript

1. What does dynamic `import()` do that a normal `import` does not?
2. What do the two loops above print, and why do they differ?
3. Why can a worker not change the page directly?

Chapter 26. Advanced React

26.1 Effects, Cleanup and Refs

You met `useEffect` in Chapter 12. The advanced rule is that effects should be **honest about their dependencies**, and should always clean up.

A **ref** (`useRef`) holds a value that survives renders *without* causing one, which is perfect for things outside React's world: a canvas, a timer id, a 3D renderer.

```
function Viewer() {
    const canvasRef = React.useRef(null);
    React.useEffect(() => {
        const renderer = new THREE.WebGLRenderer({ canvas: canvasRef.current });
        let raf = requestAnimationFrame(function loop() {
            renderer.render(scene, camera);
            raf = requestAnimationFrame(loop);
        });
        return () => { cancelAnimationFrame(raf); renderer.dispose(); };
    }, []);
    return <canvas ref={canvasRef} />;
}
```

> **WHY DO IT THIS WAY? Starting the loop in an effect and stopping it on cleanup**
>
> **What it is.** The animation loop and renderer are created after the canvas exists, and torn down when the component leaves.
>
> **Why it is done this way.** The canvas does not exist until React has rendered. And a running loop outlives the component unless you stop it.
>
> **Why it matters.** Memory and GPU resources are released.
>
> **What happens if you do not.** Every time the component appears, another loop and renderer pile up, and the page slows to a crawl.

26.2 Avoiding Wasted Renders

React re-renders a component whenever its state or props change, and by default also re-renders its children. `React.memo` skips a child whose props have not changed, and `useMemo` and `useCallback` keep a value or function the same between renders. The book's React examples (`Guide/code/react/examples.jsx`) measured both:

```
7: parent rendered 3 times, memoised child rendered 1 time(s)
9: total 6 computed 1 time(s) over 2 renders
```

A `memo` child with a fixed `label` prop rendered once while its parent rendered three times; a `useMemo` sum with an unchanged `items` dependency was computed once across two renders. Both are real savings, and both are invisible until a render is slow enough to notice.

> **WHY DO IT THIS WAY? Measuring before memoising**
>
> **What it is.** Wrapping something in `memo` only after you have *seen* a slow render in the profiler.

> **Why it is done this way.** Memoising has a cost of its own, and most renders are cheap.
>
> **Why it matters.** The code stays simple where it can.
>
> **What happens if you do not.** You scatter `useMemo` everywhere "just in case", and it makes the code harder to read without making it faster (Chapter 2's rule: measure, do not guess).

26.3 Custom Hooks and Context

When two components share the same logic, move it into a **custom hook**, a function whose name starts with `use` that calls other hooks. **Context** lets a value be read by any descendant without passing props through every level.

```
const Theme = createContext('light');
function Deep() { return <em>{useContext(Theme)}</em>; }
function Middle() { return <section><Deep /></section>; }
// <Theme.Provider value="dark"><Middle /></Theme.Provider>  renders  <section><em>d
      ark</em></section>
```

`Middle` passes nothing down, yet `Deep` reads `dark`: the provider makes the value available to the whole subtree. Use it for things that genuinely belong to the whole tree (the theme, the signed-in user, the current units), not as a way to avoid thinking about where state should live.

> **WHY DO IT THIS WAY? Custom hooks for repeated logic**
>
> **What it is.** `useWindowWidth()` wraps the resize effect from Chapter 12's exercises.
>
> **Why it is done this way.** Logic, like data, should live in one place.
>
> **Why it matters.** Fix a bug once and every component benefits.
>
> **What happens if you do not.** The same effect is copied into six components, and you fix it in five.

Check Yourself: Advanced React

1. Why does an effect that starts a loop need to return a cleanup function?
2. When is `useRef` the right tool?
3. Why is it wrong to wrap everything in `useMemo`?

Chapter 27. Advanced Canvas

27.1 Layers and Offscreen Canvases

A chart often has a **static** part (axes, grid, labels) and a **moving** part (curves, markers). Redrawing the static part every frame wastes time. Draw it once to an offscreen canvas, then copy it in with `drawImage`.

```
const bg = document.createElement('canvas');     // an offscreen canvas, not in the
      page
bg.width = w * dpr; bg.height = h * dpr;
drawAxes(bg.getContext('2d'));                   // done once, or when the size chan
      ges

function frame() {
    ctx.drawImage(bg, 0, 0, w, h);               // stamp the axes in one fast call
    drawCurves(ctx);                             // only the moving part is redrawn
}
```

> **WHY DO IT THIS WAY? Caching the static drawing**
>
> **What it is.** The grid and axes are drawn once to a hidden canvas and stamped onto the visible one each frame.
>
> **Why it is done this way.** Copying a finished bitmap is much cheaper than recomputing hundreds of lines and labels.
>
> **Why it matters.** The cylinder-flow panels on this website do exactly this: the colour field is built once per Mach number and only the moving particles are redrawn each frame.
>
> **What happens if you do not.** You recompute things that have not changed, 60 times a second, and the frame budget disappears.

27.2 Only Draw What the Visitor Can See

An animation on a canvas that is scrolled off-screen is pure waste. An **IntersectionObserver** (Project 3) tells you whether a canvas is visible, so you can pause its loop.

> **WHY DO IT THIS WAY? Pausing off-screen animations**
>
> **What it is.** A visible flag set by an observer. The loop skips drawing while it is false.
>
> **Why it is done this way.** A page with ten animated panels only ever shows one or two at a time.
>
> **Why it matters.** Battery life, and a smooth frame rate for the panels you *can* see.
>
> **What happens if you do not.** Ten animations run at once, the laptop's fan spins up, and every panel stutters.

27.3 Avoiding Blurry Lines

A 1-pixel line centred on a whole-number coordinate is drawn across *two* device pixels at half brightness, and looks blurry. Offsetting by 0.5 puts it on one pixel.

Check Yourself: Advanced canvas

1. What belongs on an offscreen canvas?
2. Why should an off-screen animation be paused?
3. What causes a crisp-looking 1-pixel line to appear blurry?

Chapter 28. Advanced GSAP

28.1 Timelines as State Machines

A timeline can be **labelled**, **paused**, **reversed** and **seeked**. That makes it a good fit for a set of named states: closed, open, expanded.

```
const tl = gsap.timeline({ paused: true });
tl.addLabel('open')
  .to('.menu', { height: 'auto', duration: 0.4 })
  .addLabel('items')
  .from('.menu a', { opacity: 0, y: 10, stagger: 0.05 });

openBtn.onclick  = () => tl.play();
closeBtn.onclick = () => tl.reverse();
```

> **WHY DO IT THIS WAY? Reversing a timeline instead of writing a closing animation**
>
> **What it is.** `tl.reverse()` plays the same animation backwards.
>
> **Why it is done this way.** A close is usually the mirror image of an open.
>
> **Why it matters.** Half the code, and the two can never get out of sync.
>
> **What happens if you do not.** You write two animations and have to remember to update both when the design changes.

28.2 Scrubbed Scroll Animations

With `scrub`, an animation's progress follows the scrollbar. `pin: true` holds an element in place while the scrolling plays out. The advanced skill is knowing *when not to*: pinned, scrubbed sequences feel wonderful on a desktop and can be awkward on phones, so wrap them in `gsap.matchMedia()` and give small or reduced-motion screens a simpler version.

> **WHY DO IT THIS WAY? Giving small screens a simpler version**
>
> **What it is.** A scrub-and-pin animation only for wide screens with no motion restriction.
>
> **Why it is done this way.** Phones have little room, touch scrolling is inertial, and some visitors want less motion.
>
> **Why it matters.** The experience works for everyone, and nobody gets stuck in a pinned section they cannot scroll past.
>
> **What happens if you do not.** A beautiful desktop effect becomes a frustrating trap on a phone.

28.3 Cleaning Up

Animations hold references. When a page or component goes away, kill them: `tl.kill()` and `ScrollTrigger.getAll().forEach(t => t.kill())`, or use `gsap.context()` to group and revert everything at once.

Check Yourself: Advanced GSAP

1. What is the advantage of `tl.reverse()` over a separate closing animation?
2. Why wrap a pinned scroll animation in `matchMedia`?
3. Why must animations be killed when a component is removed?

Chapter 29. Advanced three.js

29.1 Physically Based Materials and Studios

Chapter 15 introduced the standard material. For realism you add **clearcoat** (a glossy lacquer layer), **bump maps** (fine surface texture without extra geometry), **sheen** (soft fabric-like edges) and a considered **environment** that supplies reflections. The Patrick robot uses clearcoat on its yellow plastic and acrylic plates, a grain bump map on its moulded parts, and a layered "printed" bump on its 3D-printed claw.

> **WHY DO IT THIS WAY? A bump map instead of real geometry**
>
> **What it is.** A grayscale image that fakes tiny height differences in the lighting calculation.
>
> **Why it is done this way.** Geometry costs triangles. A bump map costs one texture.
>
> **Why it matters.** Fine layer lines on a printed part, or the grain of moulded plastic, add realism for almost no cost.
>
> **What happens if you do not.** Plastic looks like smooth digital clay, the classic "computer-generated" giveaway.

29.2 Post-processing: Ambient Occlusion and Tone Mapping

Rendering into an intermediate image lets you add effects before the final picture. **Ambient occlusion** darkens creases and contact points. **Tone mapping** compresses brightness to the screen's range. Both run on every pixel, so they are costly.

> **WHY DO IT THIS WAY? Quality tiers**
>
> **What it is.** The robot page has three tiers (full effects, lighter, and none), and moves down a tier if its frame rate stays low.
>
> **Why it is done this way.** Visitors have wildly different hardware (Chapter 8).
>
> **Why it matters.** A phone gets a smooth, simpler picture and a desktop gets the full one.
>
> **What happens if you do not.** The page that looks gorgeous on your computer runs at five frames per second on a visitor's laptop.

29.3 Instancing and Merging

When you need many copies of something (screws, a field of particles), use an `InstancedMesh`: one geometry, many positions, **one draw call**. When you need many *different* small parts that share a material, merge their geometries into one mesh with `mergeGeometries` (Chapter 7).

29.4 Disposal and Warm-up

Call `dispose()` on what you stop using (Chapter 15). Also consider **warming up**: compiling shader programs before the first frame (`renderer.compileAsync`), so the page does not freeze for a second while the graphics driver prepares them.

> **WHY DO IT THIS WAY? Compiling shaders before the first frame**
>
> **What it is.** `renderer.compileAsync(scene, camera)` asks the driver to prepare every shader program now, in the background.
>
> **Why it is done this way.** Otherwise the first time something new appears, the driver compiles its shader *during* a frame.
>
> **Why it matters.** There is no stutter the first time you open a panel or turn on an effect.
>
> **What happens if you do not.** The page freezes for a noticeable moment the first time a new material is drawn.

Check Yourself: Advanced three.js

1. What does a bump map add, and at what cost?
2. Why have quality tiers?
3. What is the difference between instancing and merging?

Chapter 30. Advanced Accessibility and Performance

30.1 Auditing Seriously

Automated tools find about a third of accessibility problems (Chapter 8). A professional audit adds: keyboard-only navigation of every flow; a screen reader (NVDA or VoiceOver) on the main journeys; zoom to 200% and 400%; windows high-contrast mode; and testing with real users when possible.

> **WHY DO IT THIS WAY? Testing with a keyboard and a screen reader**
>
> **What it is.** Using your page without a mouse or without looking.
>
> **Why it is done this way.** Automated checkers can say a label exists, but not whether it *makes sense*.
>
> **Why it matters.** You find out that "Button 1 of 14" is not helpful, and that the tab order jumps around.
>
> **What happens if you do not.** The page passes a checker and is still unusable for some people.

30.2 Performance Budgets

A **performance budget** is a number you promise not to exceed: for example 300 KB of JavaScript on first load, or LCP under 2.5 seconds. Write it down, measure against it on every change, and treat going over it as a bug.

> **WHY DO IT THIS WAY? Writing a budget down**
>
> **What it is.** A target, tested continuously.
>
> **Why it is done this way.** Pages get slower by tiny steps, none of which seems important.
>
> **Why it matters.** A budget turns "it feels slower lately" into a number you can point to.
>
> **What happens if you do not.** After a year of small additions, the page takes eight seconds to load and nobody can say when it went wrong.

30.3 Measure on Real, Modest Hardware

Your development computer is probably the fastest device that will ever open the page. Test with the developer tools throttling the CPU (4x or 6x slower) and the network ("Fast 3G"), and, if you can, on an actual older phone.

Check Yourself: Advanced accessibility and performance

1. Name two things a manual audit finds that an automated checker cannot.
2. What is a performance budget and why is it written down?
3. Why test with CPU throttling?

Chapter 31. Advanced C++

You can already read the robot's program. This chapter is about the three things that made it *trustworthy*: a simulation that stays stable, a second program that proves the first one right, and a few habits that keep C++ honest.

31.1 When a Time Step Is Too Big

Chapter 17 said a smaller time step gives a more accurate simulation. There is a harsher truth: a step that is too big does not just give a *worse* answer. It can give an answer that explodes.

Think of a tyre as a very stiff spring pushing on a light wheel. The wheel's natural speed of vibration is the square root of (stiffness over mass). The simple update used in the book (change the speed, then change the position) is only stable if the time step times that natural frequency is less than 2.

```
const double k = 4.0e6, m = 0.4;      // sqrt(k/m) = 3162 per second
const double dt = 0.001;              // dt * 3162 = 3.2: above the limit of 2
for (int sub : {1, 4}) {
  double x = 0.001, v = 0.0, h = dt / sub;
  for (int i = 0; i < 100; i++)               // 100 ticks = 0.1 s
    for (int s = 0; s < sub; s++) { v += -k * x / m * h; x += v * h; }
  std::printf("sub-steps per tick = %d: |x| after 0.1 s = %.6g m\n", sub, std::fabs(
      x));
}
```

(You also need `#include <cmath>`, `<cstdio>` and `<initializer_list>`.) Running it prints:

```
sub-steps per tick = 1: |x| after 0.1 s = 4.70899e+86 m
sub-steps per tick = 4: |x| after 0.1 s = 0.000377145 m
```

With one step per tick, a wheel that started 1 millimetre from rest is, a tenth of a second later, 10 to the power 86 metres away: nonsense. With four steps per tick (each a quarter of a millisecond, so step times frequency is 0.79, well under 2) it is a perfectly sensible fraction of a millimetre.

> **WHY DO IT THIS WAY? Sub-stepping the wheels**
>
> **What it is.** In `sideStep`, the wheel and the robot's speed are advanced in four small steps for every one tick of the main loop.
>
> **Why it is done this way.** The tyre's grip changes extremely fast near zero slip, which behaves like that very stiff spring. Four sub-steps keep the step times the frequency under the limit, without making the whole simulation four times slower: only the wheel equations use them.
>
> **Why it matters.** This was a real bug in the robot's first version. At low speed the wheel chattered, and worse, tiny rounding differences between two programs grew into large differences, so the two could never agree. Sub-stepping fixed both the chatter and the disagreement.
>
> **What happens if you do not.** The simulation looks fine at speed and falls apart at the exact moments you most care about: starting and stopping.

> **TECHNICAL STUFF: Technical Stuff**

> The limit "step times frequency is less than 2" comes from the maths of this kind of update, which is called symplectic (or semi-implicit) Euler. A fancier method such as Runge-Kutta allows a somewhat larger step, but sub-stepping the one stiff part is simpler, cheaper, and easier to keep identical in two languages.

31.2 A Twin to Prove It Right

How do you know a physics program is correct? You cannot look at it and see. One of the best checks is to write the same program a *second* time, differently, and compare. If both agree on thousands of numbers, a mistake in one would have to be repeated, by accident, in the other.

For Patrick, the second program is the web page's JavaScript. Both write a table: one row every 10 ms, the same 13 columns (time, position, heading, servo angle, and so on). Then you compare the two tables row by row.

```
./patrick --out cpp.csv                  # the C++ run
node dump.mjs js.csv                     # the JavaScript run, same columns
tr -d '\r' < cpp.csv | cmp - js.csv      # byte-for-byte comparison
```

When it was run for this book, the two files were identical, with the single exception of Windows line endings (C++ writes a carriage return before each newline on Windows; JavaScript does not), which is why the `tr -d '\r'` is there. It was repeated for a run with a pause and a resume in the middle, which was identical too.

> **WHY DO IT THIS WAY? Comparing two programs row by row**
>
> **What it is.** Run the C++ and the JavaScript, write both to a table, compare the tables.
>
> **Why it is done this way.** A table of 9,776 rows by 13 columns checks 127,000 numbers at once, including every state change and every servo angle. No person could check that by eye.
>
> **Why it matters.** The page shows the JavaScript. The download is the C++. Because they agree exactly, what the visitor downloads is truly what they watched.
>
> **What happens if you do not.** The code you publish and the animation you show slowly drift apart, and nobody notices until someone downloads the file and finds it does something different.

> **WARNING**
>
> The comparison only works if both programs do their floating-point arithmetic in the same order. A compiler flag named `-ffast-math` lets the compiler reorder it for speed, which changes the last digits and ruins an exact comparison. The robot's build does not use it. If you ever want exact agreement between two programs, avoid that flag.

31.3 Habits That Keep C++ Honest

Make everything `const` that can be. A `const` promise is checked by the compiler. If you write `const double g = 9.80665;` and later try `g = 3;`, it refuses. Patrick's constants (`STOP_AT`, `RETREAT`, the lane positions) are all `const`, so nothing can change a tuned value by accident.

Pass big things by reference. `void draw(const Mission& m)` hands the function the original, not a copy, and the `const` promises it will not alter it. For a few numbers, copying is fine. For a struct with arrays inside, it wastes time.

Prefer the standard library. `std::min`, `std::max`, `std::hypot` and `std::vector` have been tested by millions of people. Writing your own copy invites mistakes. The robot's `clampd` is a one-line wrapper around two of them.

Turn on the tools that find mistakes. On Linux or macOS the compiler can add runtime checks: `g++ -g -fsanitize=address,undefined` makes the program stop with a clear message if it reads past the end of an array or divides by zero. Use it while developing; switch it off for the final fast build.

> **WHY DO IT THIS WAY? Compiling with -O2 for the final build only**
>
> **What it is.** `-O2` asks the compiler to make the program faster, at the cost of a slower compile and harder debugging.
>
> **Why it is done this way.** While you are hunting a bug you want slow, simple code that a debugger can step through. When you are done, you want speed.
>
> **Why it matters.** The robot's full 98-second run is about 98,000 ticks, each of which fires rays at ten obstacles. With `-O2` it finishes in well under a second.
>
> **What happens if you do not.** Nothing breaks, but test runs feel sluggish, so you run fewer of them.

31.4 Where This Goes Next

The robot's program is a *simulation* of the controller. The real robot runs the same state machine on an Arduino-class board, where `std::printf` becomes `Serial.println`, the sonar is read from a real pin, and motor commands go to real PWM outputs. The state machine, the ramps and the soft stop carry over unchanged. Only the edges (reading sensors, driving outputs) are replaced.

Check Yourself: Advanced C++

1. A stiff system explodes at one step per tick but is stable at four. What changed numerically?
2. Why does a second, independently written program make a simulation more trustworthy?
3. What does `-ffast-math` do, and why is it a problem for the comparison?
4. When would you use `-fsanitize=address,undefined`, and when would you not?
5. Which parts of Patrick's simulation would change on a real Arduino, and which would stay?

Part V. The Guided Build of the Website

The real site, one decision and one function at a time

Chapter 32. The Shape of the Whole Site

Time to open the hood. This part walks through the real website, using the real code. Every listing is copied straight from the project's files by the book's build script, with its line numbers, so what you read is exactly what runs. (Line numbers describe the files as they stood when this edition was built. If a file has since changed, the numbers will have drifted, but the explanations still hold.)

32.1 What Is in the Folder

The website is a folder of plain files. There is no server code, no database, no framework to install before you can open a page.

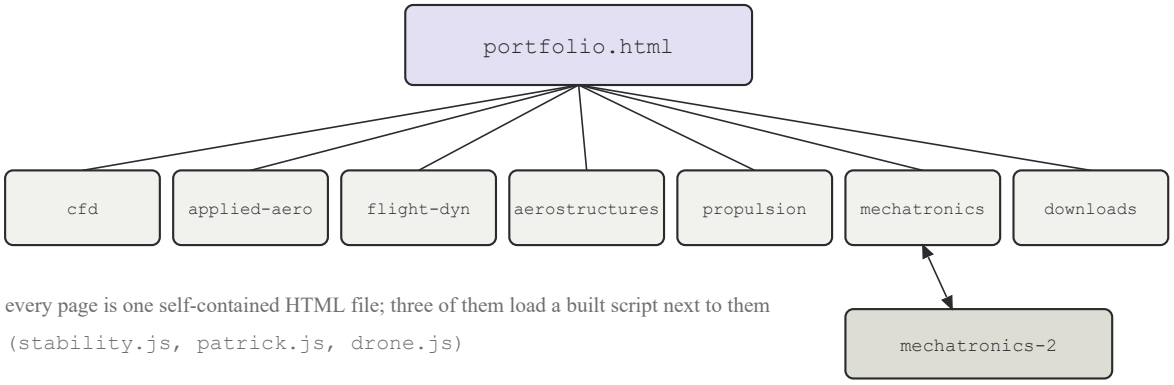


Figure 32.1. The pages and how they connect. The portfolio links to every page; the two robot pages link to each other; three pages load a bundle built from source folders next to them.

File	Size (bytes)	What it is
`portfolio.html`	45 491	The home page: a scroll-driven showcase, a selection wheel, and links to every other page.
`cfd.html`	87 861	Computational fluid dynamics page, with canvas animations.
`applied-aerodynamics.html`	130 085	Applied aerodynamics, with charts and SVG.
`flight-dynamics.html`	86 284	The spiral climb and the full stability analysis.
`aerostructures.html`	61 892	Structures, with its own chart script `fuselage-charts.js`.
`propulsion.html`	716 201	Propulsion: an interactive isometric turbofan, a flight journey and a cycle console.

File	Size (bytes)	What it is
`mechatronics.html`		The first robot page: the self-stabilising drone, with a 3D teardown, its PID controller and its graphs.
`mechatronics-2.html`		The second robot page: Patrick, in his kitchen, with a 3D teardown.
`downloads.html`	13 367	Downloads.
`PortfolioHero.jsx`	17 895	A React component that no page loads yet (Chapter 40).
`Syd Website Assets/`		Scripts, images and build tools for each page.
`Guide/`		This book.

32.2 What Each Page Is Built With

This is the honest table, taken from searching the code.

Technology	Where it is used
Plain HTML and hand-written CSS	Every page.
Canvas 2D	Charts and animations on the CFD, applied aerodynamics, flight dynamics, propulsion and mechatronics pages.
SVG	The block diagram on flight dynamics, the isometric turbofan on propulsion, icons throughout.
KaTeX (mathematics)	Aerostructures, applied aerodynamics, CFD and flight dynamics.
three.js	The Patrick robot page only, bundled with esbuild.
Tailwind CSS (from a CDN)	The portfolio page.
React	Only in `PortfolioHero.jsx`, which is not loaded by any page.
GSAP	**Nowhere.**

> **WHY DO IT THIS WAY? Telling you what is *not* used**
>
> **What it is.** The table says GSAP is used nowhere and React is not wired in.
>
> **Why it is done this way.** A book that pretends the site uses tools it does not use would teach you wrong things about your own project.
>
> **Why it matters.** When you open the code looking for GSAP, you will not waste an evening hunting for it. And it shows you a real choice: this site does its animation by hand, with the techniques of Chapter 6.
>
> **What happens if you do not.** You trust a document that is wrong, and the first thing you check contradicts it.

32.3 The Shared Design Language

Every page shares a look. It comes from a handful of decisions repeated in each stylesheet: a very dark blue ground (`#070a1a`), one violet accent (`#a78bfa`), a second blue for secondary data (`#60a5fa`), a display typeface (Chakra Petch) for headings, Inter for text, and JetBrains Mono for numbers. Because each page defines these once as custom properties (Chapter 3), the look is consistent without any shared stylesheet.

> **WHY DO IT THIS WAY? Repeating the palette in each page instead of one shared stylesheet**
>
> **What it is.** Each HTML file carries its own `<style>` block with the same `:root` variables.
>
> **Why it is done this way.** Each page is a single self-contained file that works when opened on its own, with no dependency on a file that might not be found.
>
> **Why it matters.** It is simple and robust: no broken links between files, and no "which stylesheet does this page use?".
>
> **What happens if you do not (that is, if you share one file).** You gain one place to change the palette, and you lose the guarantee that every page stands alone. For a larger site, a shared stylesheet is usually the better trade. For a small one that is edited page by page, this works well. It is a trade-off, not a rule.

Chapter 33. The HTML of a Page, Line by Line

We will take the flight dynamics page, since it has everything: a head, a navigation bar, a hero section, content sections, and scripts. Start at the top.

33.1 The Head

```
 3  <head>
 4      <meta charset="UTF-8">
 5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
 6      <title>Flight Dynamics | Gopolang Makgothi</title>
 7      <link rel="preconnect" href="https://fonts.googleapis.com">
 8      <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
 9      <link href="https://fonts.googleapis.com/css2?family=Chakra+Petch:wght@500;6
    00;700&family=Inter:wght@400;500;600&family=JetBrains+Mono:wght@400;600&display=
    swap" rel="stylesheet">
10      <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/katex@0.16.11/dist
    /katex.min.css">
11      <script defer src="https://cdn.jsdelivr.net/npm/katex@0.16.11/dist/katex.min
    .js"></script>
12      <script defer src="https://cdn.jsdelivr.net/npm/katex@0.16.11/dist/contrib/a
    uto-render.min.js"></script>
13      <script defer src="./Syd%20Website%20Assets/Flight%20Dynamics%20Page/stabili
    ty.js"></script>
14      <style>
```

Listing from `flight-dynamics.html`*, lines 3 to 14.*

Lines 4 and 5 are the two lines every page needs: the character set and the viewport (Chapters 9 and 4). **Line 6** is the title, which shows in the browser tab and in search results.

Lines 7 to 9 load fonts from Google. Look at what is there: two `preconnect` lines before the real request.

> **WHY DO IT THIS WAY? preconnect before the font stylesheet**
>
> **What it is.** `<link rel="preconnect" ...>` tells the browser to start the connection to a server early.
>
> **Why it is done this way.** Recall Chapter 1: opening a connection costs several round trips before a single byte arrives. Doing that early, in parallel with other work, hides some of the cost.
>
> **Why it matters.** Fonts appear sooner, so the text does not sit invisible or in the wrong typeface for as long.
>
> **What happens if you do not.** Nothing breaks. The page is just a little slower to show the right fonts. (The second `preconnect` has `crossorigin` because font files are fetched in a mode that needs it. Without it the early connection cannot be reused.)

Line 10 is the KaTeX stylesheet, and **lines 11 to 13** are scripts, each with `defer` (Chapter 2). Notice the *order*: `katex.min.js` first, then `auto-render.min.js`, which needs KaTeX to exist. Deferred scripts run in order, which is why this works. **Line 13** is the page's own stability script. The `%20` is how a space is written in a URL: the folder is called "Flight Dynamics Page".

> **WHY DO IT THIS WAY? Version numbers in the KaTeX URLs**
>
> **What it is.** `katex@0.16.11` in the address.
>
> **Why it is done this way.** Chapter 8: a pinned URL always returns the same file.
>
> **Why it matters.** A new KaTeX release cannot break the page's mathematics overnight.
>
> **What happens if you do not.** `katex@latest` could one day change how equations are laid out, and the page would change with no edit from you.

33.2 The Style Block

Line 14 opens the `<style>` element. The first rule defines the palette.

```
15          :root {
16              --bg: #070a1a;
17              --bg-raised: #0d1230;
18              --surface: rgba(255, 255, 255, 0.025);
19              --border: rgba(255, 255, 255, 0.08);
20              --border-accent: rgba(167, 139, 250, 0.38);
21              --text: #e6e8f5;
22              --text-muted: #9aa0c3;
23              --text-faint: #5f668f;
24              --accent: #a78bfa;
25              --accent-soft: rgba(167, 139, 250, 0.12);
26              --accent-2: #60a5fa;
27              --radius: 10px;
28              --display: 'Chakra Petch', 'Inter', system-ui, sans-serif;
29              --mono: 'JetBrains Mono', 'Courier New', monospace;
30          }
31
```

Listing from `flight-dynamics.html`*, lines 15 to 31.*

This is Chapter 3's custom properties in action. Look at what the names say: `--bg` and `--bg-raised` are two levels of background, `--text`, `--text-muted` and `--text-faint` are three levels of text, `--border` and `--border-accent` are two kinds of border. The names describe the *role*, not the colour (`--text-muted`, not `--grey`).

> **WHY DO IT THIS WAY? Naming variables by role, not by colour**
>
> **What it is.** `--accent`, not `--violet`.
>
> **Why it is done this way.** The role stays the same even if the colour changes.
>
> **Why it matters.** If the accent changes from violet to teal, `--accent` is still the right name. Had it been called `--violet`, the code would be lying.
>
> **What happens if you do not.** You end up with variables like `--blue` that hold green, which confuses everyone including you.

Line 32, just after, is the reset: `* { margin: 0; padding: 0; box-sizing: border-box; }` (Chapters 4 and 10).

33.3 The Body: Navigation and Hero

```
314  <body>
315      <div class="progress" id="progress"></div>
316
317      <nav class="topnav">
318          <a href="portfolio.html#expertise-wheel" class="back-link">← Back to Por
     tfolio</a>
319          <div class="nav-status"><span>SECTION</span> 01 — FLIGHT DYNAMICS</div>
320      </nav>
321
322      <header class="hero">
323          <div class="hero-inner">
324              <span class="eyebrow">AERODYNAMICS · FLIGHT DYNAMICS</span>
325              <h1>Climbing<br>in <span>circles.</span></h1>
326              <p>How an aircraft moves through the air: the balance of lift, weigh
     t, thrust and drag in steady manoeuvres, and what happens to it after a disturba
     nce. A stability analysis of the Carbon Cub, from the equations of motion throug
     h to the PID loops that keep it flying straight.</p>
327              <nav class="toc" aria-label="On this page">
328                  <a href="#spiral-climb">01 SPIRAL CLIMB</a>
329                  <a href="#conditions">02 FLIGHT CONDITION</a>
330                  <a href="#compressibility">03 INCOMPRESSIBLE FLOW</a>
331                  <a href="#static">04 STATIC STABILITY</a>
332                  <a href="#derivatives">05 DERIVATIVES &amp;amp; STATE SPACE</a>
333                  <a href="#modes">06 PHUGOID &amp;amp; SHORT PERIOD</a>
334                  <a href="#routh">07 ROUTH-HURWITZ</a>
335                  <a href="#control">08 PID CONTROL</a>
336              </nav>
337          </div>
338      </header>
339
340      <main>
```

Listing from `flight-dynamics.html`*, lines 314 to 340.*

Line 315 is an empty `<div class="progress">`. It is a decoration that a script fills in, a thin bar at the top that widens as you scroll. **Lines 317 to 320** are the navigation: a link back to the portfolio page and a label saying which section this is. Notice it is a `<nav>`, a landmark (Chapter 3).

Lines 322 to 338 are the hero. Look at line 325: `<h1>Climbing<br>in <span>circles.</span></h1>`. The `<span>` has no meaning here. It is a *hook* so the stylesheet can colour that one word differently.

> **WHY DO IT THIS WAY? One h1 per page**
>
> **What it is.** The page has exactly one `<h1>`, in the hero.
>
> **Why it is done this way.** The `<h1>` is the title of the whole page. Everything below is a section of it.
>
> **Why it matters.** A screen reader user hears the page's topic first, and the document's outline makes sense (Chapter 9).
>
> **What happens if you do not.** The outline has two competing "titles", and a screen reader user cannot tell which is the page.

Lines 327 to 336 are the contents list, and each link points to an `id` further down the page (`#spiral-climb`, `#conditions`, and so on). The browser scrolls to the matching element, with no

JavaScript. This uses the *fragment* of the URL you met in Chapter 1.

> **TIP**
> When you want a "jump to section" link, you need only two things: an `id` on the target, and an `<a href="#that-id">`. Everything else is done by the browser, and it works even if every script on the page fails.

33.4 The Main Content

Line 340 opens `<main>`, and inside are `<article>` elements, one for each topic, each with an `id`. Every `<article>` starts with a `<span class="proj-tag">` (the small "ANALYSIS 02" label) and an `<h2>`. The equations are written in a `<div class="equation">` as `$$ ... $$`, and KaTeX's auto-render script turns them into typeset mathematics once the page has loaded.

> **WHY DO IT THIS WAY? Writing equations as text and typesetting them later**
> **What it is.** The HTML holds the LaTeX source (`$$a = \sqrt{\gamma R T}$$`), and a script draws it.
> **Why it is done this way.** Equations as images cannot be searched, scaled or read aloud. As text, they can be.
> **Why it matters.** A search engine and a screen reader both get the actual equation text. (KaTeX also writes a hidden MathML version for screen readers.)
> **What happens if you do not.** Each equation is a picture file, which is blurry when zoomed and invisible to assistive technology. And if the script fails, the visitor still sees the equation's text and can read it.

Chapter 34. The CSS, Rule by Rule

Chapter 10 taught the ideas. Here they are in a real stylesheet. We will look at the three rules that give the page its character: the equation box, the instrument panel, and the range slider.

34.1 The Equation Box

```
115          .equation {
116              background: var(--bg-raised); border: 1px solid var(--border);
117              border-radius: var(--radius); padding: 1.2rem 1.25rem; margin: 1.25r
     em 0 1.5rem;
118              overflow-x: auto; overflow-y: hidden; scrollbar-width: none; font-si
     ze: 1.05rem; position: relative;
119          }
120          .equation::-webkit-scrollbar { display: none; }
121          .equation::before, .equation::after {
122              content: ''; position: absolute; width: 10px; height: 10px; border-c
     olor: var(--accent); border-style: solid;
123          }
124          .equation::before { top: -1px; left: -1px; border-width: 2px 0 0 2px; }
125          .equation::after { bottom: -1px; right: -1px; border-width: 0 2px 2px 0;
      }
126          .equation .katex-display { margin: 0.3rem 0; }
```

Listing from `flight-dynamics.html`*, lines 115 to 126.*

This is the card around each equation. `background: var(--bg-raised)` lifts it slightly off the page, and `border` and `border-radius` give it an outline. `overflow-x: auto` is a safety valve: a very wide equation scrolls sideways *inside* the box instead of pushing the whole page wider. `scrollbar-width: none` hides the scrollbar so the box stays tidy. And `position: relative` matters because the two small corner marks, drawn with `::before` and `::after`, are positioned relative to this box.

> **WHY DO IT THIS WAY? overflow-x auto on the equation box**
>
> **What it is.** The box scrolls horizontally if its content is too wide.
>
> **Why it is done this way.** Equations vary in width, and a phone screen is narrow.
>
> **Why it matters.** The page itself never gets a sideways scrollbar, which is the most annoying mobile layout bug.
>
> **What happens if you do not.** One long equation makes the whole page wider than the screen, and everything else on the page appears shrunk or cut off.

34.2 The Instrument Panel

```
131          .panel {
132              margin: 1.75rem 0 2.5rem;
133              border: 1px solid var(--border-accent); border-radius: 14px;
134              background: linear-gradient(180deg, #131a3f, var(--bg-raised));
135              overflow: hidden;
136          }
137          .panel-head {
138              display: flex; align-items: center; justify-content: space-between;
     gap: 1rem;
139              padding: 0.7rem 1.1rem; border-bottom: 1px solid var(--border);
140              font-family: var(--mono); font-size: 0.7rem; letter-spacing: 0.16em;
141          }
142          .panel-head .t { color: var(--accent); }
143          .panel-head .n { color: var(--text-faint); letter-spacing: 0.05em; margi
     n-left: auto; }
144          .replay {
145              appearance: none; cursor: pointer; font: inherit; letter-spacing: 0.
     1em;
146              color: var(--accent); background: var(--accent-soft); border: 1px so
     lid var(--border-accent);
147              border-radius: 6px; padding: 0.2rem 0.6rem; transition: background 1
     50ms ease, color 150ms ease;
```

Listing from `flight-dynamics.html`*, lines 131 to 147.*

Every interactive instrument sits in a `.panel`. It has a gradient background (`linear-gradient(180deg, #131a3f, var(--bg-raised))`), a violet border (`--border-accent`), and `overflow: hidden`. That last one has a job: it clips the panel's children to the rounded corners, so a canvas inside cannot poke out at the corners.

> **WHY DO IT THIS WAY? overflow hidden on a rounded panel**
>
> **What it is.** `overflow: hidden` together with `border-radius`.
>
> **Why it is done this way.** A child element's square corners do not follow its parent's rounded ones unless the parent clips.
>
> **Why it matters.** The panel looks like one clean rounded card.
>
> **What happens if you do not.** The canvas's square corners stick out past the panel's rounded ones, and it looks sloppy.

34.3 The Slider

```
164          input[type=range] {
165              -webkit-appearance: none; appearance: none;
166              width: 100%; height: 4px; border-radius: 2px; background: rgba(255,2
     55,255,0.12); outline: none;
167          }
168          input[type=range]::-webkit-slider-thumb {
169              -webkit-appearance: none; width: 14px; height: 14px; border-radius:
     3px; transform: rotate(45deg);
170              background: var(--accent); box-shadow: 0 0 10px rgba(167,139,250,0.7
     ); cursor: pointer;
171          }
172          input[type=range]::-moz-range-thumb {
173              width: 12px; height: 12px; border-radius: 3px; border: 0;
174              background: var(--accent); cursor: pointer;
175          }
176          input[type=range]:focus-visible { box-shadow: 0 0 0 3px rgba(167,139,250
     ,0.35); }
```

Listing from `flight-dynamics.html`*, lines 164 to 176.*

Sliders are notoriously hard to style, because browsers draw them differently. The first step, `-webkit-appearance: none; appearance: none;`, switches off the browser's own look so you can draw your own. The track is a thin bar; the thumb (the grabbable part) is drawn by two pseudo-elements, one for Chrome and Safari (`::-webkit-slider-thumb`) and one for Firefox (`::-moz-range-thumb`). The thumb is a small square rotated 45 degrees, a diamond that matches the diamond on the page's headings.

> **WHY DO IT THIS WAY? Two thumb rules for one slider**
>
> **What it is.** The same styling written twice, with browser-specific names.
>
> **Why it is done this way.** There is no standard name for the slider thumb that every browser accepts, so each engine has its own.
>
> **Why it matters.** Without both, the slider looks right in one browser and default-grey in another.
>
> **What happens if you do not.** Visitors on the browser you did not test see an ugly, mismatched control.

34.4 Responsive Behaviour

```
297          @media (max-width: 720px) {
298              .two-col { grid-template-columns: 1fr; }
299              .two-col .panel-canvas { border-right: 0; border-bottom: 1px solid v
     ar(--border); }
300              .two-col .panel-canvas:last-child { border-bottom: 0; }
301              .panel-canvas.tall { height: 560px; }
302              .panel-canvas.mid { height: 440px; }
303          }
304
305          footer { text-align: center; padding: 3rem 1rem; border-top: 1px solid v
     ar(--border); color: var(--text-faint); font-size: 0.85rem; }
306
307          @media (max-width: 720px) {
308              .nav-status { display: none; }
309              .panel-head .n { display: none; }
310              .panel-canvas { height: 420px; }
311          }
312      </style>
```

Listing from `flight-dynamics.html`*, lines 297 to 312.*

Two media queries at 720 pixels handle the phone case. The nav status label is hidden (it is only decoration), the canvases get shorter, and the two-column rows collapse to one column. The panel's controls need no query, because they use `repeat(auto-fit, minmax(...))` (Chapter 4).

> **TIP**
>
> Hiding something with `display: none` on a phone does not stop it being downloaded. If it is an image, the cost is still paid. Use it for decoration, not as a way to save data.

Chapter 35. The JavaScript: the Spiral Climb, Function by Function

The first instrument on the flight dynamics page is a 3D helix: an aircraft climbing in a steady turn. It is drawn on a 2D canvas, with the 3D done by hand. That makes it a perfect study in how little code a good instrument needs. We will read it in the order it runs.

35.1 Making the Canvas Sharp

```
852          function fitCanvas(canvas) {
853              const dpr = window.devicePixelRatio || 1;
854              const w = canvas.clientWidth, h = canvas.clientHeight;
855              if (canvas.width !== Math.round(w * dpr) || canvas.height !== Math.r
     ound(h * dpr)) {
856                  canvas.width = Math.round(w * dpr); canvas.height = Math.round(h
      * dpr);
857              }
858              const ctx = canvas.getContext('2d');
859              ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
860              ctx.clearRect(0, 0, w, h);
861              return { ctx, w, h };
862          }
```

Listing from `flight-dynamics.html`*, lines 852 to 862.*

This is the `fit` function from Chapter 4, written out in full. **Line 853** reads the device pixel ratio; the `|| 1` is a fallback in case it is missing. **Line 854** reads the size the canvas is *shown* at. **Lines 855 to 857** resize the drawing surface only when needed. **Line 859** scales the context so that all later drawing is in CSS pixels. **Line 860** clears the old picture. **Line 861** returns the context and the size.

> **WHY DO IT THIS WAY? Returning the context and size together**
>
> **What it is.** `return { ctx, w, h }`: three things in one object.
>
> **Why it is done this way.** The caller always needs all three, and always right after calling `fit`.
>
> **Why it matters.** The first line of every draw function is `const { ctx, w, h } = fitCanvas(canvas);`. The canvas is guaranteed to be the right size, and clear, before anything is drawn.
>
> **What happens if you do not.** Every draw function repeats the size-and-clear code, and one of them forgets to clear, and leaves ghost images.

35.2 The Physics: Where the Numbers Come From

```
873
874             let P;                          // current flight condition
875             function update() {
876                 const [vKt, phiDeg, rcFpm] = ins.map(i => +i.value);
877                 const V = vKt * KT, phi = phiDeg * DEG, rc = rcFpm * FT / 60;
878                 const omega = G * Math.tan(phi) / V;                 // rad/s
879                 const R = V * V / (G * Math.tan(phi)) / FT;          // ft
880                 const t = H / rcFpm * 60;                            // s
881                 const turns = omega * t / (2 * Math.PI);
882                 P = { R, omega, t, turns, phi, gamma: Math.asin(Math.min(rc / V,
     1)) };
883                 $('sp-v-out').textContent = vKt + ' kt';
884                 $('sp-phi-out').textContent = phiDeg + '°';
885                 $('sp-rc-out').textContent = rcFpm.toLocaleString('en-US') + ' f
    t/min';
886                 $('sp-r').textContent = Math.round(R).toLocaleString('en-US') +
    ' ft';
887                 $('sp-w').textContent = (omega / DEG).toFixed(1) + ' °/s';
888                 $('sp-n').textContent = (1 / Math.cos(phi)).toFixed(2) + ' g';
889                 $('sp-g').textContent = (P.gamma / DEG).toFixed(1) + '°';
890                 $('sp-t').textContent = Math.floor(t / 60) + ' min ' + String(Ma
    th.round(t % 60)).padStart(2, '0') + ' s';
891                 $('sp-turns').textContent = turns.toFixed(1);
892             }
```

Listing from `flight-dynamics.html`*, lines 873 to 892.*

This is the whole physics of the spiral climb, and it fits in a few lines. **Line 876** reads the three sliders (airspeed, bank angle, rate of climb) in one go, using `map` over the input elements, and unary plus (`+i.value`) to turn text into numbers. **Line 877** converts the units: knots to metres per second, degrees to radians, feet per minute to metres per second.

Line 878 is the turn rate: **ω = g tan φ / V**. It comes from resolving the lift vector. When the aircraft banks by φ, the horizontal part of lift is `L sin φ` and the vertical part is `L cos φ`. The vertical part must balance weight `W`, so `L = W / cos φ`, and the horizontal part is `W tan φ`. That is the sideways force that turns the aircraft, and dividing by mass and speed gives the turn rate.

Line 879 is the turn radius, `V² / (g tan φ)`, and the division by `FT` converts it to feet for the display. **Line 880** is the time to climb: height divided by rate of climb, times 60 to convert minutes to seconds. **Line 881** is the number of turns: the angle swept (`omega * t`) divided by one full circle (2π).

Lines 883 to 891 are the readouts: they write each number into the page, formatted with `toFixed` or `toLocaleString`.

> **WHY DO IT THIS WAY? All the physics in one update function**
>
> **What it is.** A single function, `update`, recalculates every number from the three sliders.
>
> **Why it is done this way.** Chapter 5's rule: *screen = render(state)*. The sliders are the state; the readouts and the picture are consequences.
>
> **Why it matters.** The radius, turn rate, load factor and time are always consistent with each other, because they all come from one calculation, from the same three inputs, at the same moment.

What happens if you do not. If each readout were updated by its own slider handler, moving one slider could leave another readout stale, and the page would show numbers that disagree with the drawing.

WHY DO IT THIS WAY? Converting units at the edge

What it is. Line 877 turns knots, degrees and feet per minute into metres, radians and metres per second *immediately*.

Why it is done this way. The formulas are all in SI units. Everything inside is consistent, and only the display converts back.

Why it matters. A famous space probe was lost because one team used pounds-force and another used newtons. Mixed units hide inside formulas and give answers that are wrong but look reasonable.

What happens if you do not. You plug degrees into `Math.tan`, which wants radians, and get a turn radius that is nonsense. You would not see an error, only a helix that looks slightly wrong.

35.3 The Loop That Draws It

```
1019              function tick(now) {
1020                  const dt = last === null ? 0 : Math.min((now - last) / 1000, 0.1
      );
1021                  last = now;
1022                  if (!dragging) azim += dt * 0.12;
1023                  if (frac < 1) frac = Math.min(1, frac + dt / duration());
1024                  else if ((hold += dt) > 2.5) { frac = 0; hold = 0; }
1025                  draw();
1026                  raf = visible ? requestAnimationFrame(tick) : null;
1027              }
1028              const start = () => { if (!raf && !reduceMotion) { last = null; raf
      = requestAnimationFrame(tick); } };
1029
1030              new IntersectionObserver(entries => {
1031                  visible = entries[0].isIntersecting;
1032                  if (visible) start();
1033              }, { threshold: 0.3 }).observe(panel);
1034
1035              $('spiral-replay').addEventListener('click', () => { frac = 0; hold
      = 0; if (reduceMotion) { frac = 1; draw(); } });
```

Listing from `flight-dynamics.html`*, lines 1019 to 1035.*

This is Chapter 6's animation loop. **Line 1020** computes the delta time and clamps it to 0.1 seconds. **Line 1022** rotates the camera slowly (`azim += dt * 0.12`) unless the visitor is dragging. **Lines 1023 and 1024** advance the climb (`frac` goes from 0 to 1) and, once it is complete, wait 2.5 seconds and start again. **Line 1025** draws the frame. **Line 1026** asks for the next frame, but only if the panel is visible.

Lines 1030 to 1033 create an `IntersectionObserver` (Project 3). It watches the panel; when it scrolls into view, `visible` becomes true and the loop starts.

WHY DO IT THIS WAY? The loop only runs while the panel is visible

What it is. `raf = visible ? requestAnimationFrame(tick) : null;` on line 1026.

Why it is done this way. A canvas that is scrolled off screen has nobody watching it.

Why it matters. The page may have a dozen animated instruments. Running only the visible one keeps the frame rate high and the laptop cool.

What happens if you do not. Every canvas animates all the time, the frame rate drops, and the fan starts up.

WHY DO IT THIS WAY? Clamping delta time to 0.1 seconds

What it is. `Math.min((now - last) / 1000, 0.1).`

Why it is done this way. If the tab was in the background for a minute, the next `now - last` would be 60 seconds.

Why it matters. The helix does not jump a whole minute forward the moment you return.

What happens if you do not. Switch tabs and come back, and the aircraft instantly teleports.

Chapter 36. The Stability Model, Function by Function

Behind the second half of the flight dynamics page sits a small engine of numerical code, `core.js`. It does four jobs: it builds the aircraft's equations from physical numbers, finds their characteristic polynomial, finds the polynomial's roots, and builds the Routh array. None of it knows anything about the page. It is plain maths, and that is deliberate.

36.1 Building the Equations

```
 7  function model(M, Kn) {
 8      const V = M * ATM.a, q = 0.5 * ATM.rho * V * V, W = AC.m * G;
 9      const CL = W / (q * AC.S), CD = AC.CD0 + AC.k * CL * CL, CDa = 2 * AC.k * CL
    * AC.CLa;
10      const f = ATM.rho * V * AC.S / AC.m;
11      const d = {
12          V, q, CL, CD,
13          Xu: -f * CD, Xw: 0.5 * f * (CL - CDa),
14          Zu: -f * CL, Zw: -0.5 * f * (AC.CLa + CD),
15          Mw: 0.5 * ATM.rho * V * AC.S * AC.c * (-Kn * AC.CLa) / AC.Iy,
16          Mq: 0.5 * ATM.rho * V * AC.S * AC.c * AC.c * AC.Cmq / AC.Iy,
17          Zeta: -q * AC.S * AC.CLe / AC.m,
18          Meta: q * AC.S * AC.c * AC.Cme / AC.Iy
19      };
20      d.A = [[d.Xu, d.Xw, 0, -G], [d.Zu, d.Zw, V, 0], [0, d.Mw, d.Mq, 0], [0, 0, 1
    , 0]];
21      d.B = [0, d.Zeta, d.Meta, 0];
22      return d;
23  }
```

Listing from `Syd Website Assets/Flight Dynamics Page/core.js`*, lines 7 to 23.*

The function `model(M, Kn)` takes a Mach number and a static margin and returns the aircraft's stability matrix. **Line 8** works out the speed (`V = M × a`, where `a` is the speed of sound), the dynamic pressure `q = ½ ρ V²`, and the weight. **Line 9** finds the lift coefficient needed to hold the aircraft up, `CL = W / (q S)`, then the drag coefficient from a parabolic drag polar. **Line 10** is a common factor, `ρ V S / m`, which appears in several derivatives.

Lines 11 to 19 are the **dimensional stability derivatives**, written from the formulas in section 05 of the flight dynamics page: `Xu = −f CD`, `Zw = −½ f (CLα + CD)` and so on. **Line 15** is the one the static margin controls: `Mw` contains `−Kn × CLα`, which is exactly "`Cmα = −a Kn`" from the page's derivation. **Lines 20 and 21** assemble the state matrix `A` and the input vector `B`.

> **WHY DO IT THIS WAY? model() takes Mach and Kn and nothing else**
>
> **What it is.** The function's only inputs are the two numbers a visitor can change on the page. Everything else (weight, wing area, inertia) is a fixed constant in the `AC` object above it.
>
> **Why it is done this way.** The two sliders are the only things the page varies.
>
> **Why it matters.** The function is a pure function (Chapter 11): same Mach and margin in, same matrix out. You can test it in the browser Console with `model(0.35, 0.14)` and check the numbers against a hand calculation.

What happens if you do not. If the model read values from page elements directly, you could not test it without the page, and you could not reuse it for a different panel.

36.2 The Characteristic Polynomial

```
32  function charPoly(A) {
33      const n = A.length; let Mp = zeros(n); const c = [1], Ms = [];
34      for (let k = 1; k <= n; k++) {
35          const AM = matMul(A, Mp), Mk = AM.map((r, i) => r.map((v, j) => v + (i =
    == j ? c[k - 1] : 0)));
36          const AMk = matMul(A, Mk); let tr = 0; for (let i = 0; i < n; i++) tr +=
     AMk[i][i];
37          c.push(-tr / k); Ms.push(Mk); Mp = Mk;
38      }
39      return { c, Ms };
40  }
```

Listing from `Syd Website Assets/Flight Dynamics Page/core.js`*, lines 32 to 40.*

The characteristic polynomial `det(sI − A)` has the system's poles as its roots. Computing a determinant of a matrix with the unknown `s` in it is awkward. The **Faddeev-LeVerrier algorithm** does it with nothing but matrix multiplication and traces. Each pass of the loop on **line 34** produces the next coefficient: **line 35** builds a matrix from the previous one, **line 36** takes the trace of `A` times it, and **line 37** divides by `k` to get the coefficient. The matrices it builds along the way (`Ms`) are saved too, because they also give the transfer function's numerator.

WHY DO IT THIS WAY? An algorithm that needs only multiplication and traces

What it is. A way to get the polynomial's coefficients using only matrix multiplication, addition and trace.

Why it is done this way. The alternatives are symbolic algebra, which needs a big library, or expanding a determinant by hand, which gets unmanageable for a 4 by 4 matrix and worse for the 6 by 6 closed-loop matrix.

Why it matters. It is about ten lines, it works for any size, and it never divides by a number that could be zero (apart from the small integer `k`).

What happens if you do not. You would have to load a maths library of hundreds of kilobytes for what is, here, a single small step, or write a symbolic determinant for each system size.

36.3 The Routh Array

```
64  function routh(c) {                              // c descending; returns {rows, cha
    nges, eps}
65      const n = c.length - 1, cols = Math.ceil((n + 1) / 2), rows = [[], []], eps
    = [false, false];
66      for (let i = 0; i <= n; i++) rows[i % 2].push(c[i]);
67      while (rows[1].length < cols) rows[1].push(0);
68      for (let i = 2; i <= n; i++) {
69          const p2 = rows[i - 2], p1 = rows[i - 1]; let piv = p1[0], e = false;
70          if (Math.abs(piv) < 1e-12) { piv = 1e-9; e = true; }
71          const r = []; for (let j = 0; j < cols; j++) r.push(j + 1 < cols ? (piv
    * p2[j + 1] - p2[0] * p1[j + 1]) / piv : 0);
72          rows.push(r); eps.push(e);
73      }
74      let changes = 0; for (let i = 1; i < rows.length; i++) { const a = rows[i -
    1][0], b = Math.abs(rows[i][0]) < 1e-12 ? 1e-9 : rows[i][0]; if (a * b < 0) chan
    ges++; }
75      return { rows, changes, eps };
76  }
```

Listing from `Syd Website Assets/Flight Dynamics Page/core.js`*, lines 64 to 76.*

Line 65 works out the array's shape. **Line 66** fills the first two rows with the polynomial's coefficients, alternately (even positions in one row, odd in the other). **Lines 68 to 73** compute every later row from the two rows above, using the Routh formula from the page. **Line 69** picks out the pivot, and **line 70** handles the awkward case where the pivot is zero, by replacing it with a tiny number (`1e-9`) and marking the row, which the page displays as ε. **Line 74** counts the sign changes in the first column, the number of right-half-plane poles.

> **WHY DO IT THIS WAY? Replacing a zero pivot with a tiny number**
>
> **What it is.** If a first-column entry is exactly zero, the formula would divide by zero. The code uses `1e-9` instead and flags it.
>
> **Why it is done this way.** This is the standard textbook remedy (the "epsilon method").
>
> **Why it matters.** The array can be completed, and the sign changes can still be counted, instead of the whole calculation breaking.
>
> **What happens if you do not.** The code produces `Infinity` or `NaN`, which then spreads through every following row, and the page shows nonsense with no error message.

> **WARNING**
>
> Floating-point numbers are not exact (Chapter 11). A value that "should" be zero often comes out as `1.2e-17`. That is why the test on line 70 is `Math.abs(piv) < 1e-12` and not `piv === 0`. Comparing floating-point numbers with === is almost always a bug.

Chapter 37. The Panels: How the Page Fits Together

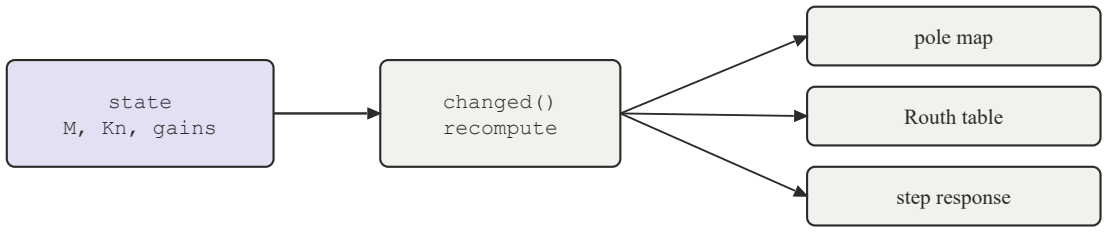
The instruments on the page are drawn by `ui.js`, which is bundled with `core.js` into the file `stability.js` that the HTML loads. Two ideas hold it together.

37.1 One Shared State and a List of Subscribers

```
 88  const S = { M: 0.35, Kn: 0.14, g: { Kp: 1.0, Ki: 0.3, Kd: 0.3 }, pid: true };
 89  let MD, CP, POLES, MODES;
 90  const subs = [];
 91  function recompute() {
 92      MD = model(S.M, S.Kn); CP = charPoly(MD.A); POLES = roots(CP.c); MODES = mod
     es(POLES);
 93  }
 94  function changed() { recompute(); subs.forEach(f => f()); }
 95  function bindKn() {
 96      const ids = ['st-kn', 'dm-kn', 'rh-kn'];
 97      ids.forEach(id => { const el = $(id); if (!el) return; el.addEventListener('
     input', () => { S.Kn = +el.value; ids.forEach(j => { $(j).value = S.Kn; }); chan
     ged(); }); });
 98  }
 99  recompute();
100
```

Listing from `Syd Website Assets/Flight Dynamics Page/ui.js`*, lines 88 to 100.*

Line 88 is the page's state: the Mach number, the static margin, the PID gains, and whether the PID is on. **Line 89** declares the results (the model, the characteristic polynomial, the poles, the modes). **Line 90** is an empty list called `subs`. Every panel that cares about the shared numbers adds a function to it. **Line 91** recomputes the results from the state. **Line 94** is the key one: `changed()` recomputes, then calls every function in `subs`. **Line 95 onwards** connects three static-margin sliders in three different panels, so moving any one moves all three.



every panel adds its redraw function to a list; a slider sets one field and calls changed(), which calls them all

Figure 37.1. The subscriber pattern. One state, one recompute, and a list of functions that redraw. A panel that wants to follow the numbers adds itself to the list and never needs to know about the sliders.

> **WHY DO IT THIS WAY? A list of subscriber functions**
>
> **What it is.** Each panel registers a "redraw me" function in `subs`. When something changes, all are called.
>
> **Why it is done this way.** The panels are independent, and none should know about the others. The static margin slider in the static stability panel should also move the pole map in a different section.

Why it matters. Adding a ninth panel means adding one function to `subs`. No existing panel changes.

What happens if you do not. Each slider has to call every panel by name, and every new panel means editing every slider's code.

37.2 Drawing Only When Visible

```
61  function runWhenVisible(el, tick, still) {
62      let raf = null, last = null, vis = false;
63      const loop = now => { const dt = last == null ? 0 : Math.min((now - last) /
    1000, 0.1); last = now; tick(dt); raf = vis ? requestAnimationFrame(loop) : null
    ; };
64      new IntersectionObserver(es => {
65          vis = es[0].isIntersecting;
66          if (vis && !reduce && !raf) { last = null; raf = requestAnimationFrame(l
    oop); }
67          else if (vis && reduce) still();
68      }, { threshold: 0.12 }).observe(el);
69      return { redraw: () => { if (!raf) still(); } };
70  }
```

Listing from `Syd Website Assets/Flight Dynamics Page/ui.js`*, lines 61 to 70.*

`runWhenVisible` is a small reusable piece of Chapter 6: it takes an element, a function to call every frame, and a function that draws a single still frame. It starts the loop when the element scrolls into view (lines 64 to 68). If the visitor has asked for reduced motion (line 67), it draws one still frame and does not animate. It hands back a `redraw` function (line 69) that the sliders call, which draws a fresh still frame only when no loop is running.

WHY DO IT THIS WAY? Passing in a "still" function

What it is. Each panel supplies two functions: an animated one and a one-shot one.

Why it is done this way. When motion is switched off, the controls must still work.

Why it matters. A visitor who asked for no animation can still move a slider and see the chart update.

What happens if you do not. With reduced motion on, the instruments freeze and appear broken, which punishes exactly the people who asked for gentleness.

Chapter 38. The Patrick Page: One Model Drives Everything

The Patrick page is the most elaborate on the site, and it rests on one rule that you have met several times already: **one source of truth.** A single model (`model.js`) knows the physics of the motors, the wheels, the arm and the sensor. The 3D robot (`robot.js`) is a picture of that model. The page (`main.js`) wires the two to the controls and the charts. Nothing is keyframed by hand.

The folder `Syd Website Assets/Mechatronics/patrick/` holds six files, and each has a single job.

File	Its job
`model.js`	The physics: motor, traction, arm linkage, ultrasonic acoustics, and the breakfast-run simulation. No drawing at all.
`robot.js`	The 3D geometry, built from code, and the `pose()` function that arranges it.
`parts.js`	The description and specifications of every part (computed from the model where possible).
`charts.js`	Small canvas charting helpers.
`main.js`	The page: renderer, camera, picking, tour, panels.
`build.mjs`	Bundles the lot into `patrick.js`.

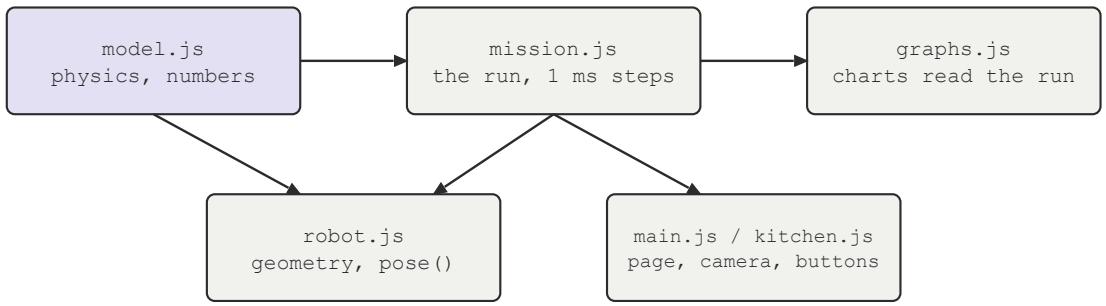


Figure 38.1. How the files depend on each other. The physics is the source; the mission runs on it; the geometry, the charts and the page read from the mission. Nothing to the right of the model is typed in by hand.

38.1 The Motor

```
22  export const MOTOR = {       // yellow TT gear motor, 1:48, at the gearbox outpu
    t
23    Vbat: 6.0, Vdrop: 1.4,     // 4 x AA, and the L298N's two saturation drops
24    Rm: 6.0, I0: 0.19, ke: 0.2326, etaG: 0.55, J: 4.0e-4,
25  };
26  MOTOR.tauF = MOTOR.etaG * MOTOR.ke * MOTOR.I0;                                  /
    / friction torque, N m
27  MOTOR.w0 = (MOTOR.Vbat - MOTOR.I0 * MOTOR.Rm) / MOTOR.ke;                       /
    / no-load speed at 6 V, rad/s
28
29  /** Motor at terminal voltage V and output speed w: current, shaft torque, power
    s. */
30  export function motor(V, w) {
31    const I = (V - MOTOR.ke * w) / MOTOR.Rm;
32    const tau = MOTOR.etaG * MOTOR.ke * I - Math.sign(w || V) * MOTOR.tauF * (Math
    .abs(w) > 0.05 || Math.abs(V) > 0.3 ? 1 : 0);
33    return {I, tau, Pel: V * I, Pmech: tau * w};
34  }
35  export const supplyV = (duty) => duty * (MOTOR.Vbat - MOTOR.Vdrop);   // the H-b
    ridge drops the same either way round
36
```

Listing from `Syd Website Assets/Mechatronics/patrick/model.js`*, lines 22 to 36.*

A small DC motor is described by a few numbers. **Line 24** holds them: winding resistance (`Rm`), no-load current (`I0`), a constant that links speed to voltage (`ke`), gearbox efficiency (`etaG`) and the inertia seen at the wheel (`J`). **Line 26** derives the friction torque, and **line 27** the no-load speed (about 200 rpm at 6 volts, which matches the motor's datasheet).

The function `motor(V, w)` on **lines 30 to 34** is the physics of the motor in three lines. The current is the voltage left after subtracting the back-EMF (`ke × w`, the voltage the spinning motor generates against you) divided by the resistance. The torque is gearbox efficiency times `ke` times current, minus friction. And the powers follow.

> **WHY DO IT THIS WAY? Computing motor numbers from a model, not typing them in**
>
> **What it is.** The stall torque, peak power and efficiency shown on the Patrick page are computed by calling `motor()`. They are never typed in.
>
> **Why it is done this way.** The charts, the part descriptions and the simulation all need these numbers.
>
> **Why it matters.** They cannot disagree. If you change the winding resistance, the power chart, the stall torque in the motor's description and the speed of the robot all change together.
>
> **What happens if you do not.** A spec sheet says "stall torque 0.07 N·m" while the chart plots 0.09, because someone updated one and forgot the other.

> **WHY DO IT THIS WAY? The supply voltage is not 6 volts**
>
> **What it is.** `supplyV(duty)` on line 35 gives `duty × (6.0 − 1.4)`, where 1.4 volts is the voltage the motor driver chip wastes.

Why it is done this way. A real motor driver (the L298N) loses about 1.4 volts as heat inside its transistors.

Why it matters. The robot is slower than the battery voltage would suggest, which matches reality, and it explains a classic beginner puzzle ("my 6 volt motors only get 4.6").

What happens if you do not. The simulated robot runs faster than the real one would, and any "performance" chart overstates what you can build.

38.2 The Wheel Grip, and the Rubber Bands

```
44  export const muPeak = (bands, surf) => (bands ? 1.0 : 0.22) * SURF[surf].k;
45  /** Brush-style friction curve: rises to its peak near 17 % slip, settles to ~80
     % of it when the wheel just spins. */
46  export const mu = (s, peak) => peak * Math.sin(1.5 * Math.atan(10 * s));
47
```

Listing from `Syd Website Assets/Mechatronics/patrick/model.js`*, lines 44 to 47.*

This is the heart of the "with and without rubber bands" story. **Line 44** gives the peak friction coefficient: 1.0 with bands, 0.22 without, scaled by the surface (a dusty desk grips less). **Line 46** is the friction curve. Friction is not constant: it rises as the wheel starts to slip, peaks at about 17 percent slip, and then falls a little as the wheel just spins. The function `mu(s, peak)` captures that with a sine of an arctangent.

WHY DO IT THIS WAY? A friction curve instead of one number

What it is. Friction depends on how much the tyre is slipping, not on a fixed coefficient.

Why it is done this way. Real tyres behave this way. A single fixed friction number would let a wheel push with a force that never falls, however hard it spins.

Why it matters. The simulated robot shows what happens in life: when the motor demands more force than the grip can supply, the wheel spins up, the grip falls, and the robot accelerates *less* than it would with enough grip.

What happens if you do not. The "bare wheels" and "with bands" curves would look almost identical, and the rubber bands would seem pointless.

38.3 The Arm: Two Circles Meeting

```
59  /** Lift servo angle phi (rad, horn from +x, counter-clockwise) -> arm angle the
    ta by circle intersection; null if it jams. */
60  export function armSolve(phi) {
61    const H = [R.servo[0] + R.horn * Math.cos(phi), R.servo[1] + R.horn * Math.sin
    (phi)];
62    const P = R.pivot, dx = H[0] - P[0], dy = H[1] - P[1], dd = Math.hypot(dx, dy)
    ;
63    const a = R.armAttach, L = R.rod;
64    if (dd > a + L || dd < Math.abs(L - a)) return null;
65    const k = (a * a - L * L + dd * dd) / (2 * dd), h = Math.sqrt(Math.max(0, a *
    a - k * k));
66    const ux = dx / dd, uy = dy / dd;
67    const cands = [[P[0] + k * ux - h * uy, P[1] + k * uy + h * ux], [P[0] + k * u
    x + h * uy, P[1] + k * uy - h * ux]];
68    const A = cands.reduce((m, c) => (c[1] > m[1] ? c : m));
    // the elbow-up solution
69    return {theta: Math.atan2(A[1] - P[1], A[0] - P[0]), A, H};
```

Listing from `Syd Website Assets/Mechatronics/patrick/model.js`, *lines 59 to 69.*

The arm is lifted by a servo through a pushrod. The servo's horn moves on a circle; the pushrod has a fixed length; the end of the pushrod is pinned to the arm, which itself can only swing on a circle about its own pivot. So the arm's angle is where two circles meet: the circle of radius "rod length" around the horn's tip, and the circle of radius "attachment distance" around the arm's pivot.

Line 60 finds the horn tip for the given servo angle. **Line 61** gets the vector and distance from the pivot to that tip. **Line 63** checks whether the circles can meet at all: if the distance is too large or too small, the linkage is **jammed**, and the function returns `null`. **Lines 64 to 66** are the standard two-circle intersection formula. **Line 67** picks the solution where the pin is higher (the "elbow up" one). **Line 68** returns the arm's angle and the two pin positions.

> **WHY DO IT THIS WAY? Solving the linkage instead of animating the arm**
>
> **What it is.** The arm's angle is *calculated* from the servo's angle by geometry. It is not set separately.
>
> **Why it is done this way.** In the real robot, the arm angle is a *consequence* of the servo angle. There is one input.
>
> **Why it matters.** The pushrod in the 3D model always connects the horn to the arm exactly, at every angle, because both ends come from the same calculation. It also tells you the robot's real limits: below about 18 degrees of servo travel, the function returns `null`, and the real linkage would jam too.
>
> **What happens if you do not.** The arm and the rod are animated separately, and at some angles the rod visibly detaches from the arm.

38.4 Posing the Robot

```
490
491   /* ---- the pose: everything from the servo angles and the wheel angles ------
     ---------------------------------------- */
492   const state = {theta: 1.0, tip: [0, 0]};
493   function pose({phi, jaw: jv, wl, wr, bands, u}) {
494     const sol = armSolve(phi);
495     if (sol) {
496       state.theta = sol.theta;
497       horn.rotation.z = phi;
498       const A = sol.A, H = sol.H;
499       rod.base.set((A[0] + H[0]) / 2, (A[1] + H[1]) / 2, 0);
500       rod.group.rotation.z = Math.atan2(A[1] - H[1], A[0] - H[0]);
501       lowBars.rotation.z = sol.theta; upBars.rotation.z = sol.theta;
502       const tx = R.pivot[0] + R.armLen * Math.cos(sol.theta), ty = R.pivot[1] +
     R.armLen * Math.sin(sol.theta);
503       state.tip = [tx, ty];
504       grip.base.set(tx, ty, 0);
505       jaw.base.set(tx + 0.0165, ty + 0.0205, 0);
506     }
507     const open = jv * 28 * Math.PI / 180;
508     jaws.jawL.rotation.y = open; jaws.jawR.rotation.y = -open;
509     wheels.L.rotation.z = -wl; wheels.R.rotation.z = -wr;
```

Listing from `Syd Website Assets/Mechatronics/patrick/robot.js`*, lines 490 to 509.*

The `pose` function takes the servo angle `phi`, the claw opening `jaw`, the wheel angles `wl` and `wr`, whether the bands are on, and how far the robot is taken apart (`u`). **Line 491** calls the linkage solver from the model. **Lines 494 to 498** turn the horn, place and aim the pushrod from the two pin positions, and swing both pairs of arm bars. **Lines 499 to 502** find the arm's tip, then put the claw and the claw servo there. **Lines 504 and 505** open the two fingers. **Lines 506 and 507** spin the wheels and the bands.

> **WHY DO IT THIS WAY? The robot is a function of its inputs**
>
> **What it is.** `pose(...)` sets every moving part's position from a handful of numbers, from scratch, each time.
>
> **Why it is done this way.** This is Chapter 5's *screen = render(state)* again, in 3D.
>
> **Why it matters.** The same function poses the robot for the mission, for a still frame when motion is reduced, and for the disassembled view. No part can be "left behind" in a wrong position.
>
> **What happens if you do not.** Each piece of code that moves something has to remember to move everything attached to it. Miss one, and the claw floats off the arm.

38.5 Looking Around and Picking

```
191    const b = new THREE.Box3().setFromObject(byId[S.sel].group), s = b.getSize(new
     V3());
192    return clamp(Math.max(s.x, s.y, s.z) * 3.2 + 0.1, 0.22, 0.7);
193  }
194  function stepCamera(dt) {
195    const k = 1 - Math.exp(-dt * 5.5);
196    const sd = selDist();
197    const td = sd ?? want.dist * (1 + 0.55 * S.u);
198    rig.az += (want.az - rig.az) * k; rig.el += (want.el - rig.el) * k; rig.dist +
     = (td - rig.dist) * k;
199    rig.t.lerp(targetPoint(), 1 - Math.exp(-dt * 6));
200    const ce = Math.cos(rig.el);
201    camera.position.set(rig.t.x + rig.dist * ce * Math.cos(rig.az), rig.t.y + rig.
     dist * Math.sin(rig.el), rig.t.z + rig.dist * ce * Math.sin(rig.az));
```

Listing from `Syd Website Assets/Mechatronics/patrick/main.js`*, lines 191 to 201.*

This is the camera. Instead of moving the camera directly, the page keeps *wanted* values (`want.az`, `want.el`, `want.dist`) and the camera glides toward them. **Line 192** is the frame-rate-independent smoothing factor from Chapter 6, with a snappiness of 5.5 per second. **Lines 195 and 196** move the current angles, distance and look-at point a fraction of the way to the wanted ones. **Lines 197 and 198** convert the orbit angles to a position, exactly as in Chapter 15.

> **WHY DO IT THIS WAY? Easing toward a wanted view**
>
> **What it is.** The camera never jumps. It always eases toward a target.
>
> **Why it is done this way.** Cutting between views is jarring. Easing feels like a camera operator.
>
> **Why it matters.** The view buttons, the part picker, the automatic tour and the visitor's own dragging can all just set the *wanted* values, and the camera takes care of the motion. None of those has to know about the others.
>
> **What happens if you do not.** Each of those features animates the camera itself, they fight each other, and the camera judders.

```
225  const ray = new THREE.Raycaster(), ndc = new THREE.Vector2();
226  let mouse = null;
227  const visibleDeep = (o) => { for (let p = o; p; p = p.parent) if (!p.visible) re
     turn false; return true; };
228  function pick(cx, cy) {
229    const r = canvas.getBoundingClientRect();
230    ndc.set(((cx - r.left) / r.width) * 2 - 1, -((cy - r.top) / r.height) * 2 + 1)
     ;
231    ray.setFromCamera(ndc, camera);
232    const list = robot.meshes.filter(visibleDeep);
233    const hit = ray.intersectObjects(list, false).find((h) => !(S.iso && S.sel &&
     h.object.userData.part !== S.sel));
```

Listing from `Syd Website Assets/Mechatronics/patrick/main.js`*, lines 225 to 233.*

This is picking, from Chapter 15. **Line 227** converts the pointer to normalised device coordinates. **Line 228** aims a ray. **Line 229** keeps only the meshes that are actually visible (a hidden mesh can still be hit by a ray). **Line 230** finds the first thing hit, but ignores everything except the selected part when "isolate" is on. **Line 231** returns the part's name.

> **WHY DO IT THIS WAY? Filtering hidden meshes before the ray test**
>
> **What it is.** `robot.meshes.filter(visibleDeep)`.
>
> **Why it is done this way.** three.js ignores the `visible` flag when raycasting.
>
> **Why it matters.** With the rubber bands off, you would otherwise click on an invisible band and get the wrong part.
>
> **What happens if you do not.** Clicks land on invisible parts, and the panel describes something you cannot see.

Chapter 39. Build and Publish

39.1 The Two Builds

Two folders need a build step. Both are small.

```
1  // Bundles entry.js (three.js + the kitchen stage + the teardown lab) into patri
   ck.js, next to the page that loads it, and copies
2  // the C++ mission program into DLC/ so the "Downloadable Files" page can offer
   it.
3  // three and esbuild come from the teardown skill's node_modules (npm install in
    ~/.claude/skills/teardown).
4  //   node build.mjs          one minified file, runs offline
5  import path from 'node:path';
6  import os from 'node:os';
7  import fs from 'node:fs';
8  import {createRequire} from 'node:module';
9  import {fileURLToPath} from 'node:url';
10 const HERE = path.dirname(fileURLToPath(import.meta.url));
11 const DEPS = process.env.TEARDOWN_DEPS || path.join(os.homedir(), '.claude', 'sk
   ills', 'teardown', 'node_modules');
12 const esbuild = createRequire(path.join(DEPS, 'x.js'))('esbuild');
13 await esbuild.build({
14   entryPoints: [path.join(HERE, 'entry.js')], outfile: path.join(HERE, 'patrick.
   js'), bundle: true, minify: true, format: 'iife',
```

Listing from `Syd Website Assets/Mechatronics/patrick/build.mjs`*, lines 1 to 14.*

This is the whole build for the Patrick page. **Lines 9 and 10** find `esbuild` in the Teardown skill's folder of installed packages and load it. **Lines 11 to 14** tell it to start at `main.js`, follow every import (three.js included), minify, wrap the result in an immediately-invoked function, and write `patrick.js`. The flight dynamics folder has the same idea, in a different form: its `build.mjs` joins `core.js` and `ui.js` into `stability.js`, wrapping both in one function so their names cannot collide with the page's own script.

> **WHY DO IT THIS WAY? Keeping source files and a generated file apart**
>
> **What it is.** You edit `main.js`, `model.js` and the rest. You never edit `patrick.js`.
>
> **Why it is done this way.** `patrick.js` is *output*, a minified file that is unreadable to humans.
>
> **Why it matters.** The sources stay readable and testable (the model can be run in Node to check numbers before it is ever drawn). The browser gets one small file.
>
> **What happens if you do not.** If you edit `patrick.js` directly, your changes are overwritten the next time anyone builds.

> **WARNING**
>
> Always run the build before you commit. A page that loads `patrick.js` shows whatever is in that file, and it will not know you changed `model.js` until you rebuild.

39.2 Publishing

The site is published by committing to Git and pushing to the repository on GitHub. The steps used for this site are the ones of Chapter 16:

```
git add flight-dynamics.html "Syd Website Assets/Flight Dynamics Page"
git commit -m "Add stability analysis to the Flight Dynamics page"
git add mechatronics.html mechatronics-2.html portfolio.html "Syd Website Assets/Mec
      hatronics/patrick"
git commit -m "Add the Control Systems and Mechatronics page ..."
git push origin main
```

> **WHY DO IT THIS WAY? Two commits for two pieces of work**
>
> **What it is.** The flight dynamics work and the Patrick work went in as separate commits.
>
> **Why it is done this way.** They are unrelated changes (Chapter 16).
>
> **Why it matters.** If the Patrick page turns out to have a problem, you can undo that commit without touching the flight dynamics work.
>
> **What happens if you do not.** One commit holds both, and undoing either means undoing both.

Chapter 40. React and GSAP in This Site

Parts II and IV taught React and GSAP properly, because you will meet them everywhere. This chapter tells you, plainly, where they stand in *this* project.

40.1 The React File

```
1  import React, { useState, useEffect } from 'react';
2  import { Menu, X, Github, Linkedin, Mail, ChevronRight, ChevronLeft } from 'luci
   de-react';
3
4  export default function PortfolioHero() {
5    const [mobileMenuOpen, setMobileMenuOpen] = useState(false);
6    const [currentSkillIndex, setCurrentSkillIndex] = useState(0);
7    const [isTransitioning, setIsTransitioning] = useState(false);
8
```

Listing from `PortfolioHero.jsx`*, lines 1 to 8.*

`PortfolioHero.jsx` is a React component for a portfolio hero section. **Line 1** imports React and two hooks, `useState` and `useEffect`. **Line 2** imports some icons from a library called lucide-react. **Line 4** declares the component. **Lines 5 to 7** are its three pieces of state: whether the mobile menu is open, which skill is showing, and whether a transition is in progress.

But here is the important fact: **no page loads it.** A search of every HTML and script file finds no reference to `PortfolioHero`. The live portfolio page does not use React. It is a plain HTML file that loads Tailwind CSS from a CDN and runs an ordinary script (line 849 onwards).

> **WHY DO IT THIS WAY? Saying plainly that the file is not connected**
>
> **What it is.** A component exists in the folder but does nothing on any page.
>
> **Why it is done this way.** It was probably written as a design experiment or a candidate replacement for the portfolio's hero.
>
> **Why it matters.** If you edit it expecting the site to change, nothing will happen, and you will lose time. And if you decide to adopt it, you now know the work involved: you would need a build step (Chapter 16) to compile its JSX, a mount point in a page, and the `react`, `react-dom` and `lucide-react` packages installed.
>
> **What happens if you do not know.** You assume the portfolio is a React app and look for build files that are not there.

40.2 What the Portfolio Does Instead: Scroll Scrubbing by Hand

The portfolio's showcase section plays a sequence of 88 image frames as you scroll, like scrubbing through a video. It is done in plain JavaScript.

```
928          function handleScroll() {
929              const showcaseRect = SHOWCASE_SECTION.getBoundingClientRect();
930              const windowHeight = window.innerHeight;
931
932              if (showcaseRect.top > windowHeight) return;
```

```
933
934             // Calculate scroll progress through the showcase section
935             const scrollProgress = Math.max(0, (windowHeight - showcaseRect.top)
      / (windowHeight + SHOWCASE_SECTION.offsetHeight));
936             const imageIndex = Math.max(1, Math.min(TOTAL_IMAGES, Math.round(scr
     ollProgress * TOTAL_IMAGES)));
937
938             updateImage(imageIndex);
939         }
940
941         // Smooth scroll event listener - plays images like video scrubbing
942         let scrollTimeout;
943         window.addEventListener('scroll', () => {
944             if (scrollTimeout) return;
945             scrollTimeout = setTimeout(() => {
946                 handleScroll();
947                 scrollTimeout = null;
948             }, 16); // ~60fps for smooth video-like playback
949         }, { passive: true });
```

Listing from `portfolio.html`*, lines 928 to 949.*

Lines 929 to 936 work out how far through the section the visitor has scrolled (a number from 0 to 1) and turn it into a frame number between 1 and 88. **Line 938** shows that frame. **Lines 943 to 949** attach the scroll listener, and throttle it with a timer so the function runs at most about once every 16 milliseconds. Notice the `{ passive: true }` on line 949: it promises the browser that this listener will not block scrolling.

> **WHY DO IT THIS WAY? A passive scroll listener**
>
> **What it is.** `{ passive: true }` as the third argument of `addEventListener`.
>
> **Why it is done this way.** Normally, the browser must wait to see whether a scroll listener will call `preventDefault()` before it scrolls. A passive listener promises it will not.
>
> **Why it matters.** Scrolling stays smooth even if the listener is slow.
>
> **What happens if you do not.** The page feels sticky on touch devices, because every scroll waits for your code.

40.3 Where GSAP Would Fit

This section of the portfolio is the perfect place to *see* what GSAP offers. The scroll scrubbing above is exactly what ScrollTrigger's `scrub` option does:

```
gsap.to({ frame: 0 }, {
    frame: TOTAL_IMAGES - 1, snap: 'frame', ease: 'none',
    scrollTrigger: { trigger: '#showcase', start: 'top top', end: 'bottom bottom', s
      crub: true },
    onUpdate() { updateImage(Math.round(this.targets()[0].frame) + 1); }
});
```

GSAP would replace the hand-written scroll maths (lines 929 to 936) and the throttle. It would also handle resizing and refreshing the trigger positions.

> **WHY DO IT THIS WAY? Not switching to GSAP just because it exists**

> **What it is.** The site's author has a working hand-written solution.
>
> **Why it is done this way.** The existing code is about twenty lines and has no dependency. GSAP would add a library to download and keep up to date.
>
> **Why it matters.** Good engineering weighs the benefit against the cost. For one scroll effect, the hand-written version is a reasonable choice. If the site grew a dozen choreographed scroll animations, GSAP would earn its place.
>
> **What happens if you add it anyway.** You pay for the whole library to replace twenty lines. If you never add anything else, that was not a good trade.

Chapter 41. The Claude Skills Behind the Site

Two Claude skills helped build parts of this site. A **skill** is a folder of instructions and code that Claude loads for a particular kind of task. The skill's code is ordinary code, and you can read it. This chapter shows what each skill is made of and how it was used here.

41.1 Anatomy: Isometric Figures

The **Anatomy** skill builds isometric SVG figures, drawings of a machine seen from a fixed angle, with shading and parts drawn in the right order. On this site, it built the turbofan engine on the propulsion page. Look at the first lines of the turbofan's build script.

```
1  // Isometric, interactive CF34-8-style turbofan on a test stand, built with the
   Anatomy kit.
2  // Writes fragment.html (style + figure + script) for propulsion.html and previe
   w.html for screenshots.
3  import { writeFileSync } from "node:fs";
4  import { dirname, join } from "node:path";
5  import { fileURLToPath } from "node:url";
6
7  const SKILL = "file:///C:/Users/thabe/.claude/skills/anatomy/";
8  const k = await import(SKILL + "kit/iso-kit.mjs");
9  const G = await import(SKILL + "kit/lathe.mjs");
10 const { kitScript } = await import(SKILL + "scripts/inline-kit.mjs");
11
12 const HERE = dirname(fileURLToPath(import.meta.url));
```

Listing from `Syd Website Assets/Propulsion Page/turbofan/build.mjs`*, lines 1 to 12.*

Lines 7 to 10 load three pieces of the skill's kit: `iso-kit` (the projection and shapes), `lathe` (shapes made by spinning a profile around an axis, such as a nacelle or a shaft), and `inline-kit` (which copies the kit's code into the finished page so the page needs no external file). The script then writes `fragment.html`, which is inserted into `propulsion.html`.

The key idea inside the kit is the isometric projection. Here is the function that turns a 3D point into a position on the page.

```
1  export const AZIMUTH = 45;
2  export const ELEVATION = 30;
3  export const SHADES = 4;
4  const RADIANS = Math.PI / 180;
5  const UNIT = Math.sqrt(1.6);
6  export function cameraOf({ scale, azimuth = AZIMUTH, elevation = ELEVATION }) {
7      const a = azimuth * RADIANS;
8      const e = elevation * RADIANS;
9      return { sinA: Math.sin(a), cosA: Math.cos(a), sinE: Math.sin(e), cosE: Math
   .cos(e), k: scale * UNIT };
10 }
11 export function iso([x, y, z], projection) {
12     const { origin } = projection;
13     const { sinA, cosA, sinE, cosE, k } = cameraOf(projection);
14     return [origin[0] + (x * sinA - y * cosA) * k, origin[1] + ((x * cosA + y *
   sinA) * sinE - z * cosE) * k];
15 }
```

Listing from `~skills/anatomy/kit/iso-kit.mjs`*, lines 1 to 15.*

Lines 1 and 2 set the default viewing angle: 45 degrees around, 30 degrees up. **Lines 6 to 10** (`cameraOf`) compute sines and cosines of those angles once, and a scale factor `k`. **Lines 11 to 15** (`iso`) are the whole projection. The screen's horizontal position is `x × sin(a) − y × cos(a)` and the vertical position is `(x cos(a) + y sin(a)) × sin(e) − z × cos(e)`, each multiplied by the scale. That is a pair of linear formulas: no division by distance, so parallel lines stay parallel. Compare it to Chapter 7's perspective formula, which divides by `−z` to make distant things small. An isometric drawing deliberately does not.

> **WHY DO IT THIS WAY? Isometric projection has no perspective**
>
> **What it is.** The `iso` function never divides by depth.
>
> **Why it is done this way.** In an engineering illustration, equal lengths must look equal, wherever they are in the picture.
>
> **Why it matters.** A reader can compare sizes and spot alignments by eye. It is the style of exploded-view diagrams in every service manual.
>
> **What happens if you do not.** A perspective view makes parts at the back look smaller, which is realistic and a poor way to explain how they fit together.

41.2 Teardown: Interactive 3D Machines

The **Teardown** skill builds interactive 3D simulators of real machines: pages you can take apart, with live gauges driven by the same model that moves the parts. Its rules shaped the Patrick page, and its installed packages (three.js and esbuild) power the Patrick build. The skill's rules are worth knowing. Among them:

1. **Start from a reference, never from imagination.** Patrick was built from the photograph of the real robot.
2. **Real numbers, one source.** Every gauge reads from the function that poses the parts.
3. **One input drives everything.** For Patrick, that is the time of the breakfast run, plus the servo and wheel angles that follow from it.
4. **The part doing the work opens up.** Hence the disassembly slider and the isolate button.
5. **Test on a weak graphics chip.** Hence the three quality tiers.

Two small files from the skill's kit show the flavour of its code. First, the easing helpers.

```
1  // Small pure helpers shared by every sim. Everything animated must be a pure
2  // function of `t` (seconds) so live playback and frame capture match.
3
4  export const clamp01 = (x) => Math.min(1, Math.max(0, x));
5  export const lerp = (a, b, k) => a + (b - a) * k;
6  export const smooth = (x) => {
7    const k = clamp01(x);
8    return k * k * (3 - 2 * k);
9  };
```

Listing from `~skills/teardown/sims/kit/util.js`*, lines 1 to 9.*

These are Chapter 6's `lerp` and `smoothstep` (here called `smooth`), with a `clamp01` that keeps the input between 0 and 1. The header comment says why they exist: *"Everything animated must be a pure function of* `t`*."* A pure function of time means you can ask for the picture at any moment, which is what lets the skill render a video frame by frame.

> **WHY DO IT THIS WAY? Everything animated is a pure function of time**
>
> **What it is.** Given the time `t`, the scene is fully determined. Nothing depends on what happened in the previous frame.
>
> **Why it is done this way.** If the picture at time 12.5 seconds depends on the history, you can only reach it by playing every earlier frame.
>
> **Why it matters.** You can jump to any moment, scrub backwards, pause, or record at an exact frame rate. The Patrick page does this by simulating the whole breakfast run once and then reading values out of the result for any time.
>
> **What happens if you do not.** Pausing and resuming gives a different run each time, and recording to video gives frames that do not match what the viewer saw.

Second, the shader warm-up.

```
29  export function warmUp(renderer, camera, jobs) {
30    // three swaps the removed PCFSoftShadowMap for PCFShadowMap inside its first
    shadow render, after any warm-up
31    if (renderer.shadowMap.type === THREE.PCFSoftShadowMap) renderer.shadowMap.typ
    e = THREE.PCFShadowMap;
32    const prev = renderer.getRenderTarget();
33    const waits = jobs.map(({scene, materials = [], target = null, lights = null,
    override = false}) => {
34      let root = scene;
35      if (!root) {
36        root = new THREE.Scene();
37        for (const m of materials) root.add(new THREE.Mesh(override || isOverride(
    m) ? MESH : TRI, m));
38      }
39      renderer.setRenderTarget(target);
40      return renderer.compileAsync(root, camera, lights);
41    });
42    renderer.setRenderTarget(prev);
43    return Promise.all(waits);
44  }
```

Listing from `~skills/teardown/sims/kit/warm.js`*, lines 29 to 44.*

This is the idea behind Chapter 15's `compileAsync`. **Line 31** applies a fix for one library quirk. **Lines 33 to 41** go through every job (a scene, or a set of materials) and ask the renderer to compile its shader programs *now*, without drawing anything. **Line 43** returns a promise that resolves when all are done.

> **WHY DO IT THIS WAY? Compiling shaders before the first frame**
>
> **What it is.** Preparing the graphics programs in advance, in parallel.
>
> **Why it is done this way.** The first time the graphics card sees a new material, it must compile a program for it. That takes a moment.

Why it matters. Without it, the page freezes for a second at the first frame. With it, the freeze happens while the page is still loading, where nobody notices.

What happens if you do not. The page seems to hang just as the 3D scene appears, which is exactly when the visitor is paying the most attention.

REMEMBER

A skill is not magic. It is code and instructions that you can read, change and learn from. The best way to understand one is to read its source, exactly as you have read this site's.

Chapter 42. Building Patrick From Scratch: The Robot's Program in Nine Stages

This is the guided project for the C++ chapters. We will build Patrick's whole program, the one that fetches five ingredients and lays them out for breakfast, starting from an empty file. We will not read a finished program from top to bottom. We will do what a person actually does: build one small thing, run it, check the answer against a number worked out by hand, and only then build the next thing on top.

There are **nine stages**. Each stage is a complete program in the folder `Guide/code/patrick` (`stage1.cpp` to `stage8.cpp`), and every one compiles and runs. Each adds a little to the one before, so by the end the file is Patrick's real program, `patrick_mission.cpp`, the same file the Downloadable Files page offers. At every step we stop and ask why.

A word of honesty first. On the website the JavaScript came first, because the page needs it: `model.js` holds the physics and `mission.js` the controller. The C++ is a careful port of that code, checked against it row by row (Stage 9). The steps below describe the C++ in building order, but the ideas are the same in both languages.

Stage	What we add	What it proves
1	One motor, spinning a free wheel	The motor law works.
2	A wheel on the ground: grip and slip	The rubber bands matter.
3	Two wheels: pose and turning	We know where the robot is.
4	The ultrasonic sensor	The robot can see.
5	Approach and brake	The robot stops where the sensor says.
6	The arm and the claw	The robot can reach and grip.
7	One ingredient, pantry to table	The state machine works end to end.
8	Five ingredients	The whole mission.
9	The soft stop, and proving it right	Pause, resume, and the twin test.

What you need. The compiler from Chapter 17 and the folder `Guide/code/patrick`. Compile any stage with `g++ -O2 -std=c++17 stageN.cpp -o stageN` and run it with `./stageN`. Read each stage with the file open beside you.

> **REMEMBER**
>
> Every stage ends with a number you can check. Stage 1 predicts a speed before running. Stage 5 predicts where the robot should stop. If a stage prints something you did not expect, do not build on it: find out why first. A bug found at Stage 2 costs a minute; the same bug found at Stage 8 costs an evening.

42.1 Stage 1: One Motor

We start with the part that has the simplest physics and the most reliable answer: a DC motor. Everything the robot does begins with a voltage.

```
17  struct Robot {
18    double wheelR = 0.0325;                 // 65 mm TT wheel
19    double bandT = 0.0013;                  // rubber band thickness, added to the
     rolling radius
20    double track = 0.193;                   // wheel centre to wheel centre
21    double casterX = 0.125;                 // the front ball caster, ahead of the
     drive axle
22    double mBase = 0.55, xBase = 0.035;     // everything except the arm: kg, and
    its centre of mass ahead of the axle
23    double mArm = 0.085;
24    double sensorX = 0.145;                 // the ultrasonic sensor, ahead of the
     axle
25    double pivotX = -0.045, pivotY = 0.085; // arm pivot
26    double armLen = 0.24, armAttach = 0.05;
27    double servoX = 0.09, servoY = 0.04, horn = 0.05, rod = 0.135;   // lift servo
     shaft, horn radius, push-rod length
28    double gripLen = 0.085;
29  } R;
30
31  struct Motor {                            // the yellow TT gear motor (1:48), se
    en at the gearbox output
32    double Vbat = 6.0, Vdrop = 1.4;         // 4 x AA cells, and the two saturatio
    n drops of the L298N H-bridge
33    double Rm = 6.0, I0 = 0.19, ke = 0.2326, etaG = 0.55, J = 4.0e-4;
34    double tauF = 0;                        // friction torque
35  } MOT;
36  struct MotorOut { double I, tau, Pel, Pmech; };
37  // Motor at terminal voltage V and output speed w: current, shaft torque and pow
    ers.
38  static MotorOut motor(double V, double w) {
39    double I = (V - MOT.ke * w) / MOT.Rm;
40    double fric = (std::fabs(w) > 0.05 || std::fabs(V) > 0.3) ? 1.0 : 0.0;
41    double tau = MOT.etaG * MOT.ke * I - sgn(w != 0 ? w : V) * MOT.tauF * fric;
42    return {I, tau, V * I, tau * w};
43  }
44  static double supplyV(double duty) { return duty * (MOT.Vbat - MOT.Vdrop); }
```

Listing from `Guide/code/patrick/stage1.cpp`*, lines 17 to 44.*

Two `struct`s hold the numbers. `Robot` (**lines 17 to 29**) has the geometry and weight: wheel radius 32.5 mm, a rubber band's thickness, the distance between the wheels (the "track"), and so on. `Motor` (**lines 31 to 35**) describes the gear motor and its battery: four AA cells (6 volts), two voltage drops in the motor driver, the winding resistance, the no-load current, the motor constant, the gearbox efficiency and the moment of inertia. The words `R;` and `MOT;` after the closing braces create one variable of each type, named `R` and `MOT`, right there.

The function `motor(V, w)` (**lines 38 to 43**) takes the voltage across the motor and the speed of its output shaft, and returns the **current**, the **torque** and the electrical and mechanical power. **Line 39** is the motor law from Project 6: current equals (voltage minus back-voltage) over resistance. **Line 41** turns current into torque and subtracts friction. The function returns a `MotorOut`, a struct with four fields; a function can return only one thing, so bundling four numbers into a struct lets it return all of them.

> **WHY DO IT THIS WAY? Putting every measurement in one struct, with units in the comments**
>
> **What it is.** The robot's physical numbers live together, each with a comment giving what it is and its unit.
>
> **Why it is done this way.** These are measurements, not decisions. They come from a ruler, a scale and a datasheet, and they will be corrected as the real robot is measured better.
>
> **Why it matters.** If the track width is corrected after a better measurement, it changes on one line, and the turn controller and the path on the map both follow.
>
> **What happens if you do not.** A "0.193" hides inside a formula in four places and you correct three of them.

The test spins a wheel with nothing on it:

```
46  // Spin a free wheel (inertia J) with the motor at 70 % duty and watch it reach
    its free speed.
47  int main() {
48    MOT.tauF = MOT.etaG * MOT.ke * MOT.I0;
49    double V = supplyV(0.7), w = 0, dt = 0.001;
50    std::printf("supply %.2f V; predicted free speed %.2f rad/s\n", V, (V - MOT.I0
     * MOT.Rm) / MOT.ke);
51    for (int i = 0; i <= 600; i++) {
52      MotorOut m = motor(V, w);
53      if (i % 100 == 0) std::printf("t=%.1f s  w=%.3f rad/s  current %.3f A  torqu
    e %.4f N m\n", i * dt, w, m.I, m.tau);
54      w += m.tau / MOT.J * dt;
55    }
56    return 0;
57  }
```

Listing from `Guide/code/patrick/stage1.cpp`*, lines 46 to 57.*

Predict first. At 70 percent duty the motor sees 0.7 x (6.0 - 1.4) = 3.22 V. With no load the current falls to the no-load value I0 = 0.19 A, so the back-voltage is 3.22 - 0.19 x 6 = 2.08 V, and the speed is 2.08 / 0.2326 = **8.94 rad/s**. Now run it:

```
supply 3.22 V; predicted free speed 8.94 rad/s
t=0.0 s  w=0.000 rad/s  current 0.537 A  torque 0.0443 N m
t=0.1 s  w=6.374 rad/s  current 0.290 A  torque 0.0127 N m
t=0.2 s  w=8.205 rad/s  current 0.219 A  torque 0.0037 N m
t=0.3 s  w=8.731 rad/s  current 0.198 A  torque 0.0011 N m
t=0.4 s  w=8.882 rad/s  current 0.192 A  torque 0.0003 N m
t=0.5 s  w=8.925 rad/s  current 0.191 A  torque 0.0001 N m
t=0.6 s  w=8.937 rad/s  current 0.190 A  torque 0.0000 N m
```

It climbs to 8.937 rad/s, within 0.003 of the prediction, and the current falls from 0.537 A at the start (the stall current) to 0.190 A (the no-load current). This is the shape of every motor: strong when slow, gentle when fast. Notice **line 54**: `w += m.tau / MOT.J * dt;`, the simulation's one essential idea. The torque divided by the inertia is the acceleration, and multiplying by the time step gives the change in speed. Everything else is bookkeeping.

> **WHY DO IT THIS WAY? Checking the program against a hand calculation**

> **What it is.** The predicted free speed is computed from the formula and printed beside what the simulation reaches.
>
> **Why it is done this way.** A simulation that prints numbers gives you no way to know whether the numbers are right.
>
> **Why it matters.** If the printed speed had settled at 0.894 or 89.4, you would know at once that a unit was wrong.
>
> **What happens if you do not.** You build seven more stages on a motor that is quietly off by a factor of ten.

42.2 Stage 2: A Wheel on the Ground

A free wheel is easy. A wheel pushing a robot along a table is harder, because the wheel and the robot can move at different speeds: the wheel can *slip*. This is where the rubber bands that Patrick wears earn their keep. We add the tyre model and one routine that advances one side of the robot by one tick.

```
46  // Peak friction coefficient: bare hard plastic is slippery (0.22); wrapped in r
    ubber bands it grips (1.0). 'k' is the surface.
47  static double muPeak(bool bands, double k) { return (bands ? 1.0 : 0.22) * k; }
48  // Brush-style friction curve: rises to its peak near 17 % slip, settles to abou
    t 80 % of the peak when the wheel just spins.
49  static double mu(double s, double peak) { return peak * std::sin(1.5 * std::atan
    (10.0 * s)); }
```

Listing from `Guide/code/patrick/stage2.cpp`*, lines 46 to 49.*

Line 47 says how grippy the wheel can be at best: 0.22 for bare plastic and 1.0 with rubber bands, times a factor for the surface. **Line 49** is the **friction curve**: it rises with slip, peaks near 17 percent, then settles lower when the wheel just spins, using `sin` and `atan` to draw that shape in one line. The routine that uses it:

```
51  struct Side { bool bands = true; double w = 0, v = 0, slip = 0, F = 0, tau = 0,
    Pm = 0, I = 0, Pel = 0, rEff = 0; };
52  // One side of the robot (one wheel, half the mass) advanced by dt.
53  // The tyre's grip changes very fast with slip at low speed, so one 1 ms step is
     too coarse (the wheel would chatter): take four.
54  static void sideStep(Side& s, double V, bool shorted, double N, double peak, dou
    ble dt, double mass) {
55    double rEff = R.wheelR + (s.bands ? R.bandT : 0);
56    const int n = 4; double h = dt / n;
57    double sl = 0, F = 0, tau = 0;
58    for (int k = 0; k < n; k++) {
59      double slipDen = std::max({std::fabs(s.w * rEff), std::fabs(s.v), 0.02});
60      sl = (s.w * rEff - s.v) / slipDen;                              // slip ratio:
     0 = pure rolling, 1 = wheel spinning in place
61      F = N * mu(sl, peak);                                          // the grip fo
    rce the tyre passes to the table
62      if (shorted) tau = -MOT.etaG * MOT.ke * MOT.ke * s.w / MOT.Rm - sgn(s.w) * M
    OT.tauF * (std::fabs(s.w) > 0.05 ? 1.0 : 0.0);   // terminals shorted: electrica
    l brake
63      else tau = motor(V, s.w).tau;
64      double roll = 0.03 * N * (std::fabs(s.v) > 0.003 ? sgn(s.v) : s.v / 0.003);
         // rolling resistance
65      s.w += (tau - F * rEff) / MOT.J * h;                           // the wheel:
    motor torque against the tyre's reaction
66      s.v += (F - roll) / mass * h;                                  // the robot:
```

```
    grip force minus rolling resistance
67    }
68    s.slip = sl; s.F = F; s.tau = tau; s.Pm = shorted ? 0 : std::max(0.0, tau * s.
    w);
69    MotorOut m = shorted ? MotorOut{0, 0, 0, 0} : motor(V, s.w);
70    s.I = shorted ? 0 : m.I; s.Pel = shorted ? 0 : m.Pel; s.rEff = rEff;
71  }
```

Listing from `Guide/code/patrick/stage2.cpp`*, lines 51 to 71.*

`sideStep` advances one side (one wheel and half the mass) by `dt`. Each pass of the loop on **line 58** does the following. It works out the **slip ratio** (how much faster the wheel's surface moves than the robot, as a fraction), turns slip into a **grip force** with the friction curve, finds the motor's torque (or the braking torque when the terminals are shorted), subtracts rolling resistance, then updates the wheel's spin (motor torque pushes it forward, the ground's reaction holds it back) and the robot's speed (grip pushes it, rolling resistance slows it).

Look at `const int n = 4;` on **line 56** and the loop around everything. The whole thing is repeated four times per tick, each with a quarter of the step.

> **WHY DO IT THIS WAY? Four sub-steps inside a tick**
>
> **What it is.** The wheel and speed are advanced in four quarter-millisecond steps for each one-millisecond tick.
>
> **Why it is done this way.** Section 17.15 and the Advanced C++ chapter showed that a stiff system explodes if the step is too big. Near zero speed the tyre behaves like an extremely stiff spring.
>
> **Why it matters.** With one step per tick, the wheels chattered at low speed, and two programs run side by side drifted apart from rounding errors that kept growing. Four sub-steps fixed both.
>
> **What happens if you do not.** The robot looks fine while driving and juddery exactly when it starts or stops, and your comparison against the other implementation fails for reasons you cannot find.

The test applies full throttle at once, the hard case for a tyre, with and without bands:

```
74  int main() {
75    MOT.tauF = MOT.etaG * MOT.ke * MOT.I0;
76    const double mass = 0.5 * (R.mBase + R.mArm), N = 0.5 * 0.72 * (R.mBase + R.mA
    rm) * GRAV, dt = 0.001;
77    for (bool bands : {false, true}) {
78      Side s; s.bands = bands; double peak = muPeak(bands, 1.0), maxSlip = 0, t60
    = -1;
79      for (int i = 0; i < 2000; i++) {
80        sideStep(s, supplyV(1.0), false, N, peak, dt, mass);
      // full command at once: the hard test for the tyres
81        maxSlip = std::max(maxSlip, s.slip);
82        if (t60 < 0 && s.v > 0.25) t60 = i * dt;
83      }
84      std::printf("bands %-3s: speed after 2 s = %.3f m/s, peak slip %.2f, reached
     0.25 m/s at %.2f s\n", bands ? "on" : "off", s.v, maxSlip, t60);
85    }
86    return 0;
87  }
```

Listing from `Guide/code/patrick/stage2.cpp`*, lines 74 to 87.*

The weight on each side is *N = 0.5 x 0.72 x (0.55 + 0.085) x 9.80665 = 2.24 N* (half the robot, with 72 percent of its weight on the drive wheels), and its mass is half the robot's, 0.3175 kg. Running it:

```
bands off: speed after 2 s = 0.465 m/s, peak slip 0.76, reached 0.25 m/s at 0.22 s
bands on : speed after 2 s = 0.486 m/s, peak slip 0.06, reached 0.25 m/s at 0.11 s
```

There is the whole story of the rubber bands in two lines. Without them, the wheel spins: a **peak slip of 0.76** means the wheel surface was moving at four times the speed of the robot, and it takes 0.22 s to reach 0.25 m/s. With them, slip never exceeds 0.06 and the robot gets there in 0.11 s, twice as fast. The motor and the battery are identical; only the grip changed.

42.3 Stage 3: Two Wheels, and Where the Robot Is

Now two sides, left and right. A two-wheeled robot moves by one rule: the **average** of the two wheel speeds is the forward speed, and the **difference** divided by the track width is the turning rate. Integrate those and you know the robot's position and heading.

```
74  int main() {
75    MOT.tauF = MOT.etaG * MOT.ke * MOT.I0;
76    const double mass = 0.5 * (R.mBase + R.mArm), N = 0.5 * 0.72 * (R.mBase + R.mA
    rm) * GRAV, dt = 0.001, peak = muPeak(true, 1.0);
77    for (int test = 0; test < 2; test++) {
78      Side L, Rr; L.bands = Rr.bands = true;
79      double x = 0, z = 0, yaw = 0;
80      for (int i = 0; i < 2000; i++) {
81        double ramp = clampd(i * dt / 0.35, 0, 1), uL = test == 0 ? 0.7 * ramp : -
    0.5 * ramp, uR = 0.7 * ramp * (test == 0 ? 1 : 0.7143);
82        sideStep(L, supplyV(uL), false, N, peak, dt, mass); sideStep(Rr, supplyV(u
    R), false, N, peak, dt, mass);
83        double v = 0.5 * (L.v + Rr.v), om = (Rr.v - L.v) / R.track;
84        yaw += om * dt; x += v * std::cos(yaw) * dt; z += v * std::sin(yaw) * dt;
85      }
86      std::printf("%s: after 2 s x = %.3f m, z = %.3f m, heading %.1f deg\n", test
     == 0 ? "both wheels forward  " : "wheels opposite (turn)", x, z, yaw * 180 / PI
    );
87    }
88    return 0;
89  }
```

Listing from `Guide/code/patrick/stage3.cpp`*, lines 74 to 89.*

Lines 83 and 84 are the heart of it: `v` is the mean of the two speeds, `om` the difference over the track, and the position follows from `yaw`, `x` and `z`. Test one is both wheels forward; test two is the wheels turning opposite ways, a turn on the spot. The result:

```
both wheels forward  : after 2 s x = 0.459 m, z = 0.000 m, heading 0.0 deg
wheels opposite (turn): after 2 s x = 0.000 m, z = 0.000 m, heading 143.3 deg
```

Driving straight, the robot covers 0.459 m in two seconds (with its soft start) and does not drift sideways (z is 0.000). Turning on the spot, it does not move at all (x and z are both 0.000) but its heading changes by 143.3 degrees. That is exactly what the rule predicts, and both results come from the same two lines.

> **WHY DO IT THIS WAY? Position from the two wheel speeds**

> **What it is.** Forward speed is the mean of the two wheel speeds, and turning rate is their difference over the track width.
>
> **Why it is done this way.** This is how every two-wheeled differential-drive robot works, from this one to a robot vacuum.
>
> **Why it matters.** The path that Patrick traces on the Mission Map comes from these lines, so a turn on the spot (equal and opposite wheel speeds) really rotates without moving the centre.
>
> **What happens if you do not.** You fake the path with scripted moves, and the picture no longer follows from the physics.

42.4 Stage 4: The Robot Learns to See

The ultrasonic sensor reports the distance to the nearest thing in front of it. We reproduce that with **ray casting**: fire invisible rays and ask how far each travels before it enters an obstacle. The obstacles here are the five wooden pads in the pantry and the five plates in the breakfast area, all boxes.

```
 89  struct Box { double x, z, hx, hz; };
 90  static std::vector<Box> BOXES;                                    // what the so
     nar can see: every pad and every breakfast spot
 91
 92  // Where a ray from (ox,oz) in direction (dx,dz) first enters a box (distance in
      the ray's own units), or infinity.
 93  static double rayBox(double ox, double oz, double dx, double dz, const Box& b) {
 94    double t0 = 0, t1 = 1e9;
 95    double os[2] = {ox, oz}, ds[2] = {dx, dz}, lo[2] = {b.x - b.hx, b.z - b.hz}, h
     i[2] = {b.x + b.hx, b.z + b.hz};
 96    for (int i = 0; i < 2; i++) {
 97      if (std::fabs(ds[i]) < 1e-9) { if (os[i] < lo[i] || os[i] > hi[i]) return IN
     FINITY; }
 98      else {
 99        double a = (lo[i] - os[i]) / ds[i], c = (hi[i] - os[i]) / ds[i];
100        if (a > c) std::swap(a, c);
101        t0 = std::max(t0, a); t1 = std::min(t1, c);
102        if (t0 > t1) return INFINITY;
103      }
104    }
105    return t0;
106  }
107  // What the ultrasonic sensor reads: the nearest surface in a narrow cone ahead
     (five rays, 6 degrees apart), metres.
108  static double sonarRead(double x, double z, double yaw) {
109    double ox = x + std::cos(yaw) * R.sensorX, oz = z + std::sin(yaw) * R.sensorX,
      best = SONAR_MAX;
110    for (int i = -2; i <= 2; i++) {
111      double a = yaw + i * 6 * D2R, dx = std::cos(a), dz = std::sin(a);
112      for (const Box& b : BOXES) { double t = rayBox(ox, oz, dx, dz, b); if (t < b
     est) best = t; }
113      if (dx > 1e-6) { double t = (1.10 - ox) / dx; if (t > 0 && t < best) best =
     t; }   // the far wall of the arena at x = 1.10
114    }
115    return best;
116  }
```

Listing from `Guide/code/patrick/stage4.cpp`*, lines 89 to 116.*

`rayBox` (**lines 92 to 106**) answers: if I fire a ray from this point in this direction, how far does it travel before it enters this rectangle? It tests one axis at a time, finding where the ray enters and leaves the

rectangle's slab on each, and a hit exists only if the ranges overlap. `sonarRead` (**lines 107 to 116**) fires **five rays**, 6 degrees apart, across the sensor's narrow cone, tests them against every box and against the far wall, and keeps the shortest distance.

> **WHY DO IT THIS WAY? Five rays, not one**
>
> **What it is.** A fan of five rays instead of a single straight ray.
>
> **Why it is done this way.** A real ultrasonic sensor does not see along a line. It hears echoes from anywhere in a cone of about 15 degrees each side.
>
> **Why it matters.** A single ray could slip between two obstacles or miss the corner of a pad, so the robot would not brake when a real sensor would. The cone behaves like the real thing.
>
> **What happens if you do not.** The simulated robot is blind to things that the real robot sees, and the tuned braking distance would not carry over to hardware.

The test drives straight at the first pad and prints what the sensor reads. Nothing brakes yet:

```
119  int main() {
120    MOT.tauF = MOT.etaG * MOT.ke * MOT.I0;
121    for (int i = 0; i < NITEM; i++) BOXES.push_back({PANTRY_X, LANES[i], PAD_HX, P
     AD_HZ});
122    const double mass = 0.5 * (R.mBase + R.mArm), N = 0.5 * 0.72 * (R.mBase + R.mA
     rm) * GRAV, dt = 0.001, peak = muPeak(true, 1.0);
123    Side L, Rr; L.bands = Rr.bands = true; double x = START_X;
124    for (int i = 0; i < 4000; i++) {
125      double u = 0.7 * clampd(i * dt / 0.35, 0, 1);
126      sideStep(L, supplyV(u), false, N, peak, dt, mass); sideStep(Rr, supplyV(u),
     false, N, peak, dt, mass);
127      x += 0.5 * (L.v + Rr.v) * dt;
128      if (i % 250 == 0) std::printf("t=%.2f s  axle x=%.3f m  sonar reads %.3f m\n
     ", i * dt, x, sonarRead(x, START_Z, 0.0));
129    }
130    return 0;
131  }
```

Listing from `Guide/code/patrick/stage4.cpp`*, lines 119 to 131.*

```
t=0.00 s  axle x=-0.050 m  sonar reads 0.465 m
t=0.25 s  axle x=-0.048 m  sonar reads 0.463 m
t=0.50 s  axle x=-0.011 m  sonar reads 0.426 m
t=1.00 s  axle x=0.124 m  sonar reads 0.291 m
t=1.50 s  axle x=0.266 m  sonar reads 0.149 m
t=1.75 s  axle x=0.338 m  sonar reads 0.077 m
t=2.00 s  axle x=0.409 m  sonar reads 0.006 m
t=2.25 s  axle x=0.481 m  sonar reads 0.000 m
t=2.50 s  axle x=0.552 m  sonar reads 0.403 m
```

Check the first reading by hand. The pad's front face is at x = 0.60 - 0.04 = 0.56 m. The sensor sits 0.145 m ahead of the axle, so at x = -0.05 it is at 0.095 m, and 0.56 - 0.095 = **0.465 m**. The program printed 0.465. Then the robot drives straight through the pad (nothing physical stops it), the reading falls to zero, and once it is past the pad it sees the far wall instead: at x = 0.552, the wall at 1.10 m is 1.10 - (0.552 + 0.145) = 0.403 m away. Both the numbers and the behaviour are right. What is wrong is that nothing makes the robot stop. That is Stage 5.

42.5 Stage 5: Approach, Brake, and Stop Where the Sensor Says

A robot cannot know how long a drive will take, but it can ask its sensor "am I close enough yet?" That is a **state machine**: in APPROACH it drives; when the sensor reads 10.8 cm or less it switches to BRAKE; in BRAKE it shorts the motors until the robot is nearly still.

```
118  enum Phase { APPROACH, BRAKE, STOPPED };
119
120  int main() {
121    MOT.tauF = MOT.etaG * MOT.ke * MOT.I0;
122    for (int i = 0; i < NITEM; i++) BOXES.push_back({PANTRY_X, LANES[i], PAD_HX, P
     AD_HZ});
123    const double mass = 0.5 * (R.mBase + R.mArm), N = 0.5 * 0.72 * (R.mBase + R.mA
     rm) * GRAV, dt = 0.001, peak = muPeak(true, 1.0);
124    Side L, Rr; L.bands = Rr.bands = true; double x = START_X, t = 0; Phase phase
     = APPROACH; double tBrake = 0;
125    for (int i = 0; i < 8000 && phase != STOPPED; i++, t += dt) {
126      double sonar = sonarRead(x, START_Z, 0.0), u = 0; bool shortIt = false;
127      switch (phase) {
128        case APPROACH: u = 0.7 * clampd(t / 0.35, 0, 1); if (sonar <= STOP_AT) { p
     hase = BRAKE; tBrake = t; std::printf("t=%.2f s: sonar %.3f m <= %.3f m, BRAKE (
     speed %.3f m/s)\n", t, sonar, STOP_AT, 0.5 * (L.v + Rr.v)); } break;
129        case BRAKE:    shortIt = true; if (std::fabs(L.v) + std::fabs(Rr.v) < 0.00
     6 || t - tBrake > 1.2) phase = STOPPED; break;
130        case STOPPED:  break;
131      }
132      sideStep(L, shortIt ? 0 : supplyV(u), shortIt, N, peak, dt, mass); sideStep(
     Rr, shortIt ? 0 : supplyV(u), shortIt, N, peak, dt, mass);
133      x += 0.5 * (L.v + Rr.v) * dt;
134    }
135    std::printf("stopped at t=%.2f s, axle x=%.3f m, sonar now reads %.3f m (braki
     ng took %.2f s and %.1f cm)\n", t, x, sonarRead(x, START_Z, 0.0), t - tBrake, (x
      - (PANTRY_X - PAD_HX - R.sensorX - STOP_AT)) * 100);
136    return 0;
137  }
```

Listing from `Guide/code/patrick/stage5.cpp`*, lines 118 to 137.*

The `case APPROACH:` line (**line 128**) ramps the throttle up over 0.35 s and changes state when `sonar <= STOP_AT`. The `case BRAKE:` line sets `shortIt = true`, which tells `sideStep` to short the motor terminals: with the terminals shorted, the spinning motor's own back-voltage drives a current that produces a torque against the motion, an electrical brake that costs nothing. It finishes when the wheels are nearly stationary (or after 1.2 s). Running it:

```
t=1.64 s: sonar 0.108 m <= 0.108 m, BRAKE (speed 0.286 m/s)
stopped at t=1.79 s, axle x=0.325 m, sonar now reads 0.090 m (braking took 0.15 s an
      d 1.8 cm)
```

It started braking at 10.8 cm, travelling at 0.286 m/s, and stopped 0.15 s and 1.8 cm later, with the sensor reading 9.0 cm. That 1.8 cm of coasting is why the threshold is 10.8 cm and not the 9 cm we really want: the threshold was tuned so that the claw would end up exactly over the item.

> **WHY DO IT THIS WAY? A state machine, with the next state decided by the world**
>
> **What it is.** Each state does one thing and decides, from timers or sensors, when to hand over to the next.

Why it is done this way. The robot cannot see the future. It cannot know how long the drive will take; it can only look at its sensor and ask "am I close enough yet?"

Why it matters. Because the transitions come from the sensor and not from a stopwatch, the robot adapts: slower wheels or a different friction just make APPROACH last longer, and BRAKE still fires at the same distance.

What happens if you do not. If APPROACH simply drove for 1.64 seconds, any change in battery voltage or floor friction would leave the robot short of the pad, or through it.

WHY DO IT THIS WAY? Brake states that wait for the robot to actually stop

What it is. The BRAKE state finishes when the wheels are nearly stationary (or after a timeout), not after a fixed delay.

Why it is done this way. The braking distance depends on speed, weight and grip, so no fixed delay is right.

Why it matters. The arm moves only after the robot has stopped. A reach while still rolling would knock the ingredient over.

What happens if you do not. Occasionally the claw closes on air.

42.6 Stage 6: The Arm and the Claw

The arm is raised by a servo that turns a small horn, and a rod connects the horn to the arm. Given the servo angle `phi`, where does the arm point? The horn's end sits at a known point. The rod has a fixed length, and the arm's attachment point is a fixed distance from its pivot. So the attachment point lies on **two circles at once**: one around the horn's end (radius equal to the rod) and one around the arm pivot (radius equal to the attachment distance). Where two circles cross is a pure geometry problem.

```
36  // ------------------------------------------------------------------------
    -------------------------------- arm linkage
37  // Lift servo angle phi -> arm angle theta, by intersecting two circles (the hor
    n's pin and the arm's attachment point).
38  struct ArmOut { bool ok; double theta; };
39  static ArmOut armSolve(double phi) {
40    double Hx = R.servoX + R.horn * std::cos(phi), Hy = R.servoY + R.horn * std::s
    in(phi);
41    double dx = Hx - R.pivotX, dy = Hy - R.pivotY, dd = std::hypot(dx, dy);
42    double a = R.armAttach, L = R.rod;
43    if (dd > a + L || dd < std::fabs(L - a)) return {false, 0};
44    double k = (a * a - L * L + dd * dd) / (2 * dd), h = std::sqrt(std::max(0.0, a
     * a - k * k));
45    double ux = dx / dd, uy = dy / dd;
46    double c1x = R.pivotX + k * ux - h * uy, c1y = R.pivotY + k * uy + h * ux;
47    double c2x = R.pivotX + k * ux + h * uy, c2y = R.pivotY + k * uy - h * ux;
48    double Ax = c2y > c1y ? c2x : c1x, Ay = c2y > c1y ? c2y : c1y;      // the elb
    ow-up solution
49    return {true, std::atan2(Ay - R.pivotY, Ax - R.pivotX)};
50  }
51  static void armTip(double theta, double& x, double& y) { x = R.pivotX + R.armLen
     * std::cos(theta); y = R.pivotY + R.armLen * std::sin(theta); }
```

Listing from `Guide/code/patrick/stage6.cpp`*, lines 36 to 51.*

Line 43 returns `ok = false` when the circles do not touch at all: the linkage is jammed. **Lines 44 to 47** solve the intersection, which has two answers; **line 48** takes the upper one, the "elbow-up" solution. The function `armTip` (**line 51**) turns the arm angle into the position of the claw. The claw's grip is a second function:

```
67  // Jaw opening (0 closed .. 1 fully open) at which the fingers touch an item of
    the given width.
68  static double jawFor(double w) { return std::asin(clampd((w / 2 - 0.0165) / 0.03
    6, 0, 1)) / (28 * D2R); }
```

Listing from `Guide/code/patrick/stage6.cpp`*, lines 67 to 68.*

The jaw's fingers touch an item of width `w` when the opening angle satisfies a short piece of geometry, and the function returns it as a fraction from 0 (closed) to 1 (open). The test prints a table:

```
servo angle -> arm angle -> arm tip (x forward, y up)
   0 deg: the linkage jams
  10 deg: the linkage jams
  20 deg ->    0.5 deg -> (0.195, 0.087) m
  30 deg ->   17.0 deg -> (0.185, 0.155) m
  60 deg ->   51.3 deg -> (0.105, 0.272) m
  80 deg ->   71.6 deg -> (0.031, 0.313) m
STOW  servo 74.0 deg: arm 65.6 deg, tip height 0.304 m
CARRY servo 46.0 deg: arm 36.2 deg, tip height 0.227 m
LOW   servo 18.4 deg: arm -4.4 deg, tip height 0.067 m
eggs  width 34 mm -> jaw closes to 0.028 of its range
glass width 42 mm -> jaw closes to 0.256 of its range
```

(The full table has more rows.) Three servo angles are the arm's working poses: **STOW** (arm folded up at 74 degrees), **CARRY** (46 degrees, held in front) and **LOW** (18.4 degrees, reaching the table with the arm tip 6.7 cm up, exactly the height of an item's grip point). At 0 and 10 degrees the linkage jams, so the servo must never be commanded there. The jaw closes further for wider items: 0.028 of its range for the 34 mm egg carton and 0.256 for the 42 mm glass.

> **WHY DO IT THIS WAY? Solving the linkage by geometry, not by a table of angles**
>
> **What it is.** A short function that computes the arm angle from the servo angle, exactly.
>
> **Why it is done this way.** A linkage has a precise rule. No lookup table of measured angles is needed, and none could be as accurate.
>
> **Why it matters.** The same function poses the 3D arm on the page, so the arm in the picture is geometrically the real linkage. It also let a search for the right link lengths find that this servo reaches the table when the rod is 135 mm.
>
> **What happens if you do not.** You measure a dozen arm angles with a protractor, interpolate between them, and get a jerky arm.

42.7 Stage 7: One Ingredient, Pantry to Table

Now we join everything into the real controller. The state machine grows from APPROACH and BRAKE to the full sequence for one ingredient. The states are listed in an `enum`:

```
170  enum Phase { IDLE, APPROACH, BRAKE, REACH, GRIP, LIFT, TURN_CARRY, SETTLE_C, CAR
     RY, BRAKE_C, PLACE, RETREAT_P,
```

```
171               TURN_SHIFT, SETTLE_T, SHIFT, SETTLE_S, TURN_PANTRY, SETTLE_P, DONE
    };
172
173 struct Placed { int state = 0; double x = 0, z = 0, y = 0, yaw = 0; };   // 0 on
     the pantry pad, 1 in the claw, 2 on the breakfast table
```

Listing from `Guide/code/patrick/stage7.cpp`*, lines 170 to 173.*

and the controller is a `switch` on the current phase:

```
224     switch (phase) {
225       case IDLE:        if (phaseT > 0.9) go(APPROACH); break;
226       case APPROACH:    uL = uR = duty * ramp(0.35); hold(); if (sonar <= STOP_A
    T) go(BRAKE); break;
227       case BRAKE:       shortIt = true; if (settled()) { go(REACH); armTarget =
    PHI_LOW; jawTarget = 1; } break;
228       case REACH:       shortIt = true; if (phaseT > 1.5) go(GRIP); break;
229       case GRIP:        shortIt = true;
230                         if (phaseT > 0.35) jawTarget = jawFor(it.w);
231                         if (phaseT > 1.1) { held = true; items[item].state = 1;
    go(LIFT); armTarget = PHI_CARRY; } break;
232       case LIFT:        shortIt = true; if (phaseT > 1.3) { go(TURN_CARRY); yawR
    ef += PI; } break;
233       case TURN_CARRY:  turn(SETTLE_C); break;
234       case SETTLE_C:    shortIt = true; if (phaseT > 0.5) go(CARRY); break;
235       case CARRY:       uL = uR = duty * ramp(0.35); hold(); if (sonar <= STOP_A
    T) go(BRAKE_C); break;
236       case BRAKE_C:     shortIt = true; if (settled()) { go(PLACE); armTarget =
    PHI_LOW; } break;
237       case PLACE:       shortIt = true;
238                         if (phaseT > 1.5 && held) { jawTarget = 1; held = false;
     items[item].state = 2; items[item].y = 0.025 + 0.004; }   // let go
239                         if (phaseT > 2.4) { armTarget = PHI_STOW; jawTarget = 0;
     go(RETREAT_P); startX = x; startZ = z; } break;
240       case RETREAT_P:   uL = uR = -0.5 * ramp(0.2); hold();
241                         if (std::hypot(x - startX, z - startZ) >= RETREAT) {
242                           if (item == 0) go(DONE); else { go(TURN_SHIFT); yawRef
     -= PI / 2; }
243                         } break;
244       case TURN_SHIFT:  turn(SETTLE_T); break;
245       case SETTLE_T:    shortIt = true; if (phaseT > 0.4) go(SHIFT); break;
246       case SHIFT: {     // drive along the table to the next lane, slowing as th
    e gap closes
247                         double rem = LANES[item + 1] - z;
248                         uL = uR = std::max(0.4, duty * clampd(rem / 0.12, 0, 1))
     * ramp(0.3); hold();
249                         if (rem < 0.004) go(SETTLE_S);
250                         break; }
251       case SETTLE_S:    shortIt = true; if (settled()) { go(TURN_PANTRY); yawRef
     -= PI / 2; } break;
252       case TURN_PANTRY: turn(SETTLE_P); break;
253       case SETTLE_P:    shortIt = true; if (phaseT > 0.5) { item++; go(APPROACH)
    ; } break;
254       case DONE:        shortIt = true; break;
255     }
```

Listing from `Guide/code/patrick/stage7.cpp`*, lines 224 to 255.*

Read it as a story. Wait 0.9 seconds. APPROACH: drive until the sonar reads 10.8 cm. BRAKE: short the motors until still, then REACH: lower the arm and open the claw. GRIP: after 0.35 s close the claw to the width of *this* ingredient, and at 1.1 s count it as held and start lifting. LIFT, then TURN: spin the robot 180 degrees (`yawRef += PI`) with a PD controller on heading. SETTLE, then CARRY: drive to

the breakfast area, brake, lower the arm (PLACE), let go at 1.5 s, stow the arm, and RETREAT backwards 14 cm. In `stage7.cpp` the run ends after the first ingredient (one line changed from the final program: `if (item == 0) go(DONE)`). Running it and listing each time the phase changes:

```
t=  1.20  phase  1  x= -0.050 z= -0.500  holding=0     APPROACH
t=  2.85  phase  2  x=  0.309 z= -0.500  holding=0     BRAKE
t=  2.99  phase  3  x=  0.325 z= -0.500  holding=0     REACH
t=  4.49  phase  4  x=  0.324 z= -0.500  holding=0     GRIP
t=  5.60  phase  5  x=  0.324 z= -0.500  holding=1     LIFT
t=  6.90  phase  6  x=  0.323 z= -0.500  holding=1     TURN
t=  8.79  phase  7  x=  0.323 z= -0.500  holding=1     SETTLE
t=  9.29  phase  8  x=  0.323 z= -0.500  holding=1     CARRY
t= 11.95  phase  9  x= -0.330 z= -0.500  holding=1     BRAKE
t= 12.10  phase 10  x= -0.346 z= -0.500  holding=1     PLACE
t= 14.50  phase 11  x= -0.347 z= -0.500  holding=0     RETREAT
t= 15.72  phase 18  x= -0.206 z= -0.500  holding=0     DONE
finished at t = 17.72 s; items on the table: eggs(-0.613,-0.500)
```

(The words on the right are added for reading; the program prints only the numbers.) Follow the robot: it drives from x = -0.05 to the pantry (x = 0.309), picks up the eggs (`holding` becomes 1 at 5.60 s), turns, carries them the 0.65 m to the breakfast area (x = -0.330), puts them down (`holding` returns to 0 at 14.50 s) and backs away. The eggs end at x = -0.613, z = -0.500: within 7 millimetres of the centre of the first plate, which is at x = -0.62, z = -0.50.

> **WHY DO IT THIS WAY? Each phase has an exit condition, and a safety net**
>
> **What it is.** Every state leaves on a sensor reading or a timer, and the braking states also have a timeout (`settled()` gives up after 1.2 s).
>
> **Why it is done this way.** A robot stuck in one state for ever is a robot that has failed. Wheels can stick, sensors can misread.
>
> **Why it matters.** With the timeout, the worst case is a slightly late move on, never a frozen robot.
>
> **What happens if you do not.** One missed echo and the robot sits waiting for a signal that never arrives.

42.8 Stage 8: Five Ingredients

One ingredient is the whole machine once; five is the machine repeating. After PLACE and RETREAT the robot must go to the next lane. That adds four states: TURN_SHIFT (face along the table), SHIFT (drive to the next lane, slowing as the gap closes), another settle, and TURN_PANTRY (face the pantry again). The code is already in the switch from Stage 7; the only difference between `stage7.cpp` and `stage8.cpp` is the single line that made it stop after one item. The ingredients and the lane positions are tables:

```
105  static const Item ITEMS[5] = {{"eggs", 0.034, 0.045}, {"salt", 0.036, 0.055}, {"
     spice", 0.038, 0.050}, {"sauce", 0.040, 0.080}, {"glass", 0.042, 0.060}};
106  static const int NITEM = 5;
107  static const double LANES[5] = {-0.50, -0.25, 0.0, 0.25, 0.50};   // one lane pe
     r ingredient (z, to the left)
108  static const double PANTRY_X = 0.60, KITCHEN_X = -0.62;           // centre line
     s of the pantry pads and of the breakfast spots
109  static const double PAD_HX = 0.04, PAD_HZ = 0.04;
110  static const double START_X = -0.05, START_Z = -0.50;
```

```
111  static const double RETREAT = 0.14;
```

Listing from `Guide/code/patrick/stage8.cpp`*, lines 105 to 111.*

and the shift state is the cleverest bit of the controller, driving slower as the remaining gap closes:

```
245        case SHIFT: {     // drive along the table to the next lane, slowing as th
     e gap closes
246                          double rem = LANES[item + 1] - z;
247                          uL = uR = std::max(0.4, duty * clampd(rem / 0.12, 0, 1))
      * ramp(0.3); hold();
248                          if (rem < 0.004) go(SETTLE_S);
249                          break; }
```

Listing from `Guide/code/patrick/stage8.cpp`*, lines 245 to 249.*

`rem` is how far is left to the next lane; the duty is the full approach duty scaled by `rem / 0.12`, never below 0.4 (the motors will not turn below that: stiction), and the state ends when less than 4 mm remains. Running the whole mission:

```
finished at t = 97.75 s; items on the table: eggs(-0.613,-0.500) salt(-0.613,-0.252)
      spice(-0.613,-0.002) sauce(-0.613,0.248) glass(-0.613,0.498)
```

All five ingredients are on their plates. Compare each with its plate's centre (lanes at z = -0.50, -0.25, 0, 0.25, 0.50; plates at x = -0.62): every ingredient is within 7 mm of the centre in x and within 2 mm in z. The whole job took 97.75 simulated seconds, and the robot never once looked at a clock to decide where to go: every move ended on a sensor reading or on a distance check.

> **WHY DO IT THIS WAY? The Phase enum and one big switch**
>
> **What it is.** The mission is nineteen named states and one `switch`, rather than a long sequence of function calls with delays in between.
>
> **Why it is done this way.** A sequence with delays cannot react: if the robot is late, everything after it is wrong. A state machine lets each state wait for the world.
>
> **Why it matters.** The same program works whether the floor is slippery (slow starts) or the battery is low (slow moves): only the timings change, never the positions.
>
> **What happens if you do not.** The robot only works on the day you tuned the delays.

42.9 Stage 9: The Soft Stop, and Proving It Right

A pause button on a machine with momentum cannot freeze everything. Instead the program keeps a number `g`, the **go-gain**, between 0 (halted) and 1 (running). Pressing pause sets the *goal* to 0; each tick, `g` slews towards it at a fixed rate. The eased version is then used two ways: the controller's own clock slows down (so timers and servo motions stretch out and stop together) and every drive command is multiplied by it (so the motors' push fades out). The code:

```
198    // One tick (1 ms). Returns false while the robot is halted and nothing moves.
199    bool step() {
200      // the go-gain slews towards its goal: 0.6 s to resume, 0.8 s to pause
201      double rate = goal > g ? 1 / 0.6 : 1 / 0.8;
202      g += clampd(goal - g, -rate * dt, rate * dt);
203      if (g <= 0 && goal == 0) return false;
204      double e = ge(), dtc = dt * e;                  // controller time slows wi
```

```
      th the gain: timers and servos slow down smoothly
 205      t += dt; phaseT += dtc;
 206      sonar = sonarRead(x, z, yaw);
 207      const Item& it = ITEMS[item];
 208      double uL = 0, uR = 0; bool shortIt = false;
```

Listing from `Guide/code/patrick/stage8.cpp`*, lines 198 to 208.*

(The code is in the file from Stage 7 on; Stage 9 is about using it, with `--pause-at` and `--resume-at`.) Run it, pausing at 10.5 s while the robot is carrying the eggs, and resuming two seconds later:

```
./stage8 --pause-at 10.5 --resume-at 12.5 --out p.csv
```

and look at the speed (from the position column) and the gain every tenth of a second:

```
t=10.50  speed 28.7 cm/s  gain 1.00  (pause pressed)
t=10.70  speed 27.6 cm/s  gain 0.75
t=10.90  speed 19.4 cm/s  gain 0.50
t=11.00  speed 12.7 cm/s  gain 0.38
t=11.10  speed  5.1 cm/s  gain 0.25
t=11.20  speed  0.1 cm/s  gain 0.12
t=11.30  speed  0.1 cm/s  gain 0.00
...
t=11.50  speed  0.0 cm/s  gain 0.33  (resumed)
t=11.70  speed  3.1 cm/s  gain 0.67
t=11.80  speed  9.6 cm/s  gain 0.83
```

(Because no rows are written while the robot is halted, the time column carries straight on across the pause.) The robot did not stop in a tick. It eased from 28.7 cm/s to zero over about 0.7 seconds, along a smooth S-curve, and after the resume it eased back up. Everything was resumed from where it was: the mission still finished with all five ingredients in place, at 98.69 s instead of 97.75 s.

Proving it right: the twin test. How do you know the C++ is correct? You write the same program a second time, differently, and compare. For Patrick the second program is the web page's JavaScript. Both write a table, one row every 10 ms, the same 13 columns, and the two tables are compared:

```
./stage8 --out cpp1.csv
node dump.mjs js1.csv
tr -d '\r' < cpp1.csv | cmp - js1.csv
```

They were identical: 9,776 rows by 13 columns, the same in every digit. The same was done for a run with a pause at 20 s and a resume at 23.5 s, which was identical too.

It was not identical the first time. The first comparison showed the programs agreeing for almost 3 seconds, then drifting apart by a few millionths of a metre in the BRAKE state, then by centimetres a few seconds later, enough to put one program in a different state than the other. The cause was in Stage 2: with one step per tick, the tyre equation was numerically unstable at low speed, so the tiniest difference between the two languages' arithmetic was amplified instead of damped. (That is the explanation that fits the evidence: adding the four sub-steps removed the disagreement completely.) The programs have agreed to the last digit ever since.

Putting it on the page. Three small pieces carry the C++ from your folder to the visitor's screen. The build script copies `patrick_mission.cpp` into the `DLC` folder, and the Downloadable Files page lists it. The bundler is told to read `.cpp` files as plain text, so the program's own source sits inside the

page's script as a string, and the code bubble colours the keywords with a few regular expressions. And the page finds the controller's `case` lines by searching for `case NAME:`, which is why the `enum` names and the `case` labels are the same words.

> **WHY DO IT THIS WAY? The page shows the real file, not a copy**
>
> **What it is.** The code on the page is the very text of `patrick_mission.cpp`, inserted by the build.
>
> **Why it is done this way.** A second copy, pasted into the page's HTML, would be one more thing to forget to update.
>
> **Why it matters.** Edit the C++ and rebuild, and the bubble, the download and the highlighted lines all follow. What a visitor reads is, by construction, what they can download and compile.
>
> **What happens if you do not.** The code on the page quietly becomes a different program from the one in the download.

Try It Yourself

Review Problems

1. In Stage 1, change the duty to 1.0. What free speed does the program predict, and what does it reach?
2. In Stage 2, imagine only the *left* wheel has bands. What would you expect the robot to do when both sides get the same command? (Think about Stage 3.)
3. In Stage 4, add a second ray fan aimed 30 degrees to the left. What would it let the robot do?
4. In Stage 5, change `STOP_AT` from 0.108 to 0.080 and rerun the final program. What happens to the finishing time and the final positions?
5. Run `./stage8 --duty 1.0 --bands 0`. How does the result compare with the default run?
6. Add a sixth ingredient. List every place in the file that must change.
7. The robot must now *measure* the pad's width with the sonar before gripping, instead of knowing `ITEMS[i].w`. Sketch how you would do it.
8. Design your own robot: a two-wheeled cart with a bumper switch that must fetch a ball from a corner. List the states, the sensor reading that ends each one, and the numbers you would need to measure first.

Check Yourself: Building Patrick

1. What did Stage 2 prove about the rubber bands, with numbers?
2. In Stage 3, what two lines give the robot's position?
3. How do you check the first sonar reading by hand?
4. Why does Patrick brake at 10.8 cm when he wants to stop at about 9 cm?
5. What does the go-gain scale, and why both the motors and the clock?
6. How can you be sure the page's animation and the downloaded file are the same program?

Answer Key

Answers to the Check Your Knowledge questions and the Review Problems, by chapter. Try every question before you look: the value of a problem is in the attempt.

Chapter 1

Check Your Knowledge.

1. The client asks and the server answers. The words describe roles, so one computer can run both: a laptop running a browser and a local development server is both at once.
2. Scheme `https`, host `example.com`, port `8080`, path `/a/b.html`, query `x=1`, fragment `top`. The fragment is never sent to the server; the browser uses it to scroll to a place in the page.
3. The `Content-Type` header tells the browser how to interpret the body. Some browsers will not execute a script announced as plain text, and strict checks exist to stop text being run as code by mistake.
4. Latency is the time for a message to make the round trip; bandwidth is the amount of data carried per second. Latency dominates a very small file, because the transfer itself takes almost no time.
5. Advantages: nothing on the server to attack, and cheap and fast hosting. A static site cannot by itself remember or change data on the server, for example store a visitor's comment, without calling some other service.

Review Problems.

1. Size: 2.4 megabytes = 2 400 000 × 8 = 19.2 megabits; at 20 megabits per second that is 0.96 s. Set-up and request: 4 × 0.050 = 0.20 s. Total: 1.16 s.
2. 686 000 × 0.31 = 212 660 bytes = 1.701 megabits, so transfer takes 0.170 s. Total with set-up: 0.170 + 0.320 = 0.490 s.
3. Two batches of 6, each taking one round trip: 2 × 80 = 160 ms. One combined file: a single round trip, 80 ms. Combining halves the time in this case, and the saving grows with the number of files.
4. A new name is a new address, so the browser has nothing cached under it and fetches the file at once; the old name is simply no longer referenced. The old cached copies can never be shown stale, because nothing points to them.
5. Answers depend on the page and the connection. The usual first candidate is the largest image or script, because for large files the transfer time dominates.
6. Set-up and request: 4 × 0.300 = 1.200 s. Transfer is unchanged at 0.549 s, so the total is 1.749 s. This is 2.01 times the original 0.869 s. Latency scales with distance, so serving files from a location near the visitor, as content delivery networks do, reduces exactly the part of the time that no change to the file can touch.

Chapter 2

Check Your Knowledge.

1. Parse HTML into the DOM, parse CSS into the CSSOM, combine them into the render tree, layout, paint, composite.
2. The DOM holds every node of the document. The render tree holds only the nodes that will be drawn, with their computed styles. An element with `display: none` is in the DOM but not in the render tree.
3. Drawing before the styles are known would show the page unstyled and then re-arrange it. Images can be drawn into a box whose size is already known and filled in when they arrive, so the page need not wait.
4. Both fetch the script while the parser continues. An `async` script runs as soon as it arrives, in no guaranteed order. A `defer` script runs after the document has been parsed, and deferred scripts run in the order they appear.
5. `transform` on an element with its own layer can be applied during compositing and needs no layout or paint. `left` changes geometry and forces layout, then paint, then composite.
6. A long task is a stretch of main-thread work longer than about 50 ms. While it runs the page cannot respond to input, scroll or animate smoothly.

Review Problems.

1. 1000/30 = 33.33 ms; 1000/60 = 16.67 ms; 1000/90 = 11.11 ms; 1000/120 = 8.33 ms.
2. The code plus the browser takes 22 + 4 = 26 ms. The budget must be at least 26 ms, so the rate must be at most 1000/26 = 38.5 Hz. The page keeps up at 30 Hz and not at 60, 90 or 120 Hz. The highest rate is about 38 Hz.
3. Blocked in total: (120 + 10) + (80 + 5) + (200 + 30) = 130 + 85 + 230 = 445 ms. With `defer` and parallel fetches, parsing is not blocked by the fetches at all, and the scripts run after parsing, so the blocking is 0 ms (the scripts then run for 10 + 5 + 30 = 45 ms after parsing, in order).
4. 500 × 0.12 = 60 ms lost. Grouped: 0.12 ms, a saving of 59.88 ms, about 500 times faster.
5. (a) Layout, then paint, then composite. (b) Paint and composite, no layout. (c) Composite only. (d) Composite only.
6. The answer depends on the page. A typical longest task is a script start-up or a chart redraw; the aim is to see it for yourself on the timeline.

Chapter 3

Check Your Knowledge.

1. HTML describes the structure and meaning of the content; CSS describes how it looks and where it sits; JavaScript describes how the page behaves and responds.
2. A real button is focusable and operable by keyboard, is announced as a button by assistive technology, and fires activation events for Space and Enter, all without code. A `<div>` has none of that unless it is rebuilt by hand.
3. By importance (`!important` first), then by specificity, then by source order (later wins).
4. `nav ul li.active a:hover` has no IDs, two classes or pseudo-classes (`.active` and `:hover`), and four elements (`nav`, `ul`, `li`, `a`): (0, 2, 4). `#menu .item` has one ID and one class: (1, 1, 0). The second is more specific because the first number decides first.

5. Inherited: `color`, `font-family`. Not inherited: `margin`, `border`, `padding`.
6. Building a page in layers: HTML content first, then CSS, then JavaScript. If a layer fails to load, the page degrades to the one beneath and stays usable.

Review Problems.

1. `*` (0,0,0); `div p` (0,0,2); `a:hover` (0,1,1); `.card` (0,1,0); `div.card p.lead` (0,2,2); `#card` (1,0,0). Ranked from least: `*`, `div p`, `.card`, `a:hover`, `div.card p.lead`, `#card`. (Note that `a:hover` at (0,1,1) is more specific than `.card` at (0,1,0), and `div p` at (0,0,2) is less than `.card`.)
2. Both rules have the same specificity (0,1,0), so the later one wins: green. Reversed, red. With `.a.a` the first rule has specificity (0,2,0) and wins whatever the order: red.

```
:root { --accent: #a78bfa; --ink: #070a1a; }
.title { color: var(--accent); border-bottom: 2px solid var(--accent); }
.badge { background: var(--accent); color: var(--ink); }
```

4. The script is deciding appearance, which is CSS's job, and doing it every frame. Better: the script adds or removes a class (`element.classList.toggle('active', isActive)`) only when the state changes, and the stylesheet defines `.active { color: red; }` and the default grey.
5. (a) CSS, with a media query. (b) JavaScript. (c) HTML (`<blockquote>`). (d) CSS, with a transition. (e) JavaScript.
6. A correct answer has the heading, paragraph and button in HTML; `:root { --c: ...; } h1 { color: var(--c); }` in CSS; and `button.addEventListener('click', () => p.classList.add('done'))` in JavaScript, with a rule for `.done` in the CSS.

Chapter 4

Check Your Knowledge.

1. Device pixels = CSS pixels × DPR. A CSS pixel is a unit of length that is made of DPR × DPR device pixels on a dense screen.
2. 600 × 3 = 1800 device pixels. If left at 600, the browser stretches the bitmap to three times its size and it looks blurred.
3. Content, padding, border, margin.
4. It makes the declared `width` and `height` include the padding and border, so the box is exactly the size you declared.
5. It makes as many columns of at least 190 px as fit in the row, and shares any leftover space equally among them. As the container narrows, columns drop away one at a time and the rest grow to fill the row.
6. `rem` scales with the visitor's chosen default text size, whereas `px` ignores it, so `rem` respects accessibility settings.

Review Problems.

1. DPR 1: 512 × 384 = 196 608 pixels. DPR 1.5: 768 × 576 = 442 368 pixels. DPR 2: 1024 × 768 = 786 432 pixels.

2. The limits are 16 px and 32 px. The preferred value is 2.5vw + 8 px: at 360 it is 9 + 8 = 17 px; at 800 it is 20 + 8 = 28 px; at 1600 it is 40 + 8 = 48 px, which is clamped to 32 px. It stops growing when 2.5vw + 8 = 32, so 2.5vw = 24 and the width is 960 px.
3. (a) content-box: 200 + 2 × 20 + 2 × 5 + 2 × 10 = 270 px. (b) border-box: the width of 200 includes padding and border, so 200 + 2 × 10 = 220 px.
4. The number of columns n satisfies 190n + 16(n − 1) ≤ 900, so 206n ≤ 916 and n = 4 (since 4 × 206 = 824 ≤ 916 and 5 × 206 = 1030 > 916). The gaps take 3 × 16 = 48 px, leaving 852 px, so each column is 213 px wide.
5. The box is 400 × 2 = 800 device pixels square. The image has 2000 pixels across, more than 800, so it is sharp (and larger than needed). The smallest sharp square image is 800 by 800 pixels.

```
nav { display: flex; gap: 1rem; }
@media (max-width: 599px) { nav { flex-direction: column; } }
```

Chapter 5

Check Your Knowledge.

1. Advantage: only one piece of your code runs at a time, so two functions cannot change the same data simultaneously. Limitation: a long-running function blocks everything else, including input and drawing.
2. Run the oldest task to completion; run all microtasks until the queue is empty; let the browser render if it is time; repeat.
3. The task queue holds callbacks such as events and timers; the microtask queue holds follow-up work such as promise continuations. Microtasks have priority: all of them run before the next task.
4. Capture (root down to the target), target (the element itself), bubble (target back up to the root). A normal handler runs in the bubble phase.
5. Attaching one handler to a parent and examining `event.target` to see which child was clicked. It still works for later children because their events bubble up through the same parent.
6. It keeps delivering the pointer's events to one element even when the pointer leaves it, so a drag is not dropped when the cursor moves faster than the element.
7. The two copies can fall out of step, so the page shows something that contradicts its own state. Hold each fact once and derive everything else from it.

Review Problems.

1. Output: 1, 6, 3, 4, 2, 5. The script runs first (1, then 6). Then all microtasks run: 3 prints, and its follow-on 4 is queued as a microtask and runs before any task. Then the first timer task prints 2, then the second prints 5.
2. The second handler runs after the loop finishes, at about 300 ms after the first click, 200 ms after the second click. The loop occupies the main thread, so the second click's task waits in the queue; input cannot interrupt a running task.
3. Without delegation: 200 handlers. With delegation: 1. `list.addEventListener('click', e => { const li = e.target.closest('li'); if (li) handle(li); });`

4. Inner prints first, then outer, because the event bubbles. Adding `event.stopPropagation()` in the inner handler stops the outer from running.
5. For example, if a script adds the class to the button but an error prevents it from updating the panel, the two disagree. The redesign keeps a single variable `selected` and a function that, from that variable alone, sets the classes of all tabs and panels (or a single attribute on the container, with CSS selecting the matching panel).
6. The pointer leaves the element, so `pointermove` events stop arriving and the slider stops following the pointer; the drag appears to stick. Calling `setPointerCapture(e.pointerId)` on `pointerdown` makes the element keep receiving events until the button is released.

Chapter 6

Check Your Knowledge.

1. It is synchronised with the screen's refresh, so frames arrive at the right moment; it pauses in hidden tabs; and it gives a timestamp. A timer's rhythm is unrelated to the display's, so frames arrive at uneven moments.
2. Delta time is the seconds since the previous frame. Moving by speed × Δt makes the motion independent of the frame rate. It is clamped so that a long pause, such as a hidden tab, does not produce a huge jump.
3. lerp(a, b, t) = a + (b − a)t. At t = 0 it gives a, at 0.5 the midpoint, at 1 it gives b.
4. The slope is zero at both ends, so the motion starts from rest and comes to rest smoothly.
5. It makes a value follow a changing target smoothly without a fixed duration. With a fixed k per frame, a faster screen has more frames per second and so closes the gap faster; the fraction must be computed as $1 - \exp(-s\Delta t)$.
6. That the visitor wants less motion. The page should stop non-essential animation (loops, automatic motion, large movements) while leaving the content and controls fully usable.

Review Problems.

1. With delta time: 240/30 = 8 px, 240/60 = 4 px, 240/120 = 2 px per frame. With `x += 4`: 4 × 30 = 120 px/s, 4 × 60 = 240 px/s, 4 × 120 = 480 px/s. Only at 60 Hz does it match 240.
2. smoothstep(0.1) = 3(0.01) − 2(0.001) = 0.028, position 2.24. smoothstep(0.5) = 0.5, position 40. smoothstep(0.9) = 3(0.81) − 2(0.729) = 2.43 − 1.458 = 0.972, position 77.76.
3. $k = 1 - e^{-s/60} = 0.1$ gives $e^{-s/60} = 0.9$, so $s/60 = -\ln 0.9 = 0.10536$ and $s = 6.32$ per second. At 144 Hz, $k = 1 - e^{-6.32/144} = 1 - e^{-0.04390} = 1 - 0.9570 = 0.0430$.
4. The remaining distance is $100\,e^{-4t}$. At 0.25 s: $100\,e^{-1} = 36.8$. At 0.5 s: $100\,e^{-2} = 13.5$. At 1 s: $100\,e^{-4} = 1.83$.
5. Entries at 0, 7 and 12 have fired; 14.3 is before 16, so the next is the one at 16 s. The time to wait is 16 − 14.3 = 1.7 s, which is 1.7 ÷ 0.016 = 106.25 frames, so it fires on the 107th frame.
6. Use `const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;` and then `if (reduce) drawOnce(); else requestAnimationFrame(frame);`. The controls still call the draw function when the visitor changes a value, so every interaction redraws the picture; only the unrequested animation is

removed.

Chapter 7

Check Your Knowledge.

1. Retained mode keeps a record of the shapes, which can be styled and clicked individually; immediate mode paints pixels and forgets how. SVG is retained; canvas is immediate.
2. Downward. A chart must flip the axis, mapping larger data values to smaller pixel y values.
3. They save and restore the whole drawing state (colour, line width, transform), so a temporary change such as a rotation can be undone without disturbing later drawing.
4. Model space, world space, camera (view) space, clip space (normalised device coordinates), screen space.
5. x_ndc = (f / a) x / (−z) and y_ndc = f y / (−z), with f = 1 / tan(θ/2). The division by distance −z makes the apparent offset from the centre shrink as the distance grows.
6. Flickering where two surfaces are nearly at the same depth, caused by the limited precision of the depth buffer. Reduce it by setting the near plane as far and the far plane as near as the scene allows.
7. Brightness = max(0, n · l). An environment map supplies reflections of a surrounding scene, which glossy and metallic surfaces need in order to look realistic.

Review Problems.

1. f = 1 / tan 30° = 1.7321 and a = 16 / 9 = 1.7778. x_ndc = (1.7321 / 1.7778)(−0.2 / 1) = −0.1949; y_ndc = 1.7321 × 0.1 = 0.1732. x_px = (−0.1949 × 0.5 + 0.5) × 1280 = 515.3; y_px = (−0.1732 × 0.5 + 0.5) × 720 = 297.6.
2. Ten pixels from the centre means x_px − 640 = 10, so x_ndc × 0.5 × 1280 = 10 and x_ndc = 0.015625. Then 0.015625 = (1.7321 / 1.7778) × 0.5 / d = 0.9743 × 0.5 / d, so d = 0.4871 / 0.015625 = 31.2 metres.
3. After 90° the point (1, 0, 0) goes to (cos 90°, 0, −sin 90°) = (0, 0, −1). After a yaw of 30° the robot's x axis points to (cos 30°, 0, −sin 30°) = (0.866, 0, −0.5).
4. 480 × 0.02 = 9.6 ms. Merged: 48 × 0.02 = 0.96 ms, a saving of 8.64 ms. A 60 Hz frame is 16.67 ms, so the saving is 8.64 / 16.67 = 52% of the budget.
5. For (0, 0, 1): 1. For (0, 0.6, 0.8): 0.8. For (1, 0, 0): 0, so the light from the side leaves the surface black.
6. With n = 0.1, f = 100: (f + n)/(n − f) = 100.1 / (−99.9) = −1.002002 and 2fn/(n − f) = 20 / (−99.9) = −0.200200. At distance 1 (z = −1): (1.002002 − 0.200200) / 1 = 0.8018. At 10 (z = −10): (10.02002 − 0.2002) / 10 = 0.9820. At 50 (z = −50): (50.1001 − 0.2002) / 50 = 0.9980. So 10 distance units use 98.2% of the depth range, all but the last 1.8%.

Chapter 8

Check Your Knowledge.

1. Native elements come with keyboard operation, focus handling and the correct role announced by assistive technology. A `<div>` has none of this unless it is rebuilt by hand.

2. It gives a text alternative to an image for those who cannot see it. A decorative image should have an empty `alt=""` so that it is skipped.
3. A contrast ratio of at least 4.5 : 1 between text and background (3 : 1 for large text).
4. LCP, the time until the largest visible element appears; INP, how quickly the page responds to an interaction; CLS, how much the layout shifts during loading.
5. XSS is an attacker getting their script to run in your page. `textContent` treats the value strictly as text, so markup in it is displayed and not executed; `innerHTML` parses it as HTML and may run script.
6. A pinned URL cannot change under you, so a compromised or buggy new release cannot reach your page silently; it also keeps behaviour reproducible.
7. A bundler follows `import` statements and combines the code into one file the browser can load. Minification shrinks the output by removing whitespace and shortening names.

Review Problems.

1. White on the background: (1.0 + 0.05) / (0.00337 + 0.05) = 1.05 / 0.05337 = 19.67, which passes. Grey on white: (1.0 + 0.05) / (0.1845 + 0.05) = 1.05 / 0.2345 = 4.48, which narrowly fails 4.5 for normal text.
2. (1.0 + 0.05) / (0.1392 + 0.05) = 1.05 / 0.1892 = 5.55. It passes AA for normal text, even though the same colour fails on the dark background; contrast depends on the pair.
3. `2rem` is 2 × 24 = 48 px; the caption stays 12 px. The `rem` heading respects the setting; the caption in `px` ignores it.
4. LCP is failing: 3.4 s against the 2.5 s target, 0.9 s too slow. INP cannot be judged from the load time alone. Fixes: compress or resize the image so it downloads faster, and preload it or load it earlier in the page.
5. The value of `name` is inserted as HTML, so `?name=<img src=x onerror=alert(1)>` runs script in the page. Safe version: `const b = document.createElement('b'); b.textContent = 'Hello, ' + name; document.getElementById('hello').replaceChildren(b);`
6. Sharp at DPR 3 needs 400 × 3 = 1200 pixels wide. On the DPR 1 screen only 400 are needed, so the pixel ratio is (1200 × 1200) / (400 × 400) = 9, nine times as many pixels as necessary (the file is typically smaller than that in bytes, but the waste is real). This is why pages offer several sizes and let the browser choose with `srcset`.

Chapter 9

Chapter 9, Practice Problems A.

1. `<!DOCTYPE html>` (modern mode, strictly before the head); `<html lang="en">`; `<meta charset="UTF-8">` (character table); the `viewport` meta (true phone width); `<title>` (tab name); a stylesheet `<link>`. (The doctype and html tag sit above the head, so a strict answer is the last four inside the head plus these two outside.)
2. The `<b>` is closed after the `<p>` (wrong nesting; it should be `<p>Welcome to <b>my site</b></p>`), `<b>` should be `<strong>` for importance, and the `<img>` has no `alt`, `width` or

`height`.

3. `<br>`, `<img>`, `<input>` and `<meta>` are void. `<p>`, `<span>` and `<li>` have content and end tags (the end tags of `<p>` and `<li>` may be omitted, but the elements are not void).
4. Two paragraphs: a `<p>` cannot contain a `<p>`, so the second `<p>` closes the first. The first holds "One", the second holds "Two".
5. `<code>if (a < b && c > 0)</code>`. The `>` may be written as is, but `>` is clearer; `<` and `&` must be escaped.
6. `<meta charset="UTF-8">` is missing (or the file was saved in another encoding). It goes in the `<head>`, as early as possible, before any text.
7. The checkbox must be ticked. `required` is a boolean attribute: its presence means required, whatever value it is given.

Chapter 9

Chapter 9, Practice Problems B.

1. `<nav><ul><li><a href="a.html">A</a></li><li><a href="b.html">B</a></li><li><a href="c.html">C</a></li></ul></nav>`
2. The levels jump (h1 to h3, then back to h2, then h5), so the outline is confusing. Use `<h1>`, `<h2>`, `<h3>`, `<h4>` in order, and change the sizes with CSS.
3. (a) `<strong>`. (b) `<button type="submit">`. (c) `<ol>`. (d) `<dl>` with `<dt>` and `<dd>`. (e) `<kbd>`. (f) `<time datetime="...">`.
4. From the folder `/docs/guide/`: `href="../index.html"` and `src="../../img/cover.png"`.
5. `sizes="(max-width: 699px) 300px, 700px"`. On the phone the slot is 300 CSS pixels at DPR 3, so 900 device pixels are needed, and the 900w candidate is the smallest that is wide enough.
6. `<table><caption>Robot parts</caption><thead><tr><th scope="col">Part</th><th scope="col">Mass (g)</th></tr></thead><tbody><tr><th scope="row">Motor</th><td>32</td></tr><tr><th scope="row">Wheel</th><td>18</td></tr><tr><th scope="row">Battery</th><td>95</td></tr></tbody></table>` (any three parts are fine).
7. `/filter?tag=robots&q=claw+2`. The space is encoded as `+` (or `%20`).
8. `<form><label for="u">Username</label><input id="u" name="u" type="text" autocomplete="username"><label for="p">Password</label><input id="p" name="p" type="password" autocomplete="current-password"><button type="submit">Log in</button></form>`
9. A `<button>` in a form defaults to `type="submit"`, so it submits the form. Add `type="button"`.

Chapter 9

Chapter 9, Practice Problems C.

1. `<meta name="description" content="...">`, `<meta property="og:title" content="...">`, `<meta property="og:description" content="...">`, `<meta property="og:image" content="https://.../preview.jpg">`. The `og:image` must be absolute, because the chat app fetches it from its own servers.
2. `aria-label="Close"`. The value is what the button *does*, not the symbol it shows.
3. `tabindex="0"` so Tab reaches it, `role="button"` so it is announced as a button, and a key handler for Enter and Space (plus `aria-pressed` for its state). You should have used `<button>`, which has all of these already.
4. Correct for decorative content that duplicates visible text, such as an icon beside a label. Harmful on anything a person needs, because it hides the element from assistive technology while sighted users still see it.
5. `hidden` removes the element for everyone (like `display: none`). `aria-hidden="true"` leaves it visible and hides it only from assistive technology.
6. `<button class="chip" data-chart="speed">Speed</button>` and so on; the script reads `button.dataset.chart`.
7. Would catch: a missing `alt`, a duplicate `id`, an unclosed element, a `<p>` containing a `<div>`, an unknown attribute, a missing `<title>`. Would not catch: a `<div>` used as a button, headings chosen for their size (the structure is valid but wrong), poor `alt` text, a bad tab order.

Chapter 10

Chapter 10, Practice Problems A.

1. `p` (0,0,1); `a:hover` (0,1,1); `p.a` (0,1,1) ties with `a:hover` (later one wins); `.a .b` (0,2,0); `#x` (1,0,0). Order: `p`, then `a:hover` and `p.a` (tied), then `.a .b`, then `#x`.
2. A link inside a list item that is a direct child of `.nav` and is not the first such item (it has a previous `li` sibling).
3. Red. `nav a` scores (0,0,2); `.menu-link` scores (0,1,0), and one class beats any number of elements, whatever the order.
4. The section is 1.2 × 16 = 19.2 px. The paragraph's `em` refers to its parent's size: 1.25 × 19.2 = 24 px.
5. `border-box`: content width is 300 − 2 × 20 − 2 × 2 = 256 px. `content-box`: total width is 300 + 40 + 4 = 344 px.
6. 50 px, the larger of the two, because vertical margins collapse. As flex items, 80 px, because margins never collapse in a flex container.
7. `color`, `font-family` and `line-height` inherit. `padding`, `border` and `background` do not.
8. `:root { --rose: #e11d48; --rose-bg: #fff1f2; } h1 { color: var(--rose); } a { border-color: var(--rose); background: var(--rose-bg); }`

Chapter 10

Chapter 10, Practice Problems B.

1. n columns fit if 150n + 10(n − 1) ≤ 500, so 160n ≤ 510 and n = 3. Gaps take 20, leaving 480, so each column is 160 px.
2. Fixed: 100 + 3 × 16 = 148, leaving 752 for four shares. One share is 188. Widths: 100, 188, 188, 376.
3. Each item needs 200 and each gap 12. Three items: 600 + 24 = 624 ≤ 700. Four: 800 + 36 > 700. Three on the first line.
4. The button's container (or the button itself) needs `position: relative`. Without a positioned ancestor, the tooltip is placed against the page.
5. `position: sticky`, with `top: 0` (an offset is required, or sticky does nothing).
6. `opacity` less than 1 creates a stacking context on `<main>`. The modal's 10000 only counts inside `<main>`; at the page level `<main>` is a single item with z-index 0 (auto), and the sidebar's 2 beats it. Fix: move the modal outside `<main>`, or remove the opacity, or give `<main>` a `z-index` above 2.
7. `.parent { display: grid; place-items: center; min-height: 100vh; }`
8. `float: left` or `float: right` in normal flow. Flex and grid items are boxes placed in tracks or along an axis; neither lets line boxes of text flow around another box.

Chapter 10

Chapter 10, Practice Problems C.

1. `hsl(213 94% 68%)`; hover: `hsl(213 94% 82%)` (lightness raised, or `88%` if you count "20 percent" as 20 points); half-transparent: `hsl(213 94% 68% / 0.5)`.
2. A unitless line-height scales with each element's own font size, so large headings get proportionally large line spacing. A fixed 24 px makes big headings' lines overlap.
3. `4vw` is 12, 32 and 56 px. Clamped between 24 and 48 px: 24 px at 300, 32 px at 800, 48 px at 1 400.
4. `rotate(90deg) translate(50px)`: the axes turn first, so the translation is along the rotated x axis: 50 px *down* (0, 50). `translate(50px) rotate(90deg)`: 50 px right, then spun in place: (50, 0), rotated.
5. `display` has no in-between values to interpolate. Animate `opacity` and `transform`, and set `display: none` only after the transition ends (or use `visibility` with a delay).
6. `@media (hover: none) { .card:hover { transform: none; } }` and `@media (prefers-reduced-motion: reduce) { *, *::before, *::after { animation: none !important; transition: none !important; } }`
7. `.widget-link.active { color: red; }`. The original was an id-anchored chain of (1,2,4) plus `!important`, which nothing could override; it depended on the exact HTML nesting; and it named the id, so it could not be reused.
8. Tokens, reset, layout, components, utilities, media queries (or queries beside their components).

Chapter 11

Chapter 11, Practice Problems A.

1. `'object'` (string); `'object'`; `12` (number); `'34'` (string); `0.30000000000000004` (number); `0`; `1000`; `''` (an empty string, because arrays convert to strings when added).
2. `0`, `''`, `NaN`, `-0` and `null` are false. `'false'` (a non-empty string), `[]` and `{}` are true.
3. `a` is 8 and `b` is 5: numbers are copied by value, so changing `a` does not affect `b`. `o2.n` is 9: objects are shared by reference, so `o1` and `o2` are two names for the same object.
4. `var`: 3, 3, 3 (one shared `i`, already 3 when the timers run). `let`: 0, 1, 2 (a fresh `i` per iteration).
5. `const grade = s => s >= 75 ? 'distinction' : s >= 50 ? 'pass' : 'fail';`
6. `||` treats `0` as missing and replaces it with 10, so a volume of zero becomes 10. Use `settings.volume ?? 10`, which only replaces `null` and `undefined`.
7. `switch (true) { case code >= 500: return 'server error'; case code >= 400: return 'client error'; case code >= 300: return 'redirect'; case code >= 200: return 'success'; default: return 'informational'; }`
8. `let x = 1, n = 0; while (x <= 1000) { x *= 2; n++; }` gives n = 10 (x reaches 1 024).

Chapter 11

Chapter 11, Practice Problems B.

1. `[4.6, 10.35, 17.25, 25.299999999999997]`; `[15, 22]`; `50`. The last decimal is ugly because numbers are binary floating point and 25.3 cannot be stored exactly. Round for display, for example with `toFixed(2)`.
2. It prints `1 2 1`. Each call to `makeCounter` makes a separate `n`, so `c2` starts at 1.
3. `const range = (a, b) => Array.from({ length: b - a + 1 }, (_, i) => a + i);` then `range(1, 9).filter(n => n % 2 === 1).map(n => n * n)` gives `[1, 9, 25, 49, 81]`.
4. `[1, 10, 100, 9]`: the default sort compares as strings, and `'10'` comes before `'9'`. Use `sort((a, b) => a - b)` to get `[1, 9, 10, 100]`.
5. `words.reduce((acc, w) => { acc[w] = (acc[w] ?? 0) + 1; return acc; }, {})` gives `{ a: 3, b: 1, c: 1 }`.
6. `p.tags` is `['x', 'y']`. Spread is a shallow copy, so `q.tags` is the same array as `p.tags`. Copy the inner array too (`{ ...p, tags: [...p.tags] }`) or use `structuredClone(p)`.
7. `const pick = (obj, keys) => Object.fromEntries(keys.filter(k => k in obj).map(k => [k, obj[k]]));`
8. `const makeAccumulator = start => { let total = start; return x => (total += x); };` With `start = 100`: `acc(5)` is 105, `acc(10)` is 115.
9. `sort` sorts in place. Use `people.toSorted(...)`, or `[...people].sort(...)`.

Chapter 11

Chapter 11, Practice Problems C.

1. `stop() { this.duty = 0; }` and `get isRunning() { return this.rpm > 0; }`. Going through the setter keeps the clamp and any other rule in one place; writing the private field directly bypasses them (and would be a second place to change if the rule changed).
2. Called loose, `describe` has no `this` (class bodies are strict), so `this.name` throws. Fix: `const f = m.describe.bind(m)`, or call it through the object, `const f = () => m.describe()`.
3. It swallows every error, including bugs, and the program continues in a broken state. `try { risky(); } catch (e) { if (e instanceof RangeOutOfBounds) handle(e); else throw e; }`
4. The script ran before the element existed (no `defer`, and the script is in the head), or the selector does not match (the id is misspelled or the element is a different one).
5. `1`, `2`, `4`, `3`. The async function runs synchronously until its first `await`, so 2 prints before 4; the continuation after `await` runs as a microtask after the main code finishes, so 3 comes last.
6. `const [a, b] = await Promise.all([getJSON(u1), getJSON(u2)]);` If either rejects, `Promise.all` rejects immediately with that error and the `await` throws; use `Promise.allSettled` if you want both results regardless.
7. `fetch` only rejects when no response arrives at all (network failure); a 404 is a response. Add `if (!res.ok) throw new Error(res.status);` before reading the body.
8. `const run = debounce(q => search(q), 300); document.querySelector('#q').addEventListener('input', e => run(e.target.value));`

Chapter 11

Chapter 11, Practice Problems D.

1. `import fitCanvas, { COL } from './charts.js';`
2. A plain script is not a module, so `import` is a syntax error. Either load it with `<script type="module" src="main.js">` (served over HTTP, since modules do not load from `file://`), or bundle it (Chapter 16) into a single plain script.
3. `text.split('\n').slice(1).map(l => { const [name, age] = l.split(','); return { name, age: +age }; })` gives `[{ name: 'Ana', age: 31 }, { name: 'Ben', age: 17 }]`.
4. `n` is `'51'`. Storage holds strings, so `getItem` returns `'5'` and `+ 1` joins text. Use `Number(localStorage.getItem('n')) + 1`, or store and read JSON.
5. `test('lerp and smoothstep', () => { assertEq(lerp(0, 200, 0.25), 50); assertEq(smoothstep(0.5), 0.5, 1e-12); });`
6. `{ ...state, filters: { ...state.filters, tag: 'aero' } }`
7. Linter: an unused variable, a missing `await`, use of ==, a variable used before declaration. Test: that `niceTicks(0, 0.0123, 4)` returns 0.005 (a wrong *result*, which no linter can know).

Chapter 12

Chapter 12, Practice Problems A.

1. `n` becomes 1: all three calls use the same `n` from that render (0), so each sets it to 1. Use the function form: `setN(c => c + 1)` three times, which gives 3.
2. `push` changes the array in place and returns the *new length* (a number), so `setItems(3)` stores a number, and the original array was mutated. Use `setItems([...items, 'c'])`.
3. `setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t))`.
4. It shows the text `0`, because `0 && x` is `0` and React renders numbers. Write `{count > 0 && <Badge n={count} />}`.
5. Price: a prop (or data loaded by an effect into state, then passed down). Quantity: state. Total: derived (price × quantity). Button enabled: derived (quantity > 0). Currency symbol: a constant or a prop.
6. `<label htmlFor="q" className="big">Search <input id="q" onChange={go} /></label>` (the handler is a function reference, and the void `<input>` must be self-closed).
7. In their nearest common parent, as `const [selected, setSelected] = useState(null)`. Each child receives `selected` as a prop; the one that can change it also receives `setSelected` (or a wrapper) and calls it.

Chapter 12

Chapter 12, Practice Problems B.

1. The keys are probably array indexes, so after the deletion React matches rows by position. Use a stable id from the data as the key.
2. `useEffect(() => { const f = () => setWidth(window.innerWidth); window.addEventListener('resize', f); return () => window.removeEventListener('resize', f); }, []);`
3. With `[]` the effect runs once, so a new `id` never refetches. Put `[id]` in the list. To be safe against fast changes, keep a `cancelled` flag set in the cleanup and ignore a reply that arrives after it (or abort the fetch with an `AbortController`).
4. `shown` can be computed from `query` and `fruits` on every render. Keeping it in state creates a second copy that can fall out of step with the first, the very problem state is meant to avoid.
5. Not a bug: StrictMode mounts, unmounts and mounts again in development to test cleanup. The cleanup must unsubscribe, so the net effect after the double run is exactly one subscription.
6. `const [chart, setChart] = useState('speed'); <select value={chart} onChange={e => setChart(e.target.value)}>{['speed', 'power', 'sound'].map(c => <option key={c} value={c}>{c}</option>)}</select>`
7. Changing a ref does not cause a render, so the screen keeps the old number. A ref is right for values the picture does not depend on: a timer id, a DOM node, the previous value of something, a render counter for debugging.
8. `npm install react react-dom` and `npx esbuild app.jsx --bundle --jsx=automatic --outfile=app.js`; then `import { createRoot } from 'react-dom/client'; import App from './App.jsx'; createRoot(document.getElementById('root')).render(<App />);`

Chapter 13

Chapter 13, Practice Problems A.

1. 400 × 2 = 800 and 250 × 2 = 500. With a DPR 2 transform, the line goes from device (20, 20) to (220, 20).
2. x = 50 + (2.5 / 10) × 400 = 150. Pixel 450 is (450 − 50) / 400 × 10 = 10.
3. The plot spans pixels 20 to 220. The y pixel is 20 + 200 − ((v + 1) / 2) × 200. For 0: 220 − 100 = 120. For 1: 220 − 200 = 20. The value 1 is smaller in pixels because pixel y increases downward, so larger data values are drawn higher up.
4. Raw 0.0925, p = 0.01, f = 9.25, step 0.1; ticks 0, 0.1, 0.2, 0.3, 0.4. For −12 to 48 with 6: raw 10, p = 10, f = 1, step 10; ticks −20 (the first multiple of 10 at or below −12), −10, 0, ..., 50.
5. `ctx.arc(50, 50, 10, 0, Math.PI * 2)` for the full circle; `ctx.arc(50, 50, 10, Math.PI, 2 * Math.PI)` for the top half (from nine o'clock clockwise over the top to three o'clock).
6. Both strokes are in the same path, so the blue stroke re-draws the red line in blue as well; after two frames the earlier line is drawn in the later colour, and every frame adds more to the path.
7. (320 − 100, 180 − 60) = (220, 120).
8. The rotation turns the axes about the canvas origin first, so the subsequent translation moves the needle along the rotated axes: as `a` changes, the needle's *pivot* swings in a circle around the canvas's top-left corner instead of staying at (cx, cy).
9. The handler runs on every pointer movement, far more often than the screen can show, and drawing there competes with the frame loop. Storing the index and drawing in the loop keeps one drawing per frame and keeps the picture a function of state.

Chapter 13

Chapter 13, Practice Problems B.

1. `M0 0 L10 0 L5 8 Z`.
2. 600 / 200 = 3 pixels per unit, so the height is 100 × 3 = 300 pixels (the aspect ratio of the viewBox is preserved).
3. `<g transform="translate(40 20) rotate(30)">...</g>` (the rotation is applied about the group's new origin, after the translation).
4. The element is created in the HTML namespace, where "rect" means nothing. Use `document.createElementNS('http://www.w3.org/2000/svg', 'rect')`.
5. `.wire { stroke-dasharray: 6 6; animation: flow 1.2s linear infinite; } @keyframes flow { to { stroke-dashoffset: -12; } }`. It is a decoration that does not depend on data, so CSS runs it off the main thread and it survives a script failure.
6. (a) canvas: too many marks for elements. (b) SVG: regions are elements that can be clicked and styled. (c) SVG with `currentColor`. (d) canvas: redrawn every frame. (e) either; SVG is simpler because each slice is a `<path>` that can take a hover tooltip without hit testing.
7. `role="img"` with an `aria-label` describing the chart, and a visually hidden table (or text summary) of the data next to it.

Chapter 14

Chapter 14, Practice Problems A.

1. The starts are 0, 0.2, 0.4, 0.6, 0.8 seconds. The last box starts at 0.8 and finishes at 1.8. The total is 1.8 s (GSAP: `total 1.80 s`).
2. Tween a: 0 to 2. Tween b: `'-=0.5'` starts 0.5 before the timeline end (2), so 1.5 to 2.5. Tween c: absolute position 0, so 0 to 1. Total: 2.5 s (GSAP: `a: 0.00..2.00 c: 0.00..1.00 b: 1.50..2.50 total 2.50 s`).
3. `x` is applied as a CSS transform, which can be done in the compositor without layout; `left` changes geometry and forces layout and paint on every frame.
4. `tl.from('h1', {opacity: 0, duration: 0.6}).from('li', {opacity: 0, stagger: 0.15, duration: 0.4}, '-=0.2');`
5. With `none` it is at 25 (a quarter of the way, linear). With `power2.in` it is below 25, about 6, because an ease-in starts slowly (t^2 at 0.25 is 0.0625).
6. `'<'` is the start of the previous tween; `'>'` is the end of the previous tween; `'+=0'` is the end of the whole timeline so far, which can be later than the previous tween's end if an earlier tween was longer.
7. `tl.play()` to open and `tl.reverse()` to close. One timeline guarantees the close mirrors the open and halves the code; if the design changes, there is one place to edit.

Chapter 14

Chapter 14, Practice Problems B.

1. 80% of 900 is 720 px from the top of the viewport. `'top center'` is 450 px. `'bottom bottom'` fires when the trigger's bottom reaches the viewport's bottom, 900 px.
2. It smooths the link between scroll and progress with about half a second of catch-up, so the animation glides instead of snapping to each wheel notch.
3. Create the pinned timeline inside `gsap.matchMedia().add('(min-width: 800px)', ...)` so phones get no pin (and give them a simpler animation in another `add`).
4. `const mm = gsap.matchMedia(); mm.add('(prefers-reduced-motion: no-preference)', () => { gsap.from('.panel', {opacity: 0, y: 40}); });` They should see the final state: the panel visible, in place, with no motion (set it with `gsap.set` or leave the CSS default visible).
5. `ScrollTrigger.refresh()`.
6. The tweens, the timeline and the ScrollTrigger instances. `ctx.revert()` on a `gsap.context()` created in the effect kills all of them.
7. (a) by hand (Project 3's observer and a CSS transition). (b) GSAP. (c) by hand (a CSS `:hover` transition, no script at all). (d) either; GSAP's ScrollTrigger with `scrub` replaces the hand-written scroll maths and throttle, but twenty lines of plain JavaScript also do it, as Part V shows on this website's portfolio page.

Chapter 15

Chapter 15, Practice Problems A.

1. Visible height = 2 × 2 × tan(25°) = 4 × 0.4663 = 1.865 m. For 1.2 m at 3 m: θ = 2 atan(0.6 / 3) = 2 atan(0.2) = 22.6°.
2. (cos 90°, sin 90° terms) x′ = 1 × 0 + 0 = 0 and z′ = −1 × 1 + 0 = −1, so the child is at (0, 0, −1). three.js: `[0, 0, -1]`.
3. `camera.updateProjectionMatrix();` the projection is stored and not recalculated automatically.
4. `bar.position.x = 0.12; arm.add(bar);` (the bar's centre is half its length from the group's origin). After `arm.rotation.z = Math.PI / 6` the centre is at (0.12 cos 30°, 0.12 sin 30°, 0) = (0.104, 0.060, 0).
5. Azimuth 90°, elevation 0°: (cos 0 cos 90°, sin 0, cos 0 sin 90°) = (0, 0, 1). Azimuth 0°, elevation 90°: (0, 1, 0), straight above the target (where `lookAt` needs an up-vector hint; keep elevation just under 90°).
6. Is there a light (a Standard material is black without one)? Is the camera outside the object and looking at it (`lookAt`)? Does the canvas have a CSS size (a canvas with no height shows nothing)? Is `renderer.render` called inside the loop? Are the near and far planes bracketing the object?
7. It turns a fixed amount per frame, so it spins twice as fast on a 144 Hz screen as on a 60 Hz one. Use `cube.rotation.y += speed * dt` with `dt` from a `Clock`.

Chapter 15

Chapter 15, Practice Problems B.

1. No environment map (so nothing to reflect): add `scene.environment`. Or roughness too high, or metalness too low: lower the roughness and raise the metalness.
2. `MeshBasicMaterial({ wireframe: true })`; `MeshLambertMaterial` (cheap, many objects); `MeshPhysicalMaterial` with `clearcoat: 1`; `MeshPhysicalMaterial` with `transmission: 1` and low roughness.
3. (3, 4, 0) has length 5, so l = (0.6, 0.8, 0); n · l = 0.8.
4. Merging turns 300 draw calls into 1, which usually makes a big difference to the frame rate. The cost is that the merged mesh is a single object: you cannot move, hide or highlight one part on its own. Instance instead when the parts are copies of the *same* geometry that must keep their own transforms (screws, particles).
5. The raycaster ignores `visible`, so the hidden mesh still intercepts the ray. Filter the list first: `ray.intersectObjects(meshes.filter(m => m.visible))`.
6. GPU memory grows by 50 textures a minute, 3 000 over an hour, until the page slows or crashes. Call `texture.dispose()` when a texture is no longer needed.
7. Measure `renderer.info.render.calls` and the frame time first. Then: cap the pixel ratio at 1 on that tier; disable post-processing and shadows (or shrink the shadow map); merge and instance to cut draw calls; share materials. Make it a tier that switches automatically when

frames stay long.

8. `texture.colorSpace = THREE.SRGBColorSpace` (a colour map must be marked sRGB; data maps must not).

Chapter 16

Chapter 16, Practice Problems A.

1. `^4.17.2` allows >=4.17.2 and <5.0.0, so 4.17.2, 4.18.0 and 4.99.9 are accepted; 5.0.0 is not.
2. `~4.17.2` allows >=4.17.2 and <4.18.0, so 4.17.9 only. `^0.186.1` allows <0.187.0, so 0.186.9 only.
3. Bundle the scripts into one file with a bundler and load that; or load it as a module with `<script type="module" src="main.js">` (and serve it over a local server, since modules do not load from a `file://` address).
4. Two, plus anything those two call. Tree shaking.
5. `--sourcemap`, which writes `app.js.map`; the browser reads it and shows the original source.
6. `"build": "esbuild main.js --bundle --minify --format=iife --outfile=app.js"`, run with `npm run build`.
7. `node_modules` is huge and can be regenerated, so storing it only bloats the repository. `package-lock.json` is small and records the exact versions, so keeping it makes builds repeatable.

Chapter 16

Chapter 16, Practice Problems B.

1. `git add fix1.js fix2.js` and `git commit -m "Fix ..."`; then `git add feature.js` and `git commit -m "Add ..."`.
2. `git switch -c experiment` creates the branch and moves to it while keeping your uncommitted changes, which are now on the new branch.
3. A first commit; a branch `experiment` with a commit "Try a wider track"; meanwhile on main a commit "Fix the typo"; then the experiment merged into main with a merge commit that has both as parents.
4. Open `style.css`, edit the block between `<<<<<<<` and `>>>>>>>` to what it should be (removing the markers), `git add style.css`, `git commit`.
5. `git revert <hash>`: it adds a new commit that removes the line and leaves the shared history intact. `git reset` would rewrite history that others may already have pulled.
6. `git restore main.js`.
7. The image paths start with `/` (root-relative), so on Pages they point at `https://you.github.io/images/...` instead of `.../site/images/...`. Use relative paths.
8. Good: "Cap the device pixel ratio at 2: a 3x phone was rendering nine pixels per CSS pixel and ran at 12 fps." Bad: "fix bug" or "update main.js": the first says nothing, the second repeats what the diff already shows, and neither says why.

Chapter 17

Chapter 17, Practice Problems A.

1. At line 11, the line before. The compiler often reports a missing semicolon at the next statement.
2. The first prints `2` (whole-number division). The second prints `2.25`.
3. 7 + 3 = 10, then 10 x 2 = 20. So `x` is `20`.
4. omega = 120 x 2 pi / 60 = 12.566 rad/s, and v = 12.566 x 0.05 = 0.628 m/s.
5. `static_cast<int>(-2.7)` is `-2` (it cuts the decimals toward zero); `std::round(-2.7)` is `-3`.
6. `double a, b; std::cin >> a >> b; std::printf("%.3f", (a + b) / 2);`, inside `main`, with `<iostream>` and `<cstdio>` included.
7. In a `printf` format string, `%` starts a code such as `%d`, so a real percent sign must be written twice.
8. `int` for the count, `double` for the voltage, `bool` for the claw.

Chapter 17

Chapter 17, Practice Problems B.

1. `"SLOW"`. 0.2 is above 0.108 so it is not BRAKE, and at or below 0.30 so it is SLOW.
2. `!a || !b`: it is false only when both `a` and `b` are true.
3. `10 7 4 1`.
4. Five times, and `n` ends at 243 (1, 3, 9, 27, 81, 243).
5. `double average(double a, double b) { return (a + b) / 2; }`
6. 2, then 5, then 9 (the static `c` keeps its value between calls).
7. `9` and `9` (the first calls the `int` version, the second the `double` version, which prints as 9.00 with `%.2f`).
8. 25.

Chapter 17

Chapter 17, Practice Problems C.

1. `v.size()` is 3, `v[1]` is 9, `v.back()` is 2.
2. Sorted it is 1 2 3 4 5, so the median is 3. A single wild reading moves the mean a long way but moves the median by at most one place.
3. (0.82 + 0.79 + 0.85) / 3 = 0.82.
4. `argc` is 3, `argv[1]` is `"--duty"`, `argv[2]` is `"0.9"` (`argv[0]` is the program's name).
5. 14.88 rad/s (about 142 rpm) and 0.77 A, from the program's output.
6. So that only the class's own functions can change them, which lets the class keep its rules (no zero resistance, for example) and means a bug can only come from a few places.
7. You cannot call it on a `const` object or through a `const` reference; the compiler stops with an error. Marking it `const` states the promise and fixes that.
8. `s` is `"abcd"` and `s.size()` is 4.

Chapter 17

Chapter 17, Practice Problems D.

1. Nothing: no `case` mentions the button while OPENING, so it is ignored.
2. Its names are scoped (`Door::OPEN`), so they cannot clash with other names, and it will not silently convert to an integer.
3. When the pointer goes out of scope, at the closing brace of its block, before `end of main` is printed.
4. `[limit]` takes a copy when the lambda is made, so later changes to `limit` are invisible to it. `[&limit]` refers to the original, so it sees changes.
5. Seven lines: the header and six data rows (the program read 6 rows).
6. `g++ -std=c++17 -Wall -Wextra -O2 main.cpp util.cpp -o prog`
7. A link error. The likeliest cause is that the source file defining `lerp` was not given to `g++`.
8. It stops the header being read twice in one file, which would declare everything twice and cause errors.

Chapter 17

Chapter 17, Practice Problems E and F.

1. The true answer swings between -1 and +1 for ever. Euler's grows without limit (to 4.47 and more) because each step adds energy; the error is not "a little off" but a different kind of behaviour.
2. To hold 0.5 m/s it must push with 1.5 + 2 x 0.5 = 2.5 N, but P pushes only when there is an error. Setting 10 x (0.5 - v) = 1.5 + 2v gives 5 - 10v = 1.5 + 2v, so v = 3.5 / 12 = 0.2917 m/s, exactly what the program printed.
3. Integrator windup: when the system is held back, the integral grows without limit, and afterwards it keeps pushing long after the error has gone, causing a large overshoot. You would notice the first time something blocks the robot and then lets go.
4. It should be 0, because standing still means zero speed. It was written wrong on purpose, to show what a failure report looks like.
5. Decimals are not stored exactly, so a calculated value can differ from the expected one by a tiny amount even when the code is right; a tolerance accepts those.
6. `CHECK_NEAR(lerp(10, 20, 0.25), 12.5, 1e-12);`
7. `Vec2{3, 4} * 0.5` is `(1.5, 2)`, and adding `(1, 2)` gives `(2.5, 4)`.
8. `5`, since the length is the square root of 9 + 16. And `clampT<int>(15, 0, 10)` returns `10`.

Chapter 17

Chapter 17, Mini project problems.

1. Add `{"mi", 1609.344}` to the `UNITS` table; `N` updates itself.
2. Two bad readings in a row mean the middle of any three consecutive readings still includes a bad one. A median of five would catch a pair (it would take three bad readings to fool it).

3. Setting y = 1 m, 1 = 1/2 x 9.80665 x t squared, so t = square root of (2 / 9.80665) = 0.4516 s.
4. The wheels lag the command, so at the moment the heading reaches the target the wheels are still turning at speed, and the robot carries on. P has no term that notices it is moving fast.
5. Mostly from the four straight legs: a right wheel 1 percent fast steers the robot slightly left, 0.01 x 0.3 / 0.193 = 0.0155 rad/s for 3.33 s on each leg, about 3 degrees a leg and 11.9 degrees over four. The turns add about 0.45 degrees each (1.8 degrees over four), making 13.7 degrees in all.
6. For example: the bouncing ball (project 3) with the sonar filter (project 2) applied to its height, to see whether a median filter would recover the bounce heights from noisy readings.

Chapter 18

Project 1 quiz.

1. Widths would no longer include padding and border, so the cards would be wider than their grid cells and might overflow or crowd each other.
2. It makes a phone use its true width, so the layout and media queries respond to the real screen size.
3. `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))`: the browser works out how many columns of at least 220 px fit, so fewer fit as the container narrows.
4. `rem` respects the visitor's text-size setting, and `clamp` lets the size scale smoothly between a minimum and a maximum without media queries.
5. `<main>`.

Chapter 19

Project 2 quiz.

1. `items` and `state` together. The cards on screen are derived from them.
2. Titles are treated as plain text, so a title containing < or a script cannot be executed (Chapter 8's XSS lesson).
3. Two cards: "Line follower" and "Claw gripper" (the two with tag `robots`).
4. Less memory, and it still works for chips added later, because their clicks bubble up to the same parent.
5. Add `<button data-tag="all">Reset</button>`. Nothing in the JavaScript changes, because `all` already shows every item.

Chapter 20

Project 3 quiz.

1. CSS describes the looks; JavaScript adds the class `shown`.
2. It runs only when visibility changes and does not force layout on every scroll tick, so scrolling stays smooth.
3. The reveal is a one-time event; stopping the observation saves work and stops the animation repeating.
4. The animation starts when the top of the element reaches 80% of the way down the viewport.

5. The by-hand version: it is small, needs no library, and the effect is simple. (GSAP pays off for complicated sequences.)

Chapter 21

Project 4 quiz.

1. $\omega_d = 2 \times \sqrt{(1 - 0.01)} = 2 \times 0.99499 = 1.990$ rad/s. Period $= 2\pi / 1.990 = 3.157$ s.
2. Each peak is $e^{-2\pi\zeta/\sqrt{(1-\zeta^2)}} = e^{-0.6314} = 0.532$ of the previous one, so only about 47% is lost per cycle.
3. A pure maths function can be checked on its own against hand calculations, and a wrong picture can then be blamed on the drawing and not the maths.
4. It maps v = 0 to the vertical middle (`h / 2`) and v = ±1 to 20 pixels from the top and bottom. The subtraction flips the axis, because canvas y increases downward.
5. It depends on the clock (`now - t0`), not on a frame count, so it moves at the same speed on any screen (Chapter 6).

Chapter 22

Project 5 quiz.

1. A raycast hit returns a mesh, and `userData` tells you which part it is and what its description is, with no search.
2. `explode`. Because every position is computed from it, any control (slider, button, tour) can drive the model by setting that one number.
3. It converts the pointer position to normalised device coordinates (−1 to 1) and flips the y axis, because the screen's y goes down and the camera's goes up.
4. `camera.updateProjectionMatrix()` after setting the aspect ratio.
5. Add one more object to `PARTS` with a size, position, explode offset, colour and description. No other code changes.

Chapter 23

Project 6 quiz.

1. The back-voltage (KE times wheel speed) grows with speed until it almost cancels the supply voltage, so the current and the force fall to nearly zero. With no force left, the cart stops accelerating and holds its top speed.
2. A spinning motor generates its own voltage. With the supply at 0 V, that voltage drives a current through the winding, and a current in a magnetic field produces a torque that opposes the rotation. It acts as a brake.
3. The brake starts at 20 cm, but the cart is moving at 0.45 m/s and needs distance to slow down (about 5.7 cm here). It stops after that coasting distance.
4. Lower the `STOP_AT` distance (brake later), or lower the duty so the cart travels more slowly and needs less distance to stop. Braking later is riskier; slower is safer.

5. Because the command is the thing that makes the motor push. Changing the speed directly would be teleporting the robot's motion, ignoring the mass; changing the command lets the real physics (mass, braking) produce a realistic, gradual halt.

Chapter 24

Advanced CSS quiz.

1. The order of precedence of three layers: rules in `c` beat `b`, which beat `a`, regardless of specificity.
2. When a component's layout should depend on the space it is given (a card in a sidebar versus the main column) and not on the screen width.
3. It follows the writing direction, so layouts mirror automatically for right-to-left languages, where "left" means the opposite side.

Chapter 25

Advanced JavaScript quiz.

1. It loads a module at the moment it is called (and returns a promise), instead of at page start.
2. `var`: 3, 3, 3 (one shared variable, already 3 when the timers fire). `let`: 0, 1, 2 (one new variable per iteration).
3. Workers have no access to the DOM; they only send and receive messages. That keeps the two threads from interfering with each other's data.

Chapter 26

Advanced React quiz.

1. Otherwise the loop keeps running after the component is gone, leaking memory and GPU resources and stacking up duplicates.
2. When you need a value that persists across renders without triggering a re-render, such as a DOM node, a timer id or a renderer.
3. Memoisation has its own cost and most renders are cheap, so it adds complexity for no gain. Use it where a profile shows a slow render.

Chapter 27

Advanced canvas quiz.

1. Anything that does not change from frame to frame: grids, axes, backgrounds, fixed labels.
2. It uses CPU, GPU and battery for a picture nobody can see.
3. The line sits on the boundary between two device pixels, so it is spread across both at half intensity. Shift it by half a pixel to land on one.

Chapter 28

Advanced GSAP quiz.

1. It is guaranteed to mirror the opening animation, and there is only one animation to maintain.

2. So that small screens and visitors who prefer reduced motion get a simpler version instead of a pinned effect that is awkward or unwelcome.
3. Otherwise they keep running and holding memory, and a scroll trigger can keep firing on elements that no longer exist.

Chapter 29

Advanced three.js quiz.

1. Fine apparent surface detail (grain, layer lines) in the lighting, at the cost of one texture rather than extra triangles.
2. Different devices have very different speed, so the page steps down its effects to stay smooth on weaker ones.
3. Instancing draws many copies of the *same* mesh with different positions in one call. Merging joins many *different* meshes that share a material into one mesh, so they cost one call between them.

Chapter 30

Advanced accessibility and performance quiz.

1. For example: whether labels make sense, a logical tab order, whether a flow can be completed with a screen reader, whether content survives 400% zoom.
2. A numeric limit (for example JavaScript size or LCP time). Written down and checked on every change, it makes gradual slowing visible.
3. Because the developer's machine is faster than most visitors', so problems only show up under a slowdown.

Chapter 31

Advanced C++ quiz.

1. The step size dropped to a quarter, so (step times natural frequency) fell from 3.2 to 0.79, below the stability limit of 2.
2. A mistake in one program would have to be repeated by accident in the other for the tables to agree. If the programs are written separately and agree on thousands of numbers, the chance of a shared mistake is tiny.
3. It lets the compiler reorder floating-point arithmetic for speed. Reordering changes the last digits, so two programs that should agree exactly would differ slightly and an exact comparison would fail.
4. During development and testing, to catch out-of-range reads and similar mistakes with a clear message. Not in the final build, because it makes the program slower.
5. The edges would change: reading the sonar and driving the motors would use the board's own functions instead of the simulation's physics. The state machine, ramps and soft stop would stay.

Chapter 42

Chapter 42, Try It Yourself.

1. At 100 percent duty the supply is 4.6 V, so the predicted free speed is (4.6 - 1.14) / 0.2326 = 14.88 rad/s, and the program reaches that, since the motor model is the same.
2. A robot with a rubber-banded left wheel and a bare right wheel gets more push on the left, so with equal commands it would curve towards the bare side. Differential drive assumes both wheels grip equally; here they would not.
3. It could detect an obstacle at the side and steer round it, or measure a wall's distance to correct its heading.
4. It still finishes, but later (99.61 s) and the items land about 2.7 cm further in (x of about -0.64), because braking later leaves the claw past the pad's centre.
5. It finishes sooner (92.05 s) because the robot drives faster, but the items land about 7 cm further in (x of about -0.68), near the edge of the plates: speed costs accuracy.
6. Add an entry to `ITEMS`, a lane to `LANES`, change the array sizes and `NITEM`, and make sure the pad and plate lists still cover every lane.
7. Drive the sonar across the item in small steps and record where the distance jumps down (the near edge) and back up (the far edge); the width is the distance travelled between them. It needs a slow sideways drive and a median filter (mini project 2).
8. For example: SEARCH (spin until the sonar sees something closer than 1 m), APPROACH (drive until the bumper closes), GRIP or PUSH (hold for 1 s), RETURN (drive back to the start, ended by a distance check), DONE. First measure: wheel radius, track, motor free speed, the sonar's cone angle and the bumper's reaction time.

Chapter 42

Building Patrick: quiz.

1. At full throttle, bare wheels reached 0.25 m/s in 0.22 s with a peak slip of 0.76; with bands, 0.11 s and a peak slip of 0.06. Same motor, double the acceleration.
2. The forward speed is the mean of the two wheel speeds and the turning rate is their difference over the track (lines 83 and 84 of `stage3.cpp`); `yaw += om * dt` then `x` and `z` follow from `v` and `yaw`.
3. The pad's front face is at 0.60 - 0.04 = 0.56 m; the sensor is 0.145 m ahead of the axle, so at x = -0.05 it is at 0.095 m, giving 0.56 - 0.095 = 0.465 m.
4. Braking takes about 1.8 cm of coasting, so he starts braking early. The threshold was tuned so the claw ends exactly over the item.
5. Every drive command and the controller's own clock, so wheels, arm, claw and timers all slow and stop together.
6. The two were run side by side, writing the same table, and compared byte for byte; they were identical, and the page embeds the very file that is offered for download.